
Appendix: SpyGlass Design Constraints

SpyGlass® Design Constraints (SGDC) are used to:

- Provide additional design information that is not apparent in RTL.
- Restrict SpyGlass analysis to a set of design objects.

Writing Constraints in an SGDC File

You can write constraints in a text file that can have any extension. However, it is recommended that you use the .sgdc extension to distinguish this file from other files.

Specifying SGDC File to SpyGlass

Specify SGDC files in any of the following ways:

- By using the `read_file -type sgdc <SGDC-file-name>` command in a project file
- By using the *Add Files(s)* option under the *Add Design Files* tab in Atrenta Console GUI

You can specify multiple SGDC files. In this case, SpyGlass processes these files in the specified order.

Working with SGDC Files

The following table describes various tasks that you can perform in SGDC files:

Task	Description
Adding comments	Use # or // to add comments.
Defining a scope for constraints	A scope defines a design unit for which the specified constraints are applicable. To define a scope, use the <code>current_design <design-unit></code> command before writing SGDC commands, where <code><design-unit></code> can be any of the following: • For Verilog: <code><module-name></code> • For VHDL: <code><entity-name></code>, <code><entity-name>.<archname></code>, <code><configuration-name></code>, <code><libname>.<configuration-name></code> For details, refer to the <i>Defining a Scope for Constraints</i> topic of the <i>Atrenta Console User Guide</i>.
Specifying multiple values to constraint arguments	You can specify multiple values to arguments in different ways. For details, refer to the <i>Specifying Multiple Values for a Constraint Argument</i> topic of the <i>Atrenta Console User Guide</i>.
Handle interdependencies between constraint arguments	When two arguments of the same constraint are interdependent, specify the exact matching number of values with each argument. For details, refer to the <i>Handling Interdependencies between Different Arguments</i> topic of the <i>Atrenta Console User Guide</i>.

Task	Description
Specify signals names in the correct format	Certain constraint arguments accept names of signals, such as clock signals and low power signals. Based on the type of signals, such as scalar or vector signals or the design hierarchy in which signals are present, you must specify signal names in a correct format so that SpyGlass can identify them correctly. For details, refer to the <i>Specifying Signal Names</i> topic of the <i>Atrenta Console User Guide</i>.
Define and use variables	Variables are used to store values that can be used as argument values of constraints. Once you define a variable and assign a value to it, you can use that variable name as the value of a constraint argument. SpyGlass internally expands that variable name to its value for that argument. For details, refer to the <i>Defining and Using Variables</i> topic of the <i>Atrenta Console User Guide</i>.
Include an SGDC file in another SGDC file	Use the <code>include</code> directive to include an SGDC file in another SGDC. The <code>include</code> directive is used in the following format: <pre>include <file-name></pre> For details, refer to the <i>Including an SGDC File in Another SGDC File</i> topic of the <i>Atrenta Console User Guide</i>.

Task	Description
Implement compilation of SGDC commands based on certain conditions	To use the same SGDC file for different functional and testing analysis modes, compile different commands from the same SGDC file based on different conditions. For details, refer to the <i>Conditionally Specifying SGDC Constraints</i> topic of the <i>Atrenta Console User Guide</i>.
Importing Block-Level SGDC Commands to Chip-Level	While integrating design blocks at chip-level, you can migrate block-level SGDC files to the chip-level for performing chip-level analysis. For details, refer to the <i>Importing Block-Level SGDC Commands to Chip-Level</i> topic of the <i>Atrenta Console User Guide</i>.
Using the <code>::</code> operator to define a scope for searching instances in design units	For details, refer to the <i>Implementing Scoping in SGDC Commands</i> topic of the <i>Atrenta Console User Guide</i>.

Handling of Duplicate Constraint Specifications

If you have given multiple specifications for a constraint that can be applied only once on a design object, the following actions occur:

- SpyGlass considers only the last specification of that constraint.
- SpyGlass reports the *SGDCWRN_115* warning and ignores the rest of the specifications of that constraint.

Consider the following example:

```
current_design top  
  
set_case_analysis -name in -value 0  
set_case_analysis -name in -value 1
```

For the above example, SpyGlass considers only the last `set_case_analysis` constraint specification and ignores the first constraint specification that sets the value of the `in` pin to 0.

Renamed Constraints

Some SGDC constraints have been renamed to improve readability and consistency with all the other constraints in SpyGlass. The original names, however, are still supported for backward compatibility.

The following table lists the old names and the corresponding new names of constraints:

Old Name	New Name
aonbuffer	<i>always_on_buffer</i>
aonbufferedsignals	<i>aon_buffered_signals</i>
apcell	<i>antenna_cell</i>
balancedClock	<i>balanced_clock</i>
cellhookup	<i>cell_hookup</i>
clockPin	<i>clock_pin</i>
clockshaper	<i>clock_shaper</i>
defineTag	<i>define_tag</i>
expectFrequency	<i>expect_frequency</i>
gatingcell	<i>gating_cell</i>
gatingcell_enable	<i>gating_cell_enable</i>
ignorepdcrossing	<i>ignore_crossing</i>
inisocell	<i>input_isocell</i>
IP_block	<i>ip_block</i>
isocell	<i>isolation_cell</i>
lpsignal	<i>assertion_signal</i>
memory3s	<i>memory_tristate</i>
memoryreadpin	<i>memory_read_pin</i>
memorytype	<i>memory_type</i>
memorywritedisable	<i>memory_write_disable</i>
memorywritepin	<i>memory_write_pin</i>
moduleByPass	<i>module_bypass</i>
modulePin	<i>module_pin</i>
nofault	<i>no_fault</i>

Renamed Constraints

Old Name	New Name
noScan	<i>force_no_scan</i>
pdsequence	<i>power_down_sequence</i>
pdsignal	<i>domain_signal</i>
pgcell	<i>pg_cell</i>
pgpins_naming	<i>pg_pins_naming</i>
pi_drive_strength	<i>input_drive_strength</i>
pinvoltage	<i>pin_voltage</i>
porttimedelay	<i>port_time_delay</i>
powerdomaininputs	<i>domain_inputs</i>
powerdomainoutputs	<i>domain_outputs</i>
powerswitch	<i>power_switch</i>
pullDown	<i>pulldown</i>
pullUp	<i>pullup</i>
raminstance	<i>ram_instance</i>
ramswitch	<i>ram_switch</i>
requirePath	<i>require_path</i>
requireValue	<i>require_value</i>
resetPin	<i>reset_pin</i>
retain_instance	<i>retention_instance</i>
retencell	<i>retention_cell</i>
scanchain	<i>scan_chain</i>
scanratio	<i>scan_ratio</i>
scantype	<i>scan_type</i>
scanwrap	<i>scan_wrap</i>
sdcschema	<i>sdc_data</i>
selectwireloadmodel	<i>select_wireload_model</i>
seqATPG	<i>seq_atpg</i>
set_black_box_power	<i>blackbox_power</i>
set_cell_pin_info	<i>cell_pin_info</i>
set_cell_tie_class	<i>cell_tie_class</i>
set_design_map_info	<i>design_map_info</i>
set_instance_for_activity_trace	<i>instance_trace</i>

Old Name	New Name
setPin	<i>set_pin</i>
shadowratio	<i>shadow_ratio</i>
smodule	<i>special_module</i>
specialcell	<i>special_cell</i>
testclock_frequency	<i>atspeed_clock_frequency</i>
testclockFrequency	<i>atspeed_clock_frequency</i>
testmode	<i>test_mode</i>
testpoint	<i>test_point</i>
voltagedomain	<i>voltage_domain</i>
wireloadselection	<i>wireload_selection</i>

SpyGlass Design Constraints

The following table lists the SpyGlass Design Constraints (SGDC) used by various SpyGlass products:

SpyGlass Auto Verify Solution		
<i>clock</i>	<i>reset</i>	<i>define_tag</i>
<i>set_case_analysis</i>	<i>special_module</i>	<i>breakpoint</i>
<i>watchpoint</i>	<i>formal_analysis_filter</i>	
SpyGlass CDC Solution		
<i>assume_path</i>	<i>breakpoint</i>	<i>cdc_false_path</i>
<i>cdc_filter_path</i>	<i>clock</i>	<i>deltacheck_stop_instance</i>
<i>define_tag</i>	<i>deltacheck_ignore_instance</i>	<i>deltacheck_ignore_module</i>
<i>deltacheck_start</i>	<i>deltacheck_stop_module</i>	<i>deltacheck_stop_signal</i>
<i>fifo</i>	<i>input</i>	<i>ip_block</i>
<i>network_allowed_cells</i>	<i>noclockcell_start</i>	<i>noclockcell_stop_instance</i>
<i>noclockcell_stop_module</i>	<i>noclockcell_stop_signal</i>	<i>noclockcell_stop_instance</i>
<i>noclockcell_stop_module</i>	<i>noclockcell_stop_signal</i>	<i>no_convergence_check</i>
<i>num_flops</i>	<i>output</i>	<i>output_not_used</i>
<i>port_time_delay</i>	<i>reset</i>	<i>set_case_analysis</i>
<i>signal_in_domain</i>	<i>watchpoint</i>	<i>quasi_static</i>
<i>define_reset_order</i>	<i>allow_combo_logic</i>	<i>abstract_port</i>
<i>sync_cell</i>	<i>sgdc</i>	<i>quasi_static_style</i>
<i>noclockcell_stop_instance</i>	<i>signal_type</i>	<i>abstract_file</i>
<i>monitor_time</i>	<i>gray_signals</i>	<i>clock_path_wrapper_module</i>
<i>clock_sense</i>	<i>cdc_matrix_attributes</i>	<i>repeater</i>
<i>cdc_attribute</i>	<i>meta_inst</i>	<i>meta_module</i>
<i>reset_filter_path</i>	<i>rdc_false_path</i>	<i>sg_clock_group</i>
<i>cdc_check_glitch</i>	<i>cdc_define_transition</i>	<i>check_clock_relation</i>

<i>meta_monitor_options</i>	<i>sdc_data</i>	<i>validation_filter_path</i>
SpyGlass Constraints Solution		
<i>assume_path</i>	<i>clock</i>	<i>domain</i>
<i>mapped_pin_map</i>	<i>sdc_data</i>	<i>block</i>
<i>blocksize</i>	<i>clock_group</i>	<i>abstract_file</i>
SpyGlass DFT Solution		
<i>assume_path</i>	<i>balanced_clock</i>	<i>bypass</i>
<i>clock</i>	<i>clock_pin</i>	<i>clock_shaper</i>
<i>complex_cell</i>	<i>dbist</i>	<i>define_illegal_input_values</i>
<i>define_legal_input_values</i>	<i>define_tag</i>	<i>dont_touch</i>
<i>dft_stitching_exception</i>	<i>force_ta</i>	<i>gating_cell</i>
<i>initialize_for_bist</i>	<i>ip_block</i>	<i>keeper</i>
<i>memory_force</i>	<i>memory_read_pin</i>	<i>memory_tristate</i>
<i>memory_type</i>	<i>memory_write_disable</i>	<i>memory_write_pin</i>
<i>module_bypass</i>	<i>module_pin</i>	<i>no_fault</i>
<i>force_no_scan</i>	<i>pll</i>	<i>pulldown</i>
<i>pullup</i>	<i>require_path</i>	<i>require_strict_path</i>
<i>require_structure</i>	<i>require_value</i>	<i>reset_async</i>
<i>reset_pin</i>	<i>rme_config</i>	<i>force_scan</i>
<i>scan_cell</i>	<i>scan_chain</i>	<i>scan_ratio</i>
<i>scan_type</i>	<i>scan_wrap</i>	<i>seq_atpg</i>
<i>set_pin</i>	<i>shadow_ratio</i>	<i>test_mode</i>
<i>test_point</i>	<i>tie_x</i>	<i>tristate_cell</i>
<i>abstract_file</i>	<i>illegal_path</i>	<i>illegal_constraint_message_tag</i>
<i>require_constraint_message_tag</i>		
SpyGlass DFT DSM Solution		
<i>abstract_port</i>	<i>atspeed_clock_frequency</i>	<i>clock</i>
<i>clock_root</i>	<i>clock_shaper</i>	<i>clockgating</i>
<i>complex_cell</i>	<i>compressor</i>	<i>decompressor</i>
<i>define_tag</i>	<i>expect_frequency</i>	<i>false_path</i>

SpyGlass Design Constraints

<i>force_ta</i>	<i>gating_cell</i>	<i>gating_cell_enable</i>
<i>ip_block</i>	<i>module_bypass</i>	<i>no_atspeed</i>
<i>no_fault</i>	<i>force_no_scan</i>	<i>pll</i>
<i>pulldown</i>	<i>pullup</i>	<i>require_pulse</i>
<i>reset</i>	<i>force_scan</i>	<i>scan_chain</i>
<i>scan_enable_source</i>	<i>scan_wrap</i>	<i>test_mode</i>
<i>test_point</i>	<i>voltage_domain</i>	<i>abstract_file</i>
<i>sg_multicycle_path</i>		
SpyGlass ERC Product		
<i>set_case_analysis</i>	<i>clock</i>	<i>reset</i>
<i>set</i>	<i>abstract_file</i>	
SpyGlass latch Product		
<i>assume_path</i>	<i>set_case_analysis</i>	<i>abstract_file</i>
SpyGlass Power Verify Solution		
<i>always_on_cell</i>	<i>always_on_pin</i>	<i>always_on_buffer</i>
<i>always_on_buffer</i>	<i>assume_path</i>	<i>cell_hookup</i>
<i>clock</i>	<i>ignore_crossing</i>	<i>input_isocell</i>
<i>isolation_cell</i>	<i>delay_buffer</i>	<i>assertion_signal</i>
<i>multivt_lib</i>	<i>non_pd_inputcells</i>	<i>power_down_sequence</i>
<i>domain_signal</i>	<i>pg_cell</i>	<i>pg_pins_naming</i>
<i>pin_voltage</i>	<i>power_down</i>	<i>domain_inputs</i>
<i>domain_outputs</i>	<i>power_switch</i>	<i>power_state</i>
<i>ram_instance</i>	<i>ram_switch</i>	<i>retention_instance</i>
<i>retention_cell</i>	<i>set_case_analysis</i>	<i>cell_pin_info</i>
<i>cell_tie_class</i>	<i>special_cell</i>	<i>supply</i>
<i>switchoff_wrapper_instance</i>	<i>voltage_domain</i>	<i>power_data</i>
<i>ignore_supply_pin</i>	<i>antenna_cell</i>	<i>isolation_wrapper</i>
<i>aon_buffered_signals</i>	<i>levelshifter</i>	<i>set_power_info</i>
<i>associate_lib</i>	<i>set_supply_node</i>	<i>disallow_upf_command</i>
<i>make_mandatory_upf_commands_options</i>	<i>reference_toplevel_isolation_signal</i>	
SpyGlass OpenMore Product		

set_case_analysis	abstract_file	
SpyGlass Power Estimation and SpyGlass Power Reduction Solutions		
activity	activity_data	blackbox_power
clock	clock_buffer	define_clock_tree
define_library_group	design_map_info	gating_cell
input_drive_strength	instance_trace	memory
memory_port	mode_condition	operating_mode_set
pg_cell	power_rail_mapping	power_state
pr_safe_clocks	repeater_buffer	rme_config
sdc_data	select_wireload_model	set_case_analysis
set_cell_allocation	set_clock_gating_type	syn_set_dont_use
ignore_nets	set_monitor_cell	set_power_scaling
set_mega_cell	spef_data	supply
ungroup_cells	use_library_group	voltage_domain
memory_inst_port	vt_mix_percentage	wireload_selection
ignore_clock_gating	disallow_modification_type	set_slew
SpyGlass STARC Product		
set_case_analysis	test_mode	abstract_file
SpyGlass STARC02 Product		
test_mode	abstract_file	
SpyGlass STARC05 Product		
test_mode	abstract_file	
SpyGlass STARCad-21 Product		
test_mode	abstract_file	
SpyGlass TXV Solution		
clock	initstate	quasi_static
reset	simulation_data	mcp_info
assume_waveform		

abstract_block_violation

Purpose

The `abstract_block_violation` constraint specifies information about the violation reported during the generation of abstract view of a design block. This constraint is generated in the SGDC file representing the abstract view.

NOTE: *SpyGlass generates this constraint for its internal use.*

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution, SpyGlass CDC solution, SpyGlass Constraints solution, SpyGlass base products

Syntax

The syntax to specify the `abstract_block_violation` constraint is as follows:

```
abstract_block_violation
  -name <rule-name>
  -sev <severity>
  [ -count <violation-count> | -waived_count
    <waived-violation-count> ]
  [ -is_builtin ]
```

Arguments

-name <rule-name>

Specifies the name of the rule that reports a violation.

-sev <severity>

Specifies the rule severity.

-count <violation-count>

Specifies the number of violations reported for the rule specified in the **-name <rule-name>** argument.

-waived_count <waived-violation-count>

Specifies the number of violations waived for the rule specified in the *-name <rule-name>* argument.

-is_builtin

Specifies that the rule specified in the *-name <rule-name>* argument is a built-in rule.

Example

Consider the following constraint generated during abstract block generation:

```
abstract_block_violation -name WarnAnalyzeBBox -sev Warn  
-count 1 -is_builtin
```

The above constraint specifies that during abstract block generation, one violation of the *WarnAnalyzeBBox* built-in rule is reported, and the severity of the rule is Warning.

abstract_file

Purpose

The `abstract_file` constraint is used to specify the product version and the block SGDC file using which the abstract view of a block was generated.

The advantage of specifying this information is that if an abstract view becomes incompatible in a particular release, you can specify the product version and the original block SGDC file of that abstract view to SpyGlass. This way, SpyGlass uses the specified abstract view during the SoC validation.

In some cases, the block SGDC file specified by this constraint refers to some additional files through any of the following specifications:

- `include <sgdc_file>`
- `sgdc -import <block_name> <block_constraint_file>`
- `sdc_data -type <sdc_file>`
- `power_data -format <cpf|upf> -file <cpf/upf-file>`
- `activity_data -format <vcd|fsdb|saif> -file <file>`

In such cases, package the additional files manually along with the block SGDC file.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution, SpyGlass CDC solution, SpyGlass Constraints solution, SpyGlass base products

Syntax

The syntax to specify the `abstract_file` constraint is as follows:

```
abstract_file
  -version <version-string>
  -scope <dft|const|base|cdc>
  [ -block_file <original-block-sgdc-files> ]
  [ -abstract_searchpath <search_paths for block-sgdc
file(s)> ]
```

Arguments

-version <version-string>

Specifies the version number of an abstract view. This version number is product-specific.

During SoC validation run, SpyGlass uses this value to perform following abstraction model version checks:

- When the current product version supports higher abstraction version than the supplied abstracted model SGDC, the SGDC_abstract_file01 rule reports the following warning message:
SoC results for product <product> as abstracted model for block <block-name> is not up-to-date. Please regenerate these abstract model using latest <product> policy
- If the supplied abstracted SGDC is generated using a product that has a higher abstraction version than the current product version, the SGDC_abstract_file02 rule reports the following error message:
Abstracted models provided for product <product> are generated with newer <product> policy, and are not compatible with the current <product> product used. Please re-generate these abstract models with current <policy-name> policy

-scope <dft|const|base|cdc>

Specifies the scope in which an abstract view is generated.

A scope specifies a product.

-block_file <original-block-sgdc-files>

Specifies a space-separated list of block SGDC files using which the abstract view was generated.

-abstract_searchpath <search_paths for block-sgdc file(s)>

Specifies a space-separated list of search paths to locate block SGDC files.

Examples

Consider the following example:

```
abstract_file -version 1.0 -scope dft  
-block_file blk1_dft.sgdc
```

The above example means that the abstract view being used during SoC validation was generated from the blk1_dft.sgdc file. This abstract view is of the 1.0 version and was generated by using SpyGlass DFT.

abstract_interface_param

Purpose

The `abstract_interface_param` constraint specifies the definition of the parameter abstracted.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution, SpyGlass CDC solution, SpyGlass Constraints solution, SpyGlass base products

Syntax

The syntax to specify the `abstract_interface_param` constraint is as follows:

```
abstract_interface_param
  -name <param-name>
  -value <param-value>
```

Arguments

-name <param-name>

Specifies the name of the parameter.

-value <param-value>

Specifies the value of the parameter.

Example

Consider the following RTL specification:

```
module reorder_bits(in,out);
  parameter SIZE = 10;
  parameter FLIPBIT = ((SIZE + 1) >> 2);
  input [SIZE:0] in;
  output reg [SIZE:0] out;
```

SpyGlass Design Constraints

```
endmodule
```

```
abstract_interface_parameter -name "SIZE" -value "10"
```

```
abstract_interface_parameter -name "FLIPBIT" -value "((SIZE  
+ 1) >> 2)"
```

abstract_interface_port

Purpose

The `abstract_interface_port` constraint specifies the definition of the port abstracted.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution, SpyGlass CDC solution, SpyGlass Constraints solution, SpyGlass base products

Syntax

The syntax to specify the `abstract_interface_port` constraint is as follows:

```
abstract_interface_port
  -name <port-name>
  -definition <port-def>
```

Arguments

-name <port-name>

Specifies the name of the port net.

-definition <port-def>

Specifies the way it is defined in the RTL.

Example

Consider the following RTL specification:

```
module reorder_bits(in,out);
  parameter SIZE = 10;
  parameter FLIPBIT = ((SIZE + 1) >> 2);
  input [SIZE:0] in;
  output reg [SIZE:0] out;
```

SpyGlass Design Constraints

```
endmodule
```

```
abstract_interface_port -name "in"    -definition "input  
[SIZE:0] in; "
```

```
abstract_interface_port -name "out" -definition "output reg  
[SIZE:0] out; "
```

abstract_port

Purpose

The `abstract_port` constraint is used to define abstracted information for block ports.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution, SpyGlass CDC solution, SpyGlass Constraints solution, SpyGlass base products

Syntax

The syntax to specify the `abstract_port` constraint is as follows:

```
abstract_port
  -module <module-name>
  -ports <port-name-list>
  -clock <clock-port-list >
  [ -reset <reset-name> ]
  [ -combo <yes | no | unknown> ]
  [ -sync <active | inactive>
    -from <src-clk list> -to <dest-clk list>
    [ -seq <yes | no> ]
    [ -sync_names <sync-names> ]
  ]
  [ -related_ports <related-ports> ]
  [ -scope <dft | cdc | const | base> ]
  [ -mode <mode-name> ]
  [ -connected_inst <instance-name> ]
  [ -inst_master <instance-master> ]
  [ -inst_pin <instance-pin> ]
  [ -path_logic <path-logic> ]
  [ -path_polarity <path-polarity> ]
  [ -phase_list <min|max|rise|fall|setup|hold|start|end> ]
  [ -multiplier_value <multiplier-value> ]
  [ -path_constraint <min_delay | max_delay> ]
  [ -ignore ]
  [ -combo_ifn <clock_port> ]
```



```
[ -start ]
[ -end ]
[ -direction <input/output> ]
```

NOTE: *The SpyGlass DFT DSM solution uses only the following options of this constraint:*

-  `-module`
-  `-ports`
-  `-clock`

Arguments

-module <module-name>

Specifies the name of a module for which this constraint is being specified.

-ports <port-name-list>

Specifies a space-separated list of port names of a module specified in the [-module <module-name>](#) argument.

-clock <clock-port-list>

Specifies clock input, inout, or output ports of a module by which the ports specified by the [-ports <port-name-list>](#) argument is driven. You can also specify a clock as a virtual clock.

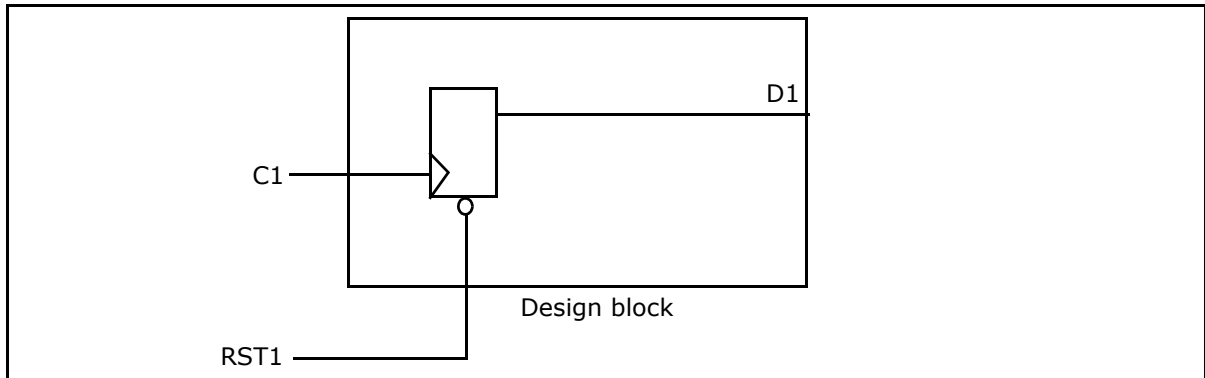
-reset <reset-name>

Specifies the reset name assigned to the port of an abstract view.

While creating abstract views for design blocks, if a sequential element with a reset pin reaches a block port, the `Ac_abstract01` and `Ac_blksgdc01` rules dump the `-reset <reset-name>` argument to the [abstract_port](#) constraint generated for the abstract view. However, if an input port of the design block reaches a sequential element having a reset pin, the `Clock_info15` rule dumps this argument.

NOTE: *This functionality works only if the `enable_soc_rdc` parameter is set to `yes`.*

Consider the following scenario when the `enable_soc_rdc` parameter is set to `yes`:

**FIGURE 1.**

For the above case, the *Ac_abstract01* rule generates the following *abstract_port* constraint for the block port D1:

```
abstract_port -ports D1 -clock C1 -reset RST1
```

Based on the generated reset information, the *Ac_abstract_validation01*, *Ac_abstract01*, *Ac_resetcross01*, and *Ar_resetcross01* rules perform checks related to invalid reset crossings between the abstract views and top-level modules.

-combo <yes | no | unknown>

Specifies the presence of a combinational logic in the input cone of a port specified with the *-ports <port-name-list>* argument.

By default, the value of this argument is *unknown*, which means that related validation checks should not be performed.

You can set this argument to *yes* or *no*.

-sync <active | inactive>

Specifies whether a port is driven by a control synchronizer or a data synchronizer.

Set this argument to *active* if a port is driven by a control synchronizer that can act as a synchronized enable for other data crossings.

Set this argument to *inactive* if a port is driven by a data synchronizer

that cannot act as a synchronized enable for other data crossings.

-from <src-clk-list>

Specifies a space-separated list of clock ports or virtual clock names that are reaching to the source of a synchronizer.

-to <dest-clk list>

Specifies a space-separated list of clock ports or virtual clock names that are reaching to the destination of a synchronizer.

-seq <yes | no>

Specifies whether sequential elements exist in the input cone of a port between the synchronizer and the port specified by using the *-ports <port-name-list>* argument.

You can set this argument to yes or no. By default, it is set to no.

-sync_names <sync-names>

Specifies a space-separated list of hierarchical net or hierarchical pin names of synchronizers.

-related_ports <related-ports>

Specifies a list of ports that have a sequential path to the ports specified by the *-ports <port-name-list>* argument. The direction of related ports should either be inout or reverse of the direction of the ports specified by the *-ports <port-name-list>* argument.

For example, if you specify *-ports P[1:0]* and *-related_ports in1[2:0]* for this constraint, it implies that 0,1, and 2 bits of the in1 port are driving both the P[0] and port P[1] ports.

NOTE: The *-related_ports* and *-sync <active | inactive>* arguments of this constraint are mutually exclusive.

-scope <dft | cdc | const | base>

Specifies the scope in which an abstract view should be generated. A scope specifies a product.

-mode <mode-name>

Specifies the mode for which the Interface Logic Model (ILM) modeling information is generated.

The following table describes the mode for the scopes specified by the *-scope <dft | cdc | const | base>* argument:

Scope	Mode for the Scope
dft	The modes can be shift, capture, and atspeed under which SpyGlass DFT analysis is performed. The abstract_port modeling is generated for these modes.
cdc	The mode is set_case_analysis because SpyGlass CDC analysis is based on this constraint.
base	The mode is set_case_analysis because SpyGlass Base analysis is based on this constraint.
const	The mode is the same as the one specified by the sdc_data constraint. The abstract_port modeling is done for each mode of SDC data.

-connected_inst <instance-name>

Specifies the name of the instance whose pin should be connected with the IP port. The master of this instance is specified by the *-inst_master <instance-master>* argument.

For example, consider the following figure:

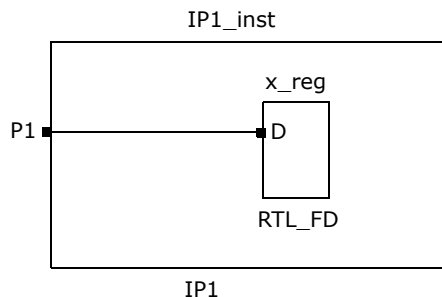


FIGURE 2. Specifying the instance name

In the above figure, *x_reg* is the instance name of the master flip-flop

SpyGlass Design Constraints

RTL_FD.

The following is the `abstract_port` constraint specification in this case:

```
current_design IP1
```

```
abstract_port -ports P1 -inst_master RTL_FD -connected_inst
x_reg -inst_pin D
```

-inst_master <instance-master>

Specifies the master type, such as RTL_FD and RTL_LD, of the instance specified by the `-connected_inst <instance-name>` argument.

For example, consider the following figure in which RTL_LD is the master of the `x_reg` latch:

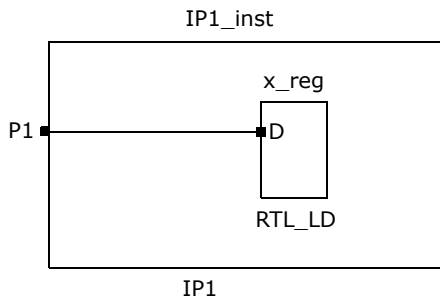


FIGURE 3. Specifying the instance master

In the above figure, specify RTL_LD in the `-inst_master <instance-master>` argument while connecting the P1 port with the D pin of the `x_reg` instance. The following is the `abstract_port` constraint specification in this case:

```
current_design IP1
```

```
abstract_port -ports P1 -inst_master RTL_LD -connected_inst
x_reg -inst_pin D
```

Specifying a Black Box Master

In the `-inst_master <instance-master>` argument, you can also specify a black box master name corresponding to the black box instance specified by the `-connected_inst <instance-name>` argument. Use the following format while specifying the black box master name:

```
-inst_master "BBOX:<black-box-master-name>"
```

For example, consider the following figure in which adder is the master of the adder_2_1 instance:

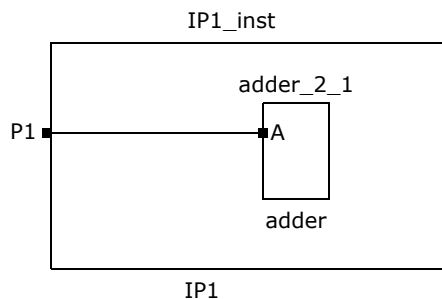


FIGURE 4. Specifying the black box master

For the above scenario, specify adder in the `-inst_master` argument while connecting the P1 port with the A pin of the adder_2_1 instance, as shown below:

```
current_design IP1
abstract_port -ports P1 -inst_master "BBOX:adder"
-connected_inst adder_2_1 -inst_pin A
```

`-inst_pin <instance-pin>`

Specifies the pin that should be connected with the port specified by the `-ports <port-name-list>` argument. This pin is present on the instance of the master specified by the `-inst_master <instance-master>` argument.

For example, consider the following figure:

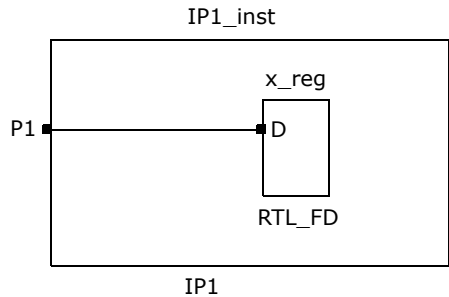


FIGURE 5. Specifying the instance pin

In the above figure, D is the instance pin present on the x_reg instance whose master is RTL_FD. In this case, specify the following constraint to connect the D pin of the x_reg instance with the P1 port:

```
current_design IP1

abstract_port -ports P1 -inst_master RTL_FD -connected_inst
x_reg -inst_pin D
```

-path_logic <path-logic>

Specifies the logic that exists between the connection from:

- An input port to an output port.
- A port (specified by *-ports <port-name-list>*) to an instance pin (specified by *-inst_pin <instance-pin>*).
- An instance pin (specified by *-inst_pin <instance-pin>*) to a port (specified by *-ports <port-name-list>*).
- A port to a hanging path by using the logic specified by the *-path_logic* argument.

You can specify any of the following values as the path logic:

Path Logic	Description
combo	Specifies that the connection is through a combinational logic.

Path Logic	Description
buf	Specifies that the connection is through a buffer.
inv	Specifies that the connection is through an inverter.

For example, consider the following figure:

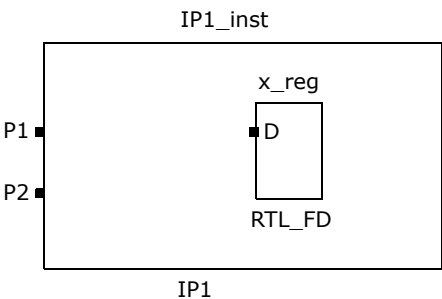


FIGURE 6. Specifying the path logic

In the above figure, consider that you want to perform the following actions:

- Create a connection from P1 to D such that a combinational logic exists between the connection. Specify the following constraints in this case:

```
current_design IP1

abstract_port -ports P1 -inst_master RTL_FD -
connected_inst x_reg -inst_pin D -path_logic combo
```

- Create a connection from the P2 port to a hanging terminal by using an inverter. Specify the following constraint in this case:

```
abstract_port -ports P2 -path_logic inv
```

After specifying the above constraints, [Figure 6](#) changes to the following:

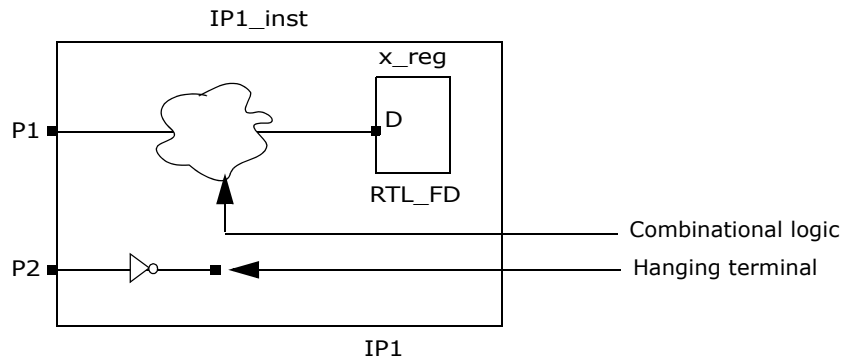


FIGURE 7. Inserting a combinational logic

Handling Multiple Connection Paths Belonging to Different Scopes

Consider the following figure:

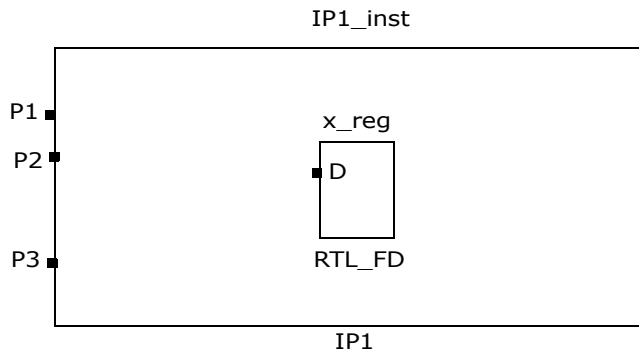


FIGURE 8. Handling Connections of Different Scopes

In the above figure, consider that you want to connect the P1, P2, and P3 ports with the D pin of the x_reg instance such that:

- The connection from P1 to D belongs to the CDC scope.

To create this connection, specify the following constraint:

```
abstract_port -ports P1 -inst_master RTL_FD
-connected_inst x_reg -inst_pin D -path_logic combo
```

```
-scope cdc
```

- The connection from P2 to D belongs to the DFT scope in the shift mode.

To create this connection, specify the following constraint:

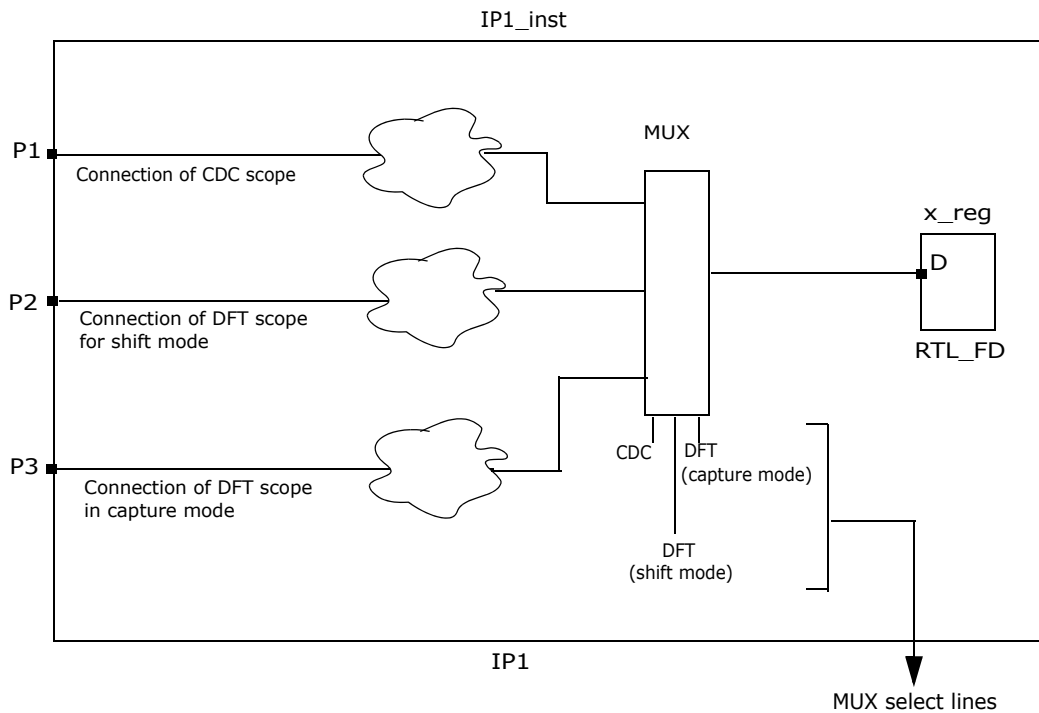
```
abstract_port -ports P2 -inst_master RTL_FD  
-connected_inst x_reg -inst_pin D -path_logic combo  
-scope dft -mode shift
```

- The connection from P3 to D belongs to the DFT scope in the capture mode.

To create this connection, specify the following constraint:

```
abstract_port -ports P3 -inst_master RTL_FD  
-connected_inst x_reg -inst_pin D -path_logic combo  
-scope dft -mode capture
```

After specifying the above constraints, SpyGlass generates connections, as shown in the following figure:

**FIGURE 9.** Handling Connections of Different Scopes

In the above figure, notice the MUX present between the connections.

SpyGlass inserts this MUX so that only one connection of a particular scope (CDC, DFT in the shift mode, or DFT in the capture mode) is active at a time. Therefore:

- When the CDC select line of the MUX is on, the connection between P1 and D is active.
- When the DFT select line of the MUX is on for the shift mode, the connection between P2 and D is active.
- When the DFT select line of the MUX is on for the capture mode, the connection between P3 and D is active.

-path_polarity <path-polarity>

Specifies the polarity of the path containing a combinational logic.

You can specify any of the following values as the path polarity:

Path Polarity	Description
buf	Specifies that the path containing the combinational logic is buffered.
inv	Specifies that the path containing the combinational logic is inverted.

NOTE: *Specify this argument only if you specify -path_logic combo in the abstract_port constraint specification.*

-phase_list <min|max|rise|fall|setup|hold|start|end>

Specifies options associated with SDC commands used by SpyGlass Constraints solution.

For example, when you set the value of this argument to `setup` for multi-cycle paths specified by the `set_multicycle_path` SDC command, SpyGlass abstracts the setup-related information while generating an abstract view.

-multiplier_value <multiplier-value>

Specifies a multiplier value for the `set_multicycle_path` SDC command.

-path_constraint <min_delay | max_delay>

Set this argument to `min_delay` or `max_delay` if a path is constrained by using the `set_max_delay` or `set_min_delay` SDC command, respectively.

-ignore

Indicates that the specified block port is not considered for SpyGlass analysis.

The `Clock_info15` rule of the SpyGlass CDC solution generates the `abstract_port -ignore` constraint if all the fan-out of an input port

are hanging or blocking. See [Example 4](#).

The `Ac_abstract01` rule of the SpyGlass CDC solution generates the `abstract_port -ignore` constraint if all the fan-in of an output port are hanging or blocking.

NOTE: *It is not mandatory to specify the `-clock` argument of the `abstract_port` constraint when you specify the `-ignore` argument of this constraint.*

[`-combo_ifn` `<clock_port>`]

Indicates the clock port associated with the control synchronizer.

[`-start`>]

Set this argument to validate the domain assumption and other CDC attributes applied on the design object by using the `abstract_port` constraint from its fanin cone.

Note that CDC verification is performed from the design object, which has the `abstract_port` constraint specified, to the output cone of the specified port/pin.

NOTE: *This argument is used by the SpyGlass CDC solution only.*

Table 1 below summarizes verification/validation with regard to CDC start points.

TABLE 1

	CDC start point
Verification	Primary Input/Blackbox output
Validation	Primary Output/Blackbox input

If the `-start` argument is not specified for an `abstract_port`, SpyGlass CDC considers the design objects specified in Table 2 as the start point.

TABLE 2

Object Type	Default	Description
Top-level input port	<code>-start</code>	
Top-level inout port	<code>-start</code> <code>-end</code>	For input side, it will be considered as <code>-start</code> and for output side, it will be treated as <code>-end</code>
Black-box output	<code>-start</code>	
Black-box inout	<code>-start</code> <code>-end</code>	For output side, it will be considered as <code>-start</code> and for input side, it will be considered as <code>-end</code>
Abstracted block output (seen at higher level of hierarchy)	<code>-start</code>	Similar to black box handling
Abstracted block input (seen at the higher level of hierarchy)	<code>-start</code>	Differs from blackbox handling as well as the description in Table 1 to avoid backward compatibility issues for existing SoC flow. SpyGlass CDC validates the abstract port and therefore consider it as CDC start point rather than as CDC end point.
Abstracted block inout (seen at the higher level of hierarchy)	<code>-start</code>	Differs from blackbox handling as well as the description in Table 1 to avoid backward compatibility issues for existing SoC flow. SpyGlass CDC validates abstract port and therefore consider it as CDC start point rather than as CDC end point.

[`-end`>]

Set this argument to validate the domain assumption and other CDC attributes applied on the design object by using the `abstract_port` constraint from its fan-out cone.

Note that CDC verification is performed from the design object, which has

the `abstract_port` constraint specified, to the input cone of the specified port/pin.

NOTE: *This argument is used by the SpyGlass CDC solution only.*

Table 3 summarizes verification/validation with regard to CDC end points.

TABLE 3

	CDC end point
Verification	Primary output/Blackbox input
Validation	Primary input/Blackbox output

If the `-end` argument is not specified for an `abstract_port`, SpyGlass CDC considers the design objects specified in Table 4 as the end point.

[`-direction`>]

Set this argument to specify direction of an inout port that is constrained by using the `abstract_port` constraint to consider it as an input port or output port. Possible values for this field are `input` and `output`.

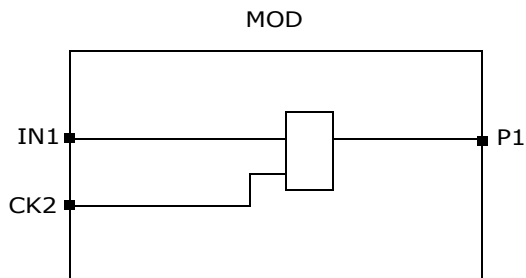
Examples

Example 1

Consider the following example:

```
abstract_port -module MOD -ports P1 -clock Ck2
-related_ports IN1 -scope cdc
```

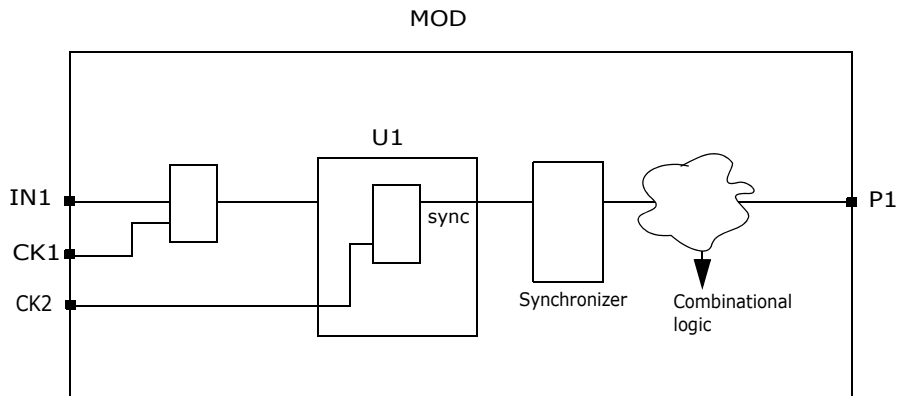
The above example implies that the P1 output port of the MOD module is driven by a flip-flop clocked by the Ck2 clock, and there is no synchronizer in an input cone of the port. The flip-flop is driven by the IN1 input port. This is shown in the following figure:

**FIGURE 10.** Example 1**Example 2**

Consider the following example:

```
abstract_port -module MOD -port P1 -clock Ck2
-combo yes -sync active -from Ck1 -to Ck2
-sync_names MOD.U1.sync -scope cdc
```

The above example implies that the P1 port of the MOD module is an output of a control synchronizer (from the source clock Ck1 to the destination clock Ck2) and there is a combinational logic present between a synchronizer and the port. This is shown in the following figure:

**FIGURE 11.** Example 2

Here, MOD.U1.sync is the hierarchical name of synchronizer output.

In this case, the port can act as a synchronized enable at a higher hierarchy for other crossings.

Example 3

Consider the following example:

```
abstract_port -module MOD -ports P1 -clock Ck2 -scope cdc
```

The above example implies that the P1 output port of the MOD module is driven by an unsynchronized crossing with the destination clock Ck2. This is shown in the following figure:

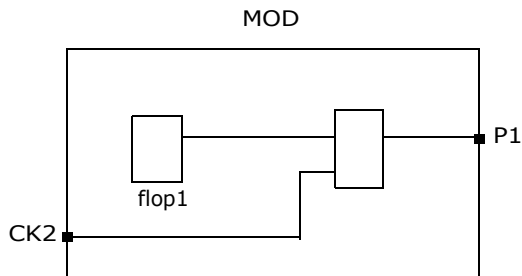


FIGURE 12. Example 3

Example 4

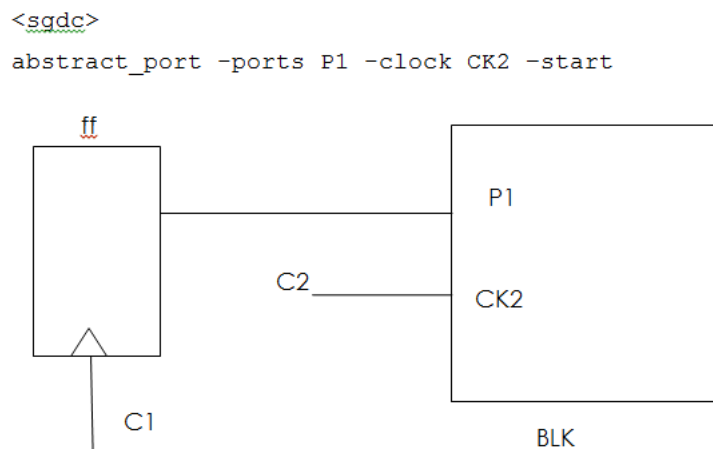
Consider the following example:

```
abstract_port -module MOD -ports P1 -ignore
```

The above example means that all the fan-out of the P1 input port are hanging or blocked.

Example 5

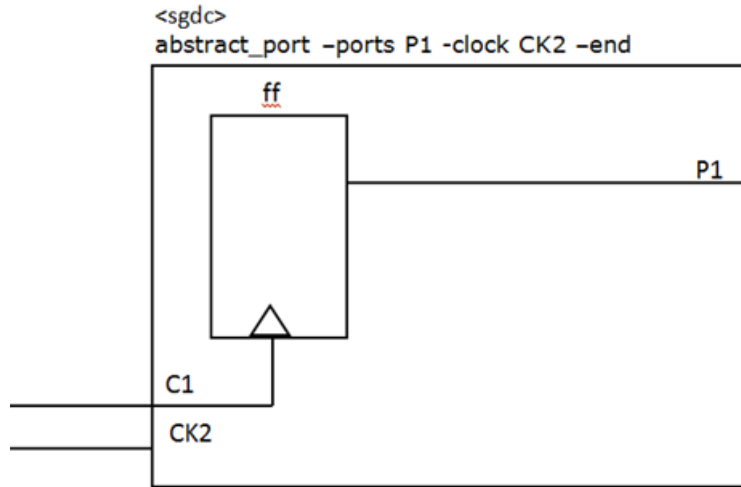
Consider the following example:

**FIGURE 13.**

In the above example, the validation check is performed for port P1 and data domain mismatch is reported by the Ac_abstract_validation01 rule.

Example 6

Consider the following example:

**FIGURE 14.**

In the above example, the `Ac_unsync01` rule reports a violation between the `ff` flop and `P1` port.

NOTE: *Note that no validation check is performed for output ports when the block is instantiated at the SoC level.*

Example 7

Consider a scenario where two constraints, one with `-start` and another with `-end` is defined for a inout port.

In this case, if the inout port is driven by a flop, then both validation and verification is performed (assuming the rules are enabled) with regard to `-start` and `-end` respectively.

Similarly if the inout port is feeding a flop, then both validation and verification is performed because the `-start/-end` arguments are specified on the same port. Therefore, it is possible that SpyGlass CDC reports three violations for the same port because validation check is not performed for the `-end` argument on output or inout ports.

Consider the following schematic:

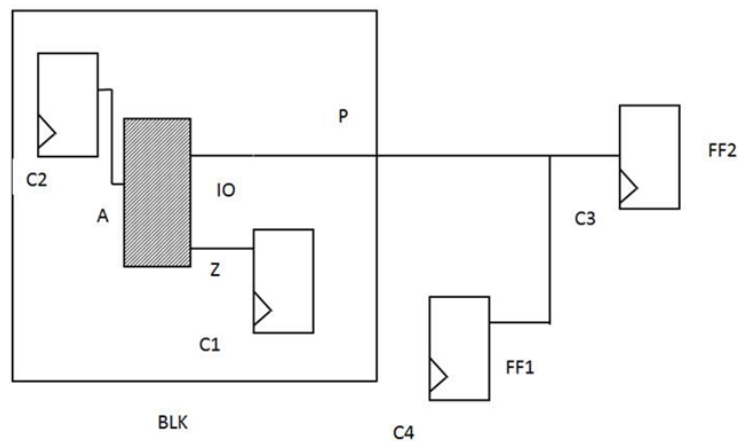


FIGURE 15.

In the above example, the P port is an inout port of the BLK block. The abstraction of the BLK block generates the following constraints:

```
abstract_port -ports P -clock C2 -start
```

At SoC level, the validation check is performed for the abstract_port constraint with the -start argument and it reports domain mismatch for the C4 clock because the FF1 flop is driving the P inout port. Similarly, a crossing is reported for C3 (where the P port is the source of the crossing) because the FF2 flop is driven by the P port.

Rules

The abstract_port constraint is used by the following rules:

SpyGlass CDC Solution		
Clock synchronization rules (except Clock_sync05)	Block constraint validation rules	Ac_abstract01
Ac_blksgdc01	Reset_sync02	Reset_sync03
Ar_unsync01	Ar_sync01	Ar_asyncdeassert01

SpyGlass Design Constraints

Ar_syncdeassert01	Ar_resetcross01	Ar_resetcross_matrix01
SpyGlass DFT Solution		
All rules		
SpyGlass DFT DSM Solution		
All rules		

activity

Purpose

Use the `activity` constraint to specify activity values and probability values of input signal used for the rules that report power/activity. The activity value for an individual non-clock signal or all non-clock signals contained in a specified instance can be specified.

The ways in which activity and probability of a net can be determined, in decreasing order of priority, are as follows:

1. The simulation file
2. The `-name` and `-instname` arguments of the activity constraint
3. The `-all_primary_input`, `-all_register_output`, `-all_blackbox_output`, `-all_register_enable`, or `-all_power_essential_signals` arguments of the activity constraint
4. The `-default_startpoints` argument of the activity constraint
5. The `-default` argument of the activity constraint

NOTE: The `pe_activity_priority` parameter is by default set to 'sim'. If it is changed to 'sgdc', points 2 and 3 take higher priority over point 1. For more information on this parameter, refer to the *SpyGlass Power Estimation and SpyGlass Power Reduction Rules Reference Guide*.

Nets for which activity and probability are not specified by any of the above points, the values are propagated from the fan-in using the SpyGlass activity propagation engine.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the activity constraint is as follows:

```
current_design <du-name>
  activity
    -name <sig-name> | -instname <inst-name> | -
default_startpoints | -default
  -prob <p-value>
```

```

-activity <a-value>
-all_primary_input
-all_register_output
-all_blackbox_output
-clock <clk-name>
-clock_enable_instname <flip-flop-inst-name>
-all_register_enable
-all_power_essential_signals

```

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

-name <sig-name>

Name of the non-clock signal for which you are specifying the activity value.

-instname <inst-name>

Name of the instance under which you are specifying the activity value for all non-clock signals. This field supports regular expressions.

The -instname field supports a hierarchical instance name and NOT leaf-level cell name or black boxes.

-default_startpoints

Indicates that the specified probability and activity values are applicable to all unannotated primary inputs and blackbox outputs.

When you specify this argument:

- You can specify only the -activity and -prob arguments.
- The -instname argument is optional because SpyGlass automatically takes the top of the design. If you specify the -instname argument, make sure the value is the name of the top of the design. Otherwise, a FATAL message is reported.

-default

Indicates the specified probability and activity values are applicable to all nets, which are non-clock nets, and for which these values have not been specified by:

- Any other argument of the `activity` constraint
- The simulation file

-prob <p-value>

Probability of a signal being high (a real value between zero and one).

-activity <a-value>

The toggle activity value (a real number representing the number of transitions from zero to one or from one to zero in a clock cycle).

-all_primary_input

When you specify this argument with `-instname` argument, the activity values are set for all the input/inout pins of the respective hierarchical instance. Otherwise, the activity values are set for all the primary input/inout pins of the top domain in the design.

-all_register_output

If this field is specified, it sets the activity values for all the nets driven by registers.

The scope of this field can be specified through the `-instname` field of `activity` constraint. If scope is not specified, it sets the activity values for all the nets driven by registers in the design. For example:

```
activity -instname top.a -activity 0.3 -prob 0.5  
-all_register_output
```

The above example sets the activity values of the nets driven by registers in the `top.a` hierarchy.

-all_blackbox_output

If this field is specified, it sets the activity values for all the nets that are driven by black boxes.

SpyGlass Design Constraints

The scope of this field can be specified through the `-instname` field of activity constraint. For example:

```
activity -instname top.a -activity 0.3 -prob 0.5
-all_blackbox_output
```

The above example sets the activity values of the nets driven by black boxes in the `top.a` hierarchy.

If the scope is not specified, it sets the activity values for all the nets that are driven by the black boxes in the design.

-clock <clk-name>

In general, all the activity values are considered with respect to the fastest clock in the design. However, if you want to specify the activity of a net with respect to a local clock of a block, use this argument.

Consider the following example:

```
clock -name "clk1" -period 10
activity -instname "Top" -activity 0.5 -prob 0.3
clock -name "INST1.clk2" -period 20
activity -name "INST1.n1" -activity 0.5 \
-clock INST1.clk2 -prob 0.4
```

Here, activity value for the `n1` net of the `INST1` block is considered with respect to the local clock `INST1.clk2` rather than `clk1`, which is the fastest clock in the design. The last command of the above specification is equivalent to the following:

```
activity -name "INST1.n1" -prob 0.40 -activity 0.25
```

The activity value for the `n1` net of the `INST1` block remains the same as it is considered with respect to `clk1` clock.

-clock_enable_instname <flip-flop-inst-name>

Name of the flip-flop instance on whose enable you want to apply an activity value or a probability value.

For example, consider the following specification of the activity constraint:

```
activity -clock_enable_instname FF1 -activity 0.5 -prob 0.3
```

The above specification implies that the activity value 0.5 and the probability value 0.3 should be applied on the enable of the FF1 flip-flop instance.

-all_register_enable

Sets the activity and probability values for all register enables.

The scope of this argument can be specified through the `-instname` argument of the `activity` constraint. The scope includes the module and all the underlying instances within the module. If the scope is not specified, it sets the activity and probability for all register enables in the design.

In the following example, activity and probability values are set:

```
activity -instname top.M1 -prob 0.9 -activity 0.1
-all_register_enable
activity -instname top.M2 -prob 0.9 -activity 0.1
-all_register_enable
```

-all_power_essential_signals

Sets the activity and probability values for all essential signals.

The scope of this argument can be specified through the `-instname` argument of the `activity` constraint. The scope includes the module and all the underlying instances within the module. If the scope is not specified, it sets the activity and probability for all Power-Essential signals in the design.

In the following example, the activity values for all Power-Essential signals of module M1 have the probability of 0.9 and activity of 0.1:

```
activity -instname top.M1 -prob 0.9 -activity 0.1
-all_power_essential_signals
```

Rules

The `activity` constraint is used by the following rules:

SpyGlass Design Constraints

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	PESTR03	PESTR05
PESTR06	PESTR13	poweraudit	

activity_data

Purpose

Specifies a simulation file for rules in the SpyGlass Power Estimation and SpyGlass Power Reduction solutions.

The `activity_data` constraint specifies the VCD, FSDB, and SAIF file that is to be translated to SpyGlass activity format used by rules in the SpyGlass Power Estimation and SpyGlass Power Reduction solutions.

The `activity_data` constraint takes a set of information that was previously supported by some `spyParameters`. However, some new features like SAIF file input and multiple VCD are not supported by these `spyParameters`. Therefore, it is recommended that you use the `activity_data` constraint for all simulation inputs.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `activity_data` constraint is as follows:

```
current_design <du-name>
  activity_data
    -format <format> -file <file-name>
    -starttime <start-time>
    -endtime <end-time>
    -weight <value> -sim_topname <simulation top>
    -instname <instance-name>
    -sim_rtl_design_n1
    [-parallel_saif <saif-file-name>]
    [-parallel_saif_topname <saif-top-name>]
```

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-format <format>

Name of the activity data format.

This argument can have any of the following values: "SAIF", "VCD", or "FSDB", depending on the type of the specified simulation file.

-file <file-name>

Name of the VCD/FSDB/SAIF file.

NOTE: *You can specify compressed VCD file that has been generated by using the gzip utility.*

-starttime <start-time>

(Optional) Start time for VCD/FSDB file analysis with a time unit. The allowed time units are *s*, *ms*, *us*, *ns*, *ps*, and *fs*. In case, you do not specify a time unit, the timescale value provided in the VCD/FSDB file is used to infer the time value.

For example, if you specify the <start-time> as three with no time unit, and the corresponding VCD file is as shown below:

```
#####
$date
    Dec 18, 2009 17:48:09
$end
$timescale
    1ns
$end
#####
```

Then, the time value will be inferred as 3ns.

If you do not specify this argument, the start time given in VCD/FSDB file is used for analysis.

-endtime <end-time>

(Optional) End time for VCD/FSDB file analysis with a time unit. If you do not specify a time unit, the timescale value provided in the VCD/FSDB file

is used to infer the time value.

If you do not specify the `<end-time>` argument, the end time given in VCD/FSDB file is used for analysis.

-weight <value>

The percentage value (an integer number between 0 and 100) denoting weights assigned to the multiple VCD/FSDB files.

When using the `<value>` argument, you should specify weights for all the VCD/FSDB files. In case you do not specify this argument, the weights are considered based on simulation times of each VCD/FSDB file.

The sum of the `<value>` arguments specified for all the `activity_data` constraints should be 100.

If multiple VCD/FSDB files are specified and the weight is not specified, the average power estimation is done based on the weighted average of the duration of each VCD/FSDB file. For example:

If VCD1 has simulation values from 0 to 500 ns and VCD2 has simulation values from 200 ns to 450 ns, the average power is calculated, assuming that in a complete duration of

$(500 \text{ ns} - 0) + (450 \text{ ns} - 200 \text{ ns}) = 750 \text{ ns}$,
VCD1 runs for 500 ns and VCD2 runs for 250 ns.

Therefore, the weight of VCD1:VCD2 will be 2:1.

However, if weights are specified for VCD/FSDB files, the duration of VCD/FSDB file is not considered while taking weighted average for power estimation.

-sim_topname <simulation top>

Name of the instantiated top module in the VCD/FSDB/SAIF file.

If you do not specify `<simulation top>`, SpyGlass attempts to automatically infer the hierarchy in the VCD/FSDB/SAIF file corresponding to the top design.

-instname <instance-name>

(Optional) Name of the hierarchical instance for which the simulation file is applicable. This field is specified when horizontal simulation file flow is used.

SpyGlass Design Constraints

-sim_rtl_design_n1

(Optional) This field should be specified if simulation file is used with gate level design.

-parallel_saif <saif-file-name>

This is an optional argument and is used to specify the Switching Activity Interchange Format (SAIF) file name with the simulation file formats VCD/FSDB.

During the annotation process, if any signal is not found in the VCD/FSDB files, that signal's activity is retrieved from the parallel SAIF when the signal is present in the parallel SAIF file.

-parallel_saif_topname <saif-top-name>

This is an optional argument and is used to specify the parallel SAIF top name. If you do not specify the parallel SAIF top name, the -sim_topname argument is considered as the parallel SAIF top name.

Rules

The activity_data constraint is used by the following rules in the SpyGlass Power Estimation and SpyGlass Power Reduction solutions:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	PESAE02	PESAE03
poweraudit	PESTR03	PESTR05	PESTR06
PESTR13	PESAE04	PESAE06	

add_fault

Purpose

The `add_fault` constraint is used to specify faults to be considered while calculating fault/test coverage. When `add_fault` is used then all faults that are not specified as `add_fault` are treated as `no_fault` and ignored while calculating fault/test coverage.

Product

SpyGlass DFT solution

Syntax

The syntax to specify the `add_fault` constraint is as follows:

```
add_fault
  -name <du-name> | <inst-name>
  [- fault <hier_pin_names> ]
  [- net <hier_net_names> ]
  [- net_input <hier_net_names> ]
  [- net_output <hier_net_names> ]
  [- clock_control <hier_net_names> ]
  [- set_control <hier_net_names> ]
  [- reset_control <hier_net_names> ]
  [- register_suffix ]
  [- instance_suffix ]
  [- module_suffix ]
```

NOTE: The `add_fault` constraint supports wildcard characters. Using wildcards, expression is expanded only within the hierarchy.

Arguments

The `add_fault` constraint has the following arguments:

-name <du-name> | <inst-name>

Specifies name of a module or instance that needs to be marked as `add_fault`.

You can specify a single or space-separated list of design unit names / hierarchical instance names:

You can specify design unit names, hierarchical instance names, or a combination of both.

-fault <hier_pin_names>

(Optional) Space-separated list of hierarchical names of pins or ports.

Do not use this argument in case of RTL design because pin names will contain generated names and will fail SGDC sanity check at the RTL.

-net <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the faults in the direct fanin or fanout of the net as add_fault.

-net_input <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the faults in the direct fanin of the net as add_fault.

-net_output <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the faults in the direct fanout of the net as add_fault.

-clock_control <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the faults associated with the registers, where clock pin is topologically driven by the specified clock, as add_fault.

-set_control <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the fault associated with the registers, where set pin is topologically driven by the specified set signal, as add_fault.

-reset_control <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the fault associated with the registers, where reset pin is topologically driven by the specified reset signal, as add_fault.

-register_suffix <suffixes>

Space-separated list of suffixes to be specified as `add_fault`. The `-register_suffix` argument should not be used along with other arguments of the `add_fault` constraint, that is, `-name`, `-clock_control`, `-set_control`, or `-reset_control`.

If the value of the `dft_treat_suffix_as_pattern` parameter is set to `on`, the `register_suffix` value is used as a pattern to be matched with the register name. The pattern may be present anywhere in the register name, excluding the path.

If the value of the `dft_check_path_name_for_register_suffix` parameter is `on`, the value of the `-register_suffix` field will be matched with the register name along with the path in which the register is present.

-instance_suffix <suffixes>

Define this field to use suffix based pattern match for all instance names.

-module_suffix <suffixes>

Define this field to use suffix based pattern match for all module names.

If the value of the `dft_treat_suffix_as_pattern` parameter is `on`, the value of the `-module_suffix` field will be matched with the module name along with the path in which the module is present.

Rules

The `add_fault` constraint is used by the following rules:

SpyGlass DFT Solution

Info_coverage	Coverage_audit
---------------	----------------

SpyGlass DFT DSM Solution

Info_transitionCoverage	Info_transitionCoverage_audit
-------------------------	-------------------------------

Example**Example 1**

Consider the following `add_fault` definition:

SpyGlass Design Constraints

```
add_fault -name sub1 -fault top.in4
add_fault constraint definition:
```

Now, consider the schematic for the same ([Figure 16](#)):

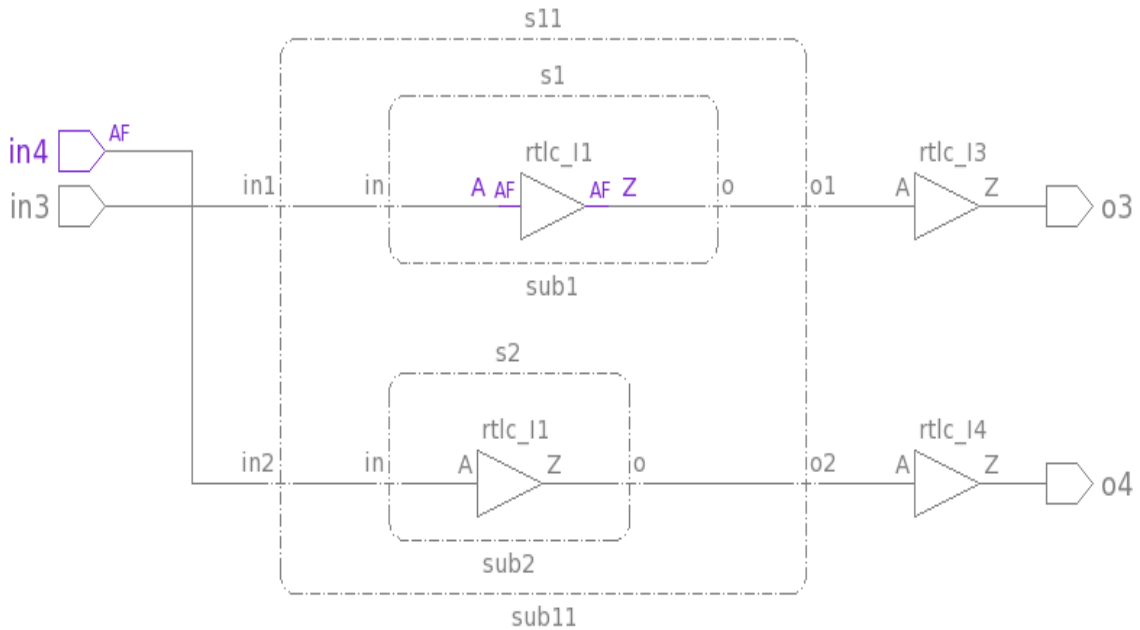


FIGURE 16. Terminals marked as add_fault

In the above schematic, instance, in4, and terminals inside sub1 are marked as add_fault. The faults of other terminals are not used for coverage calculations.

Example 2: Specifying list of suffixes using the -register_suffix argument

Consider the following example:

```
R1 (register 1) name: top.u_ctrl.u2.u1.ff1_ctrl
R2 (register 2) name: top.u_ctrl.u2.u1.ff1_state
R3 (register 3) name: top.u_core.u2.u1.ff1_state_ctrl
```

R4 (register 4) name: top.u_ctrl_state.u2.u1.ff1_ctrl_state

Now, consider the following *add_fault* descriptions:

```
add_fault -register_suffix ctrl
add_fault -register_suffix state
```

The following table lists the results when combination of values are used for the *dft_treat_suffix_as_pattern* and *dft_check_path_name_for_register_suffix* parameters:

TABLE 4 Pattern Matching for the -register_suffix argument

Value of <i>dft_treat_suffix_as_pattern</i>	Value of <i>dft_check_path_name_for_register_suffix</i>	Value of - <i>register_suffix</i>	Matched Registers
off	off	ctrl	R1, R3
		state	R2, R4
off	on	ctrl	R1, R2, R3
		state	R2, R4
on	off	ctrl	R1, R3, R4
		state	R2, R3, R4
on	on	ctrl	R1, R2, R3, R4
		state	R2, R3, R4

Example 3: Specifying list of suffixes using the -instance_suffix argument

Consider the following example:

```
R1 (register 1) name: top.u_ctrl.u2.u1.ff1_ctrl
R2 (register 2) name: top.u_ctrl.u2.u1.ff1_state
R3 (register 3) name: top.u_core.u2.u1.ff1_state_ctrl
R4 (register 4) name: top.u_ctrl_state.u2.u1.ff1_ctrl_state

I1 (instance 1) name: top.u_ctrl.u2.u1.inst1_ctrl
I2 (instance 2) name: top.u_ctrl.u2.u1.inst1_state
I3 (instance 3) name: top.u_core.u2.u1.inst1_state_ctrl
I4 (instance 4) name: top.u_ctrl_state.u2.u1.inst1_ctrl_state
```

Now, consider the following *add_fault* descriptions:

```
add_fault -instance_suffix ctrl
add_fault -instance_suffix state
```

The following table lists the results when combination of values are used for the *dft_treat_suffix_as_pattern* and *dft_check_path_name_for_instance_suffix* parameters:

TABLE 5 Pattern Matching for the -instance argument

Value of <i>dft_treat_suffix_</i> <i>as_pattern</i>	Value of <i>dft_check_path_n</i> <i>ame_for_instance</i> <i>_suffix</i>	Value of - <i>instance_suffix</i>	Matched Registers/ Instances
off	off	ctrl	R1, R3, I1, I3
		state	R2, R4, I2, I4
off	on	ctrl	R1, R2, R3, I1, I2, I3
		state	R2, R4, I2, I4
on	off	ctrl	R1, R3, R4, I1, I3, I4
		state	R2, R3, R4, I2, I3, I4
on	on	ctrl	R1, R2, R3, R4, I1, I2, I3, I4
		state	R2, R3, R4, I2, I3, I4

allow_combo_logic

Purpose

The `allow_combo_logic` constraint allows combinational logic between crossings only if the logic is within the modules specified using this constraint.

NOTE: *The `allow_combo_logic` constraint specifications will be applicable to all the clock-domain crossings in the design.*

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `allow_combo_logic` constraint is as follows:

```
current_design <du-name>
allow_combo_logic
  [ -name <space-separated-list> ]
  [ -all ]
  [ -none ]
```

Arguments

-name <space-separated-list>

(Optional) Specifies a space-separated list of modules and/or library cells. You can also use wildcard characters while specifying module/library cell names. For example, you can specify names, as shown in the following examples:

```
allow_combo_logic -name MD1 MD2 MD3
allow_combo_logic -name "MD*"
allow_combo_logic -name "MD*" "AB*" PQR
```

-all

(Optional) Specifies that all the combinational logic should be allowed in a crossing path.

-none

(Optional) Specifies that no combinational logic should be allowed in the crossing path.

If the `allow_combo_logic` constraint is specified multiple times, the order of preference (starting from the highest priority) will be as follows:

- 1. `allow_combo_logic -all`
- 2. `allow_combo_logic -none`
- 3. `allow_combo_logic -name <list>`

NOTE: Do not use the `-all` and `-none` arguments together in the same `allow_combo_logic` constraint.

Rules

The `allow_combo_logic` constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_sync01	Ac_sync02	Ac_unsync01	Ac_unsync02
Ac_glitch03	Ac_conv01	Ac_conv02	Ac_conv03
Clock_sync09			

Example

Consider an example, as shown in the following figure:

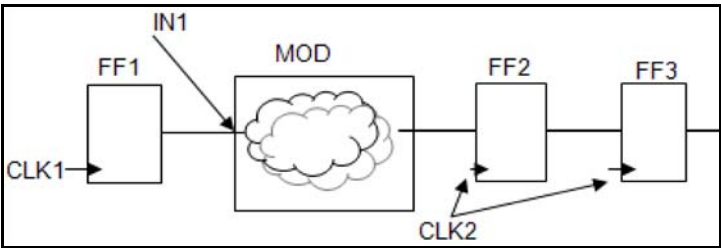


FIGURE 17. Ac_unsync01/Ac_unsync02 Rule Example

In the above example, by default, the `Ac_unsync01/Ac_unsync02` rule is

reported for the crossing between FF1 and FF2. Now consider that you specify the `allow_combo_logic` constraint, as given below:

```
allow_combo_logic -name MOD
```

In the above case, the combinational logic will be allowed if it is inside the module, `MOD`, between the crossing path. Therefore, the crossing will be reported as synchronized by the *Ac_sync01/Ac_sync02* rule.

allow_test_point

Purpose

The `allow_test_point` constraint is used to specify modules or instance which should be considered for suggesting test points.

Product

SpyGlass DFT DSM solution

Syntax

The syntax to specify the `allow_test_point` constraint is as follows:

```
current_design <du-name>
allow_test_point
  -name <module-name | instance_list>
```

Arguments

-name <module-name | instance_list>

Name of the module or the list of instances to be considered for suggesting the test points.

Rules

The `allow_test_point` constraint is used by the `Info_random_resistance` rule.

always_on_buffer

Purpose

The `always_on_buffer` constraint is used to specify always-on buffers.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `aonbuffer`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `always_on_buffer` constraint is as follows:

```
current_design <du-name>
  always_on_buffer
    -name <cell-name>
    [ -vddcpin <vddc-pin> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the always-on buffer.

-name <cell-name>

Name of the buffer cell. You can use wildcard characters while specifying the cell name.

-vddcpin <vddc-pin>

Name of the VDDC pin of the always-on buffer as used by LPPLIB11 and LPPLIB06 rules of the SpyGlass Power Verify solution.

Rules

The `always_on_buffer` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM08	LPSVM09	LPSVM10	LPSVM26
LPSVM28	LPSVM31	LPSVM40	LPSVM47
LPSVM48	LPSVM52	LPSVM53	LPSVM56
LPSVM57	LPSVM59	LPPLIB06	LPPLIB11
LPPLIB15	LPPLIB17		

always_on_cell

Purpose

The `always_on_cell` constraint is used to specify library cell names that should be instantiated in always-on domains only (that is, they should not be instantiated in switchable domains).

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `always_on_cell` constraint is as follows:

```
current_design <du-name>
  always_on_cell
    -name <cell-name-list>
    [ -locate <domain-type> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the always-on cells.

-name <cell-name-list>

Space-separated name list of always-on cells.

-locate <domain-type>

It can be AON (Always-On Domain), OFF (power domain), or BOTH (any domain).

NOTE: *You can use wildcard characters while specifying the cell names using the -name argument.*

Rules

The `always_on_cell` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM09	LPSVM10	LPSVM28	LPSVM31
LPSVM47	LPSVM54	LPSVM57	LPSVM59
LPPLIB17			

always_on_pin

Purpose

The `always_on_pin` constraint is used to specify always-on cell pins.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `always_on_pin` constraint is as follows:

```
current_design <du-name>
  always_on_pin
    -cell <name>
    -pin <pin-name-list>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the always-on pins.

-cell <name>

Name of the design unit for which always-on pins are being specified.

-pin <pin-name-list>

Space-separated name list of always-on input/inout pins of the design unit <name>.

NOTE: You can use wildcard characters while specifying the cell name using the `-cell` argument.

Rules

The `always_on_pin` constraint is used by the following rule.

SpyGlass Power Verify Solution

LPSVM53

always_on_path

Purpose

The `always_on_path` constraint is used to specify paths that should be always-on. There should not be any element that is driven by a switchable supply along the paths specified as always-on path.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `always_on_path` constraint is as follows:

```
current_design <du-name>
  always_on_path
    -from <start-point-name>
    -to <end-point-name>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the always-on cells.

-from <start-point-name>

Specifies the start point of `always_on_path`.

-to <end-point-name>

Specifies the end point of `always_on_path`.

NOTE: Here, <start-point-name>/<end-point-name> can be:

- *Top-level port (like TOP.in)*
- *Any hierarchical signal name (like TOP.u1.sig1)*
- *Pin of a hierarchical leaf level instance (like TOP.u1.LIB1.Z)*
- *Any hierarchical port of user defined module (like TOP.u1.clk_out)*

Rules

The `always_on_pin` constraint is used by the following rule.

SpyGlass Power Verify Solution

LPAON02

antenna_cell**Purpose**

The `antenna_cell` constraint is used to specify the antennae protection cells (diode cells) that should be ignored.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `apcell`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `antenna_cell` constraint is as follows:

```
current_design <du-name>
  antenna_cell
    -name <cell-name-list>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the cells.

-name <cell-name-list>

Space-separated list of the cell names. You can use wildcard characters while specifying the cell name using the -name argument.

Rules

The `antenna_cell` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM04A	LPSVM04B	LPSVM04C	LPSVM08
LPSVM09	LPSVM10		

aon_buffered_signals

Purpose

The `aon_buffered_signals` constraint is used to specify signals that should be driven by an always-on buffer.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `aonbufferedsignals`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `aon_buffered_signals` constraint is as follows:

```
current_design <du-name>
  aon_buffered_signals
```



```
-names <sig-name-list>
[ -terminatingcells <cell-pin-name-list> ]
[ -ignorecells <ignorecell-name-list> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying signals.

-names <sig-name-list>

Space-separated name list of signals driven by an always-on buffer (specified by using the [always_on_buffer](#) constraint).

-terminatingcells <cell-pin-name-list>

Space-separated cell-pin pair names list of terminating cells and their terminating pins.

NOTE: *You can use wildcard expressions while specifying this list.*

-ignorecells <ignorecell-name-list>

Space-separated name list of cells to be ignored while traversing the fan-out of the specified signals. You can use wildcard characters while specifying cells to be ignored.

Rules

The `aon_buffered_signals` constraint is used by the following rules:

SpyGlass Power Verify Solution	
LPSVM40	LPPLIB11

assertion_signal

Purpose

Specifies Power On Reset (POR) signals and low power signals.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `lpsignal`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `assertion_signal` constraint is as follows:

```
current_design <du-name>
  assertion_signal
    -name <sig-name>
    -value <0 | 1>
    -type <POR | PORCHECK>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the special signals.

-name <sig-name>

Name of the special signal.

-value <0 | 1>

The `-value` argument specifies whether the signal is active high (1) or active low (0).

-type <POR | PORCHECK>

Specify the `-type` argument with value `POR` for POR signals and with value `PORCHECK` for low-power signals.

Rules

The `assertion_signal` constraint is used by the following rule:

SpyGlass Power Verify Solution

LPSVM42

associate_lib

Purpose

The `associate_lib` constraint is used to associate the library names and library cells with the domains declared in the UPF.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `associate_lib` constraint is as follows:

```
associate_lib
  -domain <domain-name>
  -lib <list of library-name>
  -cell <list of cell-name>
```

Arguments

-domain <domain-name>

Name of a domain as declared in the UPF.

-lib <list of library-name>

List of one or more library names as defined in the .lib file with keyword:

```
library(library-name) {
}
```

-cell <list of cell-name>

List of one or more cell names as defined in the .lib file with keyword:

```
cell (cell-name) {
}
```

NOTE: Wildcards are supported for cell names. Both '-lib' and '-cell' fields are optional and can be specified together. The -domain argument is mandatory.

Rules

The `associate_lib` constraint is used by the following rule:

SpyGlass Power Verify Solution	
LPPLIB20	SGDC_lowpower118

assume_waveform

Purpose

The `assume_waveform` constraint is used to specify a user-defined waveform on a design object. This waveform is then used during the formal verification of false path (`set_false_path`) and multicycle path (`set_multicycle_path`) constraints in SpyGlass TXV.

You can specify the waveform in terms of edge list corresponding to a particular SDC `create_clock` or `create_generated_clock`.

Product

SpyGlass TXV solution

Syntax

The syntax of the `assume_waveform` keyword in a SpyGlass Design Constraints file is as follows:

```
current_design <du-name>
assume_waveform
    -clock <sdc-clock-name>
    -edgelist <list-of-edges>
    -object <port | pin | net>
```

Arguments

-clock <sdc-clock-name>

The name of an SDC `create_clock`/`create_generated_clock` as

specified in the `-name` argument of the SDC `create_clock/`
`create_generated_clock` command.

-edgelist <list-of-edges>

The waveform with reference to the SDC clock specified in the `-clock` argument.

-object <port-name | pin-name | net-name>

The design object where this waveform needs to be applied during verification of false path or multicycle path constraints. You can either specify a port, pin, or net name.

Example

If the SDC clock is specified as:

```
create_clock -name clk1 -period 10 clk
```

The corresponding `assume_waveform` can be specified as:

```
assume_waveform
    -clock "clk1"
    -edge_list "{1 3 5}"
    -object "in1"
```

Rules

The `assume_waveform` constraint is used by the following rules:

SpyGlass TXV Solution	
Txv_MCP01	Txv_FP01

assume_path

Purpose

The `assume_path` constraint is used to specify the paths that exist between the input pins and the output pins of black boxes.

NOTE: Use this constraint if you want SpyGlass rules to consider combinational path from an input to output ports of a black box, whereas use the [abstract_port](#) constraint if you want to associate or specify a clock on an input or output port.

Product

SpyGlass CDC solution, SpyGlass Constraints solution, SpyGlass latch product, SpyGlass DFT solution

Syntax

The syntax of using the `assume_path` keyword in a SpyGlass Design Constraints file is as follows:

```
current_design <du-name>
assume_path -name <bbdu-name>
           -input <input-pin-name>
           -output <output-pin-name>
```

Arguments

-name <bbdu-name>

A black box module name (for Verilog designs) or a black box entity name (for VHDL designs).

For VHDL designs, this constraint is applied to all architectures of the specified entity.

-input <input-pin-name>

The name of an input pin of the black box design unit *<bbdu-name>*.

-output <output-pin-name>

A list of output pin names of the black box design unit *<bbdu-name>*.

Examples

Consider the following example:

```
assume_path -name BBOX -input d -output q qbar
```

This constraint indicates that paths exist between input pin `d` and output pins `q` and `qbar` of black box design unit `BBOX`.

For SpyGlass CDC Solution

For the SpyGlass CDC solution, clock domain propagation through black boxes is normally limited, because the exact relationship between input pins and output pins is not known. If only one clock signal is propagated to a black box instance, it is assumed that all pins of the black box instance are in the domain of that clock signal and further clock domain propagation is performed. However, if more than one clock signal propagates to the instance, it is not possible to assume any clock domain information.

One way to extend the clock domain propagation through a black box instance is to specify which output pins belong to the same clock domain as a particular input pin.

The `Ac_unsync01/Ac_unsync02` and `Ac_sync01/Ac_sync02` rules allow you to specify that paths exist between input pins and output pins of black boxes and, thus, clock domain propagation can traverse through these black boxes. These paths can be specified using the `assume_path` keyword in a SpyGlass Design Constraints file.

For SpyGlass Constraints Solution

For the *SpyGlass Constraints solution*, `assume_path` constraint helps in determining different paths available, for a black box module, between the specified list of input pins and output pins.

Rules

The `assume_path` constraint is used by the following rules:

SpyGlass CDC Solution
All rules
SpyGlass Constraints Solution
All rules
SpyGlass latch Product
LatchFeedback

SpyGlass DFT Solution

Clock_04

atspeed_clock_frequency

Purpose

The `atspeed_clock_frequency` constraint is used to specify frequencies associated with a test clock. The `atspeed_clock_frequency` constraint might potentially affect the way SpyGlass infers at-speed clock domain.

NOTE: *Prior to the SpyGlass 4.4.0 release, the names of this constraint were `testclock_frequency` and `testclockFrequency`.*

Product

SpyGlass DFT DSM solution

Syntax

The syntax to specify the `atspeed_clock_frequency` constraint is as follows:

```
current_design <du-name>
  atspeed_clock_frequency
    -name <testclock-node>
    -freqList <freq-symbol-list>
    [-enables <assign-node-paths>
    -values <combination1 combination2>]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the instance hierarchies.

-name <testclock-node>

Path to the test clock node. This node must be declared as a test clock.

-freqList <freq-symbol-list>

List of frequency symbols with which the test clock is associated. These can be actual numbers like 100, 200 or alphanumeric symbols like F1, f2, etc.

-enables <assign-node-paths>

(Optional) Paths to nodes that enable a particular frequency.

-values <combination1 combination2>

(Optional) Binary strings (0s and 1s) that specify a combination to be applied to the enable pins. The length of the combination should be equal to the number of enable pins.

Examples

Consider the following example to specify frequencies (symbols/numeric) at nodes of interest that are declared as test clocks:

```
clock -name top.u2.clockout -testclock -atspeed
atspeed_clock_frequency -name top.u2.clockout -freqList 100
200 400 800 -enables s1 s2 s3 s4 -values 1000 0100 0010 0001
```

NOTE: Each symbol or value in the example has a corresponding set of enabling conditions.

You can infer the following from the example:

- The test clock can have frequencies: 100, 200, 400, and 800.
- These frequencies are selected by signals on the pins s1, s2, s3, and s4.
- Frequency 100 is produced when s1, s2, s3, & s4 are 1000.
- Frequency 200 is produced when s1, s2, s3, & s4 are 0100.
- Frequency 400 is produced when s1, s2, s3, & s4 are 0010.
- Frequency 800 is produced when s1, s2, s3, & s4 are 0001.

Rules

The `atspeed_clock_frequency` constraint is used by the following rules.

SpyGlass DFT DSM Solution

All rules that use clock domain

balanced_clock

Purpose

The `balanced_clock` constraint specifies points within the clock distribution system where all clocks fed from that point are to be considered in the same clock domain.

Product

SpyGlass DFT solution

Syntax

The syntax of using the `balanced_clock` keyword in a SpyGlass Design Constraints file is as follows:

```
balanced_clock
  -name <hier-pin-name-list>
  [ -domain <domain-name> ]
```

Arguments

-name <hier-pin-name-list>

Hierarchical name of a clock pin.

The pin can be a primary pin as well as an internal pin.

You can specify a single hierarchical pin name or a space-separated list of hierarchical pin names. When you specify a clock pin list, all flip-flops triggered by these clock pins are assumed to be in the same clock domain

by the rules that check for clock domains.

-domain <domain-name>

(Optional) Specifies the domain of the specified balanced clock.

The domain name can be any valid string.

When the domain name is not specified, a default domain name is created and all balanced clocks specified in one `balanced_clock` constraint are assumed to be in this domain.

When multiple `balanced_clock` constraints are used with the same domain name, all clocks specified in these `balanced_clock` constraints are assumed to be in this (common) domain.

Thus, the following specifications create four domains: `clkA`, `clkB`, `clkC`, and `clkD U clkE`:

```
balanced_clock -name clkA
balanced_clock -name clkB
balanced_clock -name clkC
balanced_clock -name clkD -domain big
balanced_clock -name clkE -domain big
```

Examples

Consider the following example:

```
module top(...)
...
m1 u1 (clock, ...);
...
m2 u2 (... ,clock, ...);
...
endmodule
```

For example, when `top.u1.clock` is declared in a `balanced_clock` constraint, all flip-flops inside instance `u1` will be considered to be in the same clock domain and not in the same clock domain as the flip-flops in instance `u2`.

Another example of the `balanced_clock` constraint is as follows:

```
balanced_clock -name top.inst1.clkPort
```

The above specification indicates that the `clkPort` pin of the instance `inst1` in the top-level design unit `top` is the balanced clock pin.

```
balanced_clock -name clk1
```

The above specification indicates that the `clk1` port of the top-level design unit is the balanced clock pin.

Rules

The `balanced_clock` constraint is used by the following rules:

SpyGlass DFT Solution		
Clock03	Clock_10	Scan_22 (optional)

blackbox_power

Purpose

The `blackbox_power` constraint is used for modeling the black boxes during power estimation.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `set_black_box_power`.

Three models are supported for the black box modeling. Refer to the *Modeling Black Boxes in Power Estimation* section in the *SpyGlass Power Estimation and SpyGlass Power Reduction Rules Reference Guide* for details.

Different arguments of this constraint are used for a different purpose for each model, as explained below:

- Refer to the Case 1 of the *Modeling Black Boxes in Power Estimation* section for the description of this scenario.

```
current_design <du-name>  
  blackbox_power  
    -modname <mod-name> | -instname <inst-name> }
```

```
-leakage <lfloat> -internal <ifloat>
-switching <sfloat>
-input_pin_cap <ipcfloat>
```

Where:

- ❑ **<du-name>**: Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs)
 - ❑ **<mod-name>**: Name of the black box module
 - ❑ **<inst-name>**: Hierarchical instance name of the black box
 - ❑ **<lfloat>**: Leakage power of the black box (in Watts)
 - ❑ **<ifloat>**: Internal power of the black box (specified in Watts)
 - ❑ **<sfloat>**: Switching power of the black box (specified in Watts)
 - ❑ **<ipcfloat>**: Average input pin capacitance of the each pin of the black box (in Farad).
- Refer to the Case 2 of the *Modeling Black Boxes in Power Estimation* section for the description of this scenario:

```
current_design <du-name>
  blackbox_power
    -modname <mod-name> | -instname <inst-name>
    -leakage <lfloat> -internal <ifloat_list>
    -switching <sfloat_list>
    -clocks <clk_pin_list>
    -input_pin_cap <ipcfloat>
```

Where:

- ❑ **<du-name>**: Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs)
- ❑ **<mod-name>**: Name of the black box module
- ❑ **<inst-name>**: Hierarchical instance name of the black box
- ❑ **<lfloat>**: Leakage power of the black box (in Watts)
- ❑ **<ifloat_list>**: Space separated list of internal power in `Watts/Hz` contributed by the logic on clocks <clk_pin_list>

- ❑ `<sfloat_list>`: Space separated list of Switching power in `Watts/Hz` contributed by the logic on the clock `<clk_pin_list>`
- ❑ `<clk_pin_list>`: List of the clock pins in the black box
- ❑ `<ipcfloat>`: Average input pin capacitance of the each pin of the black box (in Farad).
- Refer to the Case 3 of the *Modeling Black Boxes in Power Estimation* section for the description of this scenario:

```
current_design <du-name>
  blackbox_power
    -modname <mod-name> | -instname <inst-name>
    -leakage <lfloat> -internal <ifloat_list>
    -switching <sfloat_list>
    -input_pin_cap <ipcfloat> -activity <afloat>
    -clocks <clk_pin_list>
    -equiv_nand2_count <gate-count>
    -register_count <reg-count>
```

Where:

- ❑ `<du-name>`: Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs)
- ❑ `<mod-name>`: Name of the black box module
- ❑ `<inst-name>`: Hierarchical instance name of the black box
- ❑ `<lfloat>`: Leakage power of the black box (in Watts)
- ❑ `<ifloat-list>`: Space separated list of percentage of internal power contributed by the logic on the clocks `<clk_pin_list>`
- ❑ `<sfloat_list>`: Space separated list of percentage of switching power contributed by the logic on the clocks `<clk_pin_list>`
- ❑ `<ipcfloat>`: Average input pin capacitance of the each pin of the black box (in Farad)
- ❑ `<afloat>`: Average activity of the black box
- ❑ `<clk_pin_list>`: List of the clock pins in the black box
- ❑ `<gate-count>`: NAND gate equivalent for combinational gates of the black box

❑ *<reg-count>*: Register count of the black box.

Rules

The `blackbox_power` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions	
PEPWR01	PEPWR02

block

Purpose

The `block` constraint is used to specify the blocks (design units) under the `current_design` unit.

Product

SpyGlass Constraints solution

Syntax

The syntax of the `block` constraint is as follows:

```
current_design <du_name>
block -name <block1-name> <block2-name>

or

current_design <top_design_name>
block -name <block1-name>
block -name <block2-name>
```

Arguments

<du_name>

Name of the top design unit

-name <block-name>

Name of the design partition to be synthesized.

Each *<block-name>* should name a module (for Verilog) or entity (for VHDL) appearing in the design that should be treated as a top-level partition. More than one block constraints may appear, or all blocks may be specified in one constraint.

NOTE: *The top design is also considered as a block and all block-level rules are run on it unless explicitly mentioned otherwise.*

Rules

The `block` constraint is used by the following rules:

SpyGlass Constraints			
Clk_Lat01	Clk_Lat02	Clk_Lat03	Clk_Uncert01
High_Fan04	High_Fan05	Clk_Consis05	Clk_Gen13
Clk_Gen17	Clk_Gen18	Clk_Gen21	Clk_Lat09
Check_Timing04	Clk_Lat06	Clk_Lat07	Clk_Trans04
Clk_Trans05	Clk_Trans06	Clk_Trans07	Clk_Trans08
Clk_Trans16	Clk_Trans17	Clk_Trans12	Clk_Trans11
Clk_Uncert05	Combo_Paths01	Combo_Paths02	Combo_Paths03
Combo_Paths04	Const_Struct01	Const_Struct02	Const_Struct04a
Const_Struct04b	Const_Struct05	Const_Struct09	IO_Consis02
Dont_Touch05	Block05	Block11	Clk_Gen01a
Clk_Gen01b	Clk_Gen33	Clk_Gen03	Clk_Gen06
Clk_Gen07	Clk_Gen14	Clk_Gen23	Clk_Gen24
Clk_Gen26	Clk_Gen27	Check_Timing03	Clk_Gen29
Clk_Gen30	Clk_Gen31	Clk_Gen32	Show_Clock_Propagation
Clk_Uncert03	Clk_Uncert08	Clk_Gen09	Clk_Gen02
Disable_Timing02	Block10	Combo_Paths06	SDC_Methodology29
Show_Case_Analysis	SDC_DnStrm08	DomainAnalysis	DomainInfo

SpyGlass Design Constraints

DomainError	Domain_SGDC_ Consis	Dont_Touch02	Dont_Touch03
False_Path07	False_Path08	CheckMCP	False_Path04a
False_Path04	False_Path03	High_Fan10	High_Fan02
Inp_Del03a	Inp_Del03b	Inp_Trans05	Inp_Trans01
Inp_Del01a	Inp_Del01b	Inp_Del01c	IO_Consis07
TE_Consis01	MCP04a	MCP04	MCP03
TE_Consis02	SDC_Methodology65	Op_Del01a	Op_Del01b
Op_Del01c	Op_Del03a	Op_Del03b	SDC_Case_Sanity01
SDC_Methodology66	SDC_Methodology67	SDC_Methodology68	Check_Timing02
Load02b	Load02a	Disable_Timing01	SDC_Methodology30
SDC_Methodology31	SDC_Methodology44_MG	SDC_Methodology47_MG	High_Fan06
High_Fan07	Test_Rules05	Test_Rules06	Clk_Uncert07
Clk_Gen08	High_Fan12	High_Fan01a	High_Fan01
High_Fan09	High_Fan14	High_Fan15	High_Fan08
Inp_Del05	Inp_Del07a	Inp_Del07	Inp_Trans07
Inp_Trans06	Inp_Trans01a	IO_Consis01	SDC_Methodology25
SDC_Methodology62	High_Fan11	SDC_Methodology26	SDC_Methodology63
Op_Trans01	MCP06	SDC_Methodology28	Block06
Op_Del05	Op_Del07a	Op_Del07	SDC_Methodology01
SDC_Methodology03	SDC_Methodology05a	SDC_Methodology06	SDC_Methodology07
SDC_Methodology09	SDC_DnStrm05	SDC_DnStrm06	SDC_Methodology10
SDC_Methodology11	SDC_Misc_Setup01	SDC_DnStrm07	SDC_Methodology22

SDC_Methodology23	SDC_Methodology24	SDC_Report04	SDC_Misc_Power01
SDC_Misc_Command01	Clk_Trans09	Test_Rules01	Test_Rules02
Test_Rules03	Test_Rules04	Clk_Gen05	High_Fan03a
High_Fan03b	Dont_Touch04	Inp_Del02	MCP05
Op_Del09	Op_Del02	SDC_DnStrm04a	SDC_DnStrm04
SDC_Report01	SDC_Misc_WLM01	SDC_DnStrm01	SDC_DnStrm02
SDC_DnStrm03	SDC_Methodology02	SDC_DnStrm04	SDC_DnStrm04a
SDC_Methodology12	SDC_Methodology13	SDC_Report03	SDC_Methodology27
SDC_Methodology70	SDC_Report01	Load01	XBuf01
Inp_Del14	Op_Del14		

blocksize

Purpose

The `blocksize` constraint is used to define the maximum or minimum allowed block size for each synthesizable block. The SpyGlass Constraint solution reports any block that does not lie in these limits. By default, the maximum block size is 100000 and the minimum is 5000.

Product

SpyGlass Constraints solution

Syntax

The syntax of the `block` constraint is as follows:

```
current_design <du_name>
block -name <block1-name> <block2-name>
```

```
blocksize -min <minimum-blocksize> -max <maximum-blocksize>
```

Arguments

<du_name>

Name of the top design unit

-min <minimum-blocksize>

(Optional) Specifies the minimum block size for the blocks

-max <maximum-blocksize>

(Optional) Specifies the maximum block size for the blocks

Examples

The following code snippet sets the minimum block size to 20000 and the maximum block size to 120000.

```
current_design top
    sdc_data -type top.sdc
    block -name lower1 lower2 lower3
    blocksize -min 20000 -max 120000
```

breakpoint

Purpose

The `breakpoint` constraint is used to specify breakpoints in a design. Using the `breakpoint` constraint, you can specify internal signals of a design to be considered as input ports of the design during functional analysis. The functional analysis will stop at the breakpoints.

Product

SpyGlass Auto Verify solution, SpyGlass CDC solution

Syntax

The syntax of the `breakpoint` constraint is as follow:

```
current_design <du-name>
breakpoint
  -name <pin-name-list>
```

Arguments

-name <pin-name-list>

Space-separated list of hierarchical pin names.

Example

The following constraint defines signals `sig1` and `sig2` in the design unit `top` as breakpoints:

```
breakpoint -name top.sig1 top.sig2
```

Rules

The `breakpoint` constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_cdc01a	Ac_cdc01b	Ac_cdc01c	Ac_cdc08

Ac_fifo01	Ac_handshake01	Ac_handshake0 2	Clock_sync03a
Ac_conv02			
SpyGlass Auto Verify Solution			
All rules			

bypass

Purpose

The `bypass` constraint is used to specify the design units (black boxes) that must be bypassed.

The intention is to declare design units that will not contain internal scan wrappers. The `bypass` constraint specifies modules whose output pins must be isolated from scan registers.

Use the Scan_20 rule of the SpyGlass DFT solution to check that such design units are so connected.

Product

SpyGlass DFT solution

Syntax

The syntax of the `bypass` constraint is as follows:

```
bypass -name <du-name>
```

NOTE: *The `bypass` constraint supports wildcard characters.*

Arguments

-name <du-name>

The name of the design unit to be bypassed.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are treated as bypassed.

You can specify a single design unit name or a space-separated list of design unit names.

Rules

The `bypass` constraint is used by the following rule:

SpyGlass DFT Solution

Scan_20

cdc_attribute

Purpose

The `cdc_attribute` constraint is used to specify mutually exclusive and unrelated signals such that:

- Convergence-related violations are suppressed for such signals, and
- Glitch issues are filtered if the specified signals are on a control crossing.

Product

SpyGlass CDC solution

Syntax

The syntax of the `cdc_attribute` constraint is as follows:

```
cdc_attribute
  [ -exclusive <mutually-exclusive-signal-names> ] |
  [ -unrelated <unrelated-signal-names> ]
```

Arguments

-exclusive <mutually-exclusive-signal-names>

Space-separated list of mutually exclusive source or destination signals, such as hierarchical net, hierarchical terminal, or port.

Such signals cannot toggle at the same time, that is, they are gray encoded.

-unrelated <unrelated-signal-names>

Space-separated list of unrelated source or destination signals, such as hierarchical net, hierarchical terminal, or port.

Such signals have no timing relationship (for example, interrupts).

NOTE: *For the Ac_glitch03 rule, SpyGlass CDC does not consider sources specified through -unrelated argument because it assumes that the design uses custom techniques to avoid the glitch for these sources.*

Examples

Example 1

Consider the scenario shown in the following figure:

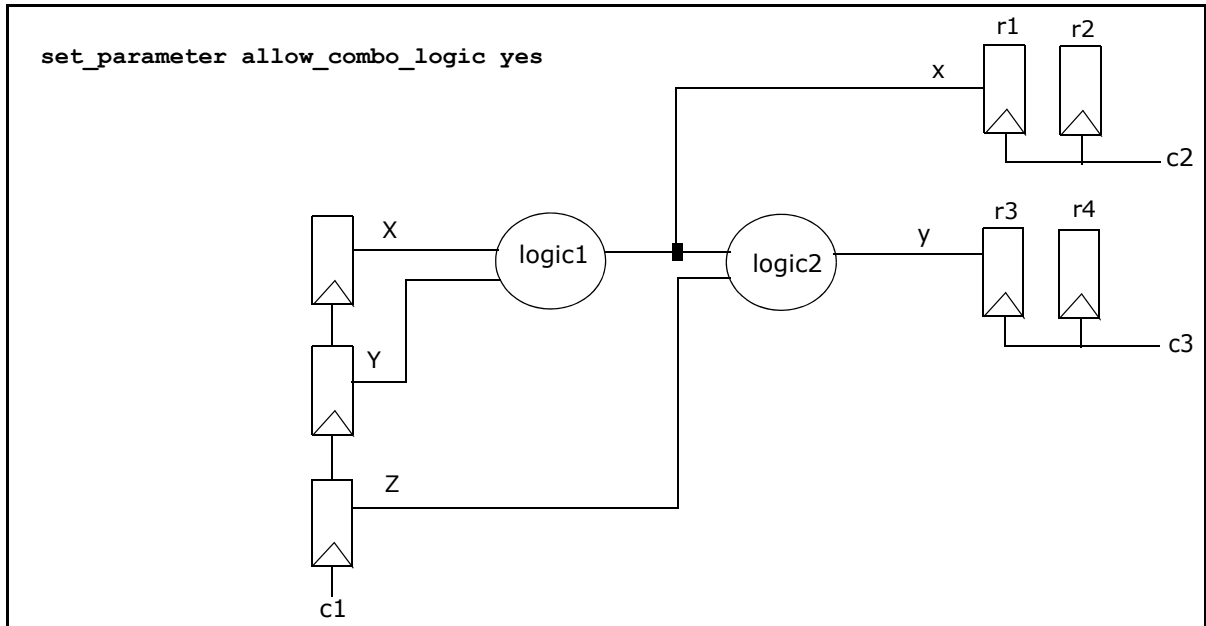


FIGURE 18.

In the above scenario, the *Ac_glitch03* rule reports convergence issues at the *r1* and *r3* destination for the *X*, *Y*, and *Z* sources that are mutually exclusive due to gray encoding.

To suppress the *Ac_glitch03* rule violations due to X, Y, and Z, specify the following constraint:

```
cdc_attribute -exclusive X Y Z
```


Example 2

Consider the following files specified for SpyGlass analysis:

```
// test.v
module top(rst, in1, in2, in3, in4, clk1,
           clk2, out1);
  input rst, in1, in2, in3, in4, clk1, clk2;
  output out1;
  wire w1,w2;
  DES des(.d0(in1), .d1(in2), .d2(in3),
           .d3(in4),.c1(clk1), .out(w1));
  DFF fReg1(.clk(clk2), .d(w1), .q(w2));
  SYNC sync(.clk(clk2), .rst(rst),
            .in(w2), .out(out1));
endmodule

module DES(d0, d1,d2, d3,c1, out);
  input d0, d1, d2, d3,c1;
  output out;
  wire f1, f2, f3, w1;
  DFF fReg1(.clk(c1), .d(d0), .q(f1));
  DFF fReg2(.clk(c1), .d(d1), .q(f2));
  DFF fReg3(.clk(c1), .d(d3), .q(f3));
  MUX mux_1(.in1(f1) , .in2(f2),
            .sel(d2), .out(w1));
  assign out = w1 & f3;
endmodule

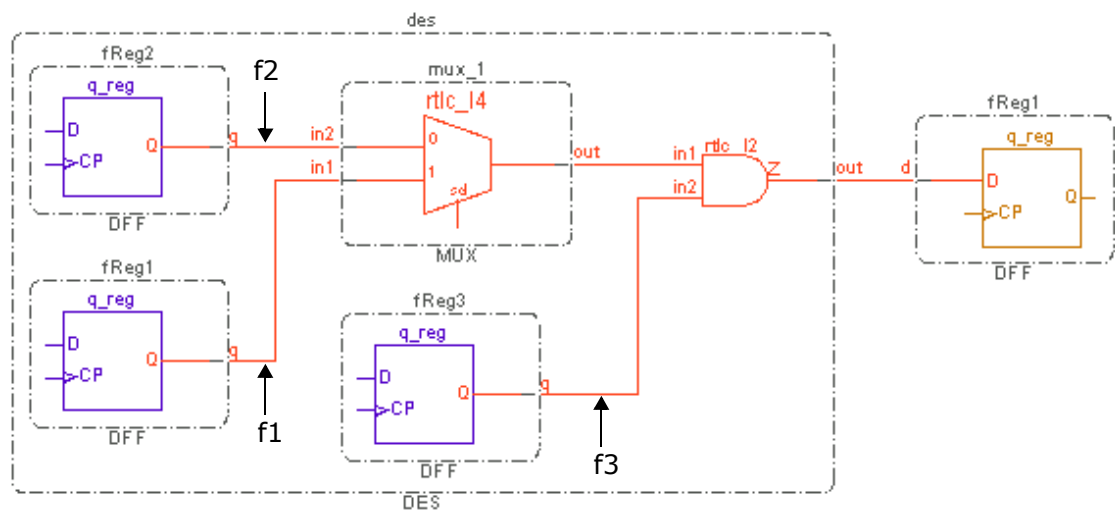
module MUX(in1 , in2, sel, out);
  input in1, in2, sel;
  output out;
  wire out;
  assign out = (sel) ? in1: in2;
endmodule

module DFF(clk, d, q);
  input clk;
  input d;
  output q;
  reg q;
  always @(posedge clk ) begin
    input clk;
```

```
// constr.sgd
current_design top
clock -name clk1 -domain d1
clock -name clk2 -domain d2

// Project File
set_parameter allow_combo_logic yes
```

For the above example, the *Ac_glitch03* rule reports the following crossing:



Violation Message:

Glitch check performed on destination flop 'top.fReg1.q' clocked by 'top.clk2' (3 source(s), 1 domain(s)). Multi-source toggling check : 'FAILED'

FIGURE 19.

In the above crossing, the f1, f2, and f3 signals are mutually exclusive. If you do not want to report the crossings containing these signals, specify the following constraint:

```
cdc_attribute -exclusive top.des.f1 top.des.f2 top.des.f3
```

Rules

The *cdc_attribute* constraint is used by the following rule:

SpyGlass CDC Solution			
Ac_glitch03	Ac_conv01	Ac_conv02	Ac_conv03
Ac_conv04			

cdc_check_glitch

Purpose

The `cdc_check_glitch` constraint is used to generate glitch expressions for any net in the design.

Product

SpyGlass CDC solution

Syntax

Use the `cdc_check_glitch` constraint as follows to generate glitch expressions:

```
cdc_check_glitch -name <net-name> -type <type>
```

Arguments

-name <net-name>

Specifies the hierarchical name of the net/pin.

-type <type>

Can be any subset {010, 101}

Examples

The following specification builds a glitch expression for the `top.A` net for the 010 glitch:

```
cdc_check_glitch -name top.A -type 010
```

The following specification builds a glitch expression for the `top.B` net for both 101 and 010 glitch:

```
cdc_check_glitch -name top.B -type 010 101
```

Rules

The `cdc_check_glitch` constraint is used by the following rule:

SpyGlass CDC Solution
Ac_glitch02

cdc_define_transition

Purpose

The `cdc_define_transition` constraint is used to specify the permitted transitions of any net.

Product

SpyGlass CDC solution

Syntax

Use the `cdc_define_transition` constraint as follows to specify the permitted transitions:

```
cdc_define_transition -name <net-name> -type <type>
```

Arguments

-name <net-name>

Specifies the hierarchical name of the net/pin.

-type <type>

Can be any subset {00, 11, 01, 10, 010, 101}, excluding {00, 11}

Examples

The following specification specifies the permitted transitions of the top.A net:

```
cdc_define_transition -name top.A -type {00, 11, 01, 10}
```

Rules

The `cdc_define_transition` constraint is used by the following rule:

SpyGlass CDC Solution
Ac_glitch02

cdc_false_path

Purpose

The `cdc_false_path` constraint is used to specify false paths so that clock-domain crossings along these paths are ignored for rule checking. Such crossings are reported in *Section C of The Clock-Reset-Detail Report* of the SpyGlass CDC.

Note the following points:

- Use this constraint instead of waivers for SpyGlass CDC rules.
- If you want to suppress rule-checking on crossings that contain static signals, you can also use the [quasi_static](#) constraint.
- The internal rule `FalsePathSetup` checks for valid `cdc_false_path` constraints.

Advantages of using the `cdc_false_path` Constraint

Following are the advantages:

- Targets violations of the SpyGlass CDC solution with `-to`, `-through`, `-from` that accommodate various schemes of filtering for crossings, for example, `-from clk1` will take out all crossings initiated at flip-flops clocked by `clk1`.
- The constraint is understood by rules (waivers are applied post process), and as a result crossings as well as convergence issues may be filtered based on the `cdc_false_path` specification.
- More portable as it does not refer to rule names or messages.
- Reduces the size of the *.vdb file.

- As compared to the *quasi_static* constraint, the `cdc_false_path` constraint provides more flexibility in specifying the type of crossing through various options (`-from`, `-to`, and `-through`).

If one `_cross_per_dest` parameter set to `yes` and if a crossing (`s1->d1`) is waived off using `cdc_false_path` constraint (source `s1` for a destination `d1`), the other crossing (`s2->d1`) of the same destination having another source `s2` is reported. In case of waivers specified for one source (`s1->d1`), none of the crossings (`s1->d1`, `s2->d1`) will be reported for the destination.

Leaf level destination pin support

When a destination net has multiple fan-out to different instance pins, `cdc_false_path` supports filtering of each destination by specifying leaf level pin with `-to` argument.

In the following schematic, to waive a crossing from `r1` to `r2`, specify instance pin `r2_reg.D`

```
cdc_false_path -from r1 -from_type data -to r2_reg.D -to_type data
```

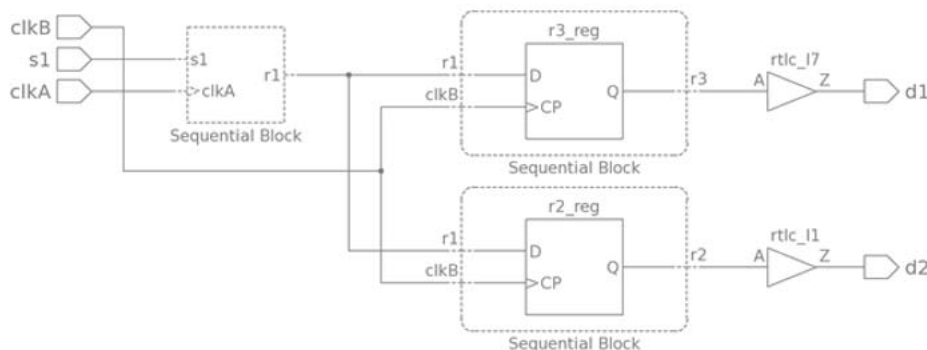


FIGURE 20.

NOTE: Leaf-level destination is not supported if:

- Hierarchical pins are specified
- Clock paths specified

- '-through' specified
- Wild card specified
- Multiple values specified
- -to_type and/or -from_type is clock

Product

SpyGlass CDC solution

Syntax

Use the `cdc_false_path` constraint as follows to specify the false paths:

```
current_design <du-name>
    cdc_false_path
        [ -from <obj1-name> ]
        [ -to <obj2-name> ]
        [ -through <obj3-name> ]
        [ -from_type <obj1-type> ]
        [ -to_type <obj2-type> ]
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs)

-from <obj1-name>

The name of a clock source or the net connected to the clock pin of sequential elements, a clock tag (specified by the `-tag` argument of the [clock](#) constraint), a master design unit, a flip-flop output net, a port of a master design unit of the source, or an input/inout port.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

Wildcard expressions are matched to clock tag names only if no nets are present in the design with the same name. If a net is found in the design, clock tag name will

not be inferred.

-to <obj2-name>

The name of a clock source or the net connected to the clock pin of sequential elements, a clock tag (specified by the `-tag` argument of the [clock](#) constraint), a master design unit, a flip-flop output net, or a port of a master design unit of the destination.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

-through <obj3-name>

Name of an internal net, master design unit, or a port of the master design unit.

Note that specifying **`cdc_false_path -through X -through Y`** is equivalent to specifying **`cdc_false_path -through X Y`**.

You can use regular expressions and wildcard characters ('*' and '?') while specifying names. For details, refer to the *Using Regular Expressions and Wildcard Characters* topic of the *Atrenta Console User Guide*.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression with the hierarchies. For example, the `top.*.n1` expression is matched to `top.u1.n1` and `top.u1.u2.n1`. By default, the `cdc_false_path` constraint matches only `top.u1.n1`.
Setting the value of the `hier_wild_card` parameter to `yes` runtime performance of the `cdc_false_path` constraint is impacted.

-from_type <obj1-type>

Type of the signal specified in the `-from <obj1-name>` argument for which the crossings need to be filtered out.

This argument accepts any of the following values:

clock	data	all (default)
-------	------	---------------

NOTE: Use this argument only when you specify the `-from <obj1-name>` argument.

-to_type <obj2-type>

Type of the signal specified in `-to <obj2-name>` for which the crossings need

to be filtered out.

This argument accepts any of the following values:

clock	data	all (default)
-------	------	---------------

NOTE: Use this argument only when you specify the *-to <obj2-name>* argument.

Specifying Arguments of the `cdc_false_path` Constraint

Please note the following points:

- You must specify at least one of the `-from`, `-to`, and `-through` arguments.
- When you specify a clock name (hierarchical net name) with the `-from` or `-to` argument, it should be a hierarchical net name. Then the applicable paths are paths originating from or terminating at flip-flops triggered by the specified clock.
- When a master design unit is specified with the `-from` or `-to` arguments, the applicable paths are paths originating from or terminating at all flip-flops in all instances of the specified design unit. When you specify a master design unit name with the `-through` argument, the applicable paths are all paths passing through all instances of the specified design unit.
- When you specify a flip-flop output net (hierarchical net name) with the `-from` or `-to` arguments, the applicable paths are paths originating from or terminating at the corresponding flip-flop.
- When you specify a pin name (of a master design unit) (in `<du-name>/<pin-name>` format) with the `-from` or `-to` arguments, the applicable paths are paths originating from or terminating at all flip-flops connected to the specified pin in all instances of the corresponding master design unit. When you specify a pin name (of a master design unit) with the `-through` argument, the applicable paths are all paths passing through the specified pin in all instances of the corresponding master design unit.
- When you specify an internal net (hierarchical net name) with the `-through` argument, the applicable paths are paths containing the specified net.

- When you specify a clock net with the `-from` or `-to` arguments, only those flip-flops that are exclusively triggered by the clock net are considered. Flip-flops where this clock net and some other clock net(s) are converging are not considered.

Specifying Object Names to the `cdc_false_path` Constraint

The following points describe different ways of specifying different objects to the `cdc_false_path` constraint:

- Specify net names as hierarchical names with respect to top level.
The top-level name is optional. For example, you can specify `top.U1.net1` or `U1.net1`.
- Specify design units (sub modules) as simple design unit names, such as `myDU1`.
- Specify top-level port names as simple names. For example, `in1`.
Specify port of a design unit (submodule) as `<du-name>/<port-name>` or `<du-name>.<port-name>`. You should **not** specify instance names or top-level design unit name.

Wild Card Support for the `cdc_false_path` Constraint

You can use the following wild card characters while specifying object names:

- Asterisk (*): To match none, one, or multiple object names
For example, `top.*.n1` matches with `top.U1.n1`.
- Question mark (?): To match none or one object name

Handling Escaped Names

You cannot specify wildcards with escaped names directly. In such cases, replace the escape character (`\`) with a wildcard character.

For example, for the source flip-flop output `top.\rdPtr1` and `top.\rdPtr2`, you cannot specify the following constraint:

```
cdc_false_path -from "top.\rdPtr*"
```

Instead, you should specify the following:

```
cdc_false_path -from "top.*rdPtr*"
```

Handling an Array of Instances

For an array of instances in a design, escaped character is appended as part of the hierarchy name. Therefore, wildcard in such names should also be specified as mentioned above.

For example, if the design `top` has an array of instances `U_INST[1:10]` and `n1` is the net inside the module of these instances. In this case, SpyGlass generates the following nets:

```
top.\U_INST[1].n1
top.\U_INST[2].n1
...
```

Now, to declare `cdc_false_path` for the `n1` net inside any of these instances, you cannot specify the name as `top.\U_INST[*].n1`. Instead, you should specify this as follows:

```
cdc_false_path -to "top.*U_INST*.n1"
```

NOTE: *The `hier_wild_card` parameter is set to `yes` and the `cdc_false_path` constraint matches all hierarchies based on the specified wildcard expression. For example, if you specify the wildcard expression `top.*.n1`, the `cdc_false_path` constraint matches the hierarchies `top.u1.n1` and `top.u1.u2.n1`. However, by default, the `cdc_false_path` constraint matches only `top.u1.n1`. In this example, if you specify the wildcard expression as `top.*.*.n1`, the `cdc_false_path` constraint matches `top.u1.u2.n1`.*

Vector Name Support in the `cdc_false_path` Constraint

You can specify vector object names as simple object names, object names with width specification, part-select, or bit-select.

You must use the *Verilog bus-width specification* format for specifying vector object names even for VHDL designs. For example,

`in1 (7 downto 0)` must be specified as `in1 [7:0]` or `in1 [0:7]`.

Similarly, a part-select should be written as say `in1 [6:3]` or `in1 [3:6]`.

A bit-select should be written as say `in1 [4]`.

You can also use wildcards in vector names. `in1 [*]` matches any bit of `in1`.

The bit-width specification using wildcards for part-select is assumed to be

higher bit first. Therefore, the order of bits is important. `in1[*:3]` matches `in1[7:3]`. `in1[3:*)` matches `in1[3:0]`. `in1[0:*)` does not match anything!

Scope of False Paths specified by the `cdc_false_path` Constraint

The scopes for different object types are described below:

- **Clock name (hierarchical net name) with the `-from` or `-to` argument:** Applicable paths are considered to originate from or terminate at the flip-flops triggered by the specified clock.
- **Master design unit specified with the `-from` or `-to` arguments:** Applicable paths are considered to originate from or terminate at all flip-flops in all instances of the specified design unit.
- **Master design unit name with the `-through` argument:** Applicable paths are considered as all paths passing through all instances of the specified design unit.
- **Flip-flop output net (hierarchical net name) with the `-from` or `-to` arguments:** Applicable paths are considered as the paths originating from or terminating at the corresponding flip-flop.
- **Net Objects:** While specifying a net object, all nets directly connected to that net object (anywhere in the top-level design) are implicitly considered, in addition to the specified net object.

For example, `top.U1.net1` implicitly specifies directly connected net `top.net1`. This effect is more profound and often comes as a surprise when combined with a wildcard net specification. For example, if you specify `top.U1.*` with the intention to match all nets connected to output pins of flip-flops within `top.U1`, that will ultimately result in matching of every net within `top.U1` and every net present anywhere in the design unit name, `top`, that is directly connected to any net within `top.U1`. The most unexpected result is that this will match any clock net specified at higher levels that are directly connected to nets within `top.U1`. The best way to match all sources of crossings (or destinations of crossings) within a sub-module is to specify the design unit name of the sub-module directly. For example, `cdc_false_path -from A` command matches all crossings that originate within the sub-module `A`.

- **Pin name with the -from or -to arguments:** When you specify a pin of a master design unit in the `<du-name>/<pin-name>` format) with the `-from` or `-to` arguments, the applicable paths are the paths originating from or terminating at all the flip-flops connected to the specified pin in all instances of the corresponding master design unit.
- **Pin name with the -through argument:** When you specify a pin of a master design unit with the `-through` argument, the applicable paths are all paths passing through the specified pin in all instances of the corresponding master design unit.
- **Internal net (hierarchical net name) with the -through argument:** Applicable paths are the paths containing the specified net.

Handling Merged Clocks with the `cdc_false_path` Constraint

When multiple clocks reach a clock pin of a flip-flop or a latch, such clocks are called merged clocks.

To waive clock crossing involving a flip-flop or a latch triggered by merged clocks, specify each clock pair to the `cdc_false_path` constraint. For example, consider the following figure:

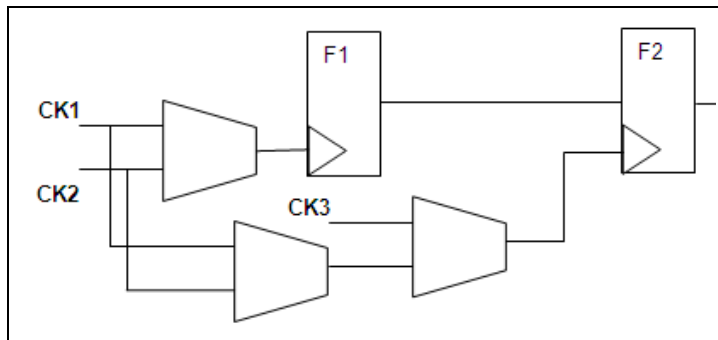


FIGURE 21. Example of merged clocks

In the above case, specify the following constraints to waive the crossing:

```
cdc_false_path -from CK1 -to CK2
cdc_false_path -from CK1 -to CK3
cdc_false_path -from CK2 -to CK1
```

```
cdc_false_path -from CK2 -to CK3
```

The crossing is reported if you do not specify all combinations of clock pairs. The violation message reports the clock pair that is not specified by using the `cdc_false_path` constraint is reported.

NOTE: *This is not supported for crossings that involve black boxes or primary ports. In such cases, specify all the clocks in the same constraint.*

Alternatively, you can specify names of all clocks from a merged set as a space-separated list by using the `-from` or `-to` argument, as shown in the following example:

```
cdc_false_path -from clk1 clk2 ...
```

You can specify the above constraint in the following manner:

```
cdc_false_path -from CK1 CK2 -to CK1 CK2 CK3
```

Clock crossing involving merged clocks are not waived if one of the clocks is not specified.

You cannot specify multiple objects that are not merged clocks. Such specification does not match any clock crossing and the `FalsePathSetup` rule reports a violation. For example, the following specification is invalid:

```
cdc_false_path ... -to top.U1.Q clk2
```

Impact of `one_cross_per_dest` Parameter on `cdc_false_path` Constraint

Setting the `one_cross_per_dest` rule parameter to `yes` (default value) results in only the first found clock crossing of a destination object to be reported.

If you set a valid `cdc_false_path` constraint on this first found clock crossing, then the next found clock crossing for the same destination object is reported. For example, there are three clock crossings between sources `s1`, `s2`, and `s3` and the destination `d1` and these clock crossings are found as say, `s1->d1`, `s2->d1`, and `s3->d1`. Normally, the crossing `s1->d1` will be reported. If you set a valid `cdc_false_path` constraint on this path, the second found crossing `s2->d1` is reported:

```
cdc_false_path -from s1 -to d1
```

If you have only specified the `-to` argument (without `-from` argument) in

a valid `cdc_false_path` constraint, all clock crossing involving the destination object are waived.

cdc_false_path Sanity Checking Rules

Following are the sanity checking rules for the `cdc_false_path` constraint:

Rule	Checks
SGDC_cdc_false_path01	Existence of object specified with the <code>-from</code> argument (without wildcards)
SGDC_cdc_false_path02	Existence of object specified with the <code>-to</code> argument (without wildcards)
SGDC_cdc_false_path03	Existence of object specified with the <code>-through</code> argument (without wildcards)
SGDC_cdc_false_path04	Existence of object specified with the <code>-from/-to/-through</code> arguments (with wildcards)
FalsePathSetup	Flags when a <code>cdc_false_path</code> constraint specification does not waive any clock crossing

Example

Example 1

```
cdc_false_path -from clk1 -to top.lower.q
```

The above specification suppresses all the paths originating from flip-flops triggered by clock `clk1` and terminating at flip-flop `top.lower.q`.

Example 2

```
cdc_false_path -through LOWER/out1
```

The above specification suppresses all the paths passing through pin `out1` of all instances of master design unit `LOWER`.

Example 3

```
cdc_false_path -from "top.b1.*" -to "block1" -from_type clock
```

The above specification suppresses all the paths that are:

- Originating from the flip-flops clocked by the clock that reaches the b1 instance.
- Terminating at the block1 block.

Example 4

```
cdc_false_path -from "top.b1.*" -to "block1" -from_type data
```

The above specification suppresses all the paths originating from the data path of the b1 instance and terminating at the block1 block.

In this case, the clocks reaching the b1 instance are not honored for suppressing the crossings.

Example 5

```
cdc_false_path -from "block1" -to "top.b1.*" -to_type clock
```

The above specification suppresses all the paths originating from the block1 block and terminating at flip-flops that are clocked by the same clock reaching the b1 instance.

Rules

The `cdc_false_path` constraint is used by the following rules:

SpyGlass CDC Solution			
Clock_sync03a	Clock_sync03b	Clock_sync08	Clock_sync08a
Clock_sync09	Ac_cdc01a	Ac_cdc08	Ac_handshake01
Ac_cdc01b	Ac_cdc01c	Ac_conv02	Ac_conv03
Ac_handshake02	Ac_conv01	Ac_unsync01	Ac_unsync02
Ac_conv04	Ac_conv05	Ac_crossing01	Ac_datahold01a
Ac_sync01	Ac_sync02	Ac_meta01	Ac_glitch01
Ac_glitch02	Ac_glitch03		

SpyGlass Design Constraints

cdc_filter_coherency

Purpose

The `cdc_filter_coherency` constraint is used to specify points at or beyond which no convergence of signals should be reported. Filtering convergence this way helps in debugging convergences-related issues.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `cdc_filter_coherency` constraint is as follows:

```
cdc_filter_coherency
[ -stop_points <stop-points> ]
[ -conv_gates <conv-points> ]
[ -unrelated <list-of-synchronizers> ]
```

Arguments

-stop_points <stop-points>

Space-separated list of hierarchical net names, hierarchical terminals, ports, or instances beyond which convergence should not be propagated.

See [Example 2](#).

-conv_gates <conv-points>

Space-separated list of hierarchical net names, hierarchical terminals, ports, or instances at which convergence should not be reported.

See [Example 1](#).

-unrelated <list-of-synchronizers>

Space-separated list of synchronizers such that the `Ac_conv01`, `Ac_conv02`, `Ac_conv03`, and `Ac_conv04` rules do not report violations on the convergences involving these synchronizers.

NOTE: Use the `cdc_attribute` constraint instead of the `cdc_filter_coherency` constraint with the `-unrelated` argument. If the `cdc_filter_coherency -unrelated` constraint is used together with the `cdc_attribute` constraint, SpyGlass CDC ignores the `cdc_filter_coherency` constraint and honors the `cdc_attribute` constraint.

See [Example 3](#).

Example

Consider the scenario shown in the following figure:

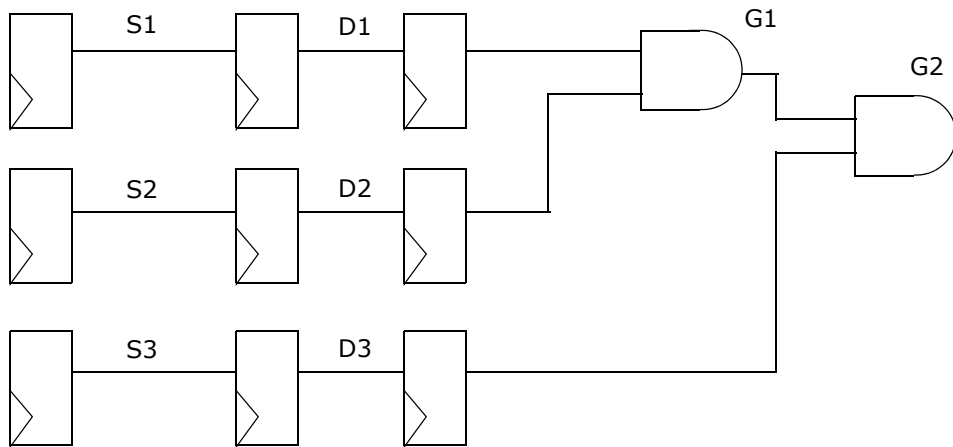


FIGURE 22.

[Example 1](#) and [Example 2](#) shows how you can filter coherence shown in the above scenario.

Example 1

Consider the following constraint specification:

```
cdc_filter_coherency -conv_gates top.G2
```

On specifying the above constraint, no convergence will be reported on G2 shown in [Figure 22](#). However, convergence on G1 will get reported, which was not reported earlier.

Example 2

Consider the following constraint specification:

```
cdc_filter_coherency -stop_points top.G1
```

On specifying the above constraint, propagation from S1 and S2 will be stopped on G1 shown in [Figure 22](#). Therefore, no convergence will be reported on G2. However, convergence will be reported at G1 for S1 and S2.

Example 3

Consider the following figure:

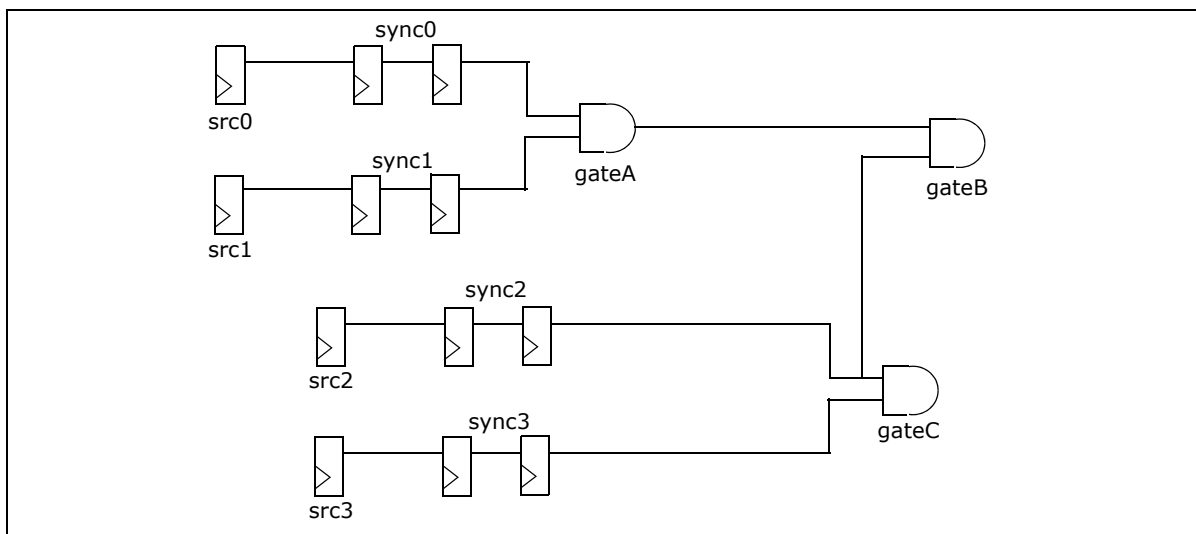


FIGURE 23. Example of -unrelated Argument of cdc_filter_coherency Constraint

In the above figure, if you do not want the *Ac_conv01*, *Ac_conv02*, *Ac_conv03*, or *Ac_conv04* rules to report convergence on gateB, specify the sync0, sync1, and sync2 synchronizers as unrelated by specifying any of the following constraints:

SpyGlass Design Constraints

```
cdc_filter_coherency -unrelated sync0 sync1 sync2
```

or

```
cdc_filter_coherency -unrelated src0 src1 src2
```

However, note that the value specified by the `-unrelated` argument is not migrated to the block-level boundary during SoC-level verification. In such cases, specify the constraint manually at the top level. For example, consider the following figure:

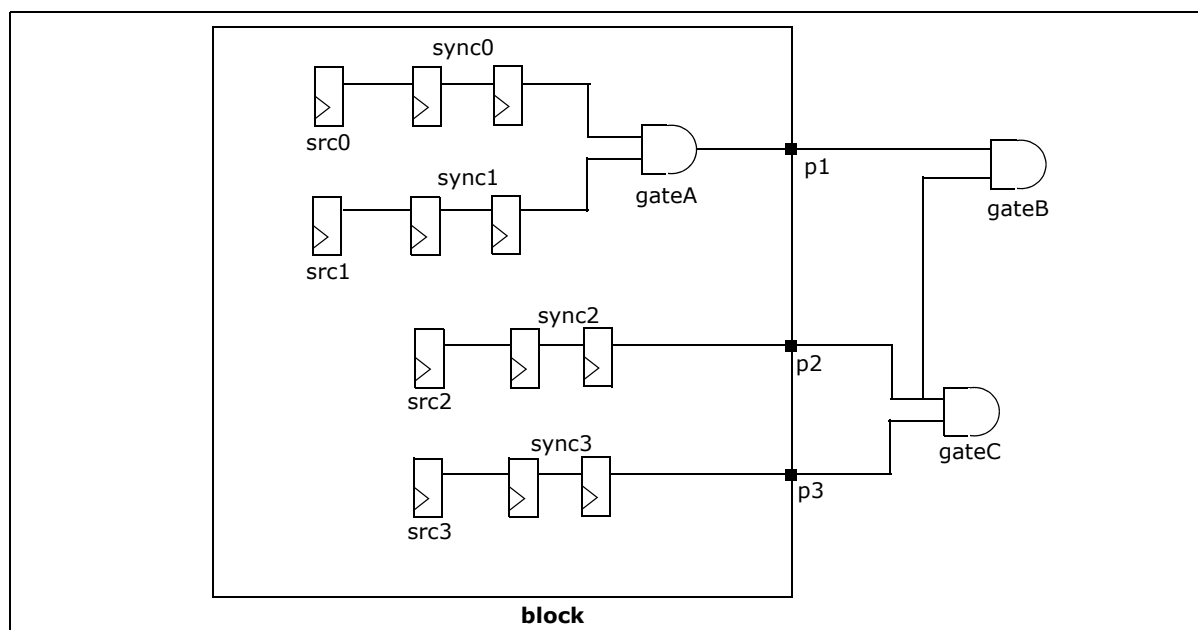


FIGURE 24.

In the above scenario, consider that you have given the following constraints:

```
cdc_filter_coherency -unrelated sync0 sync1 sync2
```

In this case, the above constraint will not automatically migrate to the block boundary. Therefore, you must manually specify the following constraint to suppress the `Ac_conv01`, `Ac_conv02`, `Ac_conv03`, or `Ac_conv04` violations at gateB:

```
cdc_filter_coherency -unrelated p1 p2
```

NOTE: The `cdc_attribute -unrelated` constraint performs the same check as the `cdc_filter_coherency -unrelated` constraint. For example, the following two constraints perform the same checks:

```
cdc_filter_coherency -unrelated sync0 sync1 sync2
cdc_attribute -unrelated sync0 sync1 sync2
```

Rules

The `cdc_filter_coherency` constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_conv01	Ac_conv02	Ac_conv03	Ac_conv04

cdc_filter_path

Purpose

The `cdc_filter_path` constraint allows you to specify paths so that the `Ac_unsync01/Ac_unsync02` and `Ac_sync01/Ac_sync02` rules of the *SpyGlass CDC* solution do not consider clock crossings along these paths. The `cdc_filter_path` constraint is the same as [cdc_false_path](#).

Product

SpyGlass CDC solution

Rules

The `cdc_filter_path` constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_sync01	Ac_sync02	Ac_unsync01	Ac_unsync02

cdc_matrix_attributes

Purpose

The `cdc_matrix_attributes` constraint is used to set a limit for SpyGlass-CDC attributes during the SpyGlass-CDC setup stage.

To check if an attribute exceeds the specified limit, view the report generated by the *Setup_req01* rule, and fix the issue that is causing the limit to exceed. This way, you can ensure that the design statistics (in the form of attributes) are good enough to proceed with SpyGlass CDC verification.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `cdc_matrix_attributes` constraint is as follows:

```
current_design <du-name>
cdc_matrix_attributes
  -src_clock_limit <int>
  -gen_clock_limit <int>
  -sync_reset_limit <int>
  -async_reset_limit <int>
  -domain_limit <int>
  -crossing_limit <int>
  -src_per_dest_limit <int>
  -crossing_per_clock_pair_limit <int>
```

NOTE: If you specify "-1" to any of the above arguments, the *Setup_req01* rule does not perform checking for the limit corresponding to that argument. You must specify -1 in double quotes (" ").

Arguments

<du-name>

Name of the design unit.

-src_clock_limit <int>

Specifies the maximum number of source clocks.

By default, this argument is set to 200.

-gen_clock_limit <int>

Specifies the maximum number of generated clocks.

By default, this argument is set to 1000.

-sync_reset_limit <int>

Specifies the maximum number of synchronous resets.

By default, this argument is set to 500.

-async_reset_limit <int>

Specifies the maximum number of asynchronous resets.

By default, this argument is set to 500.

-domain_limit <int>

Specifies the maximum number of clock domains.

By default, this argument is set to 200.

-crossing_limit <int>

Specifies the maximum number of clock-domain crossings in a design.

By default, this argument is set to 5000.

-src_per_dest_limit <int>

Specifies the maximum number of sources that reach a particular destination.

By default, this argument is set to 50.

-crossing_per_clock_pair_limit <int>

Specifies the maximum number of crossings between each clock pair.
By default, this argument is set to 500.

Example

The following example sets a limit for various SpyGlass-CDC attributes:

```
cdc_matrix_attributes -src_clock_limit "-1"
-gen_clock_limit 2 -sync_reset_limit 0 -async_reset_limit 0
-domain_limit 1 -crossing_limit 8 -src_per_dest_limit 0
-crossing_per_clock_pair_limit 0
```

Refer to the documentation of the CDC matrix report to see the information generated in this report based on the limit set by the `cdc_matrix_attributes` constraint.

Rules

The `cdc_matrix_attributes` constraint is used by the following rules:

SpyGlass CDC Solution
Setup_req01

cell_hookup

Purpose

The `cell_hookup` constraint is used to specify the special cell hookups as used by the LPSVM44 rule of the *SpyGlass Power Verify* solution.

NOTE: Prior to *SpyGlass 4.3.0* release, the name of this constraint was `cellhookup`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `cell_hookup` constraint is as follows:

```
current_design <du-name>
  cell_hookup
    -signame <sig-name>
    [ -names <cell-pin-name-list> ]
    [ -ignorecells <ignorecell-name-list> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the special cells.

-signame <sig-name>

Name of the signal to be checked for connections.

-names <cell-pin-name-list>

Space-separated cell-pin pair name list of special cells and their pins. If you need to specify more than one pin for a cell, specify the cell-pin pair more than once as in the following example:

```
... -names cell1 p1 cell1 p2 ...
```

You can use wildcard characters while specifying the cell-pin pair names using the `-names` argument.

-ignorecells <ignorecell-name-list>

Space-separated list of cells to be skipped while traversing the fan-out of the specified signals. You can use wildcard characters while specifying cells to be ignored using the `-ignorecells` argument.

Rules

The `cell_hookup` constraint is used by the following rules:

SpyGlass Power Verify Solution	
LPSVM44	LP_SGDC_CHECKS (optional)

cell_pin_info

Purpose

Specifies pin-to-supply name pairs for multi-supply cells.

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was `set_cell_pin_info`.*

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `cell_pin_info` constraint is as follows:

```
current_design <du-name>
  cell_pin_info
    -cellname <cell-name>
    -pin_name_supply <pin-sname-pair-list>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the pin-to-supply

name pairs for multi-supply cells.

-cellname <cell-name>

Simple cell name or a regular expression.

-pin_name_supply <pin-sname-pair-list>

Space-separated pair list of pin name and its supply name as in the following example:

```
cell_pin_info -cellname "INVX2*"
  -pin_name_supply VSS VSS1 VDD VDDC IN VDDC OUT VDDC
```

Rules

The `cell_pin_info` constraint is used by the following rule:

SpyGlass Power Verify Solution
LPSVM49

cell_tie_class

Purpose

Specifies tie conditions for multi-power and multi-ground cells.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `set_cell_tie_class`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `cell_tie_class` constraint is as follows:

```
current_design <du-name>
  cell_tie_class
    -cell <cell-name> | -instance <inst-name>
```

```

-tie1 <power-pin-net-name>
-tie0 <ground-pin-net-name>
[ -no_tie <no-tie-pin-name-list> ]
[ -exception <exception-list> ]

```

Arguments

<du-name>

Name of the design unit under which you are specifying tie conditions.

-cell <cell-name>

Name of the cell for which you are specifying tie conditions.

-instance <inst-name>

Name of the cell instance for which you are specifying tie conditions.

You can use wildcard characters while specifying cell names (using the -cell argument) and instance names (using the -instance argument).

For example,

```

cell_tie_class -cell "RSHL*" -tie1 vdd -tie0 vss -no_tie A
E

```

-tie1 <power-pin-net-name>

Name of the power pin of the cell *<cell-name>* when the cell is a multi-power, multi-ground cell or name of the net connected to the power pin of cell instance *<inst-name>* when the cell instance is an instance of a multi-power, multi-ground cell. The field accepts a scalar net or bit select of the vector net.

-tie0 <ground-pin-net-name>

Name of the ground pin of the cell *<cell-name>* when the cell is a multi-power, multi-ground cell or name of the net connected to the ground pin of cell instance *<inst-name>* when the cell instance is an instance of a multi-power, multi-ground cell. The field accepts a scalar net or bit select of the vector net

-no_tie <no-tie-pin-name-list>

(Optional) Specifies a space-separated name list of pins of cell *<cell-name>* or nets connected to pins of cell instance *<inst-name>* that should not be connected to the power/ground pins/nets specified using the -tie1/-tie0 arguments.

-exception <exception-list>

(Optional) Specifies the pins of cell *<cell-name>* or nets connected to pins of cell instance *<inst-name>* that should not be connected to the power/ground pins/nets specified using the -tie1/-tie0 arguments and should be connected to the specified power/ground pins/nets.

<exception-list> is a space-separated list of the following tuples for each exception pin/net:

<pin-net-name> *<pwr-pin-net-name>* *<gnd-pin-net-name>*

For example:

-exception Z top.VSS_3 top.VSS_5

Rules

The cell_tie_class constraint is used by the following rule:

SpyGlass Power Verify Solution
LPPLIB14

check_clock_relation

Purpose

The check_clock_relation constraint is used to define the constraints for verifying clock relationships.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `check_clock_relation` constraint is as follows:

```
check_clock_relation
  -phase <inv | noninv | any>
  -from_clk <src-clock> / -from_obj <src-clock-path-point>
  -to_clk <dest-clock> / -to_obj <dest-clock-path-point>
```

For the checks that are performed on the **check_clock_relation** constraint, see the [Sanity Checks on the check_clock_relation Constraint](#) section.

Arguments

-phase <inv | noninv | any>

(Mandatory) Defines phase requirements on nets/clocks specified in the following arguments:

`-from_clk/-from_obj/-to_clk/-to_obj`

For more information, see [Defining Phase Requirements](#)

-from_clk <src-clock>

(Optional) Specifies the ports/nets/terminal defined with the clock constraint, which goes to a source object of a crossing. This argument must have a corresponding `-to_clk` or `-to_obj` argument.

NOTE: *Virtual clocks are not supported.*

-to_clk <dest-clock>

(Optional) Specifies the ports/nets/terminal defined with the clock constraint, which goes to a destination object of a crossing. This argument must have a corresponding `-from_clk` or `-from_obj` argument.

NOTE: *Virtual clocks are not supported.*

-from_obj <src-clock-path-point>

(Optional) Specifies the net/terminal/port point in path of clock reaching

to source of a crossing. This argument must have a corresponding `-to_clk` or `-to_obj` argument.

`-to_obj <dest-clock-path-point>`

(Optional) Specifies the net/terminal/port point in path of clock reaching to destination of a crossing. This argument must have a corresponding `-from_clk` or `-from_obj` argument.

Examples

This section describes the following examples:

- *Example 1: Correct Specification of the `check_clock_relation` Constraint*
- *Example 2: Incorrect Specification of the `check_clock_relation` Constraint*
- *Example 3: Multiple Clocks on Both Source and Destination Side*
- *Example 4: Using the `-from_obj/-to_obj` Arguments*

Example 1: Correct Specification of the `check_clock_relation` Constraint

This example shows the correct way to define the **`check_clock_relation`** constraint. Notice that `-from_clk/-from_obj` arguments have a corresponding `-to_clk/-to_obj`.

```
check_clock_relation -from_clk C1 -to_clk n1 -phase inv
check_clock_relation -from_clk C1 -to_obj n1 -phase inv
check_clock_relation -from_obj C1 -to_clk n1 -phase inv
check_clock_relation -from_obj C1 -to_obj n1 -phase inv
```

Example 2: Incorrect Specification of the `check_clock_relation` Constraint

This example shows some incorrect **`check_clock_relation`** constraint specifications.

A violation is reported for the following specification because the `-from_obj` argument does not have a corresponding `-to_obj` or `-to_clk` argument.

```
check_clock_relation -from_obj C1 -phase inv
```

A violation is reported for the following specification because the

SpyGlass Design Constraints

-from_clk argument does not have a corresponding -to_obj or -to_clk argument.

```
check_clock_relation -from_clk C1 -from_obj C2 -phase inv
```

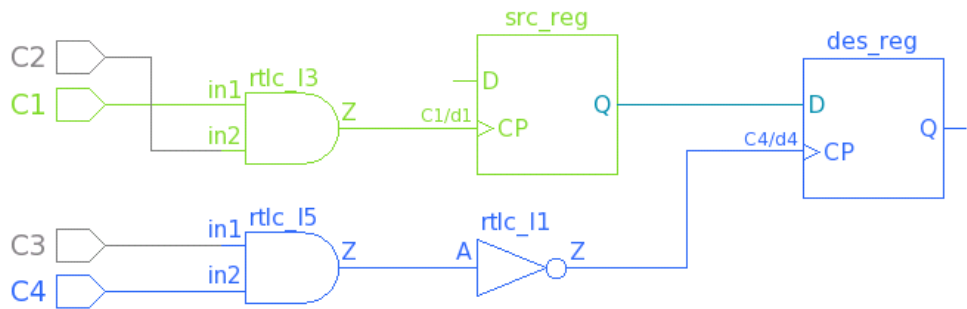
A violation is reported for the following specification because the -to_clk argument does not have a corresponding -from_obj or -from_clk argument.

```
check_clock_relation -to_obj C1 -phase noninv
```

Example 3: Multiple Clocks on Both Source and Destination Side

This example shows how to specify the **check_clock_relation** constraint for a pairwise relation of all source and destination clocks of a crossing.

Suppose, you have the following design:



If a crossing source receives two clocks, C1 and C2, and the destination receives two clocks, C3 and C4. Assuming all clocks are in the same phase, specify the **check_clock_relation** constraint as follows:

```
current_design top
```

```
clock -name C1 -domain d1
clock -name C2 -domain d2
clock -name C3 -domain d3
```

```
clock -name C4 -domain d4
```

```
check_clock_relation -from_clk C1 -to_clk C3 -phase inv
check_clock_relation -from_clk C1 -to_clk C4 -phase inv
check_clock_relation -from_clk C2 -to_clk C3 -phase inv
check_clock_relation -from_clk C2 -to_clk C4 -phase inv
```

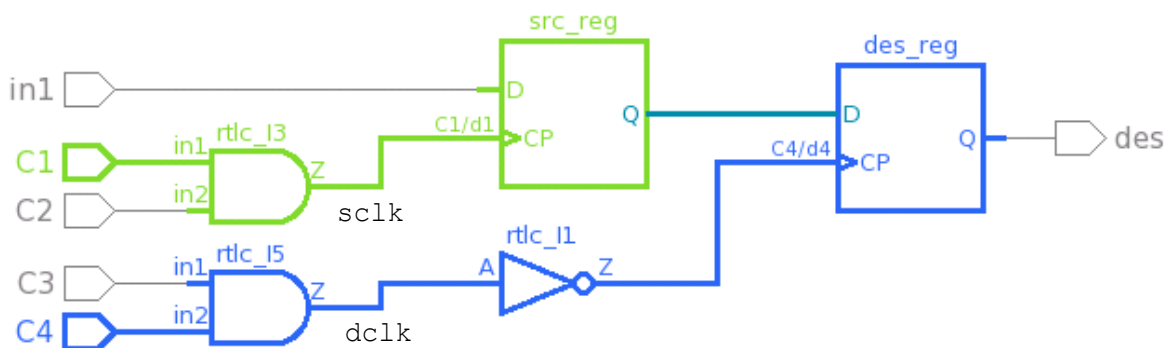
Example 4: Using the -from_obj/-to_obj Arguments

You can specify a point on the clock path between the root clock and the crossing flip-flop through the -from_obj/-to_obj arguments. However, such a point must reach directly to the end crossing point, with no other domain signal merging through the side path.

Therefore, you should specify -from_obj/-to_obj on the clock path point, which is available after the convergence of all clocks on either side of crossing.

For the following design, the **check_clock_relation** constraint specification is as follows:

```
check_clock_relation -from_obj sclk -to_obj dclk -phase inv
```



SpyGlass Design Constraints

The generated violation message specifies synchronized crossing and is as follows:

```
Ac_sync01 (1) : Checks synchronized crossing for scalar signals
Synchronized Crossing: destination flop top.des, clocked by top.C4, source flop top.src, clocked by top.C1, by method: Clock relationship satisfied [Total domains: 1]
```

The following cases describe crossings that are reported as unsynchronized:

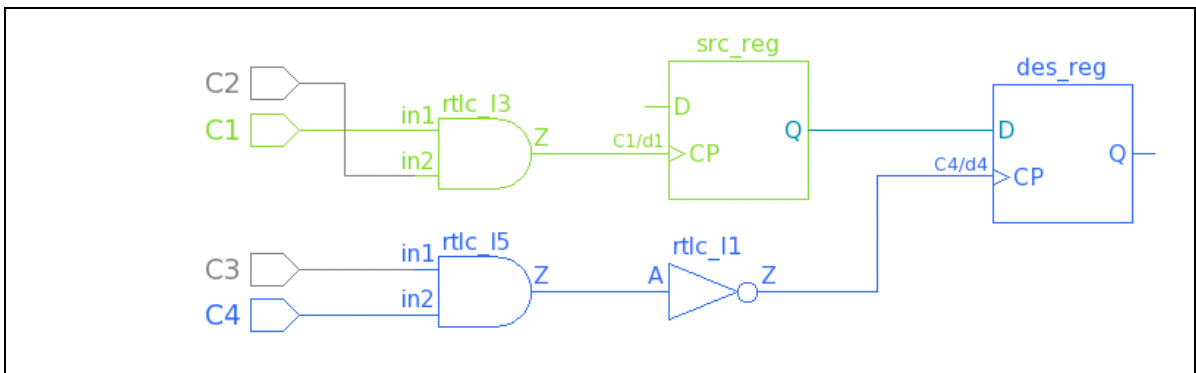
Case 1: From the point of specifying `-from_obj/-to_obj` to the `src/des` clock pins respectively, no different domain is allowed to be merged.

This means that the path from the `-from_obj` to `src` clk pin does not allow a different clock domain path to merge from side path. However, but it allows the same domain clock/path to be merged as of now. The same domain, could be due to the same net diverging-reconverging or a different clock have same domain.

Case 2: No different set of `-from_obj/-to_obj` are allowed to be merged.

This statement is true as discussed only if the domains of the `-from_obj/-to_obj` are different (from different domain). If they are from the same domain, multiple constraints can be specified and are checked in parallel, but only one of them satisfying the requirements is sufficient for synchronization.

The design is as follows:



For this design, the **check_clock_relation** constraint specification is as follows:

```
current_design top

clock -name C1 -domain d1
clock -name C2 -domain d2
clock -name C3 -domain d3
clock -name C4 -domain d4

check_clock_relation -from_obj C1 -to_obj C3 -phase inv
check_clock_relation -from_obj C1 -to_obj C4 -phase inv
check_clock_relation -from_obj C2 -to_obj C3 -phase inv
check_clock_relation -from_obj C2 -to_obj C4 -phase inv
```

The following violation is displayed:

 **Ac_unsync01 (1)** : Checks unsynchronized crossings for scalar signals
 **Unsynchronized Crossing:** destination [flop top.des](#), clocked by [top.C4](#), source [flop top.src](#), clocked by [top.C1](#). Reason: [Qualifier not found](#) [Total Source

Defining Phase Requirements

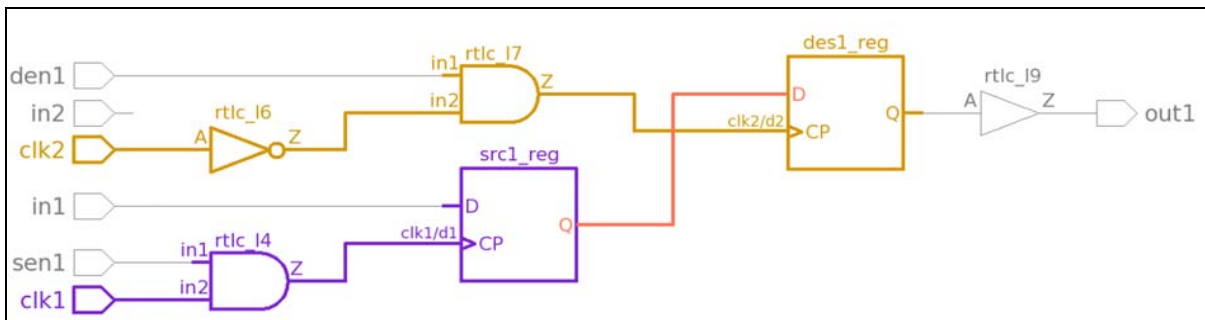
When you define the `-phase` argument, the **Ac_sync** rules compute the net phase difference of `-from_clk/-from_obj` to the source clock pin of a crossing and the net phase difference of `-to_clk/-to_obj` to the destination clock pin of crossing

You can specify either of the following values in the `-phase` argument:

- **inv**: Means that for a crossing, both "net phase from `-from_clk/-from_obj` to the source object" and "net phase from `-to_clk/-to_obj` to the destination object", should be either positive or negative, and both should be inverted to each other
- **noninv**: Means that for a crossing, both "net phase from `-from_clk/-from_obj` to the source object" and "net phase from `-to_clk/-to_obj` to the destination object", should be either positive or negative, and both should be same with reference to each other

- **any**: Means that the phase is not considered while determining synchronized or unsynchronized crossings.

In the following design, the `clk2` clock has an inverter in its path and is therefore negative. The `clk1` clock is positive. Therefore, the phase relationship is negative (inverted).



For this design, the following **check_clock_relation** constraint is defined:

```
check_clock_relation -from_clk clk1 to_clk clk2 -phase inv
```

Since the phase is specified as **inv** (inverted), the rules in the **Ac_sync** rule group report the crossing as synchronized because the phase relationship is actually negative.

Suppose, you define the **check_clock_relation** constraint as follows:

```
check_clock_relation -from_clk clk1 to_clk clk2 -phase noninv
```

Since the phase is specified as **noninv** (noninverted), the rules in the **Ac_sync** rule group report the crossing as unsynchronized because the phase relationship is actually negative. To synchronize the crossing, you would need to either:

- Invert the `clk2` clock again to change the net phase relationship to inverted
- Change the phase relationship defined in the **check_clock_relation** constraint to **inv**

Viewing the Schematics Generated by the Ac_sync Rule Group

This section shows the violation message and schematic that is generated by the *Ac_unsync01* and *Ac_sync01* rules. In this example, the following

check_clock_relation constraint is specified:

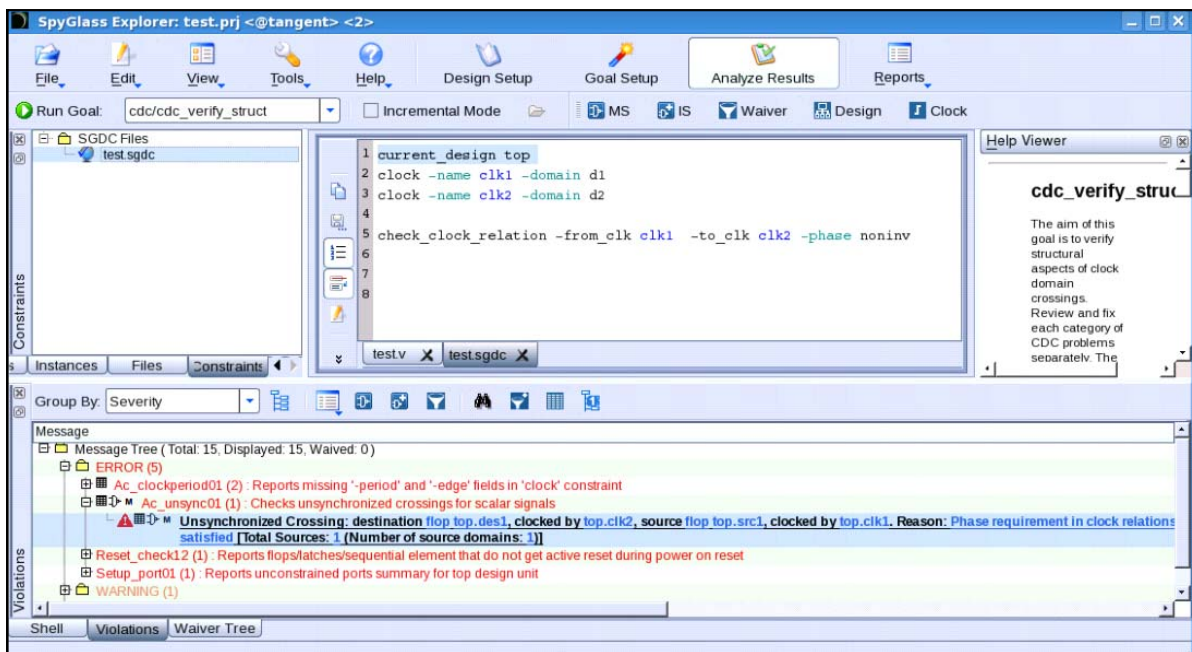
```
check_clock_relation -from_clk clk1 -to_clk clk2 -phase
noninv
```

Ac_unsync01

After you run the *Ac_unsync01* rule, the following violation is reported.

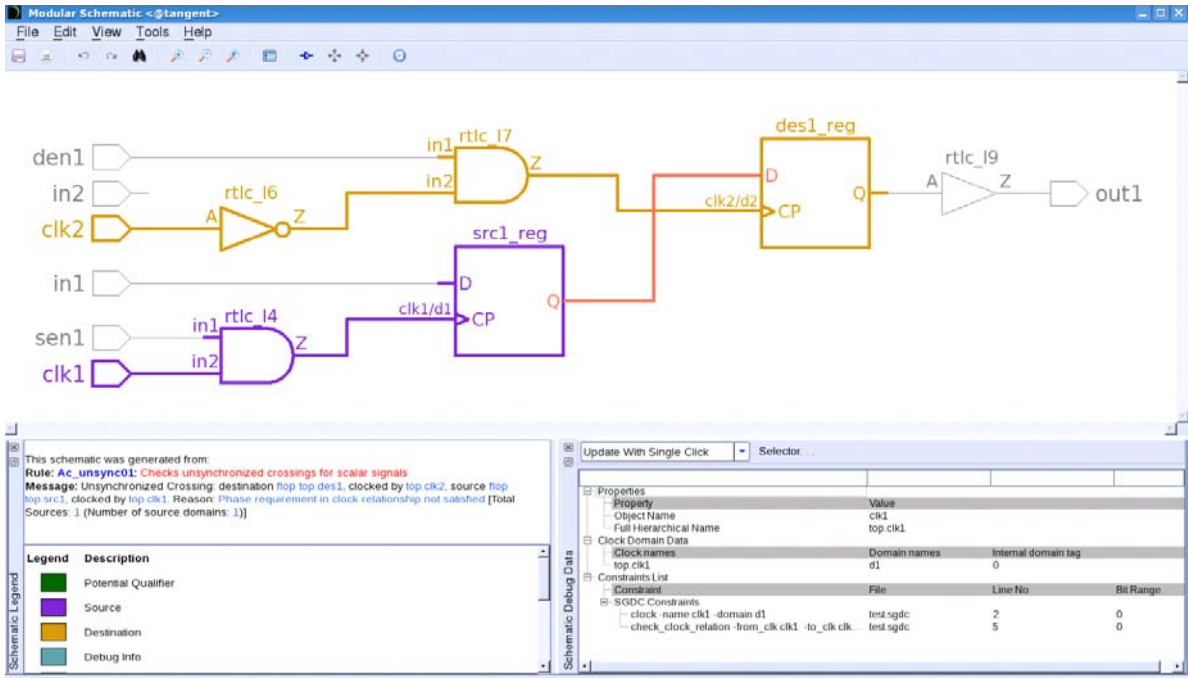
unsynchronized Crossing: destination flop top.des1, clocked by top.clk2, source flop top.src1, clocked by top.clk1. Reason: Phase requirement in clock relationship not satisfied [Total Sources: 1 (Number of source domains: 1)]

In the violation message, the reason for the unsynchronized crossing is stated.



The following schematic is generated. In this schematic, the unsynchronized crossing is highlighted between src1_reg and dest1_reg.

SpyGlass Design Constraints

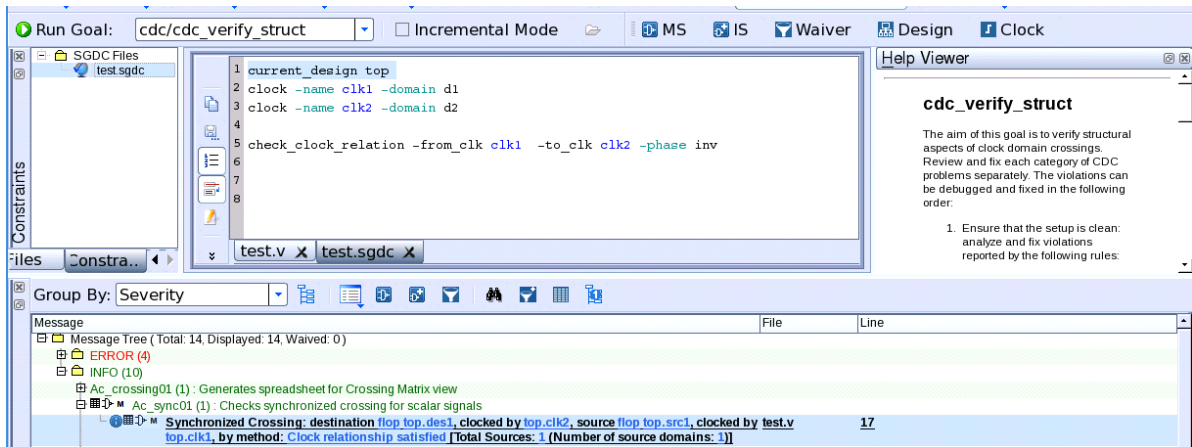


Ac_sync01

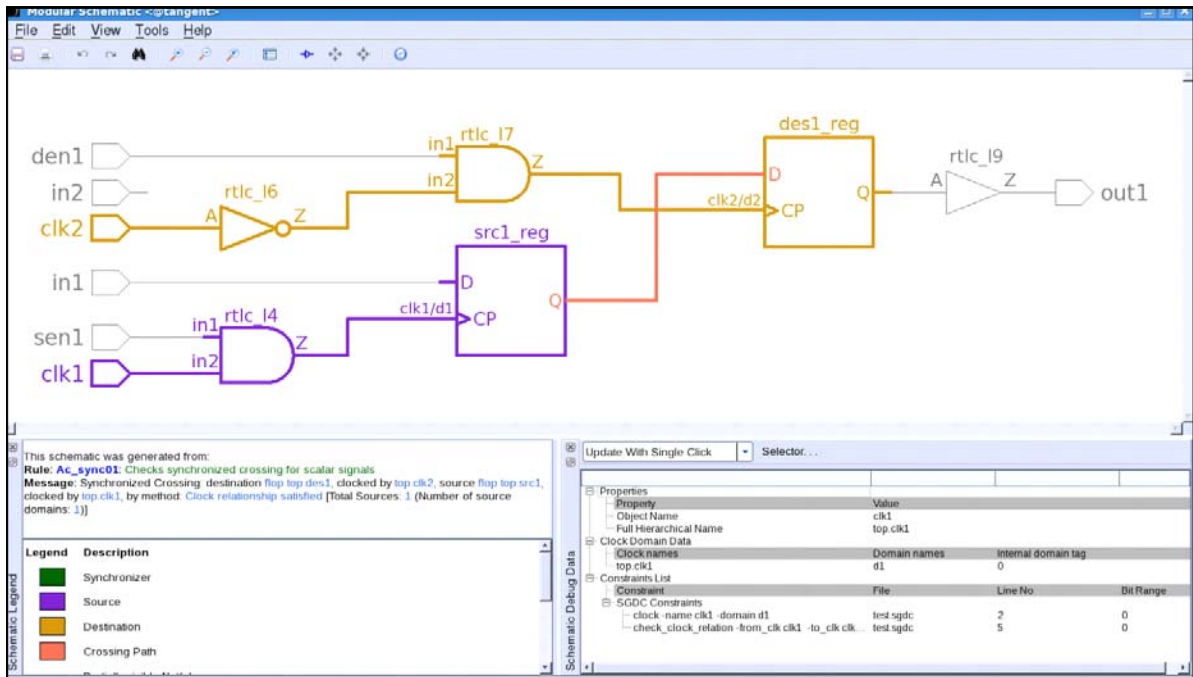
After you run the `Ac_sync01` rule, the following violation is reported:

Synchronized Crossing: destination flop top.des1, clocked by top.clk2, source flop top.src1, clocked by top.clk1, by method: clock relationship satisfied [Total Sources: 1 (Number of source domains: 1)]

In the violation message, the reason for the synchronized crossing is stated.



The following schematic is generated. In this schematic, the synchronized crossing is highlighted between src1_reg and dest1_reg.



Sanity Checks on the `check_clock_relation` Constraint

Several rules perform sanity checks on the `check_clock_relation` constraint.

The following message appears when the `check_clock_relation` constraint has not been used to synchronize any crossing in the current design:

Sanity Rule Name, Message, and Severity	How to Debug and Fix
[CheckClockRelationSetup][WARNING] 'check_clock_relation' constraint is not used to synchronize any crossing in the current design '<current-design>'	If you want to synchronize crossings by using the check_clock_relation constraint, specify the check_clock_relation constraint in the SGDC file.

The following table shows the list of rules and the rule severity that perform sanity checks on the `-from_clk` argument.

Sanity Rule Name, Message, and Severity	How to Debug and Fix
[SGDC_check_clock_relation01a][ERROR] Constraint 'check_clock_relation': clock name '<clock-name>' specified in field '-from_clk' does not exist either as a net, port, hierarchical terminal or as a virtual clock	Specify a valid clock name in the <code>-from_clk</code> argument.
[SGDC_check_clock_relation01b][ERROR] Constraint 'check_clock_relation': clock name '<clock-name>' specified in field '-from_clk' is found but it is not a valid clock	Specify a valid clock name in the <code>-from_clk</code> argument.
[SGDC_check_clock_relation01c][WARNING] Constraint 'check_clock_relation': clock '<clock-name>' specified with '-from_clk' within module '<current-design>' is a virtual clock	Virtual clocks are not supported. Specify a real clock name in the <code>-from_clk</code> argument.

The following table shows the list of rules and the rule severity that perform sanity checks on the `-to_clk` argument.

Sanity Rule Message and Severity	How to Debug and Fix
[SGDC_check_clock_relation02a][ERROR] Constraint 'check_clock_relation': clock name '<clock-name>' specified in field '-to_clk' does not exist either as a net, port, hierarchical terminal or as a virtual clock	Specify a valid clock name in the <code>-to_clk</code> argument.
[SGDC_check_clock_relation02b][ERROR] Constraint 'check_clock_relation': clock name '<clock-name>' specified in field '-to_clk' is found but it is not a valid clock	Specify a valid clock name in the <code>-to_clk</code> argument.
[SGDC_check_clock_relation02c][WARNING] Constraint 'check_clock_relation': clock '<clock-name>' specified with '-to_clk' within module '<current-design>' is a virtual clock	Virtual clocks are not supported. Specify a real clock name in the <code>-to_clk</code> argument.

The following table shows the list of rules and the rule severity that perform sanity checks on the `-from_obj` argument.

Sanity Rule Message and Severity	How to Debug and Fix
[SGDC_check_clock_relation03a][FATAL] '<signal-name>'[TopPort + Net + HierTerminal] not found on/within module '<design-name>'	To fix this violation, update the value of the <code>-from_obj</code> argument to specify a name that exists as a port, net, or hierarchical pin in the design.
[SGDC_check_clock_relation03b][ERROR] Constraint 'check_clock_relation': Point name '<source-clock-path-point>' specified in field '-from_obj' is not a valid clock path point	Check if the object specified in the <code>-from_obj</code> argument is on the data path. Specify an object in the clock path.

The following table shows the list of rules and the rule severity that

perform sanity checks on the `-to_obj` argument.

Sanity Rule Message and Severity	How to Debug and Fix
[SGDC_check_clock_relation04a][FATAL] '<signal-name>'[TopPort + Net + HierTerminal] not found on/within module '<design-name>'	To fix this violation, update the value of the <code>-to_obj</code> argument to specify a name that exists as a port, net, or hierarchical pin in the design.
[SGDC_check_clock_relation04b][ERROR] Constraint 'check_clock_relation': Point name '<destination-clock-path-point>' specified in field '-to_obj' is not a valid clock path point	Check if the object specified in the <code>-to_obj</code> argument is on the data path. Specify an object in the clock path.

Rules

The `check_clock_relation` constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_unsync01	Ac_sync01	Ac_unsync02	Ac_sync02

clock

NOTE: *The clock SGDC command is not supported in Tcl shell. Instead, you can use the `create_clock`, `create_clock_attribute`, and `create_generated_clock` commands to specify clock and its attributes in Tcl shell.*

Purpose

The `clock` constraint is used to define clocks of a design.

The definition of design clocks is of high importance for proper functionality of associated products. Wrong assumptions on clock periods/edges may cause undetected bugs or false rule violations.

Product

SpyGlass Auto Verify solution, SpyGlass Constraints solution, SpyGlass DFT solution, SpyGlass DFT DSM solution, SpyGlass Power Estimation and SpyGlass Power Reduction solutions, SpyGlass ERC product, SpyGlass Power Verify solution, and SpyGlass CDC solution.

For SpyGlass CDC solution, SpyGlass Constraints solution, and SpyGlass Auto Verify solution

Syntax

The syntax of the `clock` constraint is as follows:

```
current_design <du-name>
  clock
    -name <clk-name> | -tag <logical-clock-name>
    [ -period <period> ]
    [ -edge <edge-list> ]
    [ -domain <domain-name> ]
    [ -add ]
```

Arguments

<du-name>

Specifies any of the following:

- Module name for Verilog designs
- Design unit name in the `<entity-name>.<arch-name>` format for VHDL designs

-name <clk-name>

The clock port/pin name.

You can specify a single port/pin name or a space-separated list of port/pin names.

For top-level port/pin, `<clk-name>` can be the port's/pin's full hierarchical name or its simple name. For other pins, `<clk-name>` must be the pin's full hierarchical name.

If the clock name is an escape name, it should be enclosed in `q%` and `%`, as shown in the following example:

```
clock -name q%top.\ipcie/txbclk[0]%
```

NOTE: *If you define multiple clocks on multiple terminals that are derived by the same net, such clocks are considered as different clocks by SpyGlass CDC solution.*

-tag <logical-clock-name>

(Optional) Logical name of the clock.

Based on the specified logical name (tag), the spreadsheet of the following rules displays the source and destination clock tag names:

Ac_glitch01	Ac_glitch02	Ac_sync02	Ac_sync01
Ac_unsync02	Ac_unsync01	Ac_cdc01a	Ac_cdc01b
Ac_cdc01c			

SpyGlass reports a violation if multiple clocks have the same tag name but different characteristics, such as domain.

If you do not specify the `-name` argument of the `clock` constraint, the name specified by the `-tag` argument is considered as the virtual clock name.

-period <period>

(Optional) Clock period (in nanoseconds) to be used in clock domain crossing checks.

You can specify the clock period as any positive floating-point number. SpyGlass Auto Verify solution rounds off the specified number to the nearest multiple of 0.5.

If you do not specify the clock period, SpyGlass Auto Verify solution assumes the clock period to be 10.

-edge <edge-list>

(Optional) Clock-edge value list.

By default, clock edges are assumed 0, 50, 100, and so on.

The following command describes a 100MHz clock with posedge at zero ns and negedge at five ns.

```
clock -name top.clk -period 10 -edge {0 5}
```

-domain <domain-name>

(Optional) Clock domain name.

The domain name can be any valid string not containing whitespace characters.

If the domain name is not specified, the clock domain name is the same as the clock name.

NOTE: *The -domain argument of the clock constraint is ignored by the SpyGlass Constraints solution.*

You must provide all clock sources of a design along with their attributes (period and edges). The associated products first analyze the clocks of a design prior to any functional analysis. At least one source clock is expected for any register clock pin. If no source clock could be identified for a register, an error message is issued and no functional analysis is performed.

The clock definitions are reported through the `Av_clkinf01` rule of the SpyGlass Auto Verify solution or the `Propagate_Clocks` rule of the SpyGlass CDC solution. You can visualize and explore the clocks definitions through schematic highlight and RTL back-annotations. You can determine the missing clocks and provide their definition and re-run the analysis again.

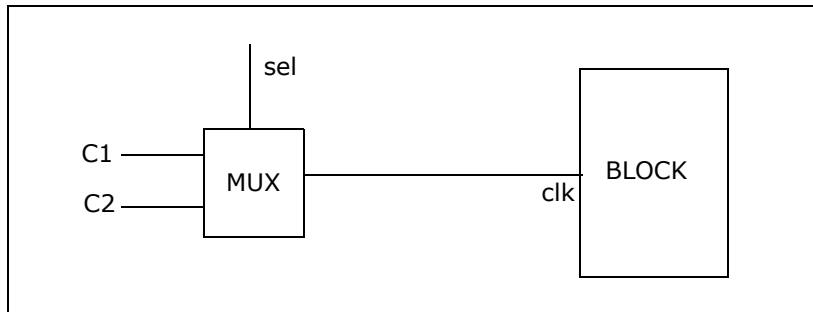
If you are unsure of the clock domains in your design, use the SpyGlass CDC solution to identify clock sources of a design.

The associated products support multiple asynchronous clocks and gated clocks. The source clocks are supposed to be linked to the register clock pin through combinational or sequential gates. SpyGlass Auto Verify solution analyzes the clock circuitry while analyzing the functionality of a design. In particular, clock dividers are participating in functional analysis.

-add

(Optional) Use this argument to specify multiple clocks on the same object. You can specify multiple clock constraints on the same object (specified by the *-name <clk-name>* argument) only if you specify the `-add` argument. This argument overrides the unique check on the *-name <clk-name>* argument.

Consider the scenario shown in the following figure:

**FIGURE 25.**

In the above scenario, the block pin `clk` can have any of the clock, `C1` or `C2`, coming through the MUX depending upon the MUX select line. This means that this block can have two different values for the `clk` clock. Therefore, it is necessary to define multiple clocks on the same object in the above case.

In such cases, use the `-add` argument in the next `clock` constraint specification on the same object, as shown in the following example:

```

clock -name top.clk -tag C1 -domain A -period 10.0
clock -name top.clk -tag C2 -domain B -period 10.0 -add
  
```

In the above example, the `-name` argument is the unique key of the `clock` constraint. That is, only one clock can be applied on a single object (in this case, `clk`). To add another clock with the same `-name` argument, the `-add` argument is used.

Primary Clocks

A primary clock is a source clock defined using the `clock` constraint. A primary clock may be directly connected to a clock pin of a register or it may pass through combinational and/or sequential logic before reaching the clock pin of a register. You should define the following potential primary clock sources:

- Primary inputs of a design

■ Black box outputs (for example, PLL outputs)

In addition, you can define an intermediate signal of a design as a primary clock. For instance, if a MUX is selecting between two clock sources running at 33MHz and 66MHz, and if you want to check the functionality of a design for 66MHz, the output of the MUX can be defined as a primary clock with 66MHz frequency. Alternatively, the 66MHz source clock can be defined as a clock and a constraint can be set for the MUX select pin to select the 66MHz clock only.

Derived Clocks

A derived clock is a clock that is generated internally from a primary clock. The primary clock could pass through a combinational logic or sequential logic or both to generate a derived clock. Assume a clock port traversing a clock divider, the clock port should be defined as a primary clock and the output of clock divider feeding some registers' clock is a derived clock (assuming you have not defined it as a primary clock).

The concept of derived clocks is important for some specific rules where the relative frequencies of the derived clocks are used to determine if the crossing is from a fast clock domain to a slow clock domain. For structural analysis (needed by some rules), the frequency seen by the derived clock is the maximum frequency of all source clocks in the fan-in cone of the derived clock. If this assumption is not appropriate, you can define a primary clock for the internal signal that is directly controlling the register.

The SpyGlass Auto Verify solution and SpyGlass CDC solution support complex gated clocks such that they can analyze the functionality of a design in the presence of such complex gated clocks. However, these products do not attempt to determine the exact phase and frequency of such clocks and do not report them. For instance, if these products carry out functional analysis on the design illustrated in the figure shown below. However, the clock report (the Av_clkinf01 rule of the SpyGlass Auto Verify solution or the Propagate_Clocks rule of the SpyGlass CDC solution) will only report the clocks defined by you that are ck1, ck2, and ck3.

Default clock

If the clock definitions of primary clocks provided by you are missing the period or edge information, default clock attributes will be assigned to them.

A default clock has a period of 10ns (100 MHz) with a rising edge at 0 and a falling edge at 5ns. While the default clock can be used with a design with

a single clock feeding all registers with a single phase, it may lead to erroneous failure or success of the rules of the associated products for gated clock or multi-clock designs.

Clock Definition Impact on Functional Analysis

The associated products determine the source clocks for all registers in a design. The functional analysis is carried out by evaluating each register at the active edge of its controlling clocks.

For example, in the figure shown below, clocks `ck1`, `ck2`, and `ck3` are defined as clocks using the `clock` constraint. Register `R1` is seeing `ck1` and `ck2` as clocks, while register `R2` is seeing `ck1`, `ck2`, and `ck3` as clocks. Therefore, register `R1` is evaluated at the active edges of `ck1` as well as the active edges of `ck2`, while register `R2` is evaluated at the active edges of all three clocks — `ck1`, `ck2`, and `ck3`.

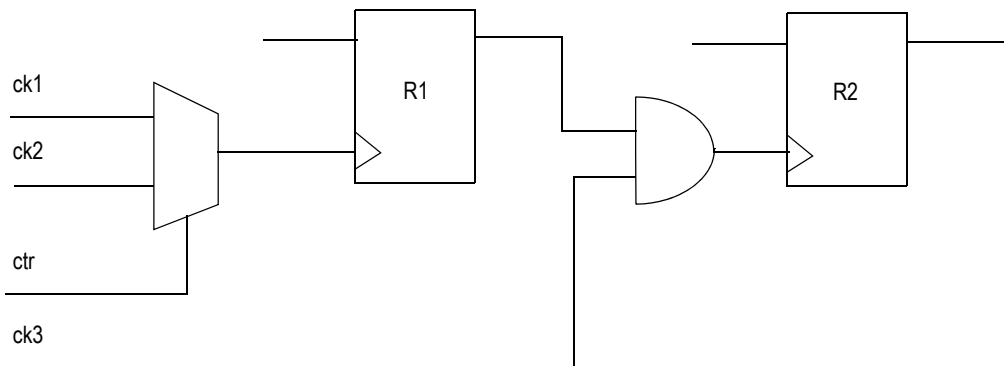


FIGURE 26. Clock Definition Impact

Bus Support

SpyGlass CDC honors all bits of a bus in the clock SGDC constraint. The following example shows the violations reported by the `Propagate_Clocks` rule when the `clock` SGDC constraint is specified on a bus. The Verilog design file and corresponding SGDC specification is shown as follows.

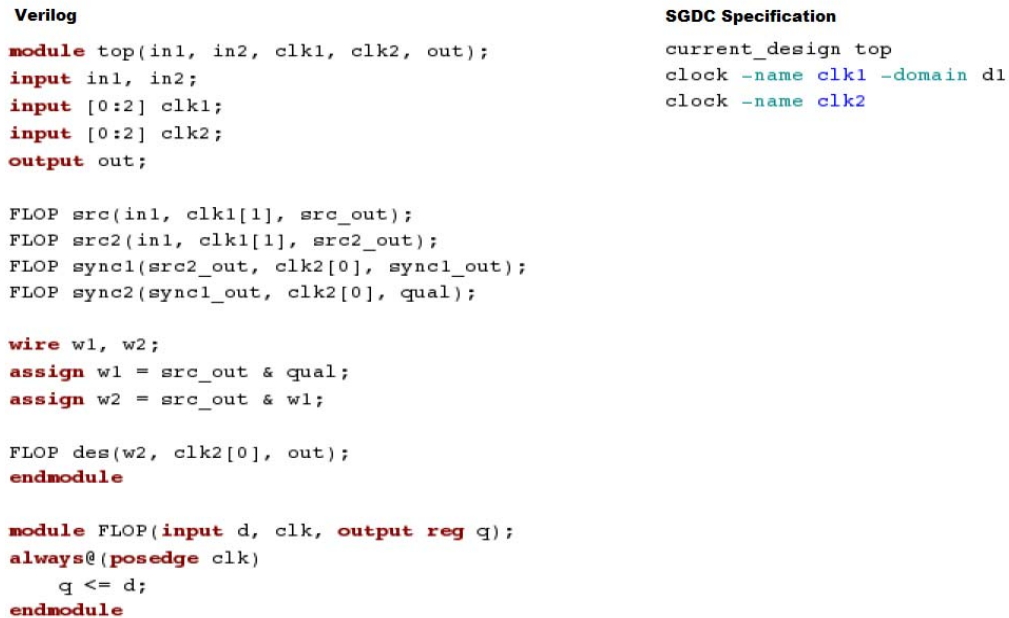


FIGURE 27.

The Propagate_Clocks rule reports the following violations. Notice that all bits of `clk1` and `clk2` are reported.

Propagate_Clocks (5) : Propagates clocks and displays a portion of the clock-tree			
For top, clock(s) 'top.clk1[1]' of domain 'd1' propagated	top.v	3	
For top, clock(s) 'top.clk2[0]' of domain 'top.clk2[0]' propagated	top.v	4	
For top, clock(s) 'top.clk1[2],top.clk1[0]' of domain 'd1' not propagated	top.sgdc	2	
For top, clock(s) 'top.clk2[1]' of domain 'top.clk2[1]' not propagated	top.sgdc	3	
For top, clock(s) 'top.clk2[2]' of domain 'top.clk2[2]' not propagated	top.sgdc	3	

Rules

The clock constraint is used by the following rules:

SpyGlass Design Constraints

SpyGlass CDC Solution			
Clock_check01	Clock_check02	Clock_check03	Clock_check04
Clock_check05	Clock_check06a	Clock_check06b	Clock_check07
Clock_converge01	Clock_delay01	Clock_delay02	Clock_glitch01
Clock_glitch02	Clock_glitch03	Clock_info02	Clock_info03a
Clock_info03b	Clock_info03c	Clock_info05	Clock_info05a
Clock_info06	Clock_info07	Clock_info14	Clock_info15
Clock_info16	Clock_Reset_check01	Clock_Reset_Info01	Clock_sync03a
Clock_sync03b	Clock_sync05	Clock_sync06	Clock_sync08
Clock_sync08a	Clock_sync09	Propagate_clocks	Ar_resetcross01
Reset_sync01	Reset_sync02	Reset_sync03	Reset_sync04
Ac_cdc01a	Ac_cdc01b	Ac_cdc01c	Ac_cdc08
Ac_fifo01	Ac_conv01	Ac_conv02	Ac_conv03
Ac_sync01	Ac_sync02	Ac_unsync02	Setup_quasi_static01
Ar_resetcross_matrix01	Ac_conv04	Ac_conv05	Ac_handshake01
Ac_handshake02			
SpyGlass Auto Verify Solution			
All rules			
SpyGlass Constraints Solution			
SDC_GenerateIncr			

For SpyGlass DFT solution, SpyGlass DFT DSM solution

Purpose

The `clock` constraint declares the clock pins used as test clocks.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

```

clock
    -name <pin-name> ]
    [ -sysclock | -testclock ]
    [ -atspeed ]
    [ -value <value> ]
    [ -fflimit <limit> ]
    [ -fflimit_percentage <percentage> ]
    [ -domain <dom-name> ]
    [ -pll_reference ]
    [ -frequency <frequency> ]
    [ -period ]

```

Arguments

-name <pin-name>

Complete hierarchical name of a clock port/pin.

The pin can be a primary pin as well as an internal pin.

You can specify a single port/pin's full hierarchical name or a space-separated list of full hierarchical port/pin names.

For primary ports, you can also specify the simple port name, as in the following example:

```

current_design top
    clock -name in15 ...

```

When you specify this argument with the *clock* constraint, all the clocks specified irrespective of the presence/absence of the *-testclock* and *-atspeed* modifiers, are considered as functional clocks.

-sysclock

Specifies that the clock is a system clock.

NOTE: *By default, the clock pin specified with the `clock` constraint is considered as a system clock. Therefore, you need not use the `-sysclock` argument unless required for clarity.*

-testclock

Specifies that the clock is a test clock.

When you specify this argument with the *clock* constraint, all the clocks specified irrespective of the presence/absence of the *-atspeed* modifier are considered as slow clocks. Such clocks are used during scan shift and stuck-at capture.

NOTE: *You should use this option for all the rules of the SpyGlass DFT DSM solution. Otherwise, the clock pin specified with the `clock` constraint is considered as a system clock. Therefore, the coverage reported is very low.*

-atspeed

Specifies that the clock operates at the system frequency.

When you specify this argument with the *clock* constraint, all the clocks specified irrespective of the presence/absence of the *-testclock* modifier are considered as *atspeed* clocks. Such clocks are used during *atspeed* test.

NOTE: *This option is not applicable for the SpyGlass DFT solution. Use this option for all the DSM rules. Otherwise, the coverage reported will be very low.*

-value <value>

Determines the edge (first or second) on which the capture will occur.

You can specify the value as *rto* or *rtz*. If the value is not specified, *rtz* is assumed.

For information on *rtz* and *rto* clocks, refer to *Test clocks* section in *SpyGlass DFT Rules Reference Guide*.

-fflimit <limit>

Specifies the maximum number of flip-flops that can be driven by one test clock, as checked by the *Clock_25* rule of the *SpyGlass DFT* solution.

NOTE: *This option is not applicable for the SpyGlass DFT DSM solution.*

-fflimit_percentage <percentage>

This signifies the upper limit on the percentage of total flip-flops driven by a given test clock. If the specified limit is crossed, the *Clock_25* rule generates a violation message.

NOTE: *This option is not applicable for the SpyGlass DFT DSM solution.*

-domain <dom-name>

Specifies the domain name. This option provides a mechanism to merge two or more clocks into the same domain.

NOTE: *This option is not applicable for the SpyGlass DFT solution. This option is only applicable for the at-speed clocks. All the at-speed clocks not merged in the same domain are considered as asynchronous.*

-pll_reference

Specifies that the clock should be used only as a pll reference clock and should not be used for any other purpose.

NOTE: *Ensure that the -pll_reference modifier is specified for the PLL_03 rule to run.*

-frequency <frequency>

Specifies frequencies (in MHz) or a logical frequency name (as a string) associated with a test clock.

-period

Specifies period, in nanoseconds, associated with a test clock.

Notes

- If the same pin is used as a system clock and a test clock, use two clock constraints — one with `-sysclock` or no argument and the other with the `-testclock` argument.
- If a clock constraint is a proper subset of another clock constraint, then it is recommended to ignore the sub-set clock constraint.

Consider the following example:

```
clock -name clk -testclock
clock -name clk -testclock -atspeed // ignore above
```

In the example above, you can ignore the first clock definition as it a sub-set of the second one.

Using multiple definitions for the same clock constraint, generates multiple messages for the same clock.

Example

For SpyGlass DFT DSM solution:

Example 1

If the following constraints are used as shown in the diagram in the illustration shown below, all flip-flops in D1 and in D2 will be considered as the same at-speed domain.

```
clock -name Ca -testclock -atspeed -domain CC
clock -name Cb -testclock -atspeed -domain CC
```

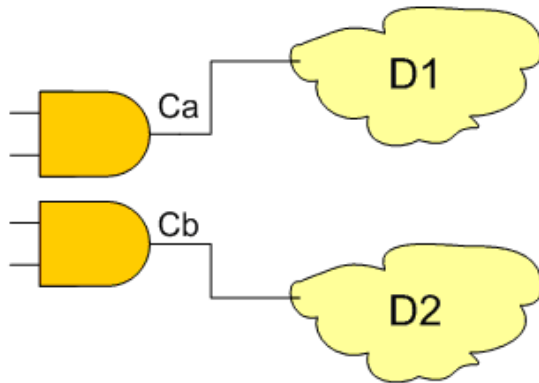


FIGURE 28. Flip-Flops in same at-speed domain

Example 2

Consider the following example:

```
clock -name clk1 -atspeed -testclock -period 10
```

In the above example, the clock, `clk1`, is assumed to be running with a 10 nanosecond period in at-speed mode.

Example 3

The following table lists some examples on various definitions of the clock constraint.

Example	Description
<code>clock -name</code>	Used as a functional clock
<code>clock -name clk</code> <code>-testclock</code>	Used as a functional and a slow clock
<code>clock -name clk -atspeed</code>	Used as a functional and an atspeed clock
<code>clock -name clk</code> <code>-testclock -atspeed</code>	Used as a functional, slow, and an atspeed clock

Rules

The `clock` constraint is used by the following rules.

SpyGlass DFT DSM Solution			
Atspeed_01	Atspeed_02	Atspeed_03	Atspeed_04
Atspeed_05	Atspeed_06	Atspeed_07	Atspeed_08
Atspeed_09	Atspeed_11	Diagnose_01	Diagnose_03
Diagnose_04	Info_transitionCoverage	Info_transitionCoverageAudit	Info_atSpeedClock
PLL_01	PLL_02	Atspeed_12	Atspeed_13
Atspeed_14	PLL_03	Atspeed_26	Atspeed_27
Info_enabledFlops			
SpyGlass DFT Solution			
clock_01	clock_02	clock_03	clock_04
clock_05	clock_06	clock_08	clock_14
clock_18	clock_21	clock_26	Info_blackboxDriver
Latch_04	Topology_11		

For SpyGlass Power Verify solution, SpyGlass ERC Product, and SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Purpose

The `clock` constraint declares the clocks used in the design.

Product

SpyGlass Power Verify solution, SpyGlass Power Estimation and SpyGlass Power Reduction solutions, and SpyGlass ERC product

Syntax

The syntax to specify the `clock` constraint is as follows:

```
current_design <du-name>
  clock
    -name <clk-name>
    [ -period <period> ]
```

NOTE: *The SpyGlass ERC product uses only the `-name` argument of the `clock` constraint to specify the names of the clock pin, ports, or nets. The `-period` argument is ignored by the SpyGlass ERC product.*

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-name <clk-name>

Name of the clock port, pin, or net.

-period <period>

The clock period (in nanoseconds).

You can specify multiple `clock` constraints.

Rules

The `clock` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
poweraudit	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	PESTR03	PESTR05
PESTR06	PESTR07	PESTR08	PESTR09
PESTR10	PESTR11	PESTR12	PESTR13
SpyGlass ERC Product			
clockPinsConnectedToClockNets			
SpyGlass Power Verify Solution			
LPSVM42	LPSVM43		

NOTE: The information provided by the `clock` constraint overrides the simulation information provided by the [activity_data](#) constraint.

clock_buffer

Purpose

Specifies a buffer library cell that should be used in clock lines while estimating a clock tree.

If you do not specify any buffer using the `clock_buffer` constraint, the related rules heuristically infer the best buffer cell to be used.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `clock_buffer` constraint is as follows:

```
current_design <top-du-name>
  clock_buffer
    -cellname <cell-name>
```

```
[ -libname <lib-name> ]
[ -maxfanout <num> ]
```

Arguments

<top-du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

-cellname <cell-name>

Name of the buffer cell of library <lib-name>.

NOTE: *The -libname argument is optional.*

Specify a library name using the optional -libname argument only when a cell with the same name is available in more than one library.

NOTE: *If you do not use the -libname argument, the library that contains the specified cell name will be taken.*

-maxfanout <num>

<num> is the maximum allowed fan-out for the specified buffer cell. Then, SpyGlass creates cascade of sets of buffer cell instances, each set having the specified number of buffer cell instances, to create the required buffering. If you do not specify the -maxfanout argument, SpyGlass creates the required buffering based on the max_capacitance attribute value for the buffer cell.

Example

Consider the following example, the clock_buffer constraint is specified for a net having a fan-out of 500:

```
clock_buffer -cellname BUFX4 -maxfanout 16,
```

Then SpyGlass will assume $500/16 = 31.25 \sim 32$ (rounded off) buffers at the first stage.

At the second stage, it will assume $32/16 = 2$ buffers.

At the third stage, it will assume one root buffer. Therefore, a total of $32 + 2 + 1 = 35$ buffers will be assumed.

Rules

The `clock_buffer` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	poweraudit	

clock_group

Purpose

The `clock_group` constraint is used to specify the clock relationship (domain, synchronous/asynchronous, exclusive). In this document, the domain term is used to denote the clock relationship. The wildcard support is provided for the `clock_group` constraint.

NOTE: The *domain* constraint has been renamed to `clock_group`. For backward compatibility, the *domain* constraint is currently still available.

Product

SpyGlass Constraints solution

Syntax

The `clock_group` constraint is used in the following syntax:

```
current_design <du-name>
  clock_group -name <clock_group-name>
  [ -clock_pin <clk-pin-name-list> ]
  [ -clock <SDC-clk-name-list> ]
```

Arguments

<du-name>

The top-level module name (for Verilog designs) or the top-level entity name (for VHDL designs) or a synthesis partition name specified using the `block` keyword.

-name <clock_group-name>

(Mandatory) Specifies a clock group name. It can be any valid string.

-clock_pin <clk-pin-name-list>

Specifies space-separated list of design clock pin names.

If this argument is used, all clocks specified on the clock pin are considered synchronous.

-clock <SDC-clk-name-list>

Specifies a space-separated list of SDC clock names.

If both options are specified, then the `clock_group` will include all the clocks specified with `-clock` option and all the clocks defined on the clock pin specified with the `-clock_pin` option.

Example

Let the clock group information be provided as follows:

```
current_design top
  clock_group -name d1 -clock { C1 C2 }
  clock_group -name d2 -clock { C3 }
```

In the above example, clocks C1 and C2 are synchronous, and clocks C1 and C3, C2 and C3 are asynchronous. You cannot specify the same clock in two different `clock_group` constraints. If you do so then the DomainSanityCheck rule reports a FATAL message.

If the same `clock_group` is specified in multiple lines of an SGDC file, the union of clocks (specified in multiple lines) will be in the same `clock_group`. For example:

```
clock_group -name d1 -clock {C1 C2}
```

```
clock_group -name d1 -clock {C4}
```

Here, clocks C1, C2, C4 will be assumed as synchronous.

All create clocks and their derived clocks will be assumed to be synchronous if you have specified the `clock_group` constraint among create clocks and their derived clocks. You can even specify clock and its derived clocks in a different `clock_group`. For example:

C1 -> GC1 -> GC2 where GC2 is derived from GC1 and so on.

```
// C1, GC1, GC2 will be assumed to be synchronous unless the
// user overrides it
```

```
clock_group -name d1 -clock C1
```

```
// User specified clock GC2 with a different group name
clock_group -name d2 -clock GC2
```

The `domainsanitycheck` rule reports a violation if you specify create clock and its derived clock in different clock groups.

NOTE: All SGDC-dependent rules will be impacted.

Rules

The `clock_group` constraint is used by the following rules:

SpyGlass Constraints Solution			
Clk_Gen05	Clk_Lat03	Clk_Uncert03	False_Path07
False_Path08	Domain_SGDC_Consis		

clock_path_wrapper_module

Purpose

The `clock_path_wrapper_module` constraint is used to exclude modules from the checks performed by the *Clock_hier01*, *Clock_hier02*, and *Clock_hier03* rules.

Product

SpyGlass CDC solution

Syntax

The syntax of the `clock_path_wrapper_module` constraint is as follows:

```
current_design <du-name>  
  clock_path_wrapper_module -names <module-list>
```

Arguments

The `clock_path_wrapper_module` constraint has the following arguments:

<du-name>

The name of the design unit from which you want to exclude modules from the checks performed by the *Clock_hier01*, *Clock_hier02*, and *Clock_hier03* rules.

-names <module-list>

Space separated list of module names.

Example

If you want to exclude specific modules, specify the module names in a space separated list, as shown in the following:

```
clock_path_wrapper_module -names MOD1 MOD2
```

In this example, the checks performed by the *Clock_hier01*, *Clock_hier02*,

and *Clock_hier03* will exclude MOD1 and MOD2.

Rules

The `clock_path_wrapper_module` constraint is used by the following rules:

SpyGlass CDC Solution		
Clock_hier01	Clock_hier02	Clock_hier03

clock_pin

Purpose

The `clock_pin` constraint is used to specify black box pins that should be assumed clock pins.

Then the `Clock_11` rule, which uses clock source analysis, treats a pin specified with the `clock_pin` constraint just as it does clock pins on flip-flops. The source of such a pin must be a test clock controlled just as any other clock source.

This is particularly useful to ensure that the clock logic connected to a black box is designed for SpyGlass DFT solution even before the box itself is available.

Product

SpyGlass DFT solution

Syntax

The syntax of the `clock_pin` constraint is as follows:

```
clock_pin
  -name <du-name>.<port-name>
  [ -value <value> ]
```

Arguments

The `clock_pin` constraint has the following arguments:

<du-name>

The name of the design unit (black box) for which you are specifying the clock pin.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name `<du-name>` can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are processed.

You can specify a single design unit name or a space-separated list of

design unit names.

<port-name>

Name of the clock port on the design unit (black box).
You can specify only a single port name.

-value <value>

(Optional) The active value for this clock port *<port-name>*. This argument can take one of the following values: *rtz* or *rto*
For information on *rtz* and *rto* clocks, refer to the *Test clocks* section in *SpyGlass DFT Rules Reference Guide*.

Rules

The `clock_pin` constraint is used by the following rules:

SpyGlass DFT Solution	
Clock_11	Clock_11_capture

clock_root

Purpose

The `clock_root` constraint is used to specify the starting node of a clock tree for checking whether clock tree is balanced.

NOTE: Prior to *SpyGlass 4.3.0* release, the name of this constraint was `clockgen`.

Product

SpyGlass DFT DSM solution

Syntax

```
clock_root
  -name <1st_signal_names>
```

Arguments

The `clock_root` constraint has the following arguments:

-name <lst_signal_names>
 List of signal names

Rules

The `clock_root` constraint is used by the following rule:

SpyGlass DFT DSM Solution
Atspeed_08

clock_sense

Purpose

Use the `clock_sense` constraint to stop propagation of clocks from the specified pins.

Product

SpyGlass CDC solution

Syntax

```
clock_sense
  -pins <pins-list>
  [ -tag <tag-names> ]
```

Arguments

The `clock_sense` constraint has the following arguments:

-pins <pins-list>
 Specifies the pins at which clock propagation should stop.

-tag <tag-names>

Specifies tags so that the propagation of the clocks associated with the specified tags is stopped at the pins specified by the *-pins <pins-list>* argument.

If you do not specify this argument, SpyGlass stops the propagation of all the clocks at the pins specified by the *-pins <pins-list>* argument.

Example

Consider the following constraints specification:

```
current_design top
clock -name top.clk1 -domain D1 -tag C1
clock -name top.clk1 -domain D2 -tag C2
clock_sense -pins AND.in1 -tag C1
clock_sense -pins AND.in2
```

In this case:

- Propagation of the `clk1` clock is stopped at the `in1` pin of the AND gate.
- Propagation of all the clocks is stopped at the `in2` pin of the AND gate.

Rules

The `clock_sense` constraint is used by the following rule:

SpyGlass CDC Solution

All clock-checking rules

clock_shaper**Purpose**

The `clock_shaper` constraint is a module that controls clock pulse propagation in a design by enabling/disabling it under certain conditions or by modifying frequencies. Here, clock shaping means changing the

frequency, that is, pulse shape, duty cycle, and enabling condition of the individual clock propagation unit.

However, a clock gating cell (CGC) can only enable or disable a clock. It is not responsible for clock shaping.

Consider the following figure:

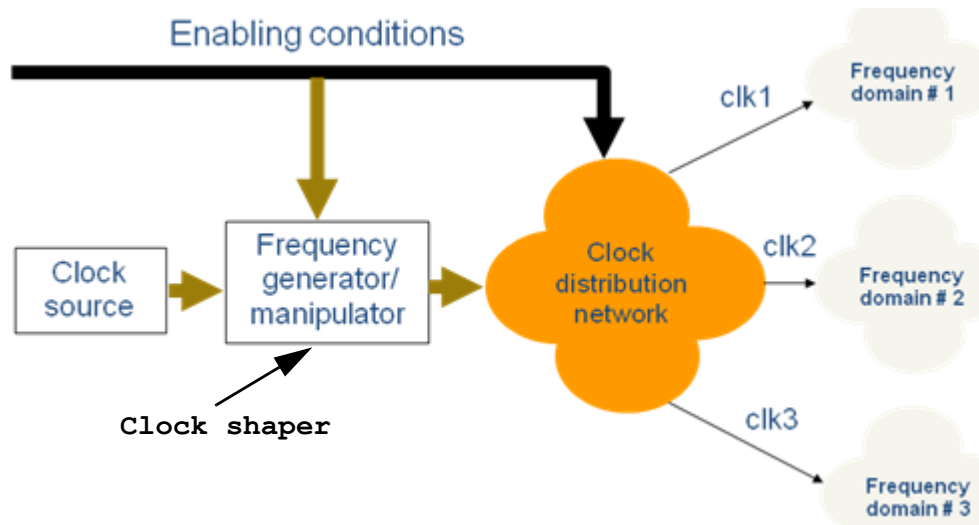


FIGURE 29. A clock shaper

The above figure represents a clock distribution network. The clock shaper is used as a key element in the network.

The structure and the use model of a clock shaper can be very versatile, ranging from a simple clock buffer to a complex programmable structure. The following are some uses of a clock shaper:

- PLL (clock generator)
- Frequency multiplier
- Frequency divider
- Flip-flop based divide-by-2 counter
- Crossbar switch between set of input and output clock pins with possible frequency manipulation

A module must be declared as `clock_shaper` and proper constraint options must be given for proper functioning in this way. Clock shapers are treated as black boxes by simulation.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `clockshaper`.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

```
clock_shaper
  -name <module-name>
  [-clkin <clkin-pin-list>]
  [-clkout <clkout-pin-list>]
  [-reset <reset-pin> -resetvalue <reset-value>]
  [-enable <en-pin-list> -envalue <en-value-list>]
  [-freqcontrols <freq-ctrl-pins> -freqcontrolvalues
<values_string>]
  [-freqmult <mult-values>]
  [-clockout_reference_inputclock <clkout pins, mapped
clkin pins>]
  [-clockout_frequency_multiplier <clkout pins, mapped freq
multiplier>]
  [-register]
  [-pll]
  [-divider]
  [-maskcaptureATspeed]
  [-scan_clock <name>]
  [-scan_clock_value <value>]
  [-scan_set <name>]
  [-scan_set_value <name>]
  [-scan_reset <name>]
  [-scan_reset_value <value>]
```

NOTE: The `clock_shaper` constraint supports wildcard characters.

NOTE: See [Arguments used by the dftDsmConstraintCheck_01 rule](#) to view the arguments used by the `dftDsmConstraintCheck_01` rule.

Arguments

General Arguments

The `clock_shaper` constraint has the following arguments:

-name <module-name>

The name of the module to be declared as a clock shaper. You can also specify an instance name as an input, if you have specified `-register` argument. This implies that instance-based `clock_shapers` are allowed only for flip-flops. Module names specified using this argument, are allowed for any user module. However, if the module is not a black box, then software forces it to be treated as black box and you would see the *InfoAnalyzeBBox* rule violation for that module.

-clkin <clkin-pin-list>

List of clock input pins of the clock shaper. Do not specify this argument, if you have specified the `-register` argument.

-clkout <clkout-pin-list>

List of clock output pins of the clock shaper. Do not specify this argument, if you have specified the `-register` argument.

-reset <reset-pin>

(Optional) Reset pin of the clock shaper. When this is activated, the clock shaper does not propagate any clock pulse through its output.

-enable <en-pin-list>

(Optional) List of control pins of the clock shaper. These pins should be activated for the clock shaper to propagate clock pulses. By default, a clock shaper is considered enabled.

-freqcontrols <freq-ctrl-pins>

(Optional) Pins that control or modify frequency of the clock pulse(s) propagating through the clock shaper.

-freqmult <mult-values>

(Optional) Multiplying values for a frequency of clock pulses propagating

through a clock shaper.

-resetvalue <reset-value>

A single 0 or 1 value. The value specifies the signal that activates the corresponding reset pin.

-envalue <en-value-list>

List of a space-separated single 0,1, or X (don't care) values where each value is for corresponding enable pin given through the -enable switch. For such values, clock shaper is enabled when each enable get its corresponding value.

You can also specify a list of space-separated string of values where each value string is of size of number of enable pins given. For such values, clock shaper is enabled when value at the enable pins satisfies any one value string.

-freqcontrolvalues <values_string>

Binary strings (0s and 1s) that specify a combination to be applied to their respective pins. The length of the combination should be equal to the number of pins that it refers.

-clockout_reference_inputclock <clkout pin, related clkin pins>

Specifies a relation of clkout pin with clkin pins, based on frequency control values in case of multi-pin support.

-clockout_frequency_multiplier <clkout pins, mapped freq multiplier>

Specifies a multiplicative factor to an input frequency based on frequency control value so that clkout has a modified frequency.

-register

Applies the clock_shaper constraint on hierarchical flip-flop instances or output net of the flip-flops. When you specify this option, you do not need to provide -clockin and -clockout fields because the clock pin of the flip-flop is treated as clockin and the output pin of the flip-flop is treated as clockout. In addition, the default value of the multiplication factor is 0.5. You can change the value of the multiplication factor by using the -clockout_frequency_multiplier option along with the

-register option. Also, you need to specify this argument to specify the instance name for the -name argument.

-pll

(Optional) Enables the clock shaper to work as a PLL. All the PLL checks are applied to this constraint.

-divider

(Optional) Enables the clock shaper to work as a clock divider.

-scan_clock <name>

Specifies the name of clock shaper pin driving clock pin of flops inside clock shaper.

-scan_clock_value <value>

Accepts either `rising_edge` or `falling_edge`, as values, to specify clock pin phase.

-scan_set <name>

Specifies the name of the clock shaper pin, which is driving the set pin of flip-flops inside clock shaper.

-scan_set_value <value>

Accepts either `high` or `low`, as values, to specify set pin phase.

-scan_reset <name>

Specifies the name of the clock shaper pin, which is driving the reset pin of flip-flops inside clock shaper.

-scan_reset_value <value>

Accepts either `high` or `low`, as values, to specify set pin phase.

For more information on support for clock shaper with scannable flip-flops, refer to *Support for clock shaper with scannable flops* section in the *SpyGlass DFT Rules Reference Guide*.

Arguments used by the `dftDsmConstraintCheck_01` rule

-testmodepins

Specifies the pins that determine whether the clockshaper is enabled in a given test mode. The values of these pins for given test modes should be passed under the scanshift or capture mode.

-scanshift

Binary strings (0s and 1s) that specify a combination to be applied to their respective pins in the scanshift mode. The length of the combination should be equal to the number of pins that it refers.

-capture

Binary strings (0s and 1s) that specify a combination to be applied to their respective pins in the capture mode. The length of the combination should be equal to the number of pins that it refers.

-captureATspeed

Binary strings (0s and 1s) that specify a combination to be applied to their respective pins in the CaptureATspeed mode. The length of the combination should be equal to the number of pins that it refers.

-maskcaptureATspeed

This option masks the test mode pin values under captureATspeed. For any test mode pin, the enabling condition is not checked if the corresponding bit is one in maskcaptureATspeed.

This argument is applicable only to the *Atspeed_17_captureatspeed* rule. It does not impact atspeed clock propagation.

Rules

The `clock_shaper` constraint is used by the following rules:

SpyGlass DFT Solution

All rules

SpyGlass DFT DSM Solution		
All rules	Specifically used by:	Atspeed_12, Atspeed_13, Atspeed_17_shift, Atspeed_17_capture and Atspeed_17_captureatspeed, dftDsmConstraintCheck_01

Examples

Consider the following examples:

Clock Shaper Without an Enable Pin

Consider the following `clock_shaper` constraint specification having a clock input pin and a clock output pin:

```
clock_shaper -name cg_cell
  -clkin clk -clkout clko
```

In this case, the `clk` pin is connected to the `clko` pin with frequency multiplier 1.

Clock Shaper With a Single Enable Pin

Consider the following `clock_shaper` constraint specification having a single enable pin:

```
clock_shaper -name cg_cell
  -clkin clk -clkout clko
  -enable en -envalue 11000
```

Here, the `clk` pin is connected to the `clko` pin with frequency multiplier 1 when the `en` pin has the specified value. If the enable value is not satisfied, the `clko` pin remains at X (don't care).

Clock Shaper With Multiple Enable Pins

Example 1

Consider the following `clock_shaper` constraint specification having multiple enable pins:

```
clock_shaper -name cg_cell
  -clkin clk -clkout clko
```

```
-freqcontrols en1 en2 en3 en4 en5
-freqcontrolvalues 111xx XX011
-freqmult 1 1
```

In this case, the clock shaper is enabled in the following two cases:

- The en1, en2, and en3 pins are equal to 111 irrespective of the values on the en4 and en5 pins.
- The en3, en4, and en5 pins are equal to 011 irrespective of the values on the en1 and en2 pins.

If neither of the above cases is satisfied, the device is disabled so that the output remains at X (don't care).

The above example also illustrates the use of don't care conditions on the enable pins.

Example 2

Consider the following example:

```
clock_shaper -name cs1
-clkin clk_in1
-clkout clk_out1 clk_out2 clk_out
-enable en1 en2 en3 -envalue 0 1 0
-freqcontrols freq_cnt1 freq_cnt2 freq_cnt3 freq_cnt4
-freqcontrolvalues x000 1001 1111
-freqmult 1 1 1 1 1 1 1 1 1
```

In the above example, clock shaper is enabled when value at en1, en2 and en3 is 0, 1 and 0 respectively.

Example 3

Consider the following example:

```
clock_shaper -name cs2
-clkin clk_in1
-clkout clk_out1 clk_out2 clk_out3
-enable en1 en2 en3 -envalue 011 1xx
-freqcontrols freq_cnt1 freq_cnt2 freq_cnt3 freq_cnt4
-freqcontrolvalues x000 1001 1111
-freqmult 1 1 1 1 1 1 1 1 1
```

In the above example, clock shaper is enabled when value at en1, en2

and en3 is 0, 1 and 1 respectively or value at en1 is 1

Clock Shaper With Multiple Clock-input and Clock-output Pins

A clock shaper can have multiple clock-input and clock-output pins. The input to output mapping is illustrated in the following example:

```
clock_shaper -name cg_cell
  -clkin clk1 clk2 clk3 -clkout clko1 clko2 clko3
```

In the above example, the connections are parallel, that is, the clk1 pin is connected to the clko1 pin, the clk2 pin is connected to the clko2 pin, the clk3 pin is connected to the clko3 pin, with frequency multiplier 1.

NOTE: *If the length of the clkin pin list is not equal to the length of the clkout pin list, the clock_shaper constraint specification is ignored and a warning message is reported.*

Clock Shaper With Frequency Division

Consider the following divide-by-2 clock_shaper constraint specification:

```
clock_shaper -name shaper
  -clkin clk -clkout clko -freqmult 0.5
```

In this case, the frequency of the clko pin is half of that of the clk pin.

Now consider the following clock_shaper constraint specification which produces the same result as the above specification:

```
clock_shaper -name shaper
  -clkin clk -clkout clko
  -clockout_frequency_multiplier clko 0.5
```

NOTE: *If the length of the clkin pin list is not equal to the length of the clkout pin list, the clock_shaper constraint specification is ignored and a warning message is reported.*

The advantage of using the -clockout_frequency_multiplier argument of the clock_shaper constraint over the -freqmult argument is that it provides flexibility to handle more complex clock shapers.

NOTE: *The -clockout_frequency_multiplier argument of the clock_shaper constraint has higher priority than the -freqmult argument.*

Clock Shaper With Frequency Multiplication

Consider the following `clock_shaper` constraint specification:

```
clock_shaper -name cg_cell
  -clkin clk -clkout clko -freqmult 2.0
```

In this case, the frequency of the `clko` pin is twice that of the `clk` pin.

Clock Shaper With Multiple Clock-output Pins With the Same Frequency Multiplication

Consider the following divide-by-2 `clock_shaper` constraint specification:

```
clock_shaper -name cg_cell
  -clkin clk1 clk2 clk3 -clkout clko1 clko2 clko3
  -freqmult 0.5
```

In this case, the `cg_cell` clock shaper has multiple clockin to clockout paths and the same divide-by-2 occurs on all the paths.

NOTE: *If the length of the `clkin` pin list is not equal to the length of the `clkout` pin list, the `clock_shaper` constraint specification is ignored and a warning message is reported.*

Clock Shaper With Multiple Clock-output Pins With Different Frequency Multipliers

Consider the following `clock_shaper` constraint specification:

```
clock_shaper -name cg_cell
  -clkin clk1 clk2 clk3 -clkout clko1 clko2 clko3
  -clockout_frequency_multiplier clko1 1.2
  -clockout_frequency_multiplier clko2 1.5
  -clockout_frequency_multiplier clko3 1.7
```

In the above example, the three clock-output pins, `clko1`, `clko2`, and `clko3`, have different frequency multiplications, that is, 1.2, 1.5, and 1.7, respectively.

NOTE: *The clock-output pin that does not have a frequency multiplier specified, is assigned a multiplier of 1.0 by default.*

Clock Shaper With a Single Clock-out Pin and Selectable Frequency

Multipliers

Consider the following clock shaper with the selectable frequency multiplication capability:

```
clock_shaper -name var_multiplier
  -clkin cin -clkout cout
  -freqcontrols fc0 fc1 -freqcontrolvalues 00 01 10
  -freqmult 1.0 0.5 0.25
```

In this case, the behavior of the clock shaper is described in the following table:

fc0	fc1	Output
0	0	cout frequency = cin frequency
0	1	cout frequency = cin frequency * 0.5
1	0	cout frequency = cin frequency * 0.25
1	1	The clock_shaper constraint is disabled

NOTE: *If the length of the freqcontrols pin list is not equal to the length of the freqcontrolvalues pin list, the clock_shaper constraint specification is ignored and a warning message is reported.*

Clock Shaper With Multiple Clock-input/Clock-output Pairs

The frequency control syntax is also used for specifying the mapping between the input and output clocks for clock shapers that can handle multiple clocks simultaneously.

The following table lists the arguments used to define the clock shaper control port names, the active values on these ports, and the clockin to clockout mapping:

Argument of the clock_shaper constraint	Defines
-freqcontrols	The control pins on a clock shaper
-freqcontrolvalues	A list of values on the control pins
-clockout_reference_inputclock	An output clock pin and a list of input clock pins that can be mapped to this output clock

Please note that the number of sets of control values must match the number of input clocks. In addition, each set of values must be unique because SpyGlass searches the value list in the left to right order.

Now consider the following `clock_shaper` constraint specification with the selectable frequency multiplication capability:

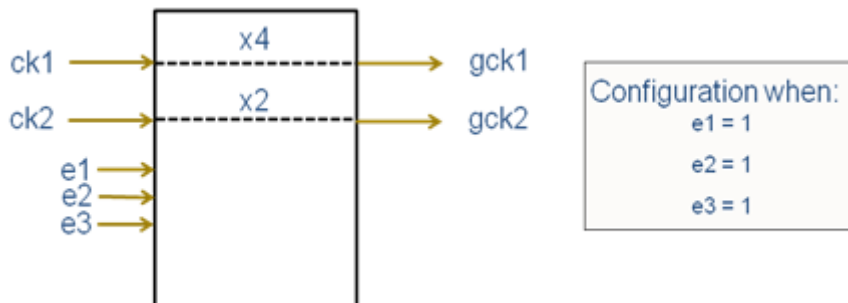
```
clock_shaper -name cg_cell
  -clkin clk1 clk2 clk3 -clkout clko1 clko2 clk03
  -freqmult 1.2 1.5 1.7
```

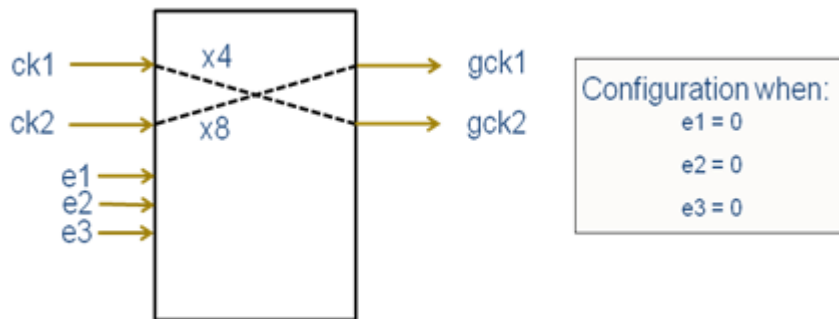
In the above example, a list of multipliers is specified. All frequency multiplications are applied in parallel, that is, the n^{th} multiplier in the list is applied to the n^{th} clkin/clockout pair.

NOTE: *If the lengths of the clkin pin list, the clkout pin list, and the freqmult list are not equal, the clock_shaper constraint specification is ignored and a warning message is reported.*

Clock Shaper With Selectable Mapping of Clock-input Pins to Clock-output Pins

```
clock_shaper -name CCN
  -clkin ck1 ck2 -clkout gck1 gck2
  -freqcontrols e1 e2 e3 -freqcontrolvalues 111 000
  -clockout_reference_inputclock gck1 ck1 ck2
  -clockout_reference_inputclock gck2 ck2 ck1
```





NOTE: If the length of the `clockout_reference_inputclock` pin list is not equal to the length of the `clkout` pin list, the `clock_shaper` constraint specification is ignored and a warning message is reported.

Clock Shaper With Selectable Clock-input/Clock-output Pairs and Different Frequencies

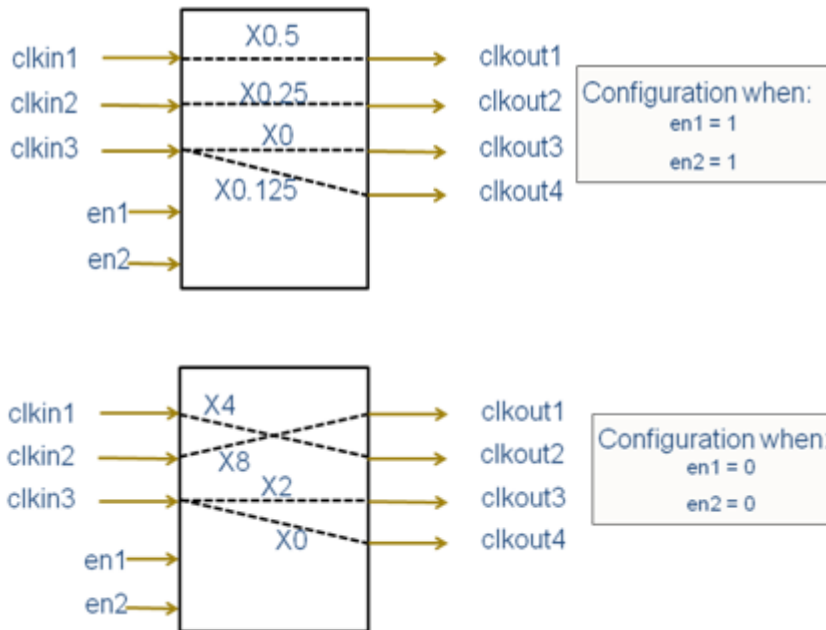
The syntax for clock shapers can be used to describe complex devices that not only have selectable clockin to clockout mapping but also have different frequency multipliers for each selection.

The syntax of the `-clockout_frequency_multiplier` argument of the `clock_shaper` constraint specifies a clockout pin and a list of frequency multipliers to be used in the same order as the values specified in the `-freqcontrolvalues` argument.

Consider the following `clock_shaper` constraint specification in which the input clocks are mapped to the output clocks with different frequency multipliers:

```
clock_shaper -name OCC
  -clkin clkin1 clkin2 clkin3
  -clkout clkout1 clkout2 clkout3 clkout4
  -clockout_reference_inputclock clkout1 clkin1 clkin2
  -clockout_reference_inputclock clkout2 clkin2 clkin1
  -clockout_reference_inputclock clkout3 clkin3 clkin3
  -clockout_reference_inputclock clkout4 clkin3 clkin3
  -clockout_frequency_multiplier clkout1 0.5 4
  -clockout_frequency_multiplier clkout2 0.25 8
  -clockout_frequency_multiplier clkout3 0 2
```

```
clkout4 0.125 0
-freqcontrols en1 en2 -freqcontrolvalues 11 00
```



Clock Shaper With PLL Support

Consider the following `clock_shaper` constraint specification in which a clock shaper is specified as a PLL:

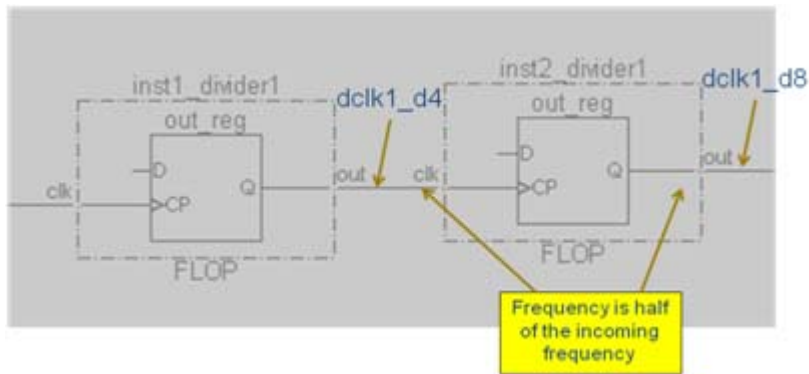
```
clock_shaper -pll -name myPLL
  -clkin ref_clk -clkout clk1 clk2
  -reset reset -resetvalue 0
```

The SpyGlass DFT DSM solution allows the use of PLLs. Please refer to rules, such as *PLL_01* and *PLL_02* for details.

Clock Shaper With Register Support

Consider the following conventional flip-flop based divide-by-2 counter:

SpyGlass Design Constraints



The above counter can be constrained by using the `-register` argument of the `clock_shaper` constraint as shown below:

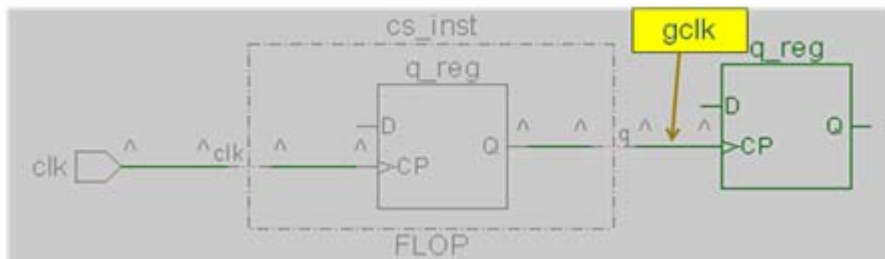
```
clock_shaper -name dclk1_d4 -register
```

```
clock_shaper -name dclk1_d8 -register
```

NOTE: Please note the following points:

- The `-register` argument of the `clock_shaper` constraint specifies that the module is a flip-flop based frequency divider.
- The default multiplication value is 0.5, which can be overridden.
- The name for the clock shaper is the net connected to the output of the flip-flop (the flip-flop is not explicitly instantiated in the RTL).
- In case of an instantiated technology library cell, the constraint can be applied on the instance.
- The `clock_shaper` constraint cannot be applied on a level-sensitive latch.

Consider the following schematic generated by the `Info_testclock` rule:



The clock_shaper constraint specification in this case is as follows:

```
clock_shaper -name gclk -register
```

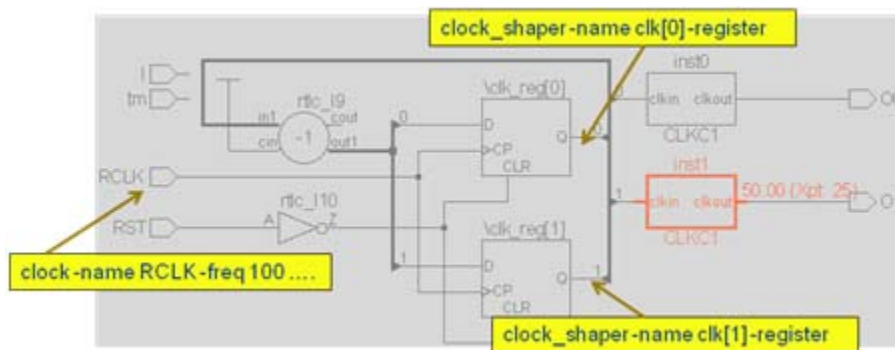
In the above schematic, the clock propagation through the flip-flop-based clock shapers is represented by the carrot (^) symbol.

Now consider the following multiplier with frequency multiplication value different from 0.5:

```
always @(posedge RCLK or negedge RST) begin
    if(!RST)
        clk <= 2'h0;
    else
        clk <= clk - 1'b1;
end
```

To model such a multiplier, use the `-clockout_frequency_multiplier` argument of the `clock_shaper` constraint.

The following figure specifies a countdown counter that can manipulate the frequency functionally:



The output frequencies of the above counter are as follows:

Clock Pin	Output Frequency of the Counter
clk[0]	50
clk[1]	25

SpyGlass Design Constraints

The above result can be modeled in the following two ways:

```
clock_shaper -name clk[1] -register
-clockout_frequency_multiplier 0.25
```

Or

```
clock_shaper -name clk[1] -register
-freqmult 0.25
```

NOTE: If both the `-clockout_frequency_multiplier` and `-freqmult` arguments of the `clock_shaper` constraint are specified, the `-clockout_frequency_multiplier` is used and the `-freqmult` argument is ignored.

Clock Shaper With External Enable Decode

```
clock_shaper -name cg -clkkin ref -clkout cout
-enable en -envalue 1
```

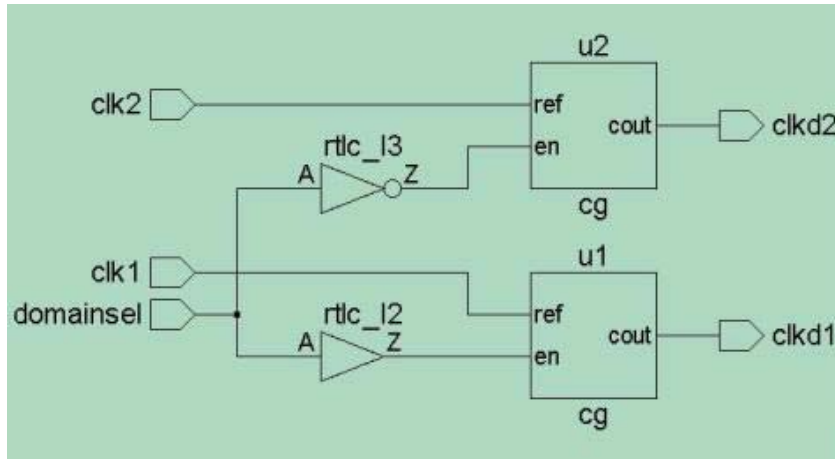


FIGURE 30. Clock shaper with external enable decode

Clock Shaper With Multiple Clock-out and Clock-in Pins

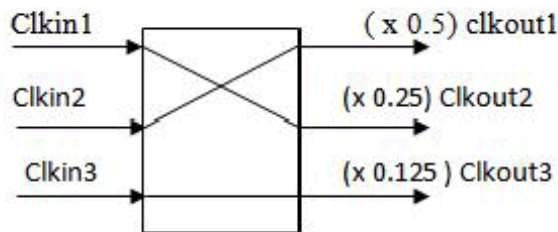
```
clock_shaper -name CST -clkkin clk1n1 clk1n2 clk1n3 -
clkout clkout1 clkout2 clkout3
-clockout_reference_inputclock clkout1 clk1n2 clk1n1 clk1n3
-clockout_reference_inputclock clkout2 clk1n1 clk1n3 clk1n3
```

```

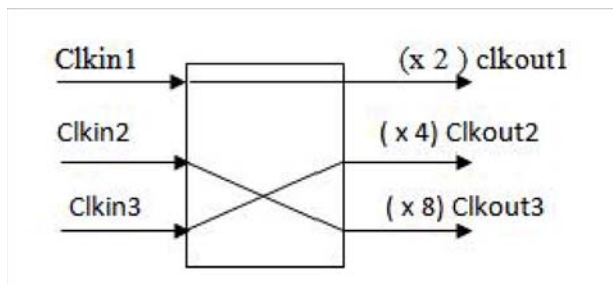
-clockout_reeference_inputclock clkout3 clkin3 clkin2 clkin1
-clockout_frequency_multiplier clkout1 0.5 2 2
-clockout_frequency_multiplier clkout2 0.25 4 4
-clockout_frequency_multprier clkout3 0.125 8 0
-freqcontrols en1 en2 en3
-freqcontrolvalues 110 001 111

```

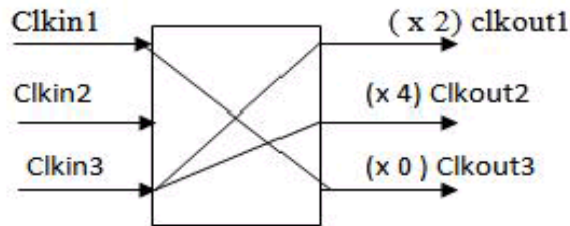
The following figure illustrates the input-output mapping of clock shaper, when -freqcontrols pins gets 110 under capture at-speed mode:



The following figure illustrates the input-output mapping of clock shaper, when -freqcontrol pins get 001 under capture at-speed mode:



The following figure illustrates the input-output mapping of clock shaper, when -freqcontrol pins get 111 under capture at-speed mode:



clockgating

Purpose

The `clockgating` constraint is used to specify gating conditions for test clocks.

NOTE: *Three test mode options are supported: `scanshift`, `capture`, and `captureATspeed`. Because these categories are not at-speed clock specific, additional data is needed to specify the gating signals that are specific to an at-speed test clock. See rule `Atspeed_07` for an example.*

Product

SpyGlass DFT DSM solution

Syntax

```
clockgating
  -name <clk-name>
  -pin <1st_signal_names>
  -value <values>
```

Arguments

The `clockgating` constraint has the following arguments:

-name <clk_name>

The name of the test clock, which is gated.

-pin <lst_signal_names>

List of signal names that act as a gating signal for the specified test clock.

-value <values>

List of values corresponding to the gating signals.

Rules

The `clockgating` constraint is used by the following rule:

SpyGlass DFT DSM Solution

Atspeed_07

complex_cell

Purpose

A complex cell is a module that controls clock pulse propagation in a design, by enabling/disabling it under different test mode conditions. Complex cells are instances of `clock_shaper`, and are treated as black boxes by simulation.

NOTE: *The `complex_cell` constraint will be deprecated in a future SpyGlass release. Please make use of [clock_shaper](#) constraint in place of this constraint.*

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

```
complex_cell
  -name <cell-name>
  -tclkinport <clkin-pin>
  -tclkoutport <clkout-pins>
  -testmodepins <tm-pins>
  -scanshift <values>
  -capture <values>
```



```
-captureStatic <values>
-captureATspeed <values>
[ -maskcaptureATspeed <values> ]
```

Arguments

The `complex_cell` constraint has the following arguments:

-name <cell-name>

Name of the module to be declared as a complex cell.

-tclkinput <clkinput-pin>

Clock input pin of the complex cell (can be one pin only).

-tclkoutput <clkoutput-pins>

Clock output pin of the complex cell (can be one pin only).

-testmodepins <tm-pins>

Pins that determine whether the complex cell is enabled in a given test mode. Values of these pins for given test modes should be passed under the arguments, such as `-scanshift` and `-capture`.

<values>

Binary strings (0 and 1) that specify a combination to be applied to their respective pins. The length of the combination should be equal to the number of pins that it refers.

For example, if there are three test mode pins, arguments, such as `-scanshift` and `-capture`, should have values of length three.

Rules

The `complex_cell` constraint is used by the following rules:

SpyGlass DFT Solution

All rules

SpyGlass DFT DSM Solution

All rules	Specifically used by:	Atspeed_12, Atspeed_13, Atspeed_17_shift, Atspeed_17_capture, and Atspeed_17_captureatspeed
-----------	-----------------------	--

compressor

Purpose

This constraint is used to specify test data compressors and data output pins.

The compressor constraint is used when data from internal chains is driving an on-chip compressor.

Product

SpyGlass DFT DSM solution

Syntax

The syntax of the `compressor` constraint is as follows:

```
compressor
  -name <mod-name| instance>
  -dataout <list of compressor module ports>
```

Arguments

-name <mod-name| instance>

Name of an instance or a module. If the `-name` argument is a module name, then all instances of that name will be defined as compressors with the same data output port names. The `-dataout` ports are assumed as the compressed scan out data.

-dataout <list of compressor module ports>

List of output ports of compressor driving primary outputs.

Examples

If the `-name` field contains a module name, then the constraint information will be applied to all instances of this name unless another compressor constraint refers to a specific instance of the same module name. For example, consider the SGDC listed below in which `top.u1` is an instance of module `compA`:

```
compressor -name compA -datain depins[31:0]
compressor -name top.u1 -datain depins[15:0]
```

In this example, `top.u1` is intended to have 16 tail registers whereas all other instances of `compA` have 32 head registers.

Rules

The `compressor` constraint is used by the following rules:

SpyGlass DFT DSM Solution				
TC_01	TC_02	TC_03	TC_04	TC_05

dbist

Purpose

The `dbist` constraint specifies the set of conditions, both pins and values, that when simulated will force the circuit into a state for DBIST mode.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `Dbist`.

Product

SpyGlass DFT solution

Syntax

The syntax of the `dbist` constraint is as follows:

```
dbist
  -name <name>
```

`-value <value>`

NOTE: The `dbist` constraint supports wildcard characters.

Arguments

The `dbist` constraint has the following arguments:

-name <name>

Complete hierarchical name of DBIST port/pin.

The pin can be a primary pin as well as an internal pin.

You can specify a single port/pin's full hierarchical name or a space-separated list of full hierarchical port/pin names. The port/pin name supports wildcard.

For primary ports, you can also specify the simple port name as in the following example:

```
current_design top
  dbist -name in15 ...
```

-value <value>

Value list for the DBIST pin.

The value list is the sequence of one or more values (each value being 0, 1, X, Z, or a combination) that when applied to the DBIST pin, will cause the circuit to enter DBIST mode.

Rules

The `dbist` constraint is used by the following rules:

SpyGlass DFT Solution

Latch_16	Tristate_17
----------	-------------

decompressor

Purpose

This constraint is used to specify a test data decompressor and its data input pins.

Product

SpyGlass DFT DSM solution

Syntax

The syntax of the `dbist` constraint is as follows:

```
decompressor
  -name <mod-name| instance>
  -datain <list of decompressor module ports>
```

Arguments

-name <mod-name| instance>

Name of an instance or a module. If the `-name` field contains a module name, the constraint information is applied to all instances of this name unless another decompressor constraint refers to a specific instance of the same module name.

-datain <list of decompressor module ports>

List of input ports of decompressor driven by primary inputs.

Examples

Consider the following example:

```
Decompressor -name decompA -datain depins[31:0]
Decompressor -name top.u1 -datain depins[15:0]
```

In the SGDC listed above, `top.u1` is an instance of module `decompA`.

Rules

The `decompressor` constraint is used by the following rules:

SpyGlass DFT DSM Solution				
TC_01	TC_02	TC_03	TC_04	TC_05

define_clock_tree

Purpose

The `define_clock_tree` constraint is used to approximate the buffering in the clock tree and to apply the correction factor on the clock pin capacitance of the flop.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `define_clock_tree` constraint is as follows:

```
define_clock_tree
    -name <clock-netname>
    -leaf_fanout <fvalue>
    -intermediate_fanout <fvalue>
    -intermediate_buffer <im-buffer/inverter-cellname>
    -leaf_buffer <leaf-buffer/inverter-cellname>
    [-leaf_cap_per_fanout <fvalue> ]
    [ -wire_cap_correction_factor <fvalue> ]
    [ -pin_cap_correction_factor <fvalue> ]
    [ -root_buffer <root-buffer/inverter-cellname> ]
    [ -root_buffer_lib <root-buffer/inverter-libname> ]

    [ -intermediate_buffer_lib <im-buffer/inverter-libname>
]
    [ -leaf_buffer_lib <leaf-buffer/inverter-libname> ]
    [ -icgc_fanout <int> ]
    [ -root_fanout <fvalue> ]
    [ -icgc_cell <icgc-cellname> ]
    [ -icgc_cell_lib <icgc-libname> ]
```

NOTE: *The `icgc_cell` and `icgc_cell_lib` arguments will be supported in a future release.*

Arguments

-name <clock-netname>

Specifies the RTL net name of the clock source.

NOTE: *This argument supports wildcard characters. Refer to [Example 4](#) for more information.*

-leaf_fanout <fvalue>

Specifies a floating-point value as the average fan-out of the leaf-level buffer.

The specified value must be greater than or equal to 2.

NOTE: *You must specify the -leaf_fanout argument if the define_clock_tree constraint is used to calculate the buffering in the clock tree.*

-leaf_cap_per_fanout <fvalue>

(Optional) Specifies the capacitance per fanout of nets connecting the leaf buffer to flip-flops. Also, range of fanout multiple is 0 to INF.

-wire_cap_correction_factor <fvalue>

(Optional) Specifies a correction factor (floating-point value) to scale the capacitance obtained from the wireload model.

The default value is 1.0.

-pin_cap_correction_factor <fvalue>

(Optional) Specifies a correction factor (floating-point value) to scale the capacitance of the fan-out clock pin of the net. Note that the fan-out clock pin of the net is also the fan-in of the flop.

The default value is 1.0.

-root_buffer <root-buffer/inverter-cellname>

(Optional) Specifies the cell name to be used as buffer/inverter at the root level.

NOTE: *You must specify the -root_fanout argument of the*

`define_clock_tree` constraint with the `-root_buffer` argument of the constraint.

-root_buffer_lib <root-buffer/inverter-libname>

(Optional) Specifies the library name of the buffer/inverter at the root level.

NOTE: You must specify the `-root_buffer` argument of the `define_clock_tree` constraint with the `-root_buffer_lib` argument of the constraint.

-intermediate_buffer <im-buffer/inverter-cellname>

Specifies the cell name to be used as buffer/inverter at the intermediate levels.

NOTE: You must specify the `-intermediate_fanout` argument of the `define_clock_tree` constraint with the `-intermediate_buffer` argument of the constraint.

-intermediate_buffer_lib <im-buffer/inverter-libname>

(Optional) Specifies the library name of the buffer/inverter at the intermediate levels.

NOTE: You must specify the `-intermediate_buffer` argument of the `define_clock_tree` constraint with the `-intermediate_buffer_lib` argument of the constraint.

-leaf_buffer <leaf-buffer/inverter-cellname>

Specifies the cell name to be used as buffer/inverter at the leaf level.

NOTE: You must specify the `-leaf_fanout` argument of the `define_clock_tree` constraint with the `-leaf_buffer` argument of the constraint.

-leaf_buffer_lib <leaf-buffer/inverter-libname>

(Optional) Specifies the library name of the buffer/inverter at the leaf level.

NOTE: You must specify the `-leaf_buffer` argument of the `define_clock_tree` constraint with the `-leaf_buffer_lib` argument of the constraint.

NOTE: *If a root buffer or inverter is not specified, SpyGlass uses an intermediate buffer/inverter specified in the `define_clock_tree` constraint.*

-icgc_fanout <int>

(Optional) Specifies the maximum fan-out of all the ICGCs, where all fan-outs are going to sink, in the netlist. If there are no such ICGCs, the `leaf_fanout` is taken as the `icgc_fanout`.

-root_fanout <fvalue>

(Optional) Specifies a floating-point value as the average fan-out of the root level buffers/inverters

-intermediate_fanout <fvalue>

Specifies a floating-point value as the average fan-out of the intermediate level buffers/inverters

Examples

Example 1

Consider the following `define_clock_tree` constraint specification:

```
define_clock_tree
  -name CLKNET
  -leaf_fanout 16.00
  -wire_cap_correction_factor 2.0
  -root_fanout 4.00
  -intermediate_fanout 4.00
  -intermediate_buffer BUFI
  -leaf_buffer BUFL
```

In the above `define_clock_tree` constraint, the root-level buffer/inverter is not specified. In this case, SpyGlass uses the intermediate-level buffer/inverter (BUFI) as the root-level buffer/inverter.

Therefore, the buffers/inverters at each level will be as follows:

Buffer/Inverter at Level	Buffer/Inverter
Intermediate-level(s)	BUFI
Leaf-level	BUFL
Root-level	BUFI

Example 2

Consider a clock tree having 1000 pins with the following specification of the `define_clock_tree` constraint:

```
define_clock_tree
  -name CLKNET
  -leaf_fanout 16.00
  -wire_cap_correction_factor 2.0
  -intermediate_fanout 4.00
  -root_fanout 4.00
  -root_buffer BUFR
  -intermediate_buffer BUFI
  -leaf_buffer BUFL
```

In this case, the number of drivers required at each level will be as follows:

- **Level 0:** Number of BUFL drivers = $1000/16 = 63$
- **Level 1:** Number of BUFI drivers = $63/(16/4) = 16$
- **Level 2:** Number of BUFI drivers = $16/4 = 4$
- **Level 3:** Number of BUFR driver = $4/4 = 1$

Now, consider that the wireload for nets with fan-out as 16 is 4 pF and the wire capacitance correction factor is 2. In this case, the capacitance of nets will be $4 * 2 = 8$ pF.

Example 3

Consider a clock tree having 1000 pins with the following specification of the `define_clock_tree` constraint:

```
define_clock_tree
  -name CLKNET
  -leaf_fanout 16.00
```

SpyGlass Design Constraints

```
-wire_cap_correction_factor 2.0  
-root_fanout 4.00  
-intermediate_fanout 4.00  
-root_buffer BUFR  
-intermediate_buffer BUFI  
-leaf_buffer BUFL  
-pin_cap_correction_factor 3.5
```

In this case, the specified pin cap correction for the CLKNET is 3.5.

Example 4

Consider the following example that shows the extracted `define_clock_tree` constraint from a netlist by the PECLKTREE rule:

```
define_clock_tree
  -name "*"
  -root_buffer BUFX8
  -intermediate_buffer BUFX2
  -leaf_buffer BUFXL
  -leaf_fanout 16.00
  -intermediate_fanout 8.00
  -root_fanout 2.00
  -leaf_cap_per_fanout 0.100
```

In this example, the clock tree in the netlist design has on average 16.00 as leaf fan-out and the average capacitance of every fan-out of the leaf nets is 0.10pf. The netlist has on average an intermediate fan-out of 8.00 and the root fan-out of 2.00. In addition, the buffer names specified in this example are determined from the clock tree of the netlist.

Rules

SpyGlass Power Estimation and SpyGlass Power Reduction solutions				
PEPWR01	PEPWR02	PEPWR03	PEPWR05	PEPWR13
PEPWR14	poweraudit			

define_illegal_input_values

Purpose

If illegal values are specified on a set of inputs, all input values not specified for that set of inputs are considered legal.

Product

SpyGlass DFT solution

Syntax

The syntax of the `define_illegal_input_values` is as follows:

```
define_illegal_input_values -name
    -name <node_names>
    -value <values>
```

Arguments

-name <node_names>

Specifies the list of node names. The names may refer to internal nodes or top-level ports.

-value <values>

Specifies the list of illegal value combinations. It is the only set of invalid simulation sequence.

Examples

Consider the following example:

```
define_illegal_input_values -name I1 I2 -values 00 11
```

In the above example, for nodes, I1 and I2, the invalid set of value combination is 00 and 11. All the values except 00 and 11 are considered valid.

Rules

The `define_illegal_input_values` constraint is used by the following rules:

SpyGlass DFT Solution
Async_06

define_legal_input_values

Purpose

If legal input values are specified for a set of inputs, all input values not specified for that set of inputs are considered illegal.

Product

SpyGlass DFT solution

Syntax

The syntax of the `define_legal_input_values` is as follows:

```
define_legal_input_values -name  
    -name <node_names>  
    -value <values>
```

Arguments

-name <node_names>

Specifies the list of node names. The names may refer to internal nodes or top-level ports.

-value <values>

Specifies the list of legal value combinations. It is the set of valid simulation sequence.

Examples

Consider the following example:

```
define_legal_input_values -name I1 I2 -values 10 01
```

In the above example, for nodes, I1 and I2, only the value combinations, 10 or 01, are allowed. All the values except 10 and 01 are considered invalid.

Rules

The `define_legal_input_values` constraint is used by the following

rules:

SpyGlass DFT Solution

Async_06

define_library_group

Purpose

Defines the groups of Synopsys technology libraries at different PVT values. Library groups need to be defined because same cell definition is possible in multiple libraries at different PVT values.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `define_library_group` constraint is as follows:

```
current_design <top-du-name>
  define_library_group
    -name <lib-grp-name>
    -libname <lib-name-list>
```

Arguments

<top-du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

-name <lib-grp-name>

Name of the library group. The specified name should be unique.

-libname <lib-name-list>

Name of the library/libraries that you want to add in the specified library group.

NOTE: You can also use the `-libname` argument as `-libnames` to specify a single library or a group of libraries.

Consider the following example, for the libraries: L1, L2, L3, L4 and L5, use the `define_library_group` constraint as:

```
define_library_group -name G1 -libname L1 L2
define_library_group -name G2 -libname L3 L4
```

NOTE: Refer to the SGDC command [use_library_group](#) to specify library groups to various design hierarchies.

Rules

The `define_library_group` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	poweraudit	

define_reset_order

Purpose

Specifies the reset order, which determines the flow of data from one reset to another reset.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `define_reset_order` constraint is as follows:


```
current_design <du-name>
define_reset_order
  -from <from-rst-list>
  -to <to-rst-list>
```

Arguments

-from <from-rst-list>

Specifies the name of the source resets in the reset ordering. You can specify a space-separated list of hierarchical reset net names in this argument.

This argument supports wildcard characters.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

-to <to-rst-list>

Specifies the name of the destination resets in the reset ordering. You can specify a space-separated list of hierarchical reset net names in this argument.

This argument supports wildcard characters.

NOTE: The resets specified in the above arguments should match with one of the asynchronous resets specified in the `reset` constraint or inferred through the `use_inferred_resets` parameter.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression with the hierarchies. For example, the `top.*.rst1` expression is matched to `top.u1.rst1` and `top.u1.u2.rst1`. By default, the `define_reset_order` constraint matches only `top.u1.rst1`. Setting the value of the `hier_wild_card` parameter to `yes` runtime performance of the `define_reset_order` constraint is impacted.

Examples

Consider the following examples:

Example1

```
define_reset_order -from R1 -to R2  
define_reset_order -from R1 -to R3
```

The above example defines the flow of data from R1 to R2 and from R1 to R3.

Example2

```
define_reset_order -from RST1 -to RST2  
define_reset_order -from RST2 -to RST1
```

The above example shows bidirectional reset ordering. Such cases are reported by the *SGDC_05* rule.

Example3

Consider an example in which the f1 flip-flop has the R1 reset pin, and the f2 flip-flop has the R2 reset pin. Also, consider that R1 comes from primary resets, r1, r2, and r3, and R2 comes from r1, r2, r4, and r5. In this case, the design is fine if every reset that exists in fan-in of R1 but not R2 (R1-R2) happens before (through a `define_reset_order` constraint) every reset in the fan-in of R2.

This guarantees that asynchronously resetting flip-flop, f1, does not cause metastability on flip-flop, f2. If the specified condition is not met, a violation will be reported. For example, the reset ordering is fine in the following case:

```
define_reset_order -from r3 -to r1  
define_reset_order -from r3 -to r2  
define_reset_order -from r3 -to r4  
define_reset_order -from r3 -to r5
```

NOTE: A violation is reported if the first flip-flop has preset/clear pins and the second flip-flop does not have any of them.

Example4

The following example specifies a reset crossing from `top.RST1` to `top.RST2`:

```
define_reset_order -from top.RST1 -to top.RST2
```

Example5

The following example specifies reset ordering from top.RST1 to top.RST2 and top.RST3:

```
define_reset_order -from top.RST1 -to top.RST2 top.RST3
```

Example6

The following example specifies a reset ordering between flip-flops where source receives RST1 and RST2 and destination receives RST3 and RST4:

```
define_reset_order -from RST1 -to RST3 RST4
define_reset_order -from RST2 -to RST3 RST4
```

Example7

The following example specifies a reset ordering between flip-flops where source receives RST1 and RST2 and destination receives RST2 and RST3:

```
define_reset_order -from RST1 -to RST2 RST3
define_reset_order -from RST2 -to RST3
```

Rules

The define_reset_order constraint is used by the following rules:

SpyGlass CDC solution		
Ac_resetcross01	Ar_resetcross01	Ar_resetcross_matrix01

define_tag

Purpose

The define_tag constraint is used to define a named condition for application of certain stimulus at the top port or an internal node. The connectivity checks are made by applying one or more of the user-defined conditions such that circuit can be simulated into the desired mode.

The define_tag constraint can be used in the following different ways:

- 1. Provide initial state of a design

To provide an initial state for a design, each register needs to be initialized using the `define_tag` constraint. The following example initializes a register `foo` of design unit `top` to the value 0:

```
define_tag -tag initState -name top.foo -value 0
```

The following example initializes a vector register `foo1[7:0]` of design unit `top` to the value `h aa`:

```
define_tag -tag initState -name top.foo1[7:0]
           -value {h aa}
```

2. Provide an initialization vector

A simulation vector that would initialize registers of a design. The following example initializes registers by applying three different vectors on three inputs of a design:

```
define_tag -tag initSeq -name top.reset1
           -value 1 1 1 x x x
define_tag -tag initSeq -name top.reset2
           -value x x x 1 1 1
define_tag -tag initSeq -name top.d
           -value 0 0 0 0 0 0
```

In this example, `reset1` is asserted for three cycles to initialize some set of registers, `reset2` is asserted to initialize another set of registers while some other registers are initialized by providing 0 on a data line "d" for six cycles. Note that `reset1` and `reset2` can be asserted simultaneously if they can reset registers independently.

The value can be 0, 1, or x (case-insensitive). Note that each value is separated by a blank.

Vectors can also be initialized as in the following example:

```
define_tag -tag initSeq -name top.foo[0:3]
           -value {h 0 6}
```

The above specification indicates that the vector `top.foo` would be assigned 0 (in Hex) or 0000 (in binary) in the first cycle and 6 (in Hex) or 0110 (in binary) in the second cycle.

NOTE: When you specify an initial sequence using the `define_tag` constraint, other initial values specified with the `reset` constraints are ignored.

3. Specifying the bit-stuck values for nets

NOTE: *The `define_tag` constraint is used for specifying the bit-stuck values only for SpyGlass Auto Verify solution. This feature is not used by SpyGlass CDC solution.*

Bit-stuck values for nets are checked by the `Av_bitstuck01` rule of the *SpyGlass Auto Verify* solution.

The `Av_bitstuck01` rule of the *SpyGlass Auto Verify* solution checks only those nets that are specified with the `-name` argument of a `define_tag` constraint defining a tag named `netBitStuck` as in the following example:

```
define_tag -tag netBitStuck -name top.var[0]
          -value 0
```

Then, the `Av_bitstuck01` rule checks the stuck-at condition based on the value of the `-value` argument. If you do not specify the `-value` argument, both stuck-at 0 and stuck-at 1 conditions are checked.

Using Wildcards

You can use wildcards with the `-value` argument while specifying the `define_tag` constraints.

You can specify the sequences of ones, zeros, and x's using the `*` wildcard character that indicates the number of times the particular value on its left is to be applied.

Consider the following `define_tag` specification:

```
define_tag -tag initSeq -name top.rst1 -value 0 0 1 1
```

The above specification can be rewritten using the wildcard character as follows:

```
define_tag -tag initSeq -name top.rst1
          -value <2*0> <2*1>
```

Please note the following:

- You must enclose the values containing wildcards in `<>`.
- The format to use the wildcard is `<N*V>` (no spaces) where `V` is the value to be applied and `N` is the number of times the value is to be

repeated. The value V can be 0,1, x, or X and N can be any non-zero positive integer number).

- The one set of operations with the wildcard character must be separated by a blank space with the next operation. For example:

```
-value <3*1> 000 <50*x> 101
```

Here, the sequence applied will be three 1's, three 0's, fifty x's, sequence 101.

Product

SpyGlass Auto Verify solution, SpyGlass CDC solution, SpyGlass DFT solution, and SpyGlass DFT DSM solution

For SpyGlass Auto Verify solution and SpyGlass CDC solution

Syntax

The syntax of the `define_tag` constraint is as follows:

```
current_design <du-name>
define_tag
  -tag <initState | initState | netBitStuck>
  -name <flop-name> | <net-name>
  -value <value> | <value-list>
```

NOTE: `netBitStuck` is used for the `-tag` argument only for the SpyGlass Auto Verify solution. This feature is not used by the SpyGlass CDC solution.

Arguments

-tag <initState | initState | netBitStuck>

The `-tag` argument accepts only a predefined list of values (case-sensitive).

-name <flop-name> | <net-name>

The `-name` argument accepts a hierarchical net name (scalar or vector).

-value <value> | <value-list>

The -value argument accepts values as follows:

Value of -tag argument	Signal Type	Allowed value of -value argument
initState, netBitStuck	Scalar	1, 0, or x
	Vector	Binary or hexadecimal number
initSeq	Scalar	Sequence of 1,0, and x
	Vector	Sequence of Binary or Hexadecimal numbers

NOTE: *The first row is only applicable for SpyGlass Auto Verify solution. This feature is not used by SpyGlass CDC solution.*

Rules

The define_tag is used by the following rules:

SpyGlass CDC Solution			
Ac_cdc01a	Ac_cdc01b	Ac_cdc01c	Ac_cdc08
Ac_fifo01	Ac_handshake01	Ac_handshake02	Clock_sync03a
Ac_conv02			
SpyGlass Auto Verify Solution			
Av_initstate01	Av_bitstuck01		

For SpyGlass DFT solution and SpyGlass DFT DSM solution

Purpose

The define_tag constraint is used to define a named condition for application of certain stimulus at the top port or an internal node. The connectivity checks are made by applying one or more of the user-defined conditions such that circuit can be simulated into the desired mode.

This condition is defined in one of the following methods:

1. Application of logic value on a specified node

This method is used to define a named condition under which given value is applied to specified node (top module port or internal node) for one or more LE simulation cycles. For example:

```
define_tag -tag s1 -name top.EN -value 0101
```

This statement defines a condition by name `s1` to mean that logic value (or sequence) `0101` be applied on pin `top.EN`. The semantics of the `define_tag` constraint is similar to the `test_mode` constraint.

Multiple pin-value pairs can be associated under the same condition name. For example,

```
define_tag -tag s1 -name top.EN1 -value 0101
```

```
define_tag -tag s1 -name top.EN2 -value 0100
```

However, please note these should not be mutually conflicting as shown below.

```
define_tag -tag s1 -name top.EN1 -value 0101
```

```
define_tag -tag s1 -name top.EN1 -value 0100
```

In addition, there is no precedence between various pin-value specifications under the same condition name, and all of these are simulated in parallel by SpyGlass LE engine.

Name can be a top-module port, any internal net name, or terminal name. It may also be bit-select or part-select.

You can specify repeat sequences for the `define_tag` constraint.

For fields that require repeat sequence, specify the values as `<I*S>`. Here, `S` is any string that does not contain the `<`, `>`, and `*` characters. However, `S` can contain another `<I*S>` expression. `I` is an integer that is always interpreted as a decimal value. The expression `<I*S>` means that the sequence `S` will be repeated `I` number of times.

NOTE: *Tagging for nesting is not allowed. For example, the following `define_tag` statements are not allowed:*

```
define_tag -name sub_seq -value <5*01>
```

```
define_tag -name main_seq -value <100*sub_seq>
```

However, you can achieve the same result by using the `setvar` command.

2. Group various conditions together to create a new condition name

This method is used to group one or more previously defined conditions under a new condition name. The semantics of this method are equivalent to creating a new condition name to represent all pin-value specifications defined under the conditions being merged. For example:

```
define_tag -tag smodel100  
    -merge <list-of-previously-defined-conditions>
```

This statement defines a condition by name `smodel100` to mean the grouping of all individual pin-value specifications as specified after the `-merge` argument. It should be noted that condition names require a 'define before use' model and therefore, use of any undefined condition name will report a syntax error in reading SGDC file.

The affect of merging multiple condition names into a single condition is the same as defining multiple pin-value specifications under the same condition name. As previously described, these should not be mutually conflicting. In addition, there is no precedence between various pin-value specifications coming from the same or different condition names, and all of these are simulated in parallel by SpyGlass LE engine.

The above two methods of defining a condition cannot be used simultaneously. You can either define a condition by pin-value specification or by merge specification. A condition created by merge method can be defined only once.

Further, a condition cannot be modified (or appended to with additional pin-value specification) after it has been used. This limitation has been defined to avoid any ambiguity in SGDC specification, and ensure WYSIWYG advantage for end users.

The condition creation by the merge method can be used for aliasing purposes also by specifying only one condition name after the `-merge` argument. This is a very useful feature because now all subsequent connectivity check commands can use the alias name and the alias name can be changed to point to different condition group(s) by a very simple edit at one location only.

Now, you can run SpyGlass analysis under a different condition group by making just one small change in the SpyGlass Design Constraints file.

NOTE: *The scope of a condition is restricted to the current_design specification only.*

You can also specify the `define_tag` constraint for a bus signal as

shown below.

```
define_tag -tag T1 -name top.in[2:0] -value {b 001}
```

Where `in` is the `2:0` vector net.

When to use the define_tag constraint

You should use `define_tag` constraint for the connectivity specific (Soc_xx rules) and scan chain-specific rules only. The idea is that if different parts of a circuit operate under a different set of simulation conditions, you can do the connectivity checks under a different simulation condition in a single run of SpyGlass analysis.

You should also use `define_tag` constraint when the test initialization sequence is complex (such as, a TAP controller controlling the scan chains). In such cases, setting up proper test mode may take multiple runs of SpyGlass analysis. Thus, `define_tag` constraint enables you to try out different initialization sequences using different `define_tag` statements, so that you can evaluate and infer which initialization sequence is appropriate in a single run of SpyGlass using the `require_value` constraint.

You should use `test_mode` constraint to define the final set of simulation condition under which chip is tested. The `define_tag` constraint is used by all rules except the SoC rules.

Syntax

The syntax of the `define_tag` constraint is as follows:

```
define_tag
  -tag <any-string-name>
  -name <net-name>
  -value <value> | <value-list>
  -merge <tag> | <tag-list>
```

Arguments

-tag <any-string-name>

The `-tag` argument accepts any string name.

-name <net-name>

The -name argument accepts a hierarchical net name| port | hierarchical terminal name (scalar or vector).

-value <value> | <value-list>

The -value argument accepts values of any string constituted using 1, 0, X, Z.

-merge <tag> | <tag-list>

The -merge argument accepts tag name or a list of tag names. Please note that tags specified as a value to -merge field must be defined using `define_tag`.

Examples

Consider the following examples:

Example1

```
define_tag -name abc -value "<5*10>"
```

The above example will be expanded as follows:

```
define_tag -name abc -value 1010101010
```

Example2

```
define_tag -name abc -value "11<5*10>010"
```

The above example will be expanded as follows:

```
define_tag -name abc -value 111010101010010
```

Example3

```
define_tag -name abc -value "<50*11<5*10>>010"
```

The above example will be expanded as follows:

```
define_tag -name abc -value 111010101010...(repeated 50 times
followed by 010)
```

You can also set a variable using the command `setvar` to obtain the above result as follows:

```
setvar x 11<5*10>
```

```
define_tag -name abc -value "<50*${x}>010"
```

The above example will be expanded as follows:

```
define_tag -name abc -value 111010101010...(repeated 50 times
followed by 010)
```

Consider the following example that has three conditions (C1, C2, and C3) and three condition groups (CG1, CG2, and CG3):

```
define_tag -tag C1 ...
define_tag -tag C2 ...
define_tag -tag C3 ...
define_tag -tag CG1 -merge C1 C2
define_tag -tag CG2 -merge C2 C3
define_tag -tag CG3 -merge C3 C1
```

Now, you want to apply the condition groups one-by-one. Normally, you would need to create the constraints of each group separately. However, you can create an alias condition (say TM) with the first condition group CG1 as follows:

```
define_tag -tag TM -merge CG1
```

Use the alias condition TM in subsequent constraints and run SpyGlass analysis. Next, modify the alias condition TM to the next condition group CG2 as follows:

```
define_tag -tag TM -merge CG2
```

Rules

The `define_tag` constraint is used by the following rules:

SpyGlass DFT Solution			
Info_scanchain	Scan_22	Scan_24	Scan_25
Scan_26	Scan_29	Diagnose_Scan Chain	
SpyGlass Connectivity Verify			
Soc_01	Soc_02	Soc_04	
SpyGlass DFT DSM Solution			
Atspeed_21	Info_Atspeed_21		

delay_buffer

Purpose

The `delay_buffer` constraint is used to specify the delay buffers.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `delay_buffer` constraint is as follows:

```
current_design <du_name>
delay_buffer
  -names <cell-name-list>
```

Arguments

<du_name>

Name of the design unit under which you are specifying the cells.

-names <cell-name-list>

Space-separated list of the cell names. You can use wildcard characters while specifying the cell name using the `-names` argument.

Rules

The `delay_buffer` constraint is used by the following rules:

SpyGlass Power Verify Solution	
LPPSW03	LPPSW04

deltacheck_ignore_instance

Purpose

Specifies the instance to be ignored for delta delay value checking.

Then, the DeltaDelay01 rule does not report the delta delay values for all flip-flops/latches in the specified instance. However, the actual delta delay value for the specified instance is used for further calculations.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `deltacheck_ignore_instance` constraint is as follows:

```
current_design <du-name>
    deltacheck_ignore_instance
        -name <inst-name>
```

Arguments:

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the instance to be ignored.

-name <inst-name>

The hierarchical name of the instance to be ignored.

You can specify multiple instances to be ignored using multiple `deltacheck_ignore_instance` constraints.

NOTE: For VHDL design units, instance names are case-insensitive.

Rules

The `deltacheck_ignore_instance` constraint is used by the following rule:

SpyGlass CDC Solution

DeltaDelay01

deltacheck_ignore_module

Purpose

Specifies the design unit to be ignored for delta delay value checking.

Then, the DeltaDelay01 rule does not report the delta delay values for all flip-flops/latches in all instances of the specified design unit. However, the actual delta delay values for these instances are used for further calculations.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `deltacheck_ignore_module` constraint is as follows:

```
current_design <du-name>
    deltacheck_ignore_module
        -name <ignr-du-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the design unit to be ignored

-name <ignr-du-name>

The name of the design unit to be ignored.

You can specify multiple design units to be ignored using multiple

deltacheck_ignore_module constraints.

NOTE: For VHDL design units, names of the design units to be ignored are case-insensitive.

Rules

The `deltacheck_ignore_module` constraint is used by the following rule:

SpyGlass CDC Solution
DeltaDelay01

deltacheck_start

Purpose

Specifies the start points (clock ports, clock pins, or clock nets) for checking by the DeltaDelay01 rule.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `deltacheck_start` constraint is as follows:

```
current_design <du-name>
  deltacheck_start
    -name <sig-name>
    [ -value <value> ]
```

Arguments

<du-name>

Module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the start point clock port/pin/net.

-name <sig-name>

Hierarchical name of a clock port/pin/net.

-value <value>

Expected delta delay value at all flip-flops/latches triggered by the clock port/pin/net specified with the `-name` argument.

You can specify multiple start point signals using multiple `deltacheck_start` constraints.

Examples

The following example specifies net `clk1` of design unit `top` as the start point:

```
current_design top
  deltacheck_start -name top.clk1
```

If you do not specify any `deltacheck_start` constraints, the `DeltaDelay01` rule infers all clock source ports/nets/pins and uses them as the start points.

The `-value` argument is an optional argument used for clock pin balancing. When you do not specify the `-value` argument, the `DeltaDelay01` rule expects the delta delay value for all flip-flops/latches triggered by the specified clock to be the same. When you specify the `-value` argument, the `DeltaDelay01` rule expects the delta delay value for all flip-flops/latches triggered by the specified clock to be the same and equal to the specified value.

The following example specifies net `clk1` of design unit `top` as the start point and expects all flip-flops/latches to have a delta delay value of 1:

```
current_design top
  deltacheck_start -name top.clk1 -value 1
```

When you want all flip-flops/latches in the design to have the same delta delay value, specify the (same) expected value with the `-value` argument for all specified clocks.

Rules

The `deltacheck_start` constraint is used by the following rule:

SpyGlass CDC Solution

DeltaDelay01

deltacheck_stop_instance

Purpose

Specifies the instance where the DeltaDelay01 rule should stop further traversal along the clock tree when the clock pin of the specified instance is hit.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `deltacheck_stop_instance` constraint is as follows:

```
current_design <du-name>
  deltacheck_stop_instance
    -name <inst-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the stop point instance

-name <inst-name>

The hierarchical name of the stop point instance.

You can specify multiple stop point instances using multiple `deltacheck_stop_instance` constraints.

NOTE: For VHDL design units, instance names are case-insensitive.

Examples

The following example specifies instance I21 as the stop point instance under the design unit top:

```
current_design top
  deltacheck_stop_instance -name top.I21
```

Here, the DeltaDelay01 rule stops further traversal along the clock tree when the clock pin of instance top.I21 is hit.

The clock traversal automatically stops at instances of a stopped design unit (stopped using the `set_option stop <du-name>` command in the project file or the equivalent in the GUI). Therefore, you need not specify such design unit instances with the `deltacheck_stop_instance` constraint.

Rules

The `deltacheck_stop_instance` constraint is used by the following rule:

SpyGlass CDC Solution
DeltaDelay01

deltacheck_stop_module

Purpose

Specifies the design unit where the DeltaDelay01 rule should stop further traversal along the clock tree when the clock pin of an instance of the specified design unit is hit.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `deltacheck_stop_module` constraint is

as follows:

```
current_design <du-name>  
  deltadelay_stop_module  
    -name <stp-du-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in *<entity-name>.<arch-name>* format (for VHDL designs) under which you are specifying the stop point design unit.

-name <stp-du-name>

The name of the stop point design unit.

You can specify multiple stop point design units using multiple `deltacheck_stop_module` constraints.

NOTE: For VHDL design units, names of stop point design units are case-insensitive.

Examples

The following example specifies design unit `m1` as the stop point design unit under the design unit `top`:

```
current_design top  
  deltacheck_stop_module -name m1
```

Here, the `DeltaDelay01` rule stops further traversal along the clock tree when the clock pin of an instance of design unit `m1` is hit.

The clock traversal automatically stops at instances of a stopped design unit (stopped using the `set_option stop <du-name>` command in the project file or the equivalent option in the Atrienta Console GUI).

Therefore, you need not specify such design units with the `deltacheck_stop_module` constraint.

Rules

The `deltacheck_stop_module` constraint is used by the following rule:

SpyGlass CDC Solution

DeltaDelay01

deltacheck_stop_signal

Purpose

Specifies the design points (ports, pins, or nets) where the DeltaDelay01 rule should stop further traversal along the clock tree.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `deltacheck_stop_signal` constraint is as follows:

```
current_design <du-name>
  deltacheck_stop_signal
    -name <sig-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the stop point.

-name <sig-name>

The hierarchical name of the end point port, pin, or net.

You can specify multiple stop points using multiple `deltacheck_stop_signal` constraints.

NOTE: For VHDL design units, signal name is case-insensitive.

Examples

The following example specifies pin d1 of instance I31 in design unit top as the stop point:

```
current_design top
  deltacheck_stop_signal -name top.I31.d1
```

Here, the DeltaDelay01 rule stops further traversal along the clock tree when the pin top.I1.d1 is hit.

The clock traversal automatically stops at the design points (ports, pins, or nets) that belong to instances of a stopped design unit (stopped using the `set_option stop <du-name>` command in the project file or the equivalent option in the Atrenta Console GUI). Therefore, you need not specify such design points with the `deltacheck_stop_signal` constraint.

Rules

The `deltacheck_stop_signal` constraint is used by the following rule:

SpyGlass CDC Solution
DeltaDelay01

design_map_info

Purpose

The `design_map_info` constraint can be used to improve RTL to RTL simulation file annotation as well as to improve netlist to RTL simulation file annotation. You can specify either of the following while using the `design_map_info` constraint:

- The `match_point_rtl` and the `match_point_gate` arguments to map netlist to RTL simulation data (`sim_file` is optional).

The `design_map_info` constraint is used for mapping RTL matchpoints with the corresponding gate-level matchpoints. Currently,

only sequential cell outputs are considered for matching. Use the `match_point_rtl` argument to specify the name of sequential cell output used in the RTL. To specify the name of the corresponding sequential cell output used in the gate-level netlist, use the `match_point_gate` argument.

Please note that it is recommended to use matchpoints instead of net names to match RTL and gate-level netlist.

- The `gatenet` and the `rtlnet` arguments to map netlist to RTL simulation data

In addition, the `design_map_info` constraint is also used to specify a mapping between the RTL net names and gate-level net names. You can use the `design_map_info` constraint to manually map a simulation hierarchy with a design hierarchy. SpyGlass PE considers both the simulation and design hierarchies and tries to create a mapping between the simulation hierarchy and the design hierarchy. However, if there is a mismatch in name or a change in the hierarchy, the signals present in this hierarchy are not mapped automatically.

- The `sim_hier_name`, `design_hier_name` arguments to map RTL to RTL simulation data (`sim_file` is optional)

The `design_map_info` constraint is also used for mapping the hierarchical instance name in the RTL with the corresponding hierarchical name in the simulation file. Use the `design_hier_name` argument to specify the hierarchical instance name in the RTL and the `sim_hier_name` argument to specify the corresponding hierarchical name in the simulation file. If you have multiple simulation files for the same design and if the mapping is applicable to only one simulation file, use the `sim_file` argument to specify the simulation file. Note that the `sim_file` argument is optional.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `set_design_map_info`.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `design_map_info` constraint is as follows:

```
current_design <du-name>  
  design_map_info  
    -match_point_rtl <matchpoints_rtl>  
    -match_point_gate <matchpoints_gate-level>
```

OR

```
current_design <du-name>  
  design_map_info  
    -rtl原因 <rtl-netname>  
    -gatenet <gate-netname>
```

OR

```
design_map_info  
  -sim_hier_name <simulation_hierarchy>  
  -design_hier_name <design_hierarchy>  
  -sim_file <simulation_file>
```

Arguments

<matchpoints_rtl>

Specifies the hierarchical name of sequential cell output as used in the RTL.

<matchpoints_gate-level>

Specifies the hierarchical name of sequential cell output as used in the gate-level netlist.

<rtl-netname>

Specifies the hierarchical RTL net name.

<gate-netname>

Specifies the gate-level net name corresponding to the RTL net name specified in the `<rtl-netname>` field.

<simulation_hierarchy>

Specifies the name of the simulation hierarchy to be mapped to the design hierarchy specified in the design_hierarchy argument.

<design_hierarchy>

Specifies the name of the design hierarchy to be mapped to the simulation hierarchy specified in the simulation_hierarchy argument.

< simulation_file >

Specifies the name of the simulation file. Relative or absolute paths can be used to specify the file name. The name specified in this argument should be any of the names specified in the -file argument of the activity_data constraint.

If this name is not provided, SpyGlass searches for the simulation hierarchy in all the simulation files specified by the activity_data constraint. In case no match is found, SpyGlass exits with a FATAL error. If multiple matches are found, the mapping is applied to all the matching files.

Rules

The design_map_info constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	PEPWR20	PEPWR21
PEPWR22	PEPWR23	PEPWR24	PEPWR25
PEPWR28			

dft_report_fault_list

Purpose

Use this constraint to define the modules and/or instances for which the fault list needs to be generated.

Product

SpyGlass DFT Solution

Syntax

The syntax to specify the `dft_report_fault_list` constraint is as follows:

```
dft_report_fault_list
    [-module < module_name> ]
    [-instance < hier_instance_name> ]
```

Arguments

<module_name>

Specifies the module names for which fault data is included in the fault report.

<hier_instance_name>

Specifies the hierarchical path of the instances for which fault data is included in the fault report.

Rules

All rules of the SpyGlass DFT solution use this constraint.

dft_stitching_exception

Purpose

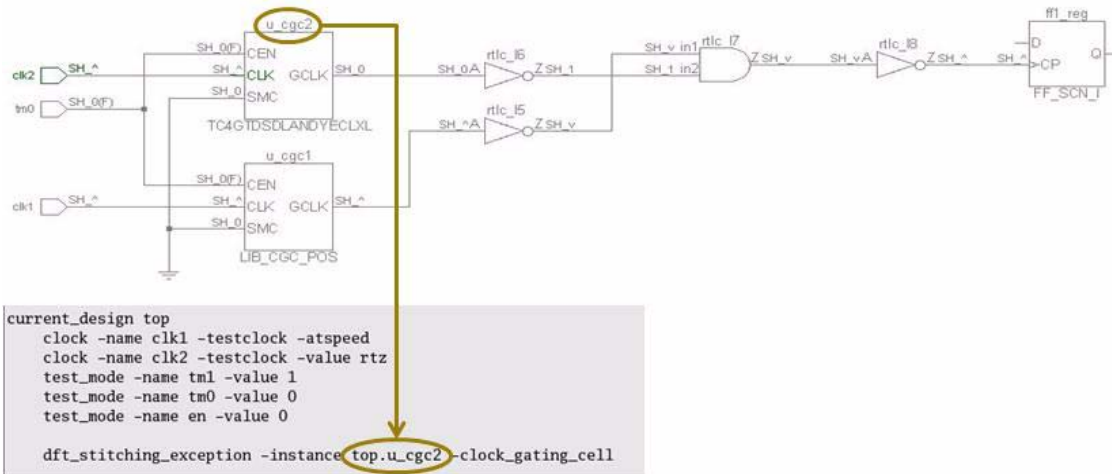
If the design state is `pre_scan_stitched`, a CGC that has a `test_enable` tied to ground (VSS), allows the test clock to propagate through it during shift mode. In this case it is assumed that after scan stitching (`-dftDesignState=post_scan_stitched`), the grounded `test_enable` pin of the CGC will be properly connected with the `SCAN_MODE` signal.

For the SoC designs containing both `pre_scan_stitched` and `post_scan_stitched` IP blocks, set the `dftDesignState` parameter to `pre_scan_stitched` and use the `dft_stitching_exception` constraint to designate the design areas as `post_scan_stitched`. For such exceptions, the CGC is treated as a regular CGC and if its `test_enable_pin` is tied to ground,

SpyGlass Design Constraints

the test clock does not propagate through it during shift mode.

For example, consider the following figure where one CGC output allows clock to propagate but other CGC output is used to drive a constant value.



Product

SpyGlass DFT solution

Syntax

The syntax to specify the dft_stitching_exception constraint is as follows:

```

dft_stitching_exception
  -clock_gating_cell
  [-module < module_name> ]
  [-instance < hier_instance_name> ]
  
```

Arguments

-clock_gating_cell

Specifies the need to apply the stitching exception to the clock gating cells that are part of the modules/instances specified using the -module/-instance arguments.

<module_name>

Specifies the name of the design unit, CGC module, or the module containing all the ignored CGCs.

<hier_instance_name>

Specifies the hierarchical path of:

- The instance containing all the ignored CGCs
- The CGC instance

Rules

All rules of the SpyGlass DFT and SpyGlass DFT DSM solution use this constraint.

disable_timing

Purpose

The `disable_timing` constraint is used to disable the timing arcs between the specified pins of a library cell.

Product

All SpyGlass products

Syntax

```
disable_timing
  -name <name>
  -from <from-pin>
  -to <to-pin>
```

Arguments

-name <name>

Name of the library cell or an instance.

-from <from-pin>

Name of the pin from which the timing arc should be disabled. The pin should be a single-bit or multi-bit pin.

-to <to-pin>

Name of the pin till which the timing arc starting from the *-from <from-pin>* should be disabled. The pin should be a single-bit or multi-bit pin.

Rules

All rules of all the SpyGlass products.

disallow_modification_type**Purpose**

The `disallow_modification_type` constraint is used to specify the type of modification to be disallowed in the RTL generated by the AutoFix feature.

Product

SpyGlass Power Reduce

Syntax

```
disallow_modification_type
[ -port_insertion ]
[ -generate_unroll ]
[ -max_hier <int> ]
[ -bus_split ]
```

Arguments**-port_insertion**

Specifies that the AutoFix feature should not perform RTL modification causing insertion of a port.

-generate_unroll

Specifies that the AutoFix feature should not perform RTL modification resulting in unroll of generate statements.

-max_hier <int>

Specifies the maximum number of hierarchies so that any signal that requires routing in the modified RTL beyond the specified hierarchy is not auto fixed.

-bus_split

Specifies that the AutoFix feature should not perform RTL modification that requires bus split of destination register.

Examples**Example 1**

Consider the following constraint:

```
disallow_modification_type -port_insertion
```

In this example, the AutoFix feature does not perform RTL modification that results in the insertion of a port.

Example 2

Consider the following constraint:

```
disallow_modification_type -max_hier 3
```

In this example, any power reduction opportunity that requires routing of a signal through more than three hierarchical boundaries are not auto fixed.

Rules

The `disallow_modification_type` constraint is used by the following rules:

PEPWR20	PEPWR21	PEPWR22	PEPWR23
PEPWR24	PEPWR25	PEPWR28	PEPWR29
PESTR20	PESTR21	PESTR22	PESTR23
PESTR24	PESTR25	PESTR28	PESTR29

disallow_upf_command

Purpose

This command is used to allow/disallow specified UPF commands and their options.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `disallow_upf_command` constraint is as follows:

```
disallow_upf_command -name <command-name> -options <option-list>
```

Arguments

-name <command-name>

(Mandatory) Specifies the name of the UPF 1.0/2.0 command. The command name is checked against UPF 1.0/2.0 for sanity checking of the constraint in the *SGDC_lowpower119* rule.

-options <option-list>

(Optional) Specifies the arguments of the UPF command. The arguments specified in the `-options` argument are not supported and all other options of that command will be supported. If you provide `disallow_upf_command <command-name>` only, without specifying `-options` argument, then the whole command will be disallowed. The specified arguments are checked against UPF 1.0/2.0 for sanity checking of

the constraint in the *SGDC_lowpower119* rule.

Rules

The `disallow_upf_command` constraint is used by the following rule:

SpyGlass Power Verify Solution		
SGDC_lowpower119	UPFSEM_42	UPFSEM_43

domain

Purpose

The `domain` constraint is used to specify the clock domain information in the design. The wildcard support is provided for the `domain` constraint.

NOTE: *The domain constraint has been renamed to [clock_group](#). For backward compatibility, the domain constraint is currently still available.*

Product

SpyGlass Constraints solution

Syntax

The `domain` constraint is used in the following syntax:

```
current_design <du-name>
  domain -name <domain-name>
  [ -clock_pin <clk-pin-name-list> ]
  [ -clock <SDC-clk-name-list> ]
```

Arguments

<du-name>

The top-level module name (for Verilog designs) or the top-level entity name (for VHDL designs) or a synthesis partition name specified using the

block keyword.

-name <domain-name>

(Mandatory) Specifies the clock domain name. It can be any valid string.

-clock_pin <clk-pin-name-list>

Specifies a space-separated list of design clock pin names. If this option is used then all clocks specified on the clock pin will be considered in the same domain.

-clock <SDC-clk-name-list>

Specifies a space-separated list of SDC clock names.

If both options are specified, then the domain will include all the clocks specified with **-clock** option and all the clocks defined on the clock pin specified with the **-clock_pin** option.

Example

Let the clock domain information be provided as follows:

```
current_design top
  domain -name d1 -clock { C1 C2 }
  domain -name d2 -clock { C3 }
```

In the above example, clocks C1 and C2 are synchronous, and clocks C1 and C3, C2 and C3 are asynchronous. You cannot specify the same clock in two different domains. If you do so, the DomainSanityCheck rule reports a FATAL message.

If the same domain is specified in multiple lines of an SGDC file, the union of clocks (specified in multiple lines) will be in the same domain. For example:

```
domain -name d1 -clock {C1 C2}
domain -name d1 -clock {C4}
```

Here, clocks C1, C2, C4 will be assumed as synchronous.

All create clocks and their derived clocks will be assumed to be synchronous if you have specified the domain among create clocks and their derived clocks. You can even specify clock and its derived rules in a different domain. For example:

```
C1 -> GC1 -> GC2  where GC2 is derived from GC1 and so on.  
// C1, GC1, GC2 will be assumed in same domain  
domain -name d1 -clock C1  
// User specified clock GC2 to d2 domain  
domain -name d2 -clock GC2
```

No domainsanitycheck violation will be thrown if you specify create clock and its derived clock in a different domain.

NOTE: All SGDC-dependent rules will be impacted.

Rules

The domain constraint is used by the following rules:

SpyGlass Constraints Solution			
Clk_Gen05	Clk_Lat03	Clk_Uncert03	False_Path07
False_Path08	Domain_SGDC_Consis		

domain_inputs

Purpose

Specifies expected values of various inputs of a power domain under a power-down condition.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was powerdomaininputs.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the domain_inputs constraint is as follows:

```
current_design <du-name>  
    domain_inputs
```

```

-name <pd-condition-name>
[-value <input-value-pairs>]
[-default <0 | 1 | 2> ]

```

Arguments

<du-name>

Name of the design unit under which you are specifying the power domain input values under power-down condition.

-name <pd-condition-name>

Name of the power-down condition specified by using the [voltage_domain](#) constraint (the -inputs argument).

-value <input-value-pairs>

Space-separated list of input-value pairs where each pair is a space-separated pair of power domain input signal name and its expected power-down value (0 for active low and 1 for active high).

-default <0 | 1 | 2>

Specifies the expected steady state value of power domain inputs or outputs. Specify 0 for active low, 1 for active high, and 2 for hold specified value.

Examples

You can specify vector inputs in one specification if all bits of the vector input are expected to attain the same value under power-down condition, as in the following example:

```
domain_inputs -name V3_PD_COND -value in[3:0] 1
```

Rules

The `domain_inputs` constraint is used by the following rule:

SpyGlass Power Verify Solution

LPSVM47

domain_outputs

Purpose

The `domain_outputs` constraint is used to specify the values of various signals under the steady state condition specified in [voltage_domain](#) constraint.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `powerdomainoutputs`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `domain_outputs` constraint is as follows:

```
current_design <du-name>
  domain_outputs
    -name <ss-condition-name>
    [ -default <0 | 1 | 2> ]
    [ -dest ]
    [ -value <signal-value-pairs> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the power domain output values under steady state.

-name <ss-condition-name>

Name of the steady state condition specified using the [voltage_domain](#) constraint.

-default <0 | 1 | 2>

Specifies the expected steady state value for all signals except those

specified with the `-value` argument. Specify 0 for active low, 1 for active high, and 2 for hold earlier value.

-dest

If `-dest` argument is specified, then isolation cells are placed in destination domain hierarchy. Otherwise, isolation cells are placed in top hierarchy.

-value <signal-value-pairs>

Space-separated list of signal-value pairs *<signal-value-pairs>* where each pair is a space-separated pair of power domain output signals and its expected steady state value (0 for active low, 1 for active high, and 2 for hold earlier value).

Rules

The `domain_outputs` constraint is used by the following rule:

SpyGlass Power Verify Solution
LPSVM23

domain_signal

Purpose

Specifies the power-up/power-down signals.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `pdsignal`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `domain_signal` constraint is as follows:

```
current_design <du-name>
```

```
domain_signal
  -name <sig-name>
  -value <0 | 1>
  [ -clocks <clk-name-list> ]
  [ -seqsignals <sig-name-list> ]
  [ -seqvalue <value-seq> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the power-up/power-down signals.

-name <sig-name>

Name of the power-up/power-down signal.

-clocks <clk-name-list>

Specifies a space-separated name list of clock sources associated with power-up/power-down.

-value <0 | 1>

Specifies whether the signal is active high (1) or active low (0).

-seqsignals <sig-name-list>

Specifies a space-separated name list of signals to be checked for value sequence specified by the `-seqvalue` argument.

-seqvalue <value-seq>

Specifies a space-separated list of value sequence for signals specified with the `-seqsignals` argument. The width of the sequence must be the same as the number of signals specified with the `-seqsignals` argument and should describe the signal values at all transitions of all signals during the power-up/power-down. In addition, use only 0 and 1 to described the sequence.

Examples

Consider the following example:

```
domain_signal -name N1 -value 0 -clocks C1
             -seqsignals N2 N3 -seqvalue 00 01 10 11
```

Here, there are two nets N2 and N3 and the expected sequence is a space-separated list of pairs of two values (0/1) indicating the expected value for each signal, respectively. Thus, both nets are expected to have a value of 0 at power-down. Then, the net N3 is expected to change from 0 to 1 at a later time while net N2 is expected to remain at 0. Then, the net N2 is expected to change from 0 to 1 and net N3 is expected to change from 1 to 0 later. Lastly, both nets are expected to attain a value of 1 before power-up. If these nets do not follow the specified sequence, the LPSVM43 rule of the *SpyGlass Power Verify* solution reports a violation when the first mismatch is encountered.

Rules

The domain_signal constraint is used by the following rule:

SpyGlass Power Verify Solution
LPSVM43

dont_touch

Purpose

The dont_touch constraint specifies the modules/nets that are not considered for AutoFix.

Product

SpyGlass DFT solution

Syntax

The syntax of the dont_touch constraint is as follows:

```
dont_touch
  -module <module-name>
  -net <net-name>
  -auto_fix
```

NOTE: The dont_touch constraint supports wildcard characters.

Arguments

The dont_touch constraint has the following arguments:

-module <module-name>

The name of the module that should not be considered for AutoFix.

-net <net-name>

Name of the net (complete hierarchical net name) that should not be considered for AutoFix.

Example

```
dont_touch -module m1 -auto_fix
dont_touch -net net1 -auto_fix
```

NOTE: The -module and -net arguments should not be used together. In addition, you should always specify the -auto_fix argument.

Rules

The dont_touch constraint is used by the following rules:

SpyGlass DFT Solution			
Async_07	Clock_11	Latch_05	Latch_08
TA_06			

expect_frequency

Purpose

The `expect_frequency` constraint is used to specify that a certain frequency value is expected at a node in the design. This constraint uses MHz as the frequency unit.

NOTE: `require_frequency` is an alias for the `expect_frequency` constraint used by the SpyGlass DFT DSM solution.

There could be a hard macro with no clockshaper inside the hard macro. Therefore, a macro top-level pin directly connects to flip-flops. In such a case, the macro designer has the `expect_frequency` constraint at this macro-top-level pin.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `expectFrequency`.

Product

SpyGlass DFT DSM solution

Syntax

The syntax to specify the `expect_frequency` constraint is as follows:

```
current_design <du-name>
  expect_frequency
    [-name <path> ]
    [-except <path to cellport | top_level_pin_name> ]
    [-type <DO_objlist> ]
    [-except_type <ExceptDO_obj_type> ]
    -freqList <freq-symbol-list>
    [-multiplier <freq multiplier> ]
    [-constraint_message_tag <value>]
```

Arguments

<du-name>

(Optional) Name of the design unit under which you are specifying the

instance hierarchies.

-name <path>

(Optional) Path to the node where the given frequency is expected, otherwise respective violation occurs.

-except <path to cellport | top_level_pin_name>

(Optional) Same as <path> but defines design nodes whose path need not be specified.

-type <DO_objlist>

(Optional) Same as <path> but it takes only macros as inputs.

-except_type <ExceptDO_obj_type>

(Optional) Same as <DO_obj_type> but it takes only macros as inputs.

-freqList <freq-symbol-list>

List of frequency symbols with which the test clock is associated. These can be actual numbers like 100, 200 or alphanumeric symbols like F1, f2, and so on.

-multiplier <freq multiplier>

(Optional) Specifies the frequency multiplier for all frequencies given through the -freqList argument.

If different multipliers are required, use multiple *expect_frequency* constraints for the same design node.

-constraint_message_tag <value>

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

Supported Macros

(Optional) To view the list of macros supported by the *expect_frequency* constraint, see [Supported Macros](#).

Examples

Example 1

Consider the following examples to specify expected frequencies (symbols/numeric) at nodes of interest:

```
expect_frequency -name u1.u7 -freqList F6
expect_frequency -name u2.u3.clk -freqlist 256
```

You can ensure the following:

- Node of interest in the design achieves 'expected' frequency.
- Node of interest in the design does not attain any other frequency.

Example 2

Consider the following example:

```
expect_frequency -name q1_reg.CP -freqList f0 f1 -multiplier
0.25
```

Above constraint implies that `q1_reg.CP` should get $f0 * 0.25$ and $f1 * 0.25$ frequencies only.

Example 3

Consider the following examples:

```
expect_frequency -name q1_reg.CP -freqList f0
-multiplier 0.25

expect_frequency -name q1_reg.CP -freqList f1 -multiplier 1.5
```

Above constraints imply that `q1_reg.CP` should get $f0 * 0.25$ and $f1 * 1.5$ frequencies only.

Rules

The `expect_frequency` constraint is used by the following rule:

SpyGlass DFT DSM Solution

Atspeed_13

false_path

Purpose

The `false_path` constraint enables you to input false and multi-cycle paths that are to be excluded from at-speed testing.

NOTE: *You can convert the `set_false_path` SDC command to `false_path` SGDC command using the SDC to SGDC conversion. For details, refer to *Atrenta Console Reference Guide*.*

Product

SpyGlass DFT DSM solution

Syntax

```
false_path
    [-from <from_list>]
    [-through <through_list>]
    [-to <to_list>]
```

NOTE: *You should specify at least one of these options for the `false_path` constraint.*

Arguments

The `false_path` constraint has the following arguments:

-from <from_list>

Specifies a list of objects that act as the start point for the multi-cycle path. A valid start point is a clock, a primary input or inout port, a sequential cell, a clock pin of a sequential cell. If a clock is specified, all registers related to the clock are considered as start points.

-to <to_list>

Specifies a list of objects that act as endpoint for the multi-cycle path. A valid endpoint is a primary output or inout port, a sequential cell, a data pin of a sequential cell.

SpyGlass Design Constraints

-through <through_list>

Specifies a list of pins or nets that you want to disable.

NOTE: *The DFT DSM policy does not support the -through option.*

Rules

The `false_path` constraint is used by the following rules:

SpyGlass DFT DSM Solution			
Info_transitionCoverage	Atspeed_05	Atspeed_06	Diagnose_03
Info_transitionCoverageAudit			

fifo

Purpose

The `fifo` constraint provides a mechanism to provide FIFO information so that SpyGlass can perform complete recognition and verification of FIFOs.

Product

SpyGlass CDC solution

Syntax

The syntax of using the `fifo` keyword in a SpyGlass Design Constraints file is as follows:

```
current_design <du-name>
fifo
  [ -memory <memory_name> ]
  [ -rd_data <read_data> -wr_data <write_data> ]
  [ -rd_ptr <read_pointer> -wr_ptr <write_pointer> ]
```

Arguments

-memory <memory_name>

Allows you to provide the memory usage of the fifo. Memory can be a black box/library cell name, hierarchical net name, hierarchical instance name, or module name of the hierarchical instance containing memory.

-rd_data <read_data> / **-wr_data** <write_data>

Can be hierarchical nets or hierarchical instance pins. These should be specified as vector signals. Scalar signals are not considered. Both <read_data> and <write_data> options should be specified together.

You can use a combination of wildcard characters ('*' and '?') when specifying hierarchical net names.

-rd_ptr <read_ptr> / **-wr_ptr** <write_ptr>

Can be hierarchical nets or hierarchical instance pins. These should be vector signals. Scalar signals are not considered. Both <read_ptr> and <write_ptr> options should be specified together.

You can use a combination of wildcard characters ('*' and '?') when specifying hierarchical net names.

Rules

The `fifo` constraint is used by the following rules:

SpyGlass CDC Solution			
Clock_sync03a	Clock_sync03b	Clock_sync08	Clock_sync08a
Clock_sync09	Ac_fifo01	Ac_unsync01	Ac_unsync02
Ac_sync01	Ac_sync02		

force_no_scan

Purpose

The `force_no_scan` constraint is used to exclude black box, latches, and flip-flops from being declared scannable even if they so qualify.

The `force_no_scan` constraint may be used when there is no intention for scan insertion in a module or circuit or when sequential ATPG tools are planned (in conjunction with the `seq_atpg` constraint.).

Specifying this constraint impacts the reported scan flip-flop percentage. That is, if you specify this constraint, specified and inferred flip-flops are excluded from the denominator while computing the scannable flip-flop percentage.

NOTE: Prior to SpyGlass 5.4.0 release, the name of this constraint was `no_scan`

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `force_no_scan` constraint is as follows:

```
force_no_scan
-name <du-name> | <net-name> | <hier-inst> |
-clock_control <signal-name> |
-set_control <signal-name> |
-reset_control <signal-name>
-register_suffix <suffixes>
-module_suffix <suffixes>
[ -black_box ]
[ -latch ]
[ -flip-flop ]
```

NOTE: The `force_no_scan` constraint supports wildcard characters. Using wildcards, expression is expanded only within the hierarchy.

Arguments

The `force_no_scan` constraint has the following arguments:

-name <du-name>

The name of the design unit from which scan should be excluded.

You can specify design units that are single flip-flops or design units where one or more flip-flops are described besides other logic. Then, all flip-flops in the specified design unit are excluded from scan.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are considered.

You can specify a single design unit name or a space-separated list of design unit names.

-name <net-name>

The name of a net that is connected to the output pin of a flip-flop.

Then, the corresponding flip-flop is excluded from scan.

You can specify a simple net name or a hierarchical net name. The net specified as simple net name is searched at the top-level.

You can specify a single net name or a space-separated list of net names.

-name <hier-inst>

The name of the hierarchical instance names that should be excluded from scan.

-clock_control <signal-name>

Flip-flops whose clock control pin is driven by the specified signal are excluded from the scan.

-set_control <signal_name>

Flip-flops whose set control pin is driven by the specified signal are excluded from the scan.

-reset_control <signal_name>

NOTE: *The traversal involved in this is structural only.*

-register_suffix <suffixes>

Space-separated list of suffixes to be specified as noscan. The

`-register_suffix` argument should not be used along with other arguments of the `force_no_scan` constraint, that is, `-name`, `-clock_control`, `-set_control`, or `-reset_control`.

If the value of the `dft_treat_suffix_as_pattern` parameter is set to on, the `register_suffix` value is used as a pattern to be matched with the register name. The pattern may be present anywhere in the register name, excluding the path.

If the value of the `dft_check_path_name_for_register_suffix` parameter is on, the value of the `-register_suffix` field will be matched with the register name along with the path in which the register is present.

`-module_suffix <suffixes>`

Define this field to use suffix based pattern match for all module names.

If the value of the `dft_treat_suffix_as_pattern` parameter is on, the value of the `-module_suffix` field will be matched with the module name along with the path in which the module is present.

`-black_box`

Marks only black boxes as no scan.

`-latch`

Marks only latches as no scan.

`-flip_flop`

Marks only flip-flops as no scan.

NOTE: *If you do not specify either `-blackbox`, `-latch`, or `-flip-flop` options, then all black boxes, latches, and flip-flops are marked as no scan.*

Examples

You can use the `force_no_scan` constraint in the following ways:

Specifying only the design unit names with the `-name` argument

By specifying a design unit name using the `-name` argument only, all instances of this design unit are considered non-scannable. The following

`force_no_scan` constraint indicates that all flip-flops within all instances of `modName1` will not be considered scannable:

```
force_no_scan -name modName1
```

Specifying only the net names with the `-name` argument

By specifying a net name using the `-name` argument only, the corresponding flip-flop will be considered non-scannable. The following `force_no_scan` constraint indicates that the flip-flop whose output pin is connected to net `reg_123` (at the top-level) will not be considered scannable:

```
force_no_scan -name reg_123
```

Specifying only the hierarchical instance names with the `-name` argument

By specifying a hierarchical instance name using the `-name` argument only, all the flip-flops inside the given hierarchy will be considered as non-scannable. The following `force_no_scan` constraint indicates that the flip-flop that lies inside the hierarchy `top.inst1` will not be considered scannable:

```
force_no_scan -name top.inst1
```

NOTE: *The `Scan_08` and `Scan_16` rules ignore flip-flops specified in a `force_no_scan` constraint. The effects of a `force_no_scan` constraint specification are visible with the `TA_01` or `TA_02` testability analysis rules.*

NOTE: *The `force_no_scan` constraint overrides the `force_scan` constraint.*

Specifying list of suffixes using the `-register_suffix` argument

Consider the following example:

```
R1 (register 1) name: top.u_ctrl.u2.u1.ff1_ctrl
R2 (register 2) name: top.u_ctrl.u2.u1.ff1_state
R3 (register 3) name: top.u_core.u2.u1.ff1_state_ctrl
R4 (register 4) name: top.u_ctrl_state.u2.u1.ff1_ctrl_state
```

Now, consider the following `force_no_scan` descriptions:

```
force_no_scan -register_suffix ctrl
force_no_scan -register_suffix state
```

The following table lists the results when combination of values are used for the `dft_treat_suffix_as_pattern` and `dft_check_path_name_for_register_suffix` parameters:

TABLE 6 Pattern Matching for the `-register_suffix` argument

Value of <code>dft_treat_suffix_as_pattern</code>	Value of <code>dft_check_path_name_for_register_suffix</code>	Value of <code>-register_suffix</code>	Matched Registers
off	off	ctrl	R1, R3
		state	R2, R4
off	on	ctrl	R1, R2, R3
		state	R2, R4
on	off	ctrl	R1, R3, R4
		state	R2, R3, R4
on	on	ctrl	R1, R2, R3, R4
		state	R2, R3, R4

Rules

The `force_no_scan` constraint is used by the following rules:

SpyGlass DFT Solution
All rules

force_ta

Purpose

The `force_ta` constraint specifies control and/or observe values for ports/pins/nets.

Use the `force_ta` constraint when it is known that various ports/pins/nets are to be connected or tied off at the next higher level of assembly in order to evaluate the effect of coverage within the current design.

If some pins of this design cannot be fully controlled or observed when instantiated in a larger design and the `force_ta` constraint is not used, the coverage for this design is reported as unrealistically high.

When `test_mode`, `force_ta`, or `test_point` constraints are specified on the same node, following is the priority among different constraints:

- `Test_mode`
- User-specified specific `force_ta` / `test_point`
- Effect of `dft_treat_primary_inputs_as_x_source` and `dft_treat_primary_outputs_as_unobservable` parameters

For example, if `test_mode 1` and `test_point control` are applied on the same node then the `test_mode` constraint will be considered.

Also, if the `test_mode`, `force_ta`, or `test_point` constraints are found in the fanout of each other, following is the priority among different constraints:

- The constraint in the fanout gets the priority
- Fanin effect is blocked by the specified / resolved constraint on the node

For example, consider that `test_mode 1` is applied on the input of buffer and `test_point control` is applied on the output of the same buffer. In this case, input will have simulation value 1 and nyn controllability but output will have yyn controllability and no simulation value.

NOTE: *The `force_ta` constraint impacts the observability for the fan-in logic-cone of the net where `force_ta` is applied. However, it does not impact the terminal. Similarly, it impacts the controllability of the fan-out logic cone of the net where `force_ta` is applied. However, it does not impact the terminal. See [Figure 45](#) and [Figure 46](#) for more information on the application of the `force_ta` constraint.*

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax for `force_ta` constraint is as follows:

```
force_ta
  -name <net-name>
    -control <ctrl-value> | -observe <obs-value>
```

Arguments

`-name <net-name>`

The port/pin/net on which control or observe value is being set.
 You can specify a single port/pin/net name or a space-separated list of port/pin/net names.
 The port names can be simple names while pin names and net names should be hierarchical names.

`-control <ctrl-value>`

The control value for a port/pin/net.
 The possible control values are as follows:

<ctrl-value>	Purpose
nnn	Not controllable
ynn	Controllable to 0
nyn	Controllable to 1
nny	Controllable to Z
yyn	Controllable to 0 and 1
yny	Controllable to 0 and Z
nyy	Controllable to 1 and Z
yyy	Controllable to 0, 1, and Z

-observe <obs-value>

The observe values for a port/pin/net.

The possible observe values are as follows:

<cvalue> or <obs-value>	Purpose
n	Not observable
y	Observable

Examples

Consider the following example:

```
force_ta -name pin1[3:2] -value ynn
force_ta -name outPin -observe n
force_ta -name inPin[1:2] -control nyy
```

Here, the port `pin1[3:2]` is being made controllable to just '0', port `outPin` is being made unobservable, and port `inPin[1:2]` is being made controllable to '1' and 'Z'.

Rules

The `force_ta` constraint is used by the following rules:

SpyGlass DFT Solution			
Bist_04	Info_uncontrollable	Info_unobservable	Info_undetectedCause
Info_path	Info_coverage	Info_untestable	Coverage_audit
TA_01	TA_02	TA_06	
SpyGlass DFT DSM Solution			
All rules			

force_probability

Purpose

The `force_probability` constraint is used to specify the probability of controllability and observability of a design node.

Product

SpyGlass DFT DSM solution

Syntax

The syntax of the `force_probability` is as follows:

```
force_probability
  -name <node_name>
  [ -control_one <ctrl1_prob> ]
  [ -control_zero <ctrl0_prob> ]
  [ -observe <obs_prob> ]
```

Arguments

-name <node_name>

Name of the pin, port, or net for which probability is specified.

-control_one <ctrl1_prob>

(Optional) Probability of controllability to 1 for the design node.

-control_zero <ctrl0_prob>

(Optional) Probability of controllability to 0 for the design node.

-observe <obs_prob>

(Optional) Probability of observability for the design node.

Rules

The `force_probability` constraint is used by the

Info_random_resistance rule of the SpyGlass DFT DSM solution:

Examples

The following examples show the usage of the `force_probability` constraint:

Example 1

```
force_probability -name in1 -control_zero 0.4 -control_one  
0.6
```

Example 2

```
force_probability -name Q -observe 0.05
```

Example 3

```
force_probability -name top.inst.in1 -control_zero 0.4  
-control_one 0.6 -observe 0.5
```


formal_analysis_filter

Purpose

The `formal_analysis_filter` constraint is used to specify the modules or hierarchies on which formal analysis should be ignored or performed.

Product

SpyGlass Auto Verify

Syntax

The syntax of the `formal_analysis_filter` constraint is as follows:

```
formal_analysis_filter
  -module_names <design-unit-names>
[ -rules <rule-names> ]
[ -hierarchical <yes | no> ]
[ -analyze <yes | no> ]
```

Arguments

-module_names <design-unit-names>

Specifies the design units on which formal analysis should be ignored or performed.

For Verilog design, a design unit name is a module name.

For VHDL design, a design unit name is an entity name (`<entity-name>`) or an architecture name (`<entity-name>.<architecture-name>`).

-rules <rule-names>

Specifies the SpyGlass Auto Verify rules on which this constraint is applicable.

By default, this constraint is applicable to all the SpyGlass Auto Verify rules.

-hierarchical <yes | no>

Set this argument is set to *yes* so that the nets present directly inside the specified module (*-module_names <design-unit-names>*) are only considered for matching with the *Start Nets* of a property.

By default, this argument is set to *yes* so that the nets present within the specified module (*-module_names <design-unit-names>*) and also its submodules are considered for matching with the *Start Nets* of a property.

-analyze <yes | no>

By default, this argument is set to *no* so that property analysis by formal engines is ignored when all the *Start Nets* of that property belong to the specified block (*-module_names <design-unit-names>*) or hierarchy (*-hierarchical <yes | no>*).

For example, consider the user-specified modules M1 and M2. In this case, by default:

- If the P1 property contains the start nets N1 (present in M1) and N2 (present in M2), P1 is ignored.
- If the P2 property contains start net N1 (present in M1) and N2 (present in M3), P2 is not ignored.

Set this argument to *yes* to start property analysis by formal engines when any one of the start nets of that property belong to the specified block or hierarchy.

Start Nets

Start nets are the nets from where the property/assertion checking should start.

For SpyGlass Auto Verify RTL rules, such as *Av_deadcod01* and *Av_range01*, start nets belong to the RTL block in which a property is modeled.

For SpyGlass Auto Verify flat rules, such as *Av_bus01*, *Av_bus02*, and *Av_staticnet01*, the driver of the net on which the property is modeled is considered as the start net.

Rules

The `formal_analysis_filter` constraint is used by the following

rules:

SpyGlass Auto Verify

All implicit property rules

fsm

Purpose

The `fsm` constraint is used to:

- Modify or remove the FSMs, FSM states, and FSM transitions.
- Add FSMs, FSM states, and FSM transition.

Product

SpyGlass Auto Verify

Syntax

The syntax of the `fsm` constraint is as follows:

Usage 1

```
fsm
  -name <fsm-logical-name>
  [ -state_value <state-values> ]
  [ -from_state_value <from-state-values> ]
  [ -to_state_value <to-state-values> ]
  [ -append | -remove ]
```

Usage 2

```
fsm
  -module <module-name>
  -state_variables <FSM-state-variables> ]
  [ -state_value <state-values> ]
  [ -from_state_value <from-state-values> ]
  [ -to_state_value <to-state-values> ]
  [ -append | -remove ]
```

Arguments

-name <fsm-logical-name>

Specifies the name of the FSM to be added, modified, or deleted.

-module <module-name>

Specifies the module in which the FSM is encoded.

It can be a top-level module or a sub module/entity in the design.

-state_variables <FSM-state-variables>

Specifies a space-separated list of synthesized nets of a module.

Some examples are as follows:

- *-state_variables state[4:0]*
- *-state_variables state*
- *-state_variables state[4] state[3] state[2] state[1] state[0]*

Note that a space-separated list is considered as concatenation. Therefore, check for the order of values specified to this argument. Consider the following example:

```
fsm -module M1
  -state_variables curr_state0 curr_state1
  -state_value 00 01
```

The equivalent RTL for the above fsm constraint is as follows:

```
case (curr_state0, curr_state1)
  2'b00: ...
  2'b01: ...
endcase
```

-state_value <state-values>

Specifies a space-separated list of state values for the FSM.

The allowed values to this argument are same as the allowed values to the -value argument of the set_case_analysis constraint. For details, refer to the documentation of the set_case_analysis constraint.

Some examples of the values to this argument are as follows:

- 1010101010
- "d 5" or "D 5"
- "h AB" or "H AB"

NOTE: *When the width of a state variable mismatches with the state value, redundant bits are removed or non-specified bits in the state value are considered as zero.*

-from_state_value <state-values> -to_state_value <state-values>

The arguments together specify the transition of an FSM.

The format of the <state-value> arguments is same as described in [-state_value <state-values>](#).

If you specify a new state value for an FSM, which is not automatically-detected, or if you specify a state value that is not already specified to the [-state_value <state-values>](#) argument, SpyGlass adds that state value to the [-state_value <state-values>](#) argument.

-append | -remove

-append is the default argument over -remove.

Use -append to modify the RTL of the automatically-detected FSM or create a new FSM.

Use -remove to remove the specified FSM state, state values (and its corresponding transition), or transition from an FSM.

Examples

Example 1

The following constraint removes the FSM detected on s[4:0] from the M1 module:

```
fsm -module M1 -state_variables s[4:0] -remove"
```

Example 2

The following constraint removes the FSM state 0000 and its corresponding transition with the FSM state variable s[4:0] from the M1 module:

```
fsm -module M1 -state_variables s[4:0] -state_value 0000 -
```

remove

Example 3

Consider the following constraint:

```
fsm -module M1 -state_variables s[4:0] -from_state_value  
0000 -to_state_value 0001 -append
```

When you specify the above constraint, one of the following occurs based on a condition:

- **Condition:** When an FSM with the `s[4:0]` state variable is not detected in the M1 module
Result: SpyGlass adds a new FSM with the specified state values transition.
- **Condition:** When an FSM with the `s[4:0]` state variable is detected in the M1 module but either of states, 0000 or 0001, is not detected
Result: SpyGlass adds the specified state values and the new transition to the FSM.
- **Condition:** When an FSM with the `s[4:0]` state variable is detected in the M1 module along with both the states, 0000 and 0001
Result: SpyGlass adds the specified transition to the FSM.
- **Condition:** When an FSM with the `s[4:0]` state variable is detected in the M1 module but both the states, 0000 and 0001, and their transition is automatically-detected
Result: SpyGlass reports the *SGDC_fsm_Setup01* violation.

gating_cell

Purpose

Specifies the user-defined clock-gating cell.

By default, a 2 input gate on the clock line or an integrated clock-gating is detected as a clock-gating cell.

You should use the `gating_cell` constraint to specify complex clock-gating cells.

Key Points

- For a white box model, `gating_cell` constraint specification takes precedence over the synthesizable logic.
- If the `gating_cell` sgdc command is specified on a lib cell then its internal functionality is completely ignored and the cell is treated like a black box with the `gating_cell` constraint.
- For SpyGlass DFT or SpyGlass DFT DSM solution, the `gating_cell` constraint is not necessary when the white box model resembles a latch-based standard integrated clock gating library cell. This automatic inference is controlled by the `dft_infer_clock_gating_cell` parameter. For details of this parameter, please refer to SpyGlass DFT Rules Reference Guide or SpyGlass DFT DSM Rules Reference Guide.
- Prior to SpyGlass 4.3.0 release, the name of this constraint was `gatingcell`.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions,
SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax to specify the `gating_cell` constraint is as follows:

```
current_design <du-name>
  gating_cell
    -name <cell-name>
    [-clkInTerm <clk-in-pin>]
    [-clkoutTerm <clk-out-pin>]
    [-enTerm <enable-pin>]
    [-enValue <enable-value> ]
    [-testenTerm <test-enable-pin>]
    [-testenValue <test-enable-value>]
    [-cgcEdgeType <positive | negative>]
    [-obsTerm <obs-pin>]
```

NOTE: The `gating_cell` constraint supports wildcard characters.

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-clkInTerm <clk-in-pin>

Name of the clock input terminal, which is a single-bit pin.

-clkoutTerm <clk-out-pin>

Name of the output clock terminal, which can be either single-bit or multi-bit.

-enTerm <enable-pin>

Name of the input enable terminal through which a clock-gating cell can be enabled or disabled. This can be either a single-bit or a multi-bit pin. The size of `<enable_pin>` should match the size of `<clk_out_pin>`. Also, if you want to enable the i^{th} bit of the `<clk_out_pin>`, ensure that the i^{th} bit of the `<enable_pin>` is set. To understand the relationship between the multi-bit system-enable and clock-out pins, refer to [Figure 31](#).

NOTE: If `<enable-pin>` is not specified, it implies that a clock-gating cell is always enabled. Therefore, a clock reaching `<clk-in-name>` always passes through `<clk-out-name>`.

-enValue <enable-value>

NOTE: This option is not applicable for the SpyGlass Power Estimation and SpyGlass Power Reduction solutions.

Value on the enable pin that disables the gating cell and allows a clock reaching the input terminal to pass through the clock output terminal.

NOTE: If `<enable-pin>` is specified and `<value>` is not specified, it implies that the signal value is active high (value 1).

-testenTerm <test-enable-pin>

Name of the input enable terminal in the test mode. The pin through which a clock-gating cell can be enabled or disabled in the test mode.

-testenValue <test-enable-value>

NOTE: This option is not applicable for the SpyGlass Power Estimation and SpyGlass Power Reduction solutions.

Value on the enable pin in the test mode that disables the gating cell and allows a clock reaching the input terminal to pass through the clock output terminal.

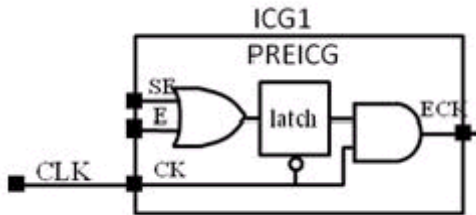
-cgcEdgeType <positive | negative>

Used to define the edge of the clock gating cell. By default, edge type is positive. You can also set the value of this argument as negative.

Positive Edge

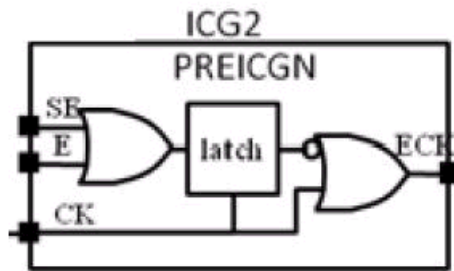
Positive edge type signifies that latch-enable pin gets the inverted clock (clock input to gating_cell).

The following figure illustrates a positive edge type:

**Negative Edge**

Negative edge type signifies that latch-enable pin gets the clock (clock input to gating_cell) without inversion.

The following figure illustrates a negative edge type:



-obsTerm <obs-pin>

Name of the observation pin using which we observe the enable signal of the cell.

Consider the following example:

```
module dpICG(I, SE, Z, EN);
    parameter width = 1;
    output [width-1:0] Z;
    input [width-1:0] EN;
    input I;
    input SE;
```

```
endmodule
```

The following figure shows the schematic for the above code:

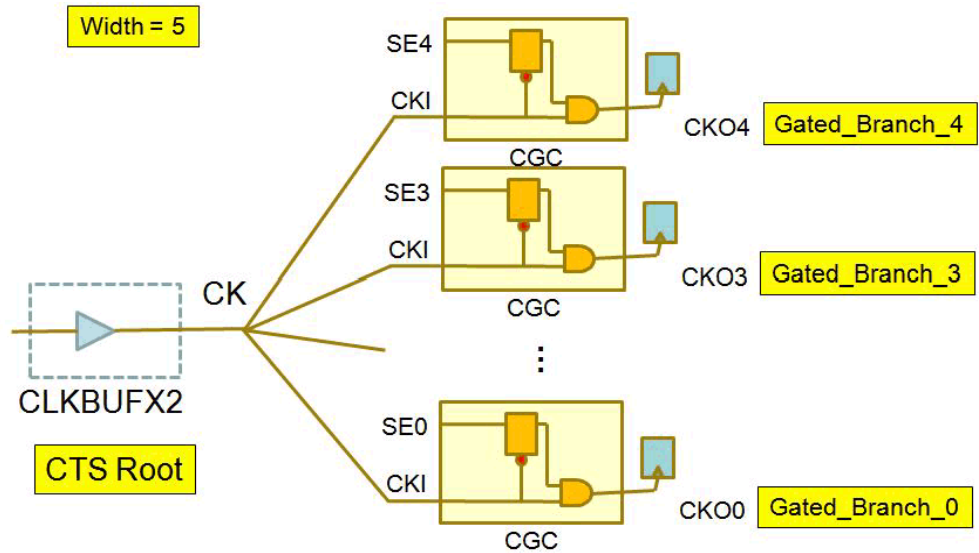


FIGURE 31. Relation Between Multi-Bit System-Enable Pin and Clock-Out Pin

The above code and figure explain the relationship between the multi-bit system-enable and clock-out pins.

Rules

The `gating_cell` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	PESTR03	PESTR05
PESTR06	PESTR07	PESTR08	PESTR09
PESTR10	PESTR11	PESTR12	PESTR13
poweraudit			
SpyGlass DFT Solution			
All rules			
SpyGlass DFT DSM Solution			
All rules			

gating_cell_enable

Purpose

The `gating_cell_enable` is used to specify the gating cell enable signal.

NOTE: Prior to SpyGlass 4.4 release, the name of this constraint was `gatingcell_enable`.

Product

SpyGlass DFT DSM solution

Syntax

The syntax to specify the `gating_cell_enable` constraint is as follows:

```
gating_cell_enable
  -name <signal-name>
  -type phased_clock_enable
  -pipeline_depth <depth> |
  -pipeline_depth_range <min max>
```

Arguments

-name <signal-name>

Signifies root level input ports, which are used to drive the test enable pin of a clock-gating cell.

-type phased_clock_enable

Specifies that the phased clocks would be used in the LSSD design style.

-pipeline_depth <depth>

Signifies the pipeline stage that is expected on `gating_cell_enable` signal. You can specify a positive integer value as an input.

NOTE: Specify either `-pipeline_depth` or `pipeline_depth_range` argument. Do not use both the arguments together.

-pipeline_depth_range <min max>

Specifies the range of pipeline stages expected on gating_cell_enable signal.

NOTE: *Specify either -pipeline_depth or pipeline_depth_range argument. Do not use both the arguments together.*

Rules

The gating_cell_enable constraint is used by the following rule:

SpyGlass DFT DSM Solution
CG_04

generated_clock

Purpose

The generated_clock constraint is used to specify the clock that traverses from the output (hierarchical pin or net) of a sequential element.

By default, when a clock reaches a sequential element, it propagates beyond that sequential element irrespective of whether the output of the element is reaching a clock pin.

Use this constraint to stop propagation of such clocks beyond sequential elements such that the clock specified by this constraint is propagated beyond the sequential element. For details, see [Examples](#).

If the generated_clock constraint is defined at the output of a combo logic that is placed just after first sequential element receiving the master clock, the generated_clock constraint is honored. For details, see [Example 3](#).

This constraint is read only if the enable_generated_clocks parameter is set to yes.

NOTE: *During SDC-to-SGDC translation, the create_generated_clocks SDC command is saved as the generated_clock constraint.*

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `generated_clock` constraint is as follows:

```
current_design <du-name>
generated_clock -name <clk-obj-name>
    -source <source-obj-name>
    [ -tag <tag-name> ]
    [ -divide_by <divide-factor> ]
    [ -multiply_by <mult-factor> ]
    [ -master_clock <source-clock-tag-name> ]
    [ -add ]
    [ all ]
```

Arguments

-name <clk-obj-name>

Specifies the name of the clock pin or net present on the output of a sequential element so that the generated clock propagates from that pin or net.

See [Example 1](#).

-source <source-obj-name>

Specifies the clock reaching the sequential element so that propagation of this clock beyond the sequential element is stopped when the `generated_clock` constraint is specified.

See [Example 1](#).

-tag <tag-name>

Specifies the tag of the generated clock.

See [Example 1](#).

-divide_by <divide-factor>

Specifies the frequency division factor.

For example, if this argument is 2, the generated clock period is twice as long as the master clock period.

-multiply_by <mult-factor>

Specifies the frequency multiplication factor.

For example, if this argument is 3, the generated clock period is one-third as long as the master clock period.

-master_clock <source-clock-tag-name>

Specifies the tag name of the clock that is the master of the source clock specified by the *-source <source-obj-name>* argument.

See [Example 1](#).

-add

Specify this argument to add multiple `generate_clock` constraints on a single object.

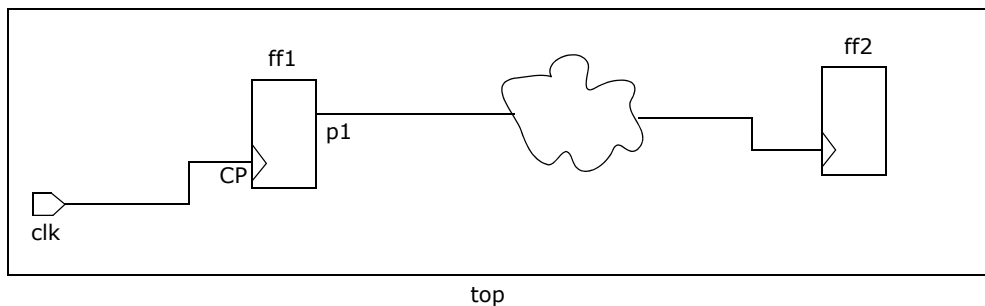
See [Example 2](#).

Bus Support

SpyGlass CDC honors all bits of a bus in the `generated_clock` SGDC constraint.

Examples**Example 1**

Consider the following figure:



```
// SGDC File:
clock -name top.clk -period 10 -domain d1 -tag T1
```

FIGURE 32. Example 1 - generated clock

In the above scenario, by default, the `clk` clock propagates beyond the `ff1` flip-flop.

To stop the propagation of `clk` beyond `ff1` and propagate another clock from the output `p1` of `ff1`, perform the following steps:

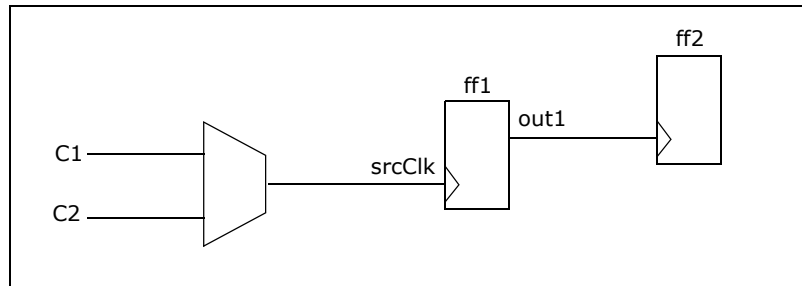
1. Set the `enable_generated_clocks` parameter to `yes`.
2. Specify the following constraint to generate a clock on the output `p1` of the `ff1` flip-flop, where the source clock `CP` is derived from the master clock `top.clk` (tag `T1`):

```
generated_clock -name top.ff1.p1 -source top.ff1.CP
-tag GT1 -divide_by 2 -master_clock T1
```

After performing the above steps, a generated clock propagates from the `p1` pin of the `ff1` flip-flop.

Example 2

Consider the following figure:



// SGDC File:

```

clock -name C1 -period 10 -domain d1 -tag T1
clock -name C2 -period 15 -domain d2 -tag T2

```

FIGURE 33. Example 2 - generated clock

In the above scenario, multiple master clocks converge on the source clock `srcClk`. In this case, to stop propagation of `srcClk` beyond `ff1` and enable a generated clock traverse from `out1` of `ff1`, perform the following steps:

1. Set the `enable_generated_clocks` parameter to `yes`.
2. Specify the following constraints:

```

generated_clock -name out1 -source srcClk -master_clock C1
-tag T1 -divide_by 2

```

```

generated_clock -name out1 -source srcClk -master_clock C2
-tag T2 -add -divide_by 2

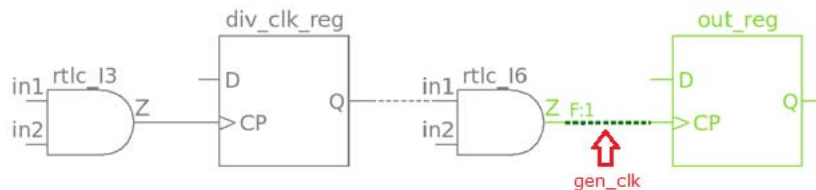
```

You should specify a `generated_clock` constraint with respect to each master clock of the source clock.

Alternatively, select one master clock by defining a mode and then specify one `generated_clock` constraint with respect to the selected master clock.

Example 3

Consider the following figure:

**//SGDC File:**

```
generated_clock -name test.gen_clk -source test.div_clk_reg.CP
-master_clock T1 -divide_by 2 -tag T4
```

FIGURE 34. Example 3 - generated clock

In the above example, if the `generated_clock` constraint is defined on the `gen_clk` net with source as `div_clk_reg.CP`, SpyGlass CDC honors this constraint and the generated clock is created on the `gen_clk` net.

gray_signals

Purpose

The `gray_signals` constraint is used to specify the signals that should be gray encoded.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `gray_signals` constraint is as follows:

```
gray_signals -name <signal-name-list>
```

Arguments

-name <signal-name-list>

Specifies a space-separated list of signals, such as hierarchical net names, hierarchical terminals, and ports.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

Rules

The `gray_signals` constraint is used by the following rules:

SpyGlass CDC Solution		
Ac_conv05	SGDC_gray_signals01	SGDC_gray_signals02
SGDC_gray_signals03		

ignore_clock_gating

Purpose

The `ignore_clock_gating` constraint is used to disable generation of clock-gating logic for:

- Enabled flip-flops driven by the specified clock nets.
- Enabled flip-flops present within the specified modules.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `ignore_clock_gating` constraint is as follows:

```
ignore_clock_gating
  -clock <clock-nets> | -module <module-list>
```

Arguments

`-clock <clock-nets>`

List of clock nets so that clock-gating logic is not generated for the enabled flip-flops driven by these clock nets. The type of clock nets supported are:

- Port clocks
- Clocks driven by black boxes
- Clocks specified using the `clock` SGDC constraint

`-module <module-list>`

List of modules so that clock-gating logic is not generated for the enabled flip-flops present within these modules.

Examples

Consider the `top` design that has:

- The `clk1` and `clk2` clocks.
- The `middle1` and `middle2` modules.

For the above design, consider you specify the following constraints:

```
ignore_clock_gating -clock "top.clk1"
ignore_clock_gating -module "middle1"
```

On specifying the above constraints, SpyGlass will not use ICGC for enabled flops present either in `middle` or driven by the `top.clk1` clock. It uses the ICGC only for the enabled flip-flops inside `middle2` and driven by the `top.clk2` clock.

Rules

The `ignore_clock_gating` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions				
PEPWR01	PEPWR02	PEPWR03	PEPWR14	PEPWR20
PEPWR21	PEPWR22	PEPWR23	PEPWR24	PEPWR25

PEPWR28	PESTR03	PESTR06	PESTR08	PESTR12
PESTR13	PESTR20	PESTR21	PESTR22	PESTR23
PESTR24	PESTR25	PESTR28	poweraudit	

ignore_crossing

Purpose

The `ignore_crossing` constraint is used to specify power domain to voltage domain crossings and power domain to power domain crossings that should be ignored by the LPSVM08, LPSVM09, LPSVM10, LPSVM23, and LPSVM47 rules of the SpyGlass Power Verify solution.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `ignorepdcrossing`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `ignore_crossing` constraint is as follows:

```
current_design <du-name>
  ignore_crossing
    -from <src-pd-name>
    -to <dest-pd-name> | <vd-name>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the crossings to be ignored

-from <src-pd-name> | -to <dest-pd-name>

Name of the power domain (source, destination)

<vd-name>

Name of the voltage domain (only destination)

Rules

The ignore_crossing constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM08	LPSVM09	LPSVM10	LPSVM22
LPSVM23	LPSVM47		

ignore_supply_pin

Purpose

The `ignore_supply_pin` constraint is used to specify specific pins of a cell on which the LPPLIB06 or LPPLIB15 rules should not report a violation.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `ignore_supply_pin` constraint is as follows:

```
current_design <du-name>
  ignore_supply_pin
    -cell <cell-name>
    -pin <pin-name-list>
```

Arguments

<du-name>

Name of the design unit

-cell <cell-name>

Name of a cell. This field supports wildcards.

-pin <pin-name-list>

Name of specific pins of a cell. This field supports wildcards.

Rules

The `ignore_supply_pin` constraint is used by the following rules:

SpyGlass Power Verify Solution	
LPPLIB06	LPPLIB15

illegal_constraint_message_tag

Purpose

The *illegal_constraint_message_tag* constraint checks the *constraint_message_tag* expression matches the expression for the design nodes. If specific combination of tags are present in the design for a particular node then it reports violations.

Product

SpyGlass DFT Product

Syntax

The syntax to specify the *illegal_constraint_message_tag* constraint is as follows:

```
illegal_constraint_message_tag
  [-name <nodename>]
  [-except <except_nodename>]
  [-except_type <exceptDo-nodename>]
  [-type <DO_nodename>]
  [-constraint_message_tag_expression
  <constraint_message_tag_expression>]
```

Argument

[-name] <nodename>

Specifies the name of the top-module port, or any internal net or terminal or leaf instance for which the specified tag expression must be satisfied.

[-except <except_nodename>]

Specifies the name of the top-module port, any internal net, terminal, or leaf instance name, which needs to be excluded from rule checking.

-except_type <exceptDo-nodename>

Specifies the name of macro for which the specified tag expression must be satisfied.

-type <DO_nodename>

Specifies the name of the macro, which needs to be excluded from rule checking.

-constraint_message_tag_expression**<constraint_message_tag_expression>**

Signifies the message tag expression specified using logical '||' and '&&' operator and their combinations and :PASS and :FAIL values of tags. You can also use braces ('(',')') when specifying message tag expression.

Examples

Consider the following example:

```
require_path -from "top.cgc_1.clkout" -to_type
FLIP_FLOP_CLOCK LATCH_ENABLE
-constraint_message_tag CGC_CHECK_1

require_path -from "top.cgc_2.clkout" -to_type
FLIP_FLOP_CLOCK LATCH_ENABLE -constraint_message_tag
CGC_CHECK_2

illegal_constraint_message_tag -type ICG -
constraint_message_tag_expression "CGC_CHECK_1:PASS ||
CGC_CHECK_2:FAIL"
```

In the above example, the `illegal_constraint_message_tag` reports violation if `CGC_CHECK_1` does not have any violation or `CGC_CHECK_2` have any violation.

illegal_path

Purpose

The `illegal_path` constraint is used to define nodes between which path should not exist.

Product

SpyGlass DFT solution

Syntax

The syntax of the `illegal_path` constraint is as follows:

```
illegal_path
[ -from <from_node_list> ] [ -to <to_node_list> ]
[ -except_from <except_from_list> ]
[ -except_to <except_to_list> ]
[ -from_type <from_type_list> ]
[ -from_one_of <fromoneof_pinlist> ]
[ -from_one_of_type <fromoneof_DO_pinlist> ]
[ -except_from_type <except_from_type_list> ]
[ -to_type <to_type_list> ]
[ -except_to_type <except_to_type_list> ]
[ -to_one_of <tooneof_pinlist> ]
[ -to_one_of_type <tooneof_DO_pinlist> ]
[ -tag <tag_name> | -use_shift | -use_capture |
  -use_captureATspeed ]
[ -path_type <buffered | sensitized | sensitizable> ]
[ -sequential_depth <value> ]/
[-constraint_message_tag <value>]
[-report_failure_as_info]
[-min_to_paths <value>]
[-max_to_paths <value>]
[ -min_from_paths ]
[ -max_from_paths ]
```

Guidelines for Using Arguments

The following combination of arguments are not allowed for the `illegal_path` constraint:

- The `-from_one_of` argument can not be used with the `-to_one_of` argument
- The `-min_from_paths` argument cannot be used with the `-max_from_paths` argument
- The `-min_to_paths` argument cannot be used with `-max_to_paths` argument

Also, The `-min_to_paths`, `max_to_paths`, `min_from_paths`, and `max_from_paths` arguments should have the corresponding `-from/from_type/from_one_of/ from_one_of_type` arguments, as well as, the `-to/to_one_of/to_type /to_one_of_type` arguments.

Arguments

`-from <from_node_list>`

(Optional) This is a list of top-module port names, internal net names, or terminal names from which path should not exist to nodes specified through `<to node list>` and `<to type list>`, if both `<to type list>` and `<to node list>` are not given then this implies that nodes given in `<from node list>` should not be driving any leaf cell/top module port.

`-to <to_node_list>`

(Optional) This is a list of top-module port names, internal net names, or terminal names to which path should not exist from nodes specified through `<from node list>` and `<from type list>`, if both `<from type list>` and `<from node list>` are not given then this implies that nodes given in `<to node list>` should not be driven by any leaf cell/top module port.

`-except_from <except_from_list>`

(Optional) Nodes specified in this list are effectively removed from list of nodes specified in `<from node list>`

-except_to <except_to_list>

(Optional) Nodes specified in this list are effectively removed from list of nodes specified in <to node list>

-from_type <from_type_list>

(Optional) Same as <from node list> but it takes only macros as inputs.

-from_one_of <fromoneof_pinlist>

(Optional) Specifies that no error message is reported, if there is at least one success case among the specified nodes.

-from_one_of_type <fromoneof_DO_pinlist>

(Optional) Same as -from_one_of <fromoneof_pinlist> but it takes only macros as inputs.

-except_from_type <except_from_type_list>

(Optional) This is a list of macros, nodes specified through this list are effectively removed from list of nodes specified through <from type list>.

-to_type <to_type_list>

(Optional) Same as <to node list> but it takes only macros as inputs.

-to_one_of <tooneof_pinlist>

(Optional) Specifies that no error message is reported, if there is at least one success case among the specified nodes.

-to_one_of_type <tooneof_DO_pinlist>

(Optional) Same as -to_one_of <tooneof_pinlist> but it takes only macros as inputs.

-except_to_type <except_to_type_list>

(Optional) This is a list of macros, nodes specified through this list are effectively removed from list of nodes specified through <to type list>.

-tag <tag name> | -use_shift | -use_capture | -use_captureATspeed

(Optional) These arguments are used to specify simulation condition under

which path is searched between from nodes and to nodes. If none of them are specified then path is searched after simulating all power-ground connections. Please note that only one of these arguments can be used to specify simulation condition in the `illegal_path` constraint.

`<tag name>` is a condition name that has been previously defined by using the `define_tag` constraint.

If `-use_shift`, `-use_capture` or `-use_captureATspeed` is specified then shift, capture or captureATspeed mode is simulated respectively.

`-path_type <buffered | sensitized | sensitizable>`

The `-path_type` argument accepts only predefined list of values: `buffered`, `sensitized` and `sensitizable`. The default value of this qualifier is `sensitizable`. This argument determines the type of path that is searched between from and to nodes.

`-sequential_depth <value>`

Specifies the number of sequential elements between end points specified on an illegal path. This means that the *illegal_path* check will go through the specified number of sequential elements. You can specify an integer value as an input to this argument.

It is recommended to use this argument when the path type is set to `sensitizable`. For other path_types, that is, `direct`, `buffered`, and `sensitized`, a sequential element is a stop-point. This is so because path through sequential cell is neither buffered nor sensitized path.

`-constraint_message_tag <value>`

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

NOTE: *This argument accepts only alpha-numeric characters and underscore.*

Supported Macros

To view the list of macros supported by the `require_strict_path` constraint, see [Supported Macros](#).

`-report_failure_as_info/-report_failures_as_info`

Reports all the failures as info severity message.

-min_to_paths <value>

(Optional) Specifies minimum number of expected successful paths. You can not specify this argument with `-from_one_of` and `-from_one_of_type` arguments.

-max_to_paths <value>

(Optional) Specifies maximum number of expected successful paths.

The following are the rules for using this argument:

- If you are using both the `-min_to_paths` and `-max_to_paths` arguments, then the value of the `-max_to_paths` argument should be greater than the `-min_to_paths` argument.
- You can not specify this argument with `-from_one_of` and `-from_one_of_type` arguments.

-min_from_paths <value>

(Optional) Specifies minimum number of expected successful paths. You can not specify this argument with `-from_one_of` and `-from_one_of_type` arguments.

-max_from_paths <value>

(Optional) Specifies maximum number of expected successful paths. The following are the rules for using this argument:

- If you are using both the `-min_from_paths` and `-max_from_paths` arguments, then the value of the `-max_from_paths` argument should be greater than the `-min_from_paths` argument.
- You can not specify this argument with `-from_one_of` and `-from_one_of_type` arguments.

Rules

The `illegal_path` constraint is used by the following rules:

SpyGlass Connectivity Verify solution

Soc_09

SpyGlass DFT Solution

Conn_09

Examples

Example 1

```
illegal_path -from top.clk1 -to_type module1:FLIP_FLOP_CLOCK
-use_shift -path_type sensitized
```

The above example implies that there should be no sensitized path from `top.clk1` to clock pin of any flip-flop inside instances of `module1` in shift mode.

Example 2

```
illegal_path -from_type top.inst1:FLIP_FLOP_OUT -to_type
top.inst2:FLIP_FLOP_DATA -tag tag1 -path_type sensitizable
```

The above example implies that there should be no sensitizable path from output pin of any flip-flop inside `test.inst1` to data pin of any flip-flop inside `top.inst2` in tag `tag1`.

Example 3

```
illegal_path -from top.ip1
```

The above example implies that `top.ip1` should not be driving any leaf cell or top module port.

Example 4

```
illegal_path -to top.op1
```

The above example implies that `top.op1` should not be driven by any leaf cell or top module port.

Example 5

```
illegal_path -from top.ip1 -to_type FLIP_FLOP_DATA
```

```
-except_to_type test.inst1:FLIP_FLOP_DATA -use_shift
```

The above example implies that there should be no sensitizable path from top.ip1 to data pin of any flip-flop outside test.inst1 in shift mode.

illegal_value

Purpose

The `illegal_value` constraint checks for the presence of illegal value on a certain node when the circuit has been simulated using the condition specified by the `-tag` argument.

When `illegal_value` is used twice for the same node, spyglass interprets as neither 0 nor 1.

Product

SpyGlass DFT solution

Syntax

The syntax for the `illegal_value` constraint is as follows:

```
illegal_value
  [-name <nodename> ]
  [ -except <except_nodename> ]
  [ -type <DO_nodename> ]
  [ -except_type <exceptDO_nodename> ]
  [ -value <value> ]
  [ -value_type <type> ]
  [ -tag <condname> | -use_shift |
    -use_capture | -use_captureATspeed ]
  [ -matchNBits <num> ]
  [-constraint_message_tag <value>]
  [-report_failure_as_info]
```

Arguments

-name <nodename>

(Optional) The name can be a top-module port, or any internal net name, or terminal name. More than one pin name can be specified, and it is effectively read as a concise description of as many individual value checks.

NOTE: *Specify either -name or -type argument.*

-except <except_nodename>

(Optional) Same as *<nodename>* but defines design nodes that are not to be used as name.

-type <DO_nodename>

Same as *<nodename>* but it takes only macros as inputs.

NOTE: Specify either *-name* or *-type* argument.

To view the list of macros supported by the *illegal_value* constraint, see [Supported Macros](#).

-except_type <exceptDO_nodename>

Same as *<DO_nodename>* but it takes only macros as inputs.

-value <value>

The value is a logic value string of 0, 1, X, Z, 1_or_0, and 0_or_1. A single-bit value means check at end of complete simulation. The X value is treated as do-not-compare. A multi-bit value means check on cycle-by-cycle simulation basis. For specification of a vector value, SGDC multi-bit specification format (same as used for the *test_mode* constraint) should be used.

You can specify repeat sequences for the *illegal_value* constraint.

For fields that require repeat sequence, you can specify the values as *<I*S>*. Here, *S* is any string that does not contain the *<*, *>*, and *** characters. However, *S* can contain another *<I*S>* expression. *I* is an integer that is always interpreted as a decimal value. The expression *<I*S>* means that the sequence *S* will be repeated *I* number of times.

value_type <type>

Specify one the following values:

- **active:** Implies 1 for non-inverting clock-pin and 0 for inverting clock-pin.
- **inactive:** Implies 0 for non-inverting clock-pin and 1 for inverting clock-pin.

NOTE: Specify either *-value* or *-value_type* argument. Otherwise, the *illegal_value* constraint is ignored for analysis.

To view the list of macros supported by the `-value_type` field of the `require_value` and `illegal_value` constraints, see [List of Macros Supported by the `require\_value` and `illegal\_value` Constraints](#).

- `FLIP_FLOP_RESET`
- `SCAN_FLIP_FLOP_RESET`
- `LATCH_RESET`
- `FLIP_FLOP_SET`
- `SCAN_FLIP_FLOP_SET`
- `LATCH_SET`
- `FLIP_FLOP_ENABLE`
- `SCAN_FLIP_FLOP_ENABLE`
- `LATCH_ENABLE`
- `FLIP_FLOP_CLOCK`
- `SCAN_FLIP_FLOP_CLOCK`

`-constraint_message_tag <value>`

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

NOTE: *This argument accepts only alpha-numeric characters and underscore.*

`-tag <condname>`

(Optional) A condition previously defined by using the [define_tag](#) constraint. It describes a stimulation condition.

Note that only one condition name can be defined in a `illegal_value` specification. However, simulation for a given condition name simulates all pin-value specifications simultaneously. The built-in power-ground connections are also simulated in this process.

`-use_shift` | `-use_capture` | `-use_captureATspeed`

For any of these modifiers, `illegal_value` simulates test mode of that particular mode.

If `-use_shift`, `-use_capture`, or `-use_captureATspeed` argument is specified, the constraint simulates all, shift, capture, or

captureAtspeed test_mode constraints, respectively.

NOTE: *If more than one of the -tag, -use_shift, -use_capture, or -use_captureAtspeed arguments is specified, an error condition occurs. You should specify exactly one of these modifiers with illegal_value constraint.*

-matchNBits <num>

(Optional) Specifies that only the <num> number of least significant bits are to be considered. If <num> is greater than <value> (specified with -value argument), the latter is padded with X to match the former's width.

-report_failure_as_info/-report_failures_as_info

Reports all the failures as info severity message.

Examples

Consider the following examples:

Example1

```
illegal_value -name abc -value "<5*10>"
```

The above example will be expanded as follows:

```
illegal_value -name abc -value 1010101010
```

Example2

```
illegal_value -name abc -value "11<5*10>010"
```

The above example will be expanded as follows:

```
illegal_value -name abc -value 111010101010010
```

Example3

```
illegal_value -name abc -value "<50*11<5*10>>010"
```

The above example will be expanded as follows:

```
illegal_value -name abc -value 111010101010...(repeated 50
times followed by 010)
```

You can also set a variable using the command setvar to obtain the above result as follows:

```
setvar x 11<5*10>
illegal_value -name abc -value "<50*${x}>010"
```

The above example will be expanded as follows:

```
illegal_value -name abc -value 111010101010...(repeated 50
times followed by 010)
```

NOTE: *Tagging for nesting is not allowed. For example, the following `illegal_value` statements are not allowed:*

```
illegal_value -name sub_seq -value <5*01>
illegal_value -name main_seq -value <100*sub_seq>
```

However, you can achieve the same result by using the `setvar` command.

Example 5

```
illegal_value -tag s1 -name top.U1.U2.SEF
-value 1010 -matchNBits 2
```

Example 6

Consider the following example:

```
-illegal_value -name p1 -value 0X110_or_110
```

The above constraint specification means:

- first bit must not be 0
- second bit is don't care
- third and fourth bits must not be 1
- fifth should be neither 0 nor 1
- sixth should not be 1
- seventh bit should not be 0

Rules

The `illegal_value` constraint is used by the following rule:

SpyGlass Connectivity Verify Solution

Soc_10

SpyGlass DFT Solution

Conn_10

initialize_for_bist

Currently, the `initialize_for_bist` constraint is merged with [test_mode](#) constraint and can be used as an argument to the `test_mode` constraint.

initstate

Purpose

The `initstate` command is used to specify the initial state sequence for the design.

You can specify the initial state sequence in a Tcl file or have the product read it from a VCD file.

NOTE: Use the [simulation_data](#) constraint instead of the `initstate` constraint because the `initstate` constraint may be deprecated in the future.

Product

SpyGlass TXV solution

NOTE: The `initstate` command is not required for combinational analysis.

Syntax

The syntax of the `initstate` constraint is as follow:

```
current_design <du-name>
  initstate
```

```

-type <tcl | vcd>
-file <file-name>
[ -mode <mode-name> ]
[ -time <value> ]
[ -scopename <block-name> ]
[ -modulename <module-name> ]

```

Arguments

<du-name>

Name of the design unit under which you are specifying the initial state sequence.

-type <tcl | vcd>

The `-type` argument specifies the input file type as `tcl` for Tcl file or `vcd` for VCD file.

-file <file-name>

The `-file` argument specifies the Tcl file or the VCD file.

-mode <mode-name>

The `-mode` argument is optional and is used to specify the applicable mode (one of the mode values specified with the `-mode` argument of the `sdc_data` constraint.) If you do not specify the `-mode` argument, the SpyGlass TXV solution assumes that the specified initial state sequence is applicable for all modes.

-time <value>

The `-time` argument is optional and is used to read the initial state by using a timestamp.

-scopename <block-name>

The `-scopename` argument is used if the given VCD file is for a full chip but you want to perform verification for a sub-block module. The

hierarchical path of the block module's instantiation (in VCD) must be specified to the `initstate` constraint.

For example, consider a VCD file with a top module `top` and a sub module `block2` that lies within another block `block1`. Therefore, while performing the verification of `block2`, the following must be defined to initialize the `block2` module from the VCD file generated for the full chip design:

```
current_design block2
initstate -type vcd -scopename
"top.block1_inst.block2_inst" -file chip.vcd
```

NOTE: *For sequential analysis, it is recommended that you either provide sufficient reset and set_case_analysis constraints so that the SpyGlass TXV solution automatically reads the initial state sequence or specify the `initstate` constraint. If the number of sequential elements initialized is high, the quality of the result produced by the SpyGlass TXV solution is good.*

-modulename <module-name>

The `-modulename` argument is used specify the top module.

Examples

Consider the example below:

```
initstate -type vcd -modulename A -scopename B -file
design.vcd
```

In the above example, A is the name of the top module and B is the sub block module name. Here, the top module name A will replace the sub block module name B and initial state sequence is read for the sub block module.

An example of the Tcl File is as follows:

```
set sig1 0xffffffff
incr a
for {set i 1} {$i<=4} {incr i} {
    force xyz [incr a 4]
    simulate 2 -clk clk1
```


SpyGlass Design Constraints

```

}
force sig2 [incr a]
simulate 2

```

Rules

The `initstate` constraint is used by the following rules:

SpyGlass TXV Solution
All rules

input

Purpose

The `input` constraint is used to specify clock domain at input ports.

Product

SpyGlass CDC solution

Syntax

The syntax of using the `input` keyword in a SpyGlass Design Constraints file is as follows:

```

current_design <du-name>
  clock -name <clk-name> -domain <domain-name>
  input -name <input-name-list>
        -clock <src-clk-name>

```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs)

-name <clk-name>

The clock signal name.

-domain <domain-name>

The clock domain name.

-name <input-name-list>

Port names in *<input-name-list>* can be scalar ports, bus ports, or wildcard names (matching against all top-level ports of appropriate type).

If you specify the same input port in multiple `input` constraint specifications, SpyGlass considers the last input constraint specification. Consider the following example in which the same input port, `in1`, is specified in two different input constraint specifications:

```
input -name in1 -clock clk2
input -name in1 -clock clk4
```

In this example, SpyGlass considers the last `input` constraint specification, which is input `in1` clocked by clock `clk4`.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

-clock <src-clk-name>

For *SpyGlass CDC* solution, this argument is the name of the effective source clock of a primary input port or a virtual clock specified by the *-tag <logical-clock-name>* argument of the *clock* constraint. For virtual clock specification, see [Example 2](#).

For *SpyGlass Auto Verify* solution, this argument is the name of the clock constraining the specified primary input ports.

NOTE: Do not specify clock domain names to this argument.

Examples

Example 1

Consider the following `input` constraint specification:

```
input -name in1 -clock clk1
```

The above specification matches input port with the name `in1`

If you want to check synchronization for paths starting from input ports, you need to provide information about the source clocks associated with those ports. The `input` constraint is used for this purpose.

Consider the following example:

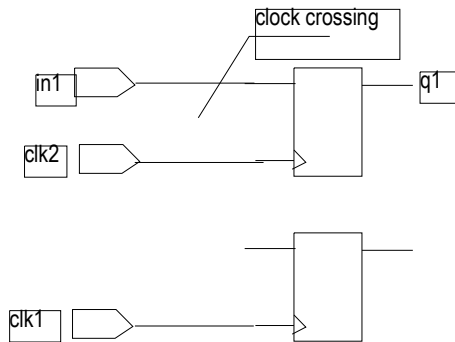


FIGURE 35. Clock Crossing

Assume that the input port `in1` is in the clock domain (say `D1`) of clock `clk1` and is connected to the data pin of flip-flop `top.q1` that is triggered by clock `clk2`. Therefore, there is clock domain crossing between primary input port `in1` (clock domain `D1`) and the flip-flop `top.q1` (clock domain `D2`). This situation can be specified as follows:

```
current_design top
  clock -name top.clk1 -domain D1
  clock -name top.clk2 -domain D2
  ...
  input -name in1 -clock top.clk1
  ...
```

NOTE: You must specify top-level simple (non-hierarchical) port names with the `-name` argument of the `input` and `output` constraints.

Example 2

You can specify input constraints using virtual clocks if the clock domain of the input port is not known or is not present in the current design. For

example, assume that the input port `in1` in [Example 1](#) is in the clock domain of the virtual clock `vclk`. This situation can be specified as follows:

```
current_design top
  clock -name top.clk2 -domain D2
  ...
  input -name in1 -clock vclk
  ...
```

Note that there is no clock constraint for the virtual clock `vclk`.

In this case, there is clock domain crossing between primary input port `in1` (clock domain `vclk`) and the flip-flop `top.q1`.

If input constraint is defined for multiple ports with the same virtual clock, these ports are considered to be in the same domain.

You can use the match many (*) and match one (?) wildcard characters with the `-name` argument of the `input` constraint by specifying the regular expression enclosed in double quotes ("").

However, do not specify a wildcard character if you want to apply the `input` constraint for the whole bus. For example, if you want to specify all bits of the vector input port, say `in[0:15]`, use the following constraint:

```
input -name "in" -clock clk
```

Rules

The `input` constraint is used by the following rules:

SpyGlass CDC Solution			
All clock synchronization rules except the Clock_sync05 rule	Reset_sync01	Reset_sync02	Reset_sync03
Reset_sync04			

input_drive_strength

Purpose

Specifies the maximum capacitance (in picofarads) that any input port can drive.

The maximum capacitance value is used by these rules for estimating the clock buffers and high fan-out buffers on input lines.

In case `input_drive_strength` constraint is not specified, the primary ports of the design will be considered to have infinite drive strength. The value of `input_drive_strength` is used to estimate the buffers for ports nets that have a high load.

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was `pi_drive_strength`.*

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `input_drive_strength` constraint is as follows:

```
current_design <top-du-name>
  input_drive_strength
    -value <value>
```

Arguments

<top-du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-value <value>

The maximum capacitance value.

You can specify only one `input_drive_strength` constraint for one top-level design unit.

Rules

The `input_drive_strength` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	poweraudit	

input_isocell

Purpose

Specifies the isolation cells at inputs of a power domain.

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was `inisocell`.*

NOTE: *Currently, a level shifter with an enable pin (clamp level shifter cell) is also treated as isolation cell.*

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `input_isocell` constraint is as follows:

```
current_design <du-name>
  input_isocell
    -names <cell-name-list>
    [ -pin <pin-name-list> ]
    [ -belongsto <pd-name> ]
    [ -input_pin <pin-name> ]
    [ -enable_pin <pin-name> ]
    [ -inhibit ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the input-side isolation cells.

-names <cell-name-list>

Space-separated name list of input-side isolation cells. You can use wildcard characters while specifying the cell names using the `-names` argument.

-pin <pin-name-list>

Space-separated name list of pins that are allowed to be connected outside of a power domain.

-belongsto <pd-name>

Name of the power domain for which you are specifying the input-side isolation cells.

-input_pin|-enable_pin <pin-name>

The `-input_pin` and `-enable_pin` arguments specify the input pin name and the enable pin name for the cells specified with other arguments. Then, these pin names are used in isolation cell instances for named association.

-inhibit

Specifies that the specified cells should not be used as input-side isolation cells.

Rules

The `input_isocell` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM48	LPSVM51	LPSVM52	LPSVM60

instance_trace

Purpose

Specifies the instances or design units for which the PESAE04 and PESAE06 rules generate the activity trace or PEPWR01 and PEPWR02 with n-cycles generate the power trace.

By default, if the `instance_trace` constraint is not specified, the respective rules dump the activity/power trace for the top design in the activity/power browser. If you specify this constraint, the `pe_cycle_power` report lists the peak power consumed and the time interval for the specified instances.

NOTE: For more information on the `pe_cycle_power` report, refer to the *SpyGlass Power Estimation and SpyGlass Power Reduction Rules Reference Guide*.

You should use the `instance_trace` constraint to specify additional hierarchies (using the `-instname` or `-meminst` argument) for which the activity/power trace needs to be created.

NOTE: A design hierarchy specified with `-meminst` argument is only used to create a power trace.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `set_instance_for_activity_trace`.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `instance_trace` constraint is as follows:

```
current_design <top-du-name>
  instance_trace
    -name <tag-name>
    [-instname <inst-name-list>]
    [-meminst <mem-inst-list>]
    [-clock <clk-name>]
```


Arguments

<top-du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

The Power Activity Product works only with designs having a single top-level design unit (checked by the PECHECK05 rule). You must specify the name of the top-level design unit.

NOTE: *The power/activity trace for the top-level design unit is always generated.*

-name <tag-name>

Symbolic name for activity/power trace to be generated. The <tag-name> is a symbolic alphanumeric string used as the activity trace label in the SpyGlass Activity/Power Browser.

Each symbolic name specified by the -name argument can correspond to multiple instance names. If there are multiple instance names specified, the symbolic name for each instance name is derived by appending an ID with the symbolic name specified.

See *Viewing Results in Activity Browser* section in the *SpyGlass Power Estimation and SpyGlass Power Reduction Rules Reference Guide* for more details.

-instname <inst-name-list>

Name of the instances for which you want the activity/power trace.

This argument supports wildcards. Therefore, you can use wildcard characters while specifying the instance names. Currently, only the asterisk (*) operator and question mark (?) are supported as wildcard characters. In addition, you must specify wildcard names enclosed in double quotes ("").

NOTE: *If you do not specify the -instname argument, respective rules generate the activity/power trace for the top-level design unit only.*

-meminst <mem-inst-list>

Memory instances or design hierarchies, for which you want the power trace. For the design hierarchies, the trace is done for all the memories

instantiated under the specified hierarchy.
You can also use wildcards for specifying the memory instances.
In case of multiple instances, a power trace is done for all memory instances in a single graph.

-clock <clk-name>

List of root clocks of the specified instance for which the graph will be populated by PESAE06 rule.

Rules

The `instance_trace` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PESAE04	PESAE06	PEPWR01	PEPWR02

ip_block

For SpyGlass CDC solution and SpyGlass Auto Verify Solution

Purpose

Your design may have IP blocks instantiated where you are only interested in clock domain crossing synchronization status at the interface to ensure that the IP block is hooked up correctly.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `IP_block`.

Product

SpyGlass CDC solution, SpyGlass Auto Verify solution

Syntax

Use the `ip_block` constraint to specify the IP blocks in your design in the following format:

`ip_block -name <du-name>`

Arguments

-name <du-name>

The module name (for Verilog designs) or the design unit name in <entity-name>.<arch-name> format (for VHDL designs). You can specify more than one IP Block design unit as a space-separated list.

Then, the clock synchronization checking rules will check for synchronization status of the clock crossing at the interface and ignore the clock crossings within the IP block.

NOTE: *Violations of any GuideWare rule of SpyGlass CDC solution are not reported for IP modules specified by the ip_block constraint.*

The following figure summarizes the above situation:

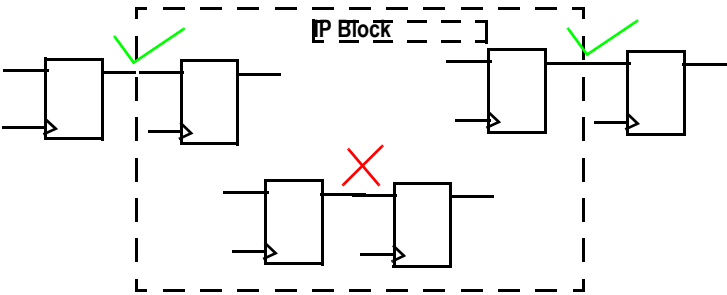


FIGURE 36. Synchronization Status Check

Rules

The ip_block constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_unsync01	Ac_unsync02	Clock_sync08	Clock_sync03a
Clock_sync03b	Ac_handshake01	Clock_sync08a	Clock_sync09
Ac_handshake02	Ac_cdc01a	Ac_cdc01b	Ac_cdc01c
Ac_cdc08	Propagate_Clocks	Ac_conv01	Ac_conv02
Ac_conv03	Ac_fifo01	Ac_sync02	Ac_sync01
SpyGlass Auto Verify Solution			
Av_bus01	Av_bus02	Av_deadcode01	Av_staticnet01

Av_case01	Av_case02	Av_bitstuck01	Av_fsm_analysis
Av_dontcare01	AV_setreset01	Av_staticreg01	Av_divide_by_zero

For SpyGlass DFT solution and SpyGlass DFT DSM solution

Purpose

Specifies the design units for which you want to generate the boundary information.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

Use the `ip_block` constraint to specify the IP blocks in your design in the following format:

```
ip_block -name <du-name>
        [-create_constraints]
        [-create_report]
        [-clock]
        [-mode]
        [-control]
        [-observe]
```

Arguments

-name <du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs). You can specify more than one IP Block design units as a space-separated list.

-create_constraints

Generates boundary information for a constraint file in an SGDC format.

SpyGlass Design Constraints

-create_report

Generates boundary information for a report file in a textual format.

-clock

Lists clock-specific boundary information for Shift mode, Capture mode, and at-speed mode in the generated report.

-mode

Lists test mode-specific boundary information for Shift mode, Capture mode, and at-speed mode in the generated report.

-control

Generates controllability-specific boundary information for the IP block in Capture mode.

-observe

Generates observable-specific boundary information for the IP block in Capture mode.

Rules

The `ip_block` constraint is used by the following rules:

SpyGlass DFT Solution
CreateDebugSGDC
SpyGlass DFT DSM Solution
Info_IP_Report

isolation_cell

Purpose

The `isolation_cell` constraint is used to specify the isolation cells in power domains.

NOTE: *Currently, a level shifter with an enable pin (clamp level shifter cell) is also treated as isolation cell.*

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was isocell.*

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `isolation_cell` constraint for the LPSVM08, LPSVM09, LPSVM10, LPSVM22, LPSVM31, LPSVM35, LPSVM48, and LPSVM51 rules of the *SpyGlass Power Verify* solution is as follow:

```
current_design <du-name>
  isolation_cell
    -names <name-list>
    [ -belongsto <dom-name> ]
```

The syntax to specify the `isolation_cell` constraint for the LPSVM23 rule of the *SpyGlass Power Verify* solution is as follows:

```
current_design <du-name>
  isolation_cell
    [ -iso_enable_val <0 | 1> ]
    [ -and_cell <cell-name> ]
    [ -or_cell <cell-name> ]
    [ -latch_cell <cell-name> ]
    [ -not_cell <cell-name> ]
    [ -and_cell_for_isosig <cell-name> ]
    [ -or_cell_for_isosig <cell-name> ]
    [ -input_pin <pin-name> ]
    [ -output_pin <pin-name> ]
    [ -enable_pin <pin-name> ]
    [ -config 1 | 2 | 3 | 4 ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the level shifters with isolation capability.

-names <name-list>

Space-separated name list of output-side isolation cells.

You can use wildcard characters while specifying the cell names using the `-names` argument. Currently, only the star (*) operator for zero or more times and question mark (?) for zero or one time are supported. In addition, you must specify wildcard names enclosed in double quotes ("").

-belongsto <dom-name>

Name of the power domain for which you are specifying the `<name-list>`.

If you do not specify the `-belongsto` argument, the specified cells are applicable for all power domains.

-iso_enable_val <0 | 1>

A value 0 or 1 of the `-iso_enable_val` argument specifies that the steady state of the power domain is active low or active high respectively.

-and_cell|-or_cell|-latch_cell <cell-name>

The cell name `<cell-name>` specified with the `-and_cell`, `-or_cell`, and `-latch_cell` arguments are the AND cell, OR cell, and Latch Cell to be used as isolation cells, respectively.

You should specify the `-and_cell`, `-or_cell`, and `-latch_cell` arguments if any of the power domain output signals has an expected steady state value of active low, active high, or hold, respectively.

-not_cell <cell-name>

The cell name `<cell-name>` specified with the `-not_cell` argument is the NOT cell to be used to change the polarity of the isolation cell.

-and_cell_for_isosig|-or_cell_for_isosig <cell-name>

If you specify multiple isolation signals with the `-isosig` argument of the `voltage_domain` constraint, specify the cell to be used for generating a single isolation signal by specifying AND or OR gates for the signals using the `-and_cell_for_isosig` argument or the

-or_cell_for_isosig argument, respectively.

When you specify the -and_cell_for_isosig argument, the AND gate is applied to the signal. Similarly, when you specify the -or_cell_for_isosig argument, the OR gate is applied to the signals. You should specify only one of these arguments

-input_pin|-output_pin|-enable_pin <pin-name>

The -input_pin, -output_pin, and -enable_pin arguments specify the input pin name, the output pin name, and the enable pin name for the cells specified with other arguments. Then, these pin names are used in isolation cell instances for named association.

-config

The -config argument specifies the order of values in the isolation cell instantiations as follows:

Value	Order
1	input-enable-output
2	output-input-enable
3	enable-input-output
4	output-enable-input

The same configuration without the enable part is also applicable for the NOT cell used for changing the polarity of the isolation signal. Thus, the configurations 1 and 3 indicate input-output and configurations 2 and 4 indicate output-input.

Configuration is useful when pin names are not provided and the instances do not have named association.

Rules

The isolation_cell constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM08	LPSVM09	LPSVM10	LPSVM22

LPSVM23	LPSVM31	LPSVM35	LPSVM48
LPSVM51			

isolation_wrapper

Purpose

The *isolation_wrapper* constraint specifies the isolation wrapper modules. The wrapper module name should be specified in the *-name* argument. Wildcard characters are also supported.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the *isolation_wrapper* constraint is as follows:

```
isolation_wrapper -name <isolation-wrapper-mod-name>
```

Arguments

-name <isolation-wrapper-mod-name>

Use this argument to specify the wrapper module name.

Rules

The *isolation_wrapper* constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM08	LPSVM22	LPSVM47	LPSVM60

keeper

Purpose

The `keeper` constraint is used to specify bus-keeper design units or nets connected to a (virtual) bus-keeper design unit instance so that various rules can take appropriate action.

For example, the `Scan_21` rule uses the `keeper` constraint to check whether a scan flip-flop exists in the fan-in of the enable pin of a bus-keeper design unit. The `Tristate_06` rule and the `Tristate_09` rules ignore those floating tristate signals that are connected to an instance of the bus-keeper design unit or are connected to the specified net.

Product

SpyGlass DFT solution

Syntax

The syntax of the `keeper` constraint for the `Scan_21` rule is as follows:

```
keeper
  -name <du-name>
  [ -pin <pin-name> -value <value> ]
```

The syntax of the `keeper` constraint for the `Tristate_06` and `Tristate_09` rules is as follows:

```
keeper
  [ -name <du-name> -pin <pin-name> -value <value> ]
  | [ -name <net-name> ]
```

NOTE: *The `keeper` constraint supports wildcard characters.*

Arguments

-name <du-name>

The name of the bus-keeper design unit (black box).

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are treated as keeper design units.

SpyGlass Design Constraints

You can specify a single design unit name or a space-separated list of design unit names.

-pin <pin-name>

(Optional) The enable pin on the bus-keeper design unit (black box). You can specify only a single pin name.

If you do not specify the enable pin, the design unit (black box) is always assumed an enabled bus-keeper.

-value <value>

(Optional) The expected value (0,1, X, or Z) on the enable pin *<pin-name>* under the shift mode.

You need to specify the value only if you have specified the pin name.

-name <net-name>

(Optional) The net that is connected to a tristate signal to be ignored.

You need to specify the hierarchical net name with respect to the top (with top name being optional).

Rules

The `keeper` constraint is used by the following rules:

SpyGlass DFT Solution		
Scan_21	Tristate_06	Tristate_09

latched_port

Purpose

The `latched_port` constraint is used to specify that a vector port is error-free.

Product

SpyGlass Power Estimation Solution

Syntax

The syntax to specify the `latched_port` constraint is as follows:

```
current_design <du-name>  
  latched_port  
    -port_name <name>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the latched port.

-port_name <name>

Name of the vector port that you want to declare as error-free.

Rules

The `latched_port` constraint is used by the PRARITH01 rule.

levelshifter

Purpose

The `levelshifter` constraint is used to specify the names of design units to be used as level shifters.

NOTE: *Currently, a level shifter with an enable pin (clamp level shifter cell) is also treated as isolation cell.*

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `levelshifter` constraint is as follows:

```
current_design <du-name>  
  levelshifter  
    -name <name-list>
```

```

-from <vd-name>
-to <vd-name>
[ -inTerm <term-name> ]
[ -enableTerm <term-name-list> ]
[ -outTerm <term-name> ]
[ -inSupplyTerm <term-name> ]
[ -outSupplyTerm <term-name> ]
[ -enableNet <en-net-name> ]
[ -locate <vd-name> | src | dest ]

```

Arguments

<du-name>

Name of the design unit under which you are specifying the level shifters.

-name <name-list>

Space separated list of names of the level shifter design units. You can use wildcard characters while specifying level shifter cell names using the -name argument. Currently, only the '*' operator for zero or more times and '?' for zero or one time are supported. You must enclose wildcard-based names in double quotes ("").

-to <vd-name>

Name of a voltage domain specified using the [voltage_domain](#) (SpyGlass Power Verify solution) constraint.

-inTerm <term-name> / -outTerm <term-name>

Name of the input/output terminal

-enableNet <en-net-name>

Name of the net to which the enable terminal is to be connected.

-enableTerm <term-name-list>

Space-separated name list of enable terminals.

When you specify the -enableTerm argument, SpyGlass assumes the

level shifter to have isolation capability (and requires you to also specify such level shifters with the `isolation_cell` constraint). Then, the LPSVM08, LPSVM09, and LPSVM10 rules of the SpyGlass Power Verify solution do not skip such level shifter instances (with isolation capability) at the output of power domains.

-outTerm <term-name>

Use the `-inSupplyTerm` and `-outSupplyTerm` arguments to specify the input-side supply terminal and output-side supply terminal respectively as used by the LPPLIB05 rule of the SpyGlass Power Verify solution.

-locate <vd-name> | src | dest

The `-locate` argument specifies the location (voltage domain) of the level shifter instance. For LPSVM30 rule of the SpyGlass Power Verify solution, you can specify the actual voltage domain or specify `src` or `dest` (case-insensitive) to indicate that the level shifter instance should be placed in the source voltage domain or destination voltage domain, respectively. If the `-locate` argument is not specified, the LPSVM30 rule of the SpyGlass Power Verify solution places all level shifter instances at the top of the design hierarchy.

Rules

The `levelshifter` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM04A	LPSVM04B	LPSVM04C	LPSVM04D
LPSVM08	LPSVM09	LPSVM10	LPSVM17
LPSVM30	LPPLIB04	LPPLIB05	LPPLIB06
LPPLIB07	LPPLIB15		

lp_ignore_cells_for_erc

Purpose

The `lp_ignore_cells_for_erc` constraint is used to specify the cells that should be ignored while checking of LPERC rules. You can use wildcard characters while specifying cell names using the `-names` argument. Currently, only the `*` operator is supported. You must enclose wildcard-based names in double quotes (`"`).

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `lp_ignore_cells_for_erc` constraint is as follows:

```
lp_ignore_cells_for_erc -names <cell-name-list>
```

Arguments

-names <cell-name-list>

A space separated list of cell names is specified.

Rules

The `lp_ignore_cells_for_erc` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPERC01A	LPERC01B	LPERC01C	LPERC02A
LPERC02B	LPERC03A	LPERC04A	LPERC04B

make_mandatory_upf_commands_options

Purpose

This command is used to make optional arguments of a UPF command as mandatory.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `make_mandatory_upf_commands_options` constraint is as follows:

```
make_mandatory_upf_commands_options -name <command-name> -
options <option-list>
```

Arguments

-name <command-name>

(Mandatory) Specifies the name of the UPF 1.0/2.0 command. The command name is checked against UPF 1.0/2.0 for sanity checking of the constraint in the *SGDC_lowpower119* rule.

-options <option-list>

(Mandatory) Specifies the arguments of the UPF command. These arguments once specified become mandatory. The specified arguments are checked against UPF 1.0/2.0 for sanity checking of the constraint in the *SGDC_lowpower119* rule.

Rules

The `make_mandatory_upf_commands_options` constraint is used by the following rule:

SpyGlass Power Verify Solution

SGDC_lowpo	UPFSTX_18
wer119	

mapped_pin_map

Purpose

When SpyGlass analyzes the RTL description, there is no knowledge about post-synthesis pin names. The design is synthesized internally, using the SpyGlass synthesis libraries that may use pin names different from the actual ones. Therefore, you need to define a pin name mapping for the different types of pins.

The `mapped_pin_map` constraint is used to define the mapping between the pin names specified in the user's library and the SpyGlass synthesis library.

Product

SpyGlass Constraints solution

Syntax

The syntax to specify the `mapped_pin_map` constraint is as follows:

```
current_design <du-name>
  mapped_pin_map
    [-clock <clock_pin_names_list>]
    [-enable <enable_pin_names_list>]
    [-data <data_pin_names_list>]
    [-out <output_pin_names_list>]
    [-preset <preset_pin_names_list>]
    [-clear <clear_pin_names_list>]
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in <entity-name>.<arch-name> format (for VHDL designs).

-clock <clock_pin_names_list>

(Optional) Specify the list of clock-pin names in the user's library.

-enable <enable_pin_names_list>

(Optional) Specify the list of enable-pin names in the user's library.

-data <data_pin_names_list>

(Optional) Specify the list of data-pin names in the user's library.

-out <output_pin_names_list>

(Optional) Specify the list of output-pin names in the user's library.

-preset <preset_pin_names_list>

(Optional) Specify the list of preset-pin names in the user's library.

-clear <clear_pin_names_list>

(Optional) Specify the list of clear-pin names in the user's library.

NOTE: *The mapped_pin_map constraint is required only when analyzing RTL descriptions. For netlists, the information is obtained from the corresponding gate library (.sglib files or .lib files).*

For example, consider the following:

```
current_design top
mapped_pin_map
  -clock CK CP -enable EN EX -data D -out Q QN
```

The above specification indicates that pins named CK or CP are clock pins in the user-specified technology library cells. Similarly, pins named EN or EX are enable pins, pins named D are data pins, and pins named Q or QN are output pins.

Rules

The mapped_pin_map constraint is not rule-specific. It is used by the whole SpyGlass Constraints solution.

mcp_info

Purpose

The mcp_info SGDC constraint is used to specify enables and other attributes to verify the SDC multicycle path constraints.

NOTE: *The mcp_info SGDC command gets priority over any parameters.*

Product

SpyGlass TXV Solution

Syntax

The syntax of the mcp_info constraint is as follows:

```
current_design <du-name>
mcp_info
  -name <name>
  [ -multiplier <MUL> ]
  [ -launch_clock <LC> ]
  [ -launch_enable <LE> ]
  [ -launch_flop <LE> ]
  [ -capture_clock <CC> ]
  [ -capture_enable <CE> ]
  [ -capture_flop <CE> ]
  [ -cpath_enable ]
  [ -start/-end ]

  -sdc_file_name <file-name>
  -sdc_line <line-number>
  [ -force_ldce <0 | 1> ]
  [ -make_pi <list-of-signals> ]
  [ -IConstrs <expression> ]
```

Arguments

-name <name>

(Mandatory) Used to match the name of the set_multicycle_path constraint defined in the SDC file. You can specify the name of a set_multicycle_path constraint in the SDC file by using the comment argument:

```
set_multicycle_path 2 -from C1 -to C2 -comment {Atrenta -name
<name1>}
```

-multiplier <MUL>

(Optional) Used to specify the path multiplier of the `set_multicycle_path` constraint.

-launch_clock <LC>

(Optional) Used to specify the SDC name of the clock that is traversing to the source point of the timing path constrained by the `set_multicycle_path` constraint.

-launch_enable <LE>

(Optional) Used to specify the start-point enable expression in terms of user specified signals in the RTL.

NOTE: *Do not use this argument to specify clock path enables.*

-launch_flop <flop-list>

(Optional) Use this argument to specify an enable for the launch flip-flop. This argument must be used with the `-launch_enable` argument.

Apart from the `-launch_enable` argument, no `mcp_info` argument is supported with the `-launch_flop` argument.

This argument supports wildcards.

See [Example 3: Specifying Enables for Launch/Capture Flip-Flops](#).

-capture_clock <CC>

(Optional) Used to specify the SDC name of the clock that is traversing to the end-point of the timing path constrained by the `set_multicycle_path` constraint.

-capture_enable <CE>

(Optional) Use to specify the end-point enable expression in terms of user specified signals in the RTL.

NOTE: *Do not use this argument to specify clock path enables.*

-capture_flop <flop-list>

(Optional) Use this argument to specify an enable for the capture flip-flop.

This argument must be used with the `-capture_enable` argument.

Apart from the `-capture_enable` argument, no `mcp_info` argument is supported with the `-capture_flop` argument.

This argument supports wildcards.

See [Example 3: Specifying Enables for Launch/Capture Flip-Flops](#).

`-cpath_enable`

(Optional) Specify this argument to detect enables only in the clock path for both source and destination.

The following `mcp_info` arguments are not supported with this argument:

- `-capture_enable`
- `-launch_enable`
- `-force_ldce`
- `-launch_flop`
- `-capture_flop`

See [Example 4: Specifying `cpath\_enable`](#).

`-start/-end`

(Optional) If the multi-cycle information is relative to the period of the start clock, specify `-start`. Otherwise, specify `-end`. If neither is specified, `-end` is taken as default. A FATAL violation message is reported if you specify both `-start` and `-end` in the same `mcp_info` constraint.

`-sdc_file_name <file-name>`

(Mandatory, if either `force_ldce`, `make_pi`, or `lConstrs` is specified)

Used to specify the name of the SDC file which contains the multicycle path specification. For this specification, the `force_ldce`, `make_pi`, or `lConstrs` arguments are applied.

The SDC file must exist in the project working directory. Do not specify the file name as an absolute or relative path.

-sdc_line <line-number>

(Mandatory, if either `force_ldce`, `make_pi`, or `lConstrs` is specified)

Used to state the line number of the SDC file specified using the `sdc_file_name` argument. This is the line number of the multicyle path specification on which the `force_ldce`, `make_pi`, or `lConstrs` arguments are applied.

-force_ldce <0 | 1>

(Optional) This argument forces multicyle path verification on the basis of data transition on the source side, instead of verification based on the source enable.

Set this argument to 1 to verify on the basis of data transition on the source side.

Options Not Supported With `force_ldce`

The following arguments cannot be specified with the `force_ldce` argument: `-launch_enable`, `-launch_clock`, or `-capture_clock`. If any of the existing options is combined with `-force_ldce`, a violation is reported. See [Example 2: The `force\_ldce` Argument Violations](#).

-make_pi <list-of-signals>

(Optional) Use to specify the signals whose fan-in cone is not to be considered during multicyle path verification. The SpyGlass TXV solution cuts off the design present in the fan-in of the specified signals and treats these signals as primary inputs.

Specify the list of signals by separating the signal names by the "@" symbol. For example, `sig1@sig2@sig3`.

-lConstrs <expression>

(Optional) Initializes signals.

This argument takes an expression, which comprises the value of the signals, so that the final expression is always true. It can take simple expressions, such as `&`, `I`, and `~`.

Example

This section contains the following examples:

- [Example 1: Flows of Multicycle Path Verification](#)
- [Example 2: The force_Idce Argument Violations](#)
- [Example 3: Specifying Enables for Launch/Capture Flip-Flops](#)

Example 1: Flows of Multicycle Path Verification

The flow of multicycle path verification is in two categories:

- Name specified in the `mcp_info` SGDC command matches with that in the SDC file
- Name specified in the `mcp_info` SGDC command does not match with that in the SDC file

Name Specified in the SGDC File Matches

In this example, the name `MCP1` matches with the name specified in the SDC file. Therefore, the values specified in the SGDC file are used to verify the `set_multicycle_path` constraint. You can use this approach to override some of the attributes or to provide missing information that SpyGlass is not able to infer.

SDC

```
set_multicycle_path 3 -from {ff1_reg/CP} -to {ff2_reg/D} -
comment {Atrenta -name <MCP1>; My comment}
```

SGDC

```
mcp_info -name MCP1 -launch_clock CLK -launch_enable
test.src_en -capture_clock CLK2 -capture_enable
test.dst_en
```

Considerations

The following considerations are applied during the verification:

- If either of the `-multiplier`, `-start/-end`, `-launch_clock`, `-launch_enable`, `-capture_clock`, or `-capture_enable` arguments is specified, the value specified in the SGDC constraint overrides that attribute for all the timing paths constrained through the `set_multicycle_path` constraint.

- For the SDC clocks specified in the `-launch_clock` and `-capture_clock` arguments, SpyGlass checks whether the clocks specified are traversing at either the source or destination. If not, those clocks are ignored.
- The enable expression specified in the `-launch_enable` or `-capture_enable` argument is not validated by SpyGlass. The verification is performed based on the assumption that these signals are the enabling condition for the start/end-points.

Name Specified in the SGDC File Does Not Match

In this example, the name MCP11 does not match with the name specified in the SDC file. You can use this approach when you do not have a `set_multicycle_path` constraint and want to validate the timing path formally.

SDC

```
set_multicycle_path 3 -from {ff1_reg/CP} -to {ff2_reg/D} -
comment {Atrenta -name <MCP1>; My comment}
```

SGDC

```
mcp_info -name MCP11 -launch_clock CLK -launch_enable
test.src_en -capture_clock CLK2 -capture_enable
test.dst_en
```

Considerations

The following considerations are applied during the verification:

- The `-name` argument would be a name that does not match with any of the existing `set_multicycle_path` constraints.
- The `-multiplier` argument is mandatory for this flow.
- If `-start` is specified, the `-launch_capture` argument is mandatory to determine the multi-cycle information relative to the period of the start clock.
- If `-end` is specified, the `-capture_clock` argument is mandatory to determine the multi-cycle information relative to the period of the end clock.
- Either the `-launch_enable` or `-capture_enable` arguments must be specified. The missing enable is taken as one.

- If either of `-launch_capture` or `-capture_clock` is not specified, the verification is done based on the multi-cycle information relative to the period of the start clock.

Example 2: The `force_ldce` Argument Violations

This example shows the violation message that is reported when the `-force_ldce` argument is specified with unsupported arguments. Consider the following `mcp_info` SGDC specification.

```
mcp_info
  -name "MCP1"
  -multiplier 2
  -launch_clock "clk"
  -capture_clock "pclk"
  -launch_enable "(pclk)&pclk"
  -capture_enable "pclk"
  -start
  -sdc_file_name "test.sdc"
  -sdc_line 8
  -force_ldce 1
```

The SpyGlass TXV solution reports the following violation because the `-force_ldce` argument cannot be specified with `-launch_enable`, `-launch_clock`, or `-capture_clock`:

For `mcp_info` command `'-force_ldce'` and `'-launch_enable'` are not supported together. `'-force_ldce'` and `'-launch_clock'` are not supported together. `'-force_ldce'` and `'-capture_clock'` are not supported together.

Example 3: Specifying Enables for Launch/Capture Flip-Flops

This section contains the following examples:

- [*Specifying launch_flop*](#)
- [*Specifying capture_flop*](#)
- [*Impact of Multiple mcp_info Commands*](#)
- [*Multiple launch_flop/capture_flop Options Specified for Same mcp_info Command*](#)

Specifying launch_flop

For the launch flip-flop, specify the hierarchical name of the output terminal (Q terminal) of the launch flip-flop, as shown in the following `mcp_info` specification.

```
mcp_info -launch_flop "F1.Q, F2.Q, F3.Q" -launch_enable "e1"  
-name MCP1
```

Specifying capture_flop

For the capture flip-flop, specify the hierarchical name of the input terminal (falling on the data path) of the launch flip-flop, as shown in the following `mcp_info` specification.

```
mcp_info -capture_flop "F5.D, F6.D, F7.D" -capture_enable  
"e2" -name MCP1
```

However, if the `mcp_info` specification is:

```
mcp_info -capture_flop "F5.D, F6.D, F7.D" -name MCP1
```

The `Txv_mcp_info` rule reported the following violation:

TXV_SGDC_MCP: For `mcp_info` command '-capture_flop' cannot be used without '-capture_enable'.

Impact of Multiple mcp_info Commands

You can specify multiple `mcp_info` commands with the same `-name` argument.

In this example, the following snippet shows one `mcp_info` specification with `launch_enable/capture_enable`, while the other specification shows without `launch_flop/capture_flop`.

```
mcp_info -name "MCP1" -launch_flop "F1.Q" -launch_enable "e1"  
mcp_info -name "MCP1" -launch_enable "e2"
```

In this case, **e1** is the enable for flip-flop **F1**, while enable **e2** is for all other launch flip-flops.

Multiple launch_flop/capture_flop Options Specified for Same mcp_info Command

For the following `set_multicycle_path` SDC commands, multiple `launch_flop/capture_flop` arguments have been specified for the same flip-flop in the same `mcp_info` command:

```
set_multicycle_path 2 -from ff1_reg -to ff2_reg -comment {Atrenta -name <MCP1>}
set_multicycle_path 2 -from ff1_reg -to ff3_reg -comment {Atrenta -name <MCP2>}
```

The `mcp_info` SGDC specification is as follows.

```
1 current_design test
2 sdc_data -file test.sdc
3
4 mcp_info -name "MCP1" -launch_flop "test.ff1_reg.Q" -launch_enable "en2"
5 mcp_info -name "MCP2" -launch_flop "test.ff1_reg.Q" -launch_enable "en3"
6 mcp_info -name "MCP2" -launch_enable "en2" -capture_flop "test.ff3_reg.D" -capture_enable "en2"
7 # Duplicate... should be ignored
8 mcp_info -name "MCP2" -multiplier 2 -launch_enable "en3" -start
9 # Valid...
10 mcp_info -name "MCP2" -multiplier 2 -capture_enable "en3" -start
11 # Duplicate... should be ignored
12 mcp_info -name "MCP2" -multiplier 2 -capture_enable "en3" -start
13 # Duplicate entry for -capture_flop ... should be ignored
14 mcp_info -name "MCP2" -capture_flop "test.ff3_reg.D" -capture_enable "en2" -start
```

The `Txv_mcp_info` rule generates the following violation messages:

TXV_SGDC_MCP:Duplicate entry for test.sgdc:8 "mcp -name MCP2 - multiplier 2 -launch_enable en3 -start " ignored for sdc constraint

TXV_SGDC_MCP:Duplicate entry for test.sgdc:12 "mcp -name MCP2 - multiplier 2 -capture_enable en3 -start " ignored for sdc constraint

TXV_SGDC_MCP:Duplicate entry for -capture_flop test.ff3_reg.D test.sgdc:14 "mcp -name MCP2 -multiplier 2 -capture_enable en2 -capture_flop test.ff3_reg.D -start " ignored for sdc constraint.

Example 4: Specifying `cpath_enable`

The following `mcp_info` specification shows how to specify the `-cpath_enable` argument:

```
mcp_info -name MCP1 -cpath_enable
```

Rules

The `mcp_info` constraint is used by the following rules:

SpyGlass Txv Solution

Txv_MCP01

memory

Purpose

The `memory` constraint is used to specify information about memory cells in the design. This information is used by PEPWR18 rule for memory splitting and PESTR11 rule for identifying the enable signals of the memory cells.

You can specify the memory details like its address ports, data ports, enable signals and their polarity using this constraint. Half-sized equivalent and quarter-sized equivalent memories can also be specified for the memory cells.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `memory` constraint is as follows:

```
memory
  -name <list-of-memory-cells>
  -address <list-of-address-ports>
  -data <list-of-data-ports>
  -halfsize <module-name1>
  -quartersize <module-name2>
  -enable <list-of-enables>
  -enable_polarity <en-polarity>
```

Arguments

-name <list-of-memory-cells>

Name of a memory cell or a list of memory cells. You can also use wildcards for specifying the memory cell names.

While specifying multiple memory cells or wildcard specification with the -name argument, you cannot use any of -address, -data, -halfsize or -quartersize argument.

-address <list-of-address-ports>

List of address ports of a memory cell.

NOTE: For the -address/-data argument of the memory constraint, generally, the address/data bus that is defined in library as `A[0:32]` is specified as 'A'. For some libraries, where the address/data bus is in form of bits like A1, A2, A3, and so on, you can collectively specify 'A%d' for all the pins. The tool infers all such pins from the library as the pins of the specified address/data bus.

-data <list-of-data-ports>

List of data ports of a memory cell.

-halfsize <module-name1>

Half-sized equivalent for a memory cell.

-quartersize <module-name2>

Quarter-sized equivalent for a memory cell.

-enable <list-of-enables>

List of enable signals of the specified list of memory cells.

-enable_polarity <en-polarity>

Polarity of the specified enable signals.

Rules

The memory constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR01	PEPWR02	PEPWR18	PESTR11
poweraudit			

memory_force

Currently, the `memory_force` constraint is merged with [test_mode](#) constraint and can be used as an argument to the `test_mode` constraint.

memory_port

Purpose

The `memory_port` constraint is used to specify information about memory cells or submodules in the design. This constraint can be specified at the technology cell level and at the wrapper level. When you have an RTL module with MBIST and other peripheral logic contained inside a wrapper, it is recommended that you specify the complete wrapper as memory using the `memory_port` constraint. In such cases, you will specify the ports of the wrapper memory in the arguments, such as `-when` and `-clock`. Refer to the [Examples](#) section for more details.

Refer to the [Rules](#) section for a list of rules that use the information specified in the `memory_port` constraint to determine memory specifications.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `memory_port` constraint is as follows:

```
memory_port
  -name <list-of-memory-cells>
  -data <data-port>
```

```

-address <address-port>
-operation <read|write|unwanted>
-posedge|-negedge <clk_pin>
-when <pin-state>
-label <label-name>
-async
-retain_output <0|1|yes|no>
-rom
-cycle_count <0|1>
-writethru
-data_out <data-out-port>
-write_mask <write-mask-port>
-write_mask_value <0|1>
-sleep_mode_when <pin-sleep-state>
-down_cycles <num-cycles-mem-down>
-wakeup_cycles <num-cycles-mem-wakeup>

```

Arguments

-name <list-of-memory-cells>

Name of a memory cell or a list of memory cells or sub-modules. A sub-module refers to a design unit that can act as a memory.

You can also use wildcards for specifying the name of memory cells and modules.

-data <data-port>

Specifies the data port of a memory module.

-address <address-port>

Specifies the address port of a memory module.

NOTE: For the -address/-data argument, the address/data bus that is defined in library as `A[0:32]` is specified as 'A'. For some libraries, where the address/data bus is in form of bits such as `A1`, `A2`, `A3`, you can collectively specify `A*` for all the pins. The tool infers all such pins from the library as the pins of the specified address/data bus.

-operation <read|write|unwanted>

Specifies the type of operation to be performed on a memory module. The `unwanted` operation represents a condition, where the memory control pins are in an unexpected state. For example, if a clock edge is applied while the chip enable for the memory is turned off.

-when <pin-state>

Lists conditions, comprising of states of pins required to trigger the specified operation on a memory module. In this option, you can specify pins, which should be asserted high and pins, which must be asserted low (with a `!` prefix). For example, if `CE` must be high and `WENB` must be low for the write operation, you need to specify: `-when "CE !WENB"`

To specify the condition, you can use any of the following operators:

`() & | ^ ! .`

-posedge|-negedge <clk_pin>

Specifies whether the specified operation occurs on rising or falling transition of the associated clock pin.

-label <label-name>

Specifies a unique label to identify a `memory_port` constraint.

-async

Specifies that the memory is an asynchronous read memory.

When this argument is specified in the *PESAE07* rule, the read frequency of the memory is calculated based on the change in the address bus.

NOTE: *Do not specify the `-async` argument with the `-posedge` and `-negedge` arguments.*

-retain_output <0|1|yes|no>

Specifies whether the output retains its value or not when the memory is switched off.

Default value is 1.

Value	Condition
0 no	Data at the read output pin does not retain its value
1 yes	Data at the read output pin retains its value

-rom

Specifies that the memory is a read-only memory.

When this argument is specified, all PE rules, except PESTR28 and PEPWR28, treat the memory cell as a read-only memory. Therefore, all write operations are ignored if the -rom argument is specified in a read operation.

NOTE: Do not specify the -rom argument with the -write argument.

-cycle_count <0|1>

Specifies the number of clock cycles for the data to be available at the memory output from the time the read condition is enabled.

Default value is 0.

-writethru

Specifies whether the memory is write-through or not.

Specifying this argument ensures that the data written to the data port (specified by the -data argument) is read and available at the data output port (specified by the -data_out argument).

-data_out <data-out-port>

Specifies the data output port corresponding to the data port, when the -writethru argument is specified.

NOTE: The -writethru and -data_out arguments must be specified together.

-write_mask <write-mask-port>

Specifies the write mask port for the data port.

The write data port is masked if the value on the port specified by the -write_mask argument is equal to the value specified by the

`-write_mask_value` argument.

`-write_mask_value <0|1>`

Specifies the value of the write mask port (specified by the `-write_mask` argument) for which the write data port is masked.

For example, consider that the write mask port is `Q[0:63]` and the write mask value is 1. In this case, the write data port will be masked only if the value on the `Q[0:63]` port is equal to 1.

NOTE: *The `-write_mask_value` and `-write_mask` arguments must be specified together.*

NOTE: *The `-writethru`, `-data_out`, `-write_mask_value`, and `-write_mask` arguments must be specified for the write operation only. For the read operation, these arguments are ignored.*

`-sleep_mode_when <pin-sleep-state>`

Specifies the states of pins required to trigger light sleep mode on a memory module.

In this option, you can specify the memory pin that should be asserted high. For example, if the `LS` pin must be high to trigger light sleep mode, specify the following expression:

`-sleep_mode_when LS`

`-down_cycles <num-cycles-mem-down>`

Specifies the number of clock cycles required to assert the light sleep pin of the memory for it to enter light sleep mode.

`-wakeupt_cycles <num-cycles-mem-wakeup>`

Specifies the number of clock cycles required to de-assert the light sleep pin of the memory for it to recover from light sleep mode.

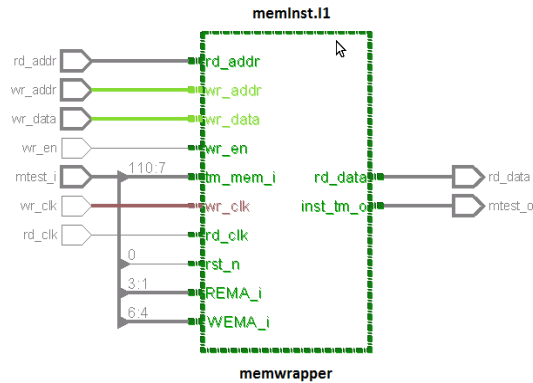
Examples

This example illustrates how to use the `memory_port` constraint at wrapper (submodule) level.

The image below, `memInst.I1` is the hierarchical boundary in the design

SpyGlass Design Constraints

for the module (wrapper) memwrapper. The mtest_i port is the test mode data and control logic.



The memory_port constraint specification for the memwrapper wrapper using the module ports as follows.

#-- Wrapper memwrapper

Read operation

```
memory_port -name " memwrapper " \
    -operation read \
    -label syncread \
    -posedge rd_clk \
    -when "!wr_en" \
    -address "rd_addr" \
    -data "rd_data" \
    -retain_output yes \
    -cycle_count 0
```

Write operation

```
memory_port -name " memwrapper " \
```

```

-operation write \
-label syncwrite \
-posedge wr_clk \
-when "wr_en" \
-address "wr_addr" \
-data "wr_data"

```

Rules

The `memory_port` constraint is used by the following rules of the SpyGlass Power Estimation and SpyGlass Power Reduction solutions:

PESAE07	PEPWR20	PEPWR21	PEPWR22	PEPWR23
PEPWR24	PEPWR25	PEPWR28	PESTR20	PESTR21
PESTR22	PESTR23	PESTR24	PESTR25	PESTR28
PEPWR02	PEPWR29	PESTR29		

memory_inst_port

Purpose

The `memory_inst_port` constraint is used to overwrite the value of the `retain_output` parameter of the `memory_port` constraint.

The `retain_output` argument of the `memory_port` constraint specifies if memory will retain its output value or not when memory is switched off. In some cases, it may happen that though the value of the `retain_output` argument specified in the `memory_port` constraint is '1' but for a particular instance of that memory, the user does not care about the output when memory is turned off. To capture this information, user can specify this constraint.

Consider the following `memory_port` constraint specifications:

```

memory_port -name mem_inst -posedge "CLKA" -data "QA" \
-when "!WEA MEA" -operation read -address "ADRA" -

```

```
retain_output 1
memory_port -name mem_inst -posedge "CLKA" -data "DA" \
  -when "WEA MEA" -operation write -address "ADRA"
memory_inst_port -term "top.m1.QA" -retain_output x
```

In the above example, the memory cell is `mem_inst` as specified by the `-name` argument of the `memory_port` constraint. The value of the `-retain_output` argument for the QA read data bus of the m1 memory instance is overridden by the `memory_inst_port` constraint. For all other instances, the value of the `-retain_output` argument will remain 1.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `memory_inst_port` constraint is as follows:

```
memory_inst_port
  -term <full-hierarchical-path-of-the-terminal>
  -retain_output <x|X>
```

Arguments

-term <full-hierarchical-path-of-the-terminal>

Specifies the name of the read data pin of the memory instance. You can use wildcards for specifying the memory terminal names. This should be the same as the name of the data bus specified in the read operation of the `memory_port` constraint.

-retain_output <x|X>

Used to override the value from '1' to 'x' for a particular instance.

NOTE: *This overriding of the value from '1' to 'x' is restricted to PESTR25 and PEPWR25 only*

memory_read_pin

Purpose

The `memory_read_pin` constraint is used to specify the read pin port name on a memory and the *inactive* value on that port.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `memoryreadpin`.

Product

SpyGlass DFT solution

Syntax

The syntax of the `memory_read_pin` constraint is as follows:

```
memory_read_pin
  -memname <mem-name>
  -readport <readpin-name-list>
  -value <value>
```

Arguments

The `memory_read_pin` constraint has the following arguments:

-memname <mem-name>

The memory design unit (black box) type name (specified using a `memory_type` constraint).

You can specify a single design unit name or a space-separated list of design unit names.

-readport <readpin-name-list>

Names of read pins in the specified memory module type.

You can specify only a single pin name or a list of pin names.

-value <value>

The inactive value (0,1, X, or Z) for this read pin `<readpin-name>`.

NOTE: *The inactive value is a single value that when applied to the read pin on this memory will disable read from the memory.*

Notes

- The `memory_read_pin` constraint requires use of a `memory_type` constraint with the same type specified. The specified pin is a read pin on this memory type.
- The `memory_read_pin` constraint may be used in conjunction with `test_mode` constraint.

Rules

The `memory_read_pin` constraint is used by the following rule:

SpyGlass DFT Solution
RAM_06

memory_tristate

Purpose

The `memory_tristate` constraint defines pins on memories and the values necessary to prevent the output of those memories from being a high impedance value.

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was `memory3s`.*

Product

SpyGlass DFT solution

Syntax

The syntax of the `memory_tristate` constraint is as follows:

```
memory_tristate
  -memname <mem-name>
  -enableport <enable-pin>
```

-value <value>

Arguments

-memname <mem-name>

The memory design unit name.

-enableport <enable-pin1>

Name of the enable pin on the memory.

-value <value>

Value (string) of the enable pin.

Rules

The memory_tristate constraint is used by the following rules:

SpyGlass DFT Solution
All Tristate rules

memory_type

Purpose

The memory_type constraint specifies the memory design unit (black box) names. Then, all instances of these design units in the design are assumed to be memory instances.

NOTE: *In addition to memory_type constraint, memory instances are automatically inferred from library cells as well.*

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was memorytype.*

Product

SpyGlass DFT solution

Syntax

The syntax of the `memory_type` constraint is as follows:

```
memory_type -name <mem-name>
[-q_pin <q-pin-name> ]
[-d_pin <d-pin-name> ]
[-path_type <combinational | sequential>]
[-clock_pin <clk-pin-name>]
```

NOTE: *The `memory_type` constraint supports wildcard characters.*

Arguments

-name <mem-name>

The memory design unit (black box) name.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name <du-name> can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are treated as memory design units.

You can specify a single design unit name or a space-separated list of design unit names.

NOTE: *The `memory_type` constraint supports wildcard expressions. The supported meta-characters are `*` (asterisk) and `?` (question mark) where `*` matches any number of characters and `?` matches only one character. The wildcard support is applicable for non-escaped names only. If the meta-characters appear inside an escaped name, they are treated as literals. For example, in the expression `"top.\mid*\bottom"`, `mid*` is considered as a literal and does expand to `"mid1, mid2, and so on"`. In addition, if you specify a hierarchical path using a wildcard, any sub-portion of this path that contains the wildcard does not cross the module boundary while searching for the expression in the design. This means that each level in the hierarchy path should be mentioned explicitly in the wildcard string. For example, the expression `"top.mid*.bottom"` will expand to `"top.mid1.bottom"` and not to `"top.mid2.bottom"`.*

NOTE: *The expression on which a wildcard is used should always be enclosed within double quotes. For example, `"top.mid*.bottom"`.*

NOTE: *The wildcard support is applicable for design objects only. For non-design objects, the support is not applicable.*

-q_pin <q-pin-name>

Q-pin of the memory design unit (black box) name.

-d_pin <d-pin-name>

Data pin of the memory design unit (black box) name.

-path_type <combinational | sequential>

When this argument is set as sequential/combinational, the path between <d-pin-name> to <q-pin-name> is treated as sequential/combinational path, respectively. By default, this is set as sequential.

-clock_pin <clk-pin-name>

Clock pin of the memory design unit.

Notes

More than one memory_type constraint may be necessary. If more than one memory_type constraint is used, they will all be processed in parallel.

Rules

The memory_type constraint is used by the following rules:

SpyGlass DFT Solution			
RAM_01	RAM_02	RAM_03	RAM_04
RAM_05	RAM_06	RAM_07	RAM_08
RAM_09	RAM_10	RAM_11	

memory_write_disable

Purpose

The memory_write_disable constraint is used to specify the top-level primary ports and values that, when simulated, will disable all write

enables to memories.

NOTE: *The memory_write_disable constraint will be deprecated in a future release.*

Product

SpyGlass DFT solution

Syntax

The syntax of the memory_write_disable constraint is as follows:

```
memory_write_disable
  -name <port-name>
  -value <value>
```

Arguments

-name <port-name>

Name of the top-level primary port required to disable all write enables to memories.

You can specify a single primary port name or a space-separated list of primary port names.

-value <value>

Value list for a primary port.

The value list is the sequence of one or more values (each value being 0,1, X, Z, or a combination) that when applied to the top-level port will disable all write enables to memories.

NOTE: See memory_write_pin constraint for more details.

Rules

The memory_write_disable constraint is used by the following rules:

SpyGlass DFT Solution		
Info_memorywritedisable	RAM_05	RAM_08

memory_write_pin

Purpose

The `memory_write_pin` constraint is used to specify the write-pin port name on a memory and the *inactive* value on that port.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `memorywritepin`.

Product

SpyGlass DFT solution

Syntax

The syntax of the `memory_write_pin` constraint is as follows:

```
memory_write_pin
  -memname <mem-name>
  -writeport <writepin-name-list>
  -value <value>
```

Arguments

-memname <mem-name>

The memory design unit (black box) type name (specified using a `memory_type` constraint).

You can specify a single design unit name or a space-separated list of design unit names.

-writeport <writepin-name-list>

Names of write pins in the specified memory module type.

You can specify a single pin name or a list of pin names.

-value <value>

The inactive value (0,1, X, or Z) for this write pin `<writepin-name>`.

Notes

- The inactive value is a single value that when applied to the write pin on this memory will disable write to memory.
- The `memory_write_pin` constraint requires use of a `memory_type` constraint with the same type specified.
- The `memory_write_pin` constraint may be used in conjunction with `memory_write_disable` and/or `test_mode` constraints.

Rules

The `memory_write_pin` constraint is used by the following rules:

SpyGlass DFT Solution		
RAM_05	RAM_06	RAM_08

meta_design_hier

Purpose

The `meta_design_hier` constraint is used to specify the top-level design name and the hierarchical name of the design with respect to the simulation testbench to be used by the `Ac_meta01` rule.

Product

SpyGlass CDC solution

Syntax

The syntax of the `meta_design_hier` constraint is as follows:

```
meta_design_hier
  -name <top-design-name>
  -inst <hier-instance-name>
```

Arguments

-name <top-design-name>

The name of the top-level module for a Verilog design or top-level entity name for a VHDL design.

-inst <hier-instance-name>

The name of the hierarchical instance of the top-level design with respect to the simulation testbench.

Example

Consider an example in which the name of the top-level design is `FSM`, which is instantiated in the testbench `top`, `tb`, as `I1`. In this case, the `meta_design_hier` constraint should be specified in the following manner:

```
meta_design_hier -name FSM -inst tb.I1
```

Rules

The `meta_design_hier` constraint is used by the following rules:

SpyGlass Auto Verify Solution			
Av_deadcode01	Av_staticnet01		
SpyGlass CDC solution			
Ac_meta01	Ac_datahold01a	Ac_cdc01	Ac_conv02

meta_inst

Purpose

The `meta_inst` constraint is used to specify instances for which monitors should be generated by the `Ac_meta01` rule.

Product

SpyGlass CDC Solution

Syntax

The syntax of the `meta_inst` constraint is as follows:

```
meta_inst
  -name <instance-name>
  -type <allow | ignore>
```

Arguments

-name <instance-name>

Specifies the hierarchical name of an instance.

-type <allow | ignore>

Set the value to `allow` to generate monitors for the signals present in the specified instance hierarchy.

Set the value to `ignore` to disable monitors generation for the signals present in the specified instance hierarchy.

Example

Specify the following constraint to generate monitors for the `top.U1.U2` instance:

```
meta_inst -name top.U1.U2 -type allow
```

Rules

The `meta_inst` constraint is used by the following rule:

SpyGlass CDC solution
Ac_meta01

meta_module

Purpose

The `meta_module` constraint is used to specify modules/entities for which monitors should be generated by the `Ac_meta01` rule.

Product

SpyGlass CDC Solution

Syntax

The syntax of the `meta_module` constraint is as follows:

```
meta_module
  -name <du-name>
  -type <allow | ignore>
```

Arguments

-name <du-name>

For Verilog design, this argument specifies a module name.

For VHDL design, this argument specifies an entity name (*<entity-name>*) or an architecture name (*<entity-name>.<architecture-name>*).

-type <allow | ignore>

Set the value to `allow` to generate monitors for the specified module/entity.

Set the value to `ignore` to disable monitors generation for the specified module/entity.

Example

Specify the following constraint to generate monitors for the MOD module:

```
meta_module -name MOD -allow
```

Rules

The `meta_module` constraint is used by the following rule:

SpyGlass CDC solution

Ac_meta01

meta_monitor_options

Purpose

The `meta_monitor_options` constraint is used to specify the attributes used during monitor generation by the `Ac_meta01` rule.

Product

SpyGlass CDC Solution

Syntax

The syntax of the `meta_monitor_options` constraint is as follows:

```
meta_monitor_options
  -setup_hold_time <setup-hold-time>
  -init_time <init-time>
  -error_inject_threshold <threshold>
  -phase_shift_control <cycle-count>
  -log_file <file-name>
  -print_setup_hold_violation <violation-type>
```

Arguments

-setup_hold_time <setup-hold-time>

Use this argument to define the setup/hold time (in percentage).

For example, if you set this argument to 10 and the period of the `clk` clock is 90, the setup/hold time for `clk` will be 9 ns. In this case, the `Ac_meta01` rule will force metastability only if the difference between the clock edge and data change is within 9 ns.

Allowed value: Integer greater than 0

Default value: 10

NOTE: You can also use the **setup_hold_time parameter** to specify the setup/hold time. However, preference is given to this constraint argument, if specified.

-init_time <init-time>

Use this argument to specify the initialization time before starting metastability analysis.

Allowed value: Integer greater or equal to 0. The value should be specified in the ns range.

Default value: 1 (which means analysis starts after 1 ns)

NOTE: You can also use the **meta_init_time parameter** to specify the initialization time. However, preference is given to this constraint argument, if specified.

-error_inject_threshold <threshold>

Use this argument to specify the threshold value for the random value above which error is injected in monitors in case of setup/hold violations.

Allowed value: Any value between 0 to 1

Default value: 0.5

NOTE: You can also use the **meta_error_inject_threshold parameter** to specify the threshold limit. However, preference is given to this constraint argument, if specified.

-phase_shift_control <cycle-count>

Use this argument to specify the number of clock cycles after which the phase of a clock should be shifted by one unit during metastability analysis.

Allowed value: Any integer greater than or equal to 0. If you specify 0, phase of the clock is never shifted.

NOTE: You can also use the **phase_shift_control parameter** to specify the phase shift. However, preference is given to this constraint argument, if specified.

-log_file <file-name>

Use this argument to specify the log file where all the messages generated during simulation are saved.

Default value: ""

-print_setup_hold_violation <violation-type>

Use this argument to specify the criteria to dump setup/hold violations on the screen output or in the file specified by *-log_file <file-name>*.

Allowed values:

error	Details of setup/hold violations when an error is injected are dumped
no_error	Details of setup/hold violations when no error is injected are dumped
both	Details of all the setup/hold violations are dumped
none	Details of none of the setup/hold violations are dumped

Default value: both

Example

The following example shows the usage of the meta_monitor_options constraint:

```
meta_monitor_options -setup_hold_time 10 -init_time 2
-error_inject_threshold 0.3 -phase_shift_control 1000
-dump_in_file -log_file "log.txt"
```

Rules

The meta_monitor_options constraint is used by the following rule:

SpyGlass CDC solution
Ac_meta01

mode_condition

Purpose

The mode_condition constraint is used to specify conditions in which a mode of the specified mode set should be active.

Refer to the *Example* section for more details.

See also the PESAE08 rule documentation.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `mode_condition` constraint is as follows:

```
mode_condition
  -name <mode_set_name>
  -value <mode_name>
  -on_condition <condition>
```

Arguments

-name <mode_set_name>

Specifies the name of a mode set.

-value <mode_name>

Specifies the name of the mode for which conditions are being defined.

-on_condition <condition>

Specifies mode conditions in which the specified mode should be active. You can specify mode condition by creating a logical expression of valid nets in a design.

Example

After you create a mode set and modes, you need to specify conditions under which a particular mode should be active by using the `mode_condition` constraint. You can create such conditions by using logical expressions.

Consider the following specification:

SpyGlass Design Constraints

```
mode_condition -name SPEED -value SLOW -on_condition "top.w1
& top.w2 | (!top.speed) "
```

The above constraint implies that the mode "SLOW" of the mode set "SPEED is active when mode condition as specified by the expression with the `on_condition` switch is **true**.

Rules

The `mode_condition` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

All rules running in EST mode

module_bypass

Purpose

The `module_bypass` constraint is used to specify modules such as memories that are designed with a bypass between data-in port and data-out port.

The `module_bypass` constraint is used to propagate the test signals through the instance of a module, such as an analog level shifter. This constraint is used by the testability analysis engine. It helps transfer the controllability values from the input side to the output side and observable figures in the reverse direction.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `module_bypass` constraint is as follows:

```
module_bypass
```

```

-name <du-name>
[ -bpin <bpin-name-list> -value <value-list> ]
-iport <port-name-list>
-oport <port-name-list>
[-invert_polarity]

```

NOTE: The `module_bypass` constraint supports wildcard characters.

Arguments

-name <du-name>

The name of the design unit (black box) to be bypassed.

The design unit must be a black box, that is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are treated as bypassed.

You can specify a single design unit name or a space-separated list of design unit names.

-bpin <bpin-name-list>

(Optional) List of names of the bypass pins on a design unit (black box).

-value <value-list>

(Optional) The active value (0 or 1) for all the bypass pins *<bpin-name-list>*.

The specified bypass pins and values are assumed to be mapped on one-to-one basis. You need to specify the exact same number of bypass pins and values with both the `-bpin` and `-value` arguments.

-iport <port-name-list> / -oport <port-name-list>

Input/Output port name list of the module being bypassed.

The port name list is a space-separated simple name list, as in the following example:

...

```
-iport in1 in2 -oport q qbar
...
```

Then, the specified input ports and output ports are assumed to be directly connected on one-to-one basis. The mapped ports transfer the controllability and simulation values from the input ports to the output ports and the observable values from the output ports to the input ports.

You need to specify the exact same number of ports with both the `-iport` and `-oport` arguments. If the number of ports with two arguments is different, the extra ports are ignored because they do not have corresponding elements to map and transfer values.

NOTE: *The module names, specified using the `-name` argument of `module_bypass` constraint, are automatically created as black boxes for SpyGlass DFT solution analysis.*

-invert_polarity

(Optional) Specify for inverting the polarity of output signals. This applies to all in port and out port specified with `iport/oport` pair in the constraint.

Rules

The `module_bypass` constraint is used by the following rules:

SpyGlass DFT Solution			
BIST_04	RAM_01	RAM_02	RAM_03
RAM_04	RAM_07	RAM_09	RAM_10
Scan_23	TA_06	Info_testclock	Info_testmode
SpyGlass DFT DSM Solution			
All rules			

module_pin

Purpose

The `module_pin` constraint is used to specify the set of pins for which the hierarchy of all instantiations is to be generated.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `modulePin`.

Product

SpyGlass DFT solution

Syntax

The syntax of the `module_pin` constraint is as follows:

```
module_pin
  -name <du-name>
  [ -pin <pin-name-list> ]
  [ -allpins ]
  [ -allinputs ]
  [ -alloutputs ]
  [ -exclude_module_list ]
```

Arguments

-name <du-name>

Name of the design unit for which you are specifying pins.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs).

You can only specify a single design unit name.

-pin <pin-name-list>

(Optional) Space-separated list of pin names under the specified design unit.

If you specify none of the arguments (`-pin`, `-allpins`, `-allinputs`, or `-alloutputs`), the Soc_06 rule generates hierarchy of all instantiations for all design unit pins.

-allpins

(Optional) Equivalent to specifying the `-pin` argument. Specify a space-separated list of all the pins on the specified design unit.

-allinputs

(Optional) Equivalent to specifying the `-pin` argument. Specify a space-separated list of all input pins on the specified design unit.

-alloutputs

(Optional) Equivalent to specifying `-pin` argument. Specify a space-separated list of all output pins on the specified design unit.

-exclude_module_list

(Optional) Space-separated list of modules to be excluded from generating the instance list.

NOTE: *This argument supports wildcard characters.*

Rules

The `module_pin` constraint is used by the following rule:

SpyGlass DFT Solution
Soc_06

monitor_time

Purpose

The `monitor_time` constraint is used to specify the design initialization time frames during simulation. The rest of the simulation time is considered as the design's functional time.

The SystemVerilog Assertions (SVA) generated for assumptions and partially proved rules during simulation are active in the design's functional time. These SVA assertions are stored in SystemVerilog files that are located in the `test_reports/clock-reset/assertions/` directory. For details, refer to `cdc_dump_assertions` parameter documentation in the *SpyGlass CDC Rules Reference Guide*.

Product

SpyGlass CDC solution

Syntax

The syntax of the `monitor_time` constraint is as follows:

```
monitor_time
  -type <time-frame-type>
  -frame <start-time> <end-time>
```

Arguments

-type <time-frame-type>

Specifies the type of time frame. The valid value is `initialization`.

-frame <start-time> <end-time>

Specifies a space-separated list of start and end times of the time frames of the specified type.

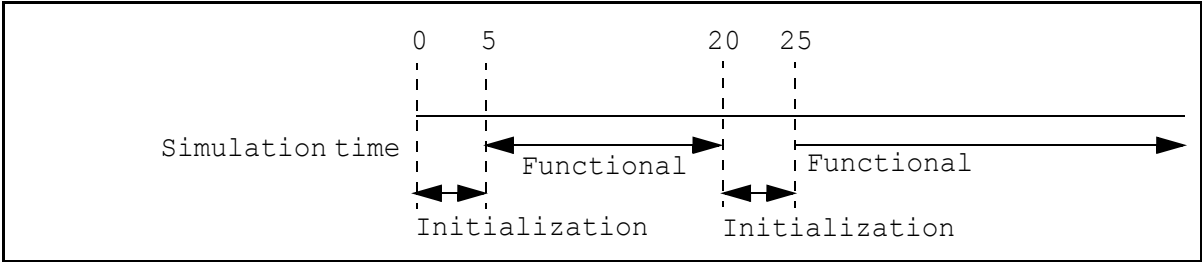
Examples

Consider the following `monitor_time` constraints:

```
monitor_time -type initialization -frame {0 5}
monitor_time -type initialization -frame {20 25}
```

The above constraints imply that the initialization time frames are from 0 to 5 seconds and then from 20 to 25 seconds.

In addition, it is also implied that the functional time frames are from 5 to 20 seconds and then from 25 seconds onwards. This is also illustrated in the following figure:



Rules

The `monitor_time` constraint is used by the following rules:

SpyGlass CDC Solution		
Ac_datahold01a	Ac_cdc01	Ac_conv02

multivt_lib

Purpose

The `multivt_lib` constraint is used to specify the VT library groups and their member libraries for the LPSVM29, LPSVM33, LPSVM33A, and LPSVM34 rules and the MTCMOS libraries for the LPSVM35 and LPSVM36 rules of the SpyGlass Power Verify solution.

Product

SpyGlass Power Verify solution

Syntax

Specifying VT Library Groups

The syntax to specify the `multivt_lib` constraint for VT Library Groups is as follows:

```
current_design <du-name>
  multivt_lib
    -type <lib-group-name>
    -names <lib-name-list>
```

Specifying MTCMOS Libraries

The syntax to specify the `multivt_lib` constraint for MTCMOS Libraries is as follows:

```
current_design <du-name>
  multivt_lib
    -type <keyword>
    -names <lib-name-list>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the VT library groups or MTCMOS libraries.

-type <lib-group-name>

Identifier for the VT library group.

<keyword>

Identifier for the MTCMOS libraries.

The supported values of *<keyword>* are:

- `mtcmOSH` (indicates an MTCMOS library with cells having supply-side sleep devices [transistors])
- `mtcomsF` (indicates an MTCMOS library with cells having ground-side sleep devices [transistors])
- `mtcmOSHf` (indicates an MTCMOS library with cells having both supply-side and ground-side sleep devices [transistors])

All other libraries are assumed to be CMOS libraries.

-name <lib-name-list>

Space-separated list of member library names (without the extension).

You can specify the names of gate (`.lib`) libraries (usually specified with the `read_file -type gateslib <file>` command in the project file) as well as the SpyGlass precompiled gate libraries (`.sglib`) libraries (usually specified by using the `read_file -type sourcelist`

`<file-name>.f` command in the project file).

Examples

Consider the following `multivt_lib` constraint specification:

```
multivt_lib -type highvt -names CORE9GPHS1 CORE9GPHS2
multivt_lib -type lowvt -names CORE9GPLL1 CORE9GPLL2
```

The above specification indicates that there is a VT library group named `highvt` of libraries named `CORE9GPHS1` and `CORE9GPHS2` (ignoring the extension). This means that you would be specifying either of the following while running SpyGlass analysis:

```
spyglass -batch ...
  -gateslib CORE9GPHS1.lib -gateslib CORE9GPHS1.lib
  ...
```

Or

```
spyglass -batch ...
  -sglib CORE9GPHS1.sglib -sglib CORE9GPHS1.sglib
  ...
```

Similarly, there is another VT library group named `lowvt` of libraries named `CORE9GPLL1` and `CORE9GPLL2` (ignoring the extension).

Rules

The `multivt_lib` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM29	LPSVM33	LPSVM33A	LPSVM34
LPSVM35	LPSVM36		

network_allowed_cells

Purpose

Specifies cells that can be allowed or disallowed in clock trees (or other

trees).

NOTE: *The network_allowed_cells constraint works for gate-level netlist cells only; all other types of cells are ignored.*

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the network_allowed_cells constraint is as follows:

```
current_design <du-name>
  network_allowed_cells
    -name <name-list>
    [ -type <type-list> ]
    [ -from <from-list> ]
    [ -disallow ]
```

Arguments

-name <name-list>

Specifies a space-separated list of names of the allowed/disallowed gate-level netlist cells.

-type <type-list>

Specifies whether only clock, reset, or both trees should be checked.

-from <from-list>

Specifies a space-separated list of names of nets from which the check should start.

You can use a combination of wildcard characters ('*' and '?') when specifying netlist cells, clock and/or reset trees, and nets.

NOTE: *Set the hier_wild_card parameter to yes to match the expression specified in this argument with hierarchies.*

-disallow

Indicates that the cells specified with the `-name` argument should be disallowed. By default, the specified cells are the only cells allowed in the network. If the `-disallow` argument is specified, these cells are the only cells NOT allowed in the network.

If you supply the `-type` argument, you must also specify the clock names, reset names, or both using the `clock` and `reset` constraints.

If you supply both the `-type` argument and the `-from` argument, the `-type` argument overrides the `-from` argument.

If you do not supply both `-type` and `-from` arguments, the complete design is checked for allowed/disallowed cells. For example, consider the following:

```
current_design top
  clock -name clk1
  network_allowed_cells -name A1234 -type clock
```

In this case, the `Clock_Reset_check01` rule reports instances of cells other than A1234 found in the clock tree of clock signal `clk1`.

Examples

Now consider the following example:

```
current_design top
  reset -name rst1
  network_allowed_cells -name A1234
    -type reset -disallow
```

In this case, the `Clock_Reset_check01` rule reports instances of A1234 found in the reset tree of reset signal `rst1`.

Another example is as follows:

```
current_design top
  network_allowed_cells -name A1234
    -from w1 -disallow
```

In this case, the `Clock_Reset_check01` rule reports instances of the A1234 found in the fan-out tree of net `w1`.

NOTE: *Only one network_allowed_cells constraint can be specified. If you want to specify constraints for multiple clocks or resets, use a single network_allowed_cells constraint.*

Rules

The network_allowed_cells constraint is used by the following rule:

SpyGlass CDC Solution
Clock_Reset_check01

no_at-speed

Purpose

The no_at-speed constraint is used to exclude flip-flops from being used in at-speed testing, even if they so qualify. The no_at-speed constraint may be used when there is no intention to use module or circuit for at-speed testing.

Product

SpyGlass DFT DSM solution

Syntax

The syntax for specifying the no_at-speed constraint is as follows:

```
no_at-speed
-name <du-name> | <net-name> | <hier-inst> |
-clock_control <signal-name> |
-set_control <signal-name> |
-reset_control <signal-name>
-register_suffix <suffixes>
-module_suffix <suffixes>
```

NOTE: *The no_at-speed constraint supports wildcard characters. Using wildcards, expression is expanded only within the hierarchy.*

Arguments

-name <du-name>

The name of the design unit from which flip-flops should be excluded in at-speed testing. You can specify design units that are single flip-flops or design units where one or more flip-flops are described besides other logic. Then, all flip-flops in the specified design unit are excluded from at-speed testing. The design unit name <du-name> can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are considered. You can specify a single design unit name or a space-separated list of design unit names.

-name <net-name>

The name of a net that is connected to the output pin of a flip-flop. Then, the corresponding flip-flop is excluded from at-speed testing. You can specify a simple net name or a hierarchical net name. The net specified as simple net name is searched at the top-level. You can specify a single net name or a space-separated list of net names.

<signal-name>

Flip-flops whose control pins (clock, set, or reset) are driven by this signal are excluded in the scan. The <signal-name> argument can have the following values:

- **CLOCK:** For the clock_control option, if the <signal-name> argument is driving CLOCK pin of the flip-flop.
- **SET:** For the set_control option, if the <signal-name> argument is driving SET pin of the flip-flop.
- **RESET:** For the reset_control option, if the <signal-name> argument is driving RESET pin of the flip-flop.

-register_suffix <suffixes>

Space-separated list of suffixes to be specified as no_atspeed. The -register_suffix argument should not be used along with other arguments of the no_atspeed constraint, that is, -name, -clock_control, -set_control, or -reset_control.

If the value of the dft_treat_suffix_as_pattern parameter is set

to on, the `register_suffix` value is used as a pattern to be matched with the register name. The pattern may be present anywhere in the register name, excluding the path.

If the value of the `dft_check_path_name_for_register_suffix` parameter is on, the value of the `-register_suffix` field will be matched with the register name along with the path in which the register is present.

`-module_suffix <suffixes>`

Define this field to use suffix based pattern match for all module names.

If the value of the `dft_treat_suffix_as_pattern` parameter is on, the value of the `-module_suffix` field will be matched with the module name along with the path in which the module is present.

Examples

You can use the `no_atspeed` constraint in the following ways:

- Specifying only the design unit names with the `-name` argument

By specifying a design unit name using the `-name` argument only, all instances of this design unit are not considered for at-speed testing. The following `no_atspeed` constraint indicates that all flip-flops within all instances of `modName1` will not be considered for at-speed testing:

```
no_atspeed -name modName1
```

- Specifying only the net names with the `-name` argument

By specifying a net name using the `-name` argument only, the corresponding flip-flop is not considered for at-speed testing. The following `no_atspeed` constraint indicates that the flip-flop whose output pin is connected to net `reg_123` (at the top-level) is not considered for at-speed testing:

```
no_atspeed -name reg_123
```

- Specifying only the hierarchical instance names with the `-name` argument

By specifying a hierarchical instance name using the `-name` argument only, all the flip-flops inside the given hierarchy are not considered for at-speed testing. The following `no_atspeed` constraint indicates that the flip-flop that lies inside the hierarchy `top.inst1` is not considered for at-

speed testing.

```
no_at-speed -name top.inst1
```

- Specifying list of suffixes using the `-register_suffix` argument:
Consider the following example:

```
current_design top_no_scan
    force_no_scan -register_suffix ff1
```

In the above example, All flip-flops with name ending with `ff1` will be marked `force_no_scan`.

Rules

The `no_at-speed` constraint is used by the following rules:

SpyGlass DFT DSM Solution		
Info_transitionCoverage	Info_noAtspeed	Atspeed_11
Info_transitionCoverage_audit		

no_convergence_check

Purpose

The `no_convergence_check` constraint is used to specify the net names that should not be checked for convergence.

If convergence is found on a gate, SpyGlass does not report a violation on that gate if you specify its output net through this constraint.

NOTE: Use the `cdc_attribute` constraint instead of the `no_convergence_check` constraint. If the `no_convergence_check` constraint is used together with the `cdc_attribute` constraint, SpyGlass CDC ignores the `no_convergence_check` constraint and honors the `cdc_attribute` constraint.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `no_convergence_check` constraint is as follows:

```
no_convergence_check -name <sig-name-list>
```

Arguments

-name <sig-name-list>

Space-separated list of hierarchical net names, hierarchical terminals, and ports that should not be checked for convergence.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

Examples

The following example shows the usage of this constraint:

```
no_convergence_check -name top.U1.net1 top.net2
```

When you specify the above constraint, SpyGlass does not check the `top.U1.net1` and `top.net2` rules for convergence.

Rules

The `no_convergence_check` constraint is used by the following rules:

SpyGlass CDC Solution		
Clock_sync03a	Clock_sync03b	Ac_conv01
Ac_conv02	Ac_conv03	

no_fault

Purpose

The `no_fault` constraint is used to prevent faulting for a module. As SpyGlass DFT solution has the means to estimate fault coverage (see

the *Info_coverage* rule), the `no_fault` constraint is useful to obtain more representative fault coverage numbers under special conditions. For example, when a design contains a synthesizable module that will be tested by some not yet implemented means, faults within each instance of that module will be included in the fault count total, thereby decreasing the fault coverage estimate. Use of the `no_fault` constraint removes such faults from consideration.

NOTE: Prior to SpyGlass 4.4 release, the name of this constraint was *nofault*.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `no_fault` constraint is as follows:

```
no_fault -name <du-name> | <inst-name>
[- fault <hier_pin_names> ]
[- net <hier_net_names> ]
[- net_input <hier_net_names> ]
[- net_output <hier_net_names> ]
[- clock_control <hier_net_names> ]
[- set_control <hier_net_names> ]
[- reset_control <hier_net_names> ]
[- register_suffix ]
[- instance_suffix ]
[- module_suffix ]
```

NOTE: The `no_fault` constraint supports wildcard characters. Using wildcards, expression is expanded only within the hierarchy.

Arguments

The `no_fault` constraint has the following arguments:

-name <du-name>

Name of the design unit whose instances are to be bypassed.

The design unit name *<du-name>* can be specified as module name (for

Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are treated as no-fault design units.

You can specify a single design unit name or a space-separated list of design unit names.

-name <inst-name>

Hierarchical name of the instance to be bypassed.

You can specify a single hierarchical instance name or a space-separated list of hierarchical instance names.

NOTE: *You can specify design unit names, hierarchical instance names, or a combination of both.*

-fault <hier_pin_names>

(Optional) Space-separated list of hierarchical names of pins or ports.

NOTE: *Do not use this argument in case of RTL design because pin names will contain generated names and will fail SGDC sanity check at the RTL.*

-net <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the faults in the direct fanin or fanout of the net as no_fault.

-net_input <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the faults in the direct fanin of the net as no_fault.

-net_output <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the faults in the direct fanout of the net as no_fault.

-clock_control <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the faults associated with the registers, where clock pin is topologically driven by the specified clock, as no_fault.

-set_control <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the

fault associated with the registers, where set pin is topologically driven by the specified set signal, as no_fault.

-reset_control <hier_net_names>

(Optional) Space-separated list of hierarchical names of nets. Mark all the fault associated with the registers, where reset pin is topologically driven by the specified reset signal, as no_fault.

-register_suffix <suffixes>

Space-separated list of suffixes to be specified as no_fault. The -register_suffix argument should not be used along with other arguments of the no_fault constraint, that is, -name, -clock_control, -set_control, or -reset_control.

If the value of the dft_treat_suffix_as_pattern parameter is set to on, the register_suffix value is used as a pattern to be matched with the register name. The pattern may be present anywhere in the register name, excluding the path.

If the value of the dft_check_path_name_for_register_suffix parameter is on, the value of the -register_suffix field will be matched with the register name along with the path in which the register is present.

-instance_suffix <suffixes>

Define this field to use suffix based pattern match for all instance names.

-module_suffix <suffixes>

Define this field to use suffix based pattern match for all module names.

If the value of the dft_treat_suffix_as_pattern parameter is on, the value of the -module_suffix field will be matched with the module name along with the path in which the module is present.

Examples

Specifying list of suffixes using the -register_suffix argument

Consider the following example:

```
R1 (register 1) name: top.u_ctrl.u2.u1.ff1_ctrl
```

```
R2 (register 2) name: top.u_ctrl.u2.u1.ff1_state
R3 (register 3) name: top.u_core.u2.u1.ff1_state_ctrl
R4 (register 4) name: top.u_ctrl_state.u2.u1.ff1_ctrl_state
```

Now, consider the following *no_fault* descriptions:

```
no_fault -register_suffix ctrl
no_fault -register_suffix state
```


The following table lists the results when combination of values are used for the *dft_treat_suffix_as_pattern* and *dft_check_path_name_for_register_suffix* parameters:

TABLE 7 Pattern Matching for the -register_suffix argument

Value of <i>dft_treat_suffix_as_pattern</i>	Value of <i>dft_check_path_name_for_register_suffix</i>	Value of -register_suffix	Matched Registers
off	off	ctrl	R1, R3
		state	R2, R4
off	on	ctrl	R1, R2, R3
		state	R2, R4
on	off	ctrl	R1, R3, R4
		state	R2, R3, R4
on	on	ctrl	R1, R2, R3, R4
		state	R2, R3, R4

Example 3: Specifying list of suffixes using the -instance_suffix argument

Consider the following example:

```

R1 (register 1) name: top.u_ctrl.u2.u1.ff1_ctrl
R2 (register 2) name: top.u_ctrl.u2.u1.ff1_state
R3 (register 3) name: top.u_core.u2.u1.ff1_state_ctrl
R4 (register 4) name: top.u_ctrl_state.u2.u1.ff1_ctrl_state

I1 (instance 1) name: top.u_ctrl.u2.u1.inst1_ctrl
I2 (instance 2) name: top.u_ctrl.u2.u1.inst1_state
I3 (instance 3) name: top.u_core.u2.u1.inst1_state_ctrl
I4 (instance 4) name:
top.u_ctrl_state.u2.u1.inst1_ctrl_state

```

Now, consider the following *no_fault* descriptions:

```
no_fault -instance_suffix ctrl
no_fault -instance_suffix state
```

The following table lists the results when combination of values are used for the *dft_treat_suffix_as_pattern* and *dft_check_path_name_for_instance_suffix* parameters:

TABLE 8 Pattern Matching for the -instance argument

Value of <i>dft_treat_suffix_</i> <i>as_pattern</i>	Value of <i>dft_check_path_n</i> <i>ame_for_instance</i> <i>_suffix</i>	Value of - <i>instance_suffix</i>	Matched Registers/ Instances
off	off	ctrl	R1, R3, I1, I3
		state	R2, R4, I2, I4
off	o	ctrl	R1, R2, R3, I1, I2, I3
		state	R2, R4, I2, I4
on	off	ctrl	R1, R3, R4, I1, I3, I4
		state	R2, R3, R4, I2, I3, I4
on	on	ctrl	R1, R2, R3, R4, I1, I2, I3, I4
		state	R2, R3, R4, I2, I3, I4

Rules

The `no_fault` constraint is used by the following rules:

SpyGlass DFT Solution	
Info_coverage	Coverage_audit
SpyGlass DFT DSM Solution	
Info_transitionCoverage	Info_transitionCoverage_audit

no_test_point

Purpose

The `no_test_point` constraint is used to exclude modules or instance, which should not be considered for suggesting test points.

Product

SpyGlass DFT DSM solution

Syntax

The syntax of the `no_test_point` constraint is as follows:

```
no_test_point
  -name <module-name | <instance_list>
```

Arguments

The `no_test_point` constraint has the following arguments:

-name <module-name | <instance_list>

Name of the module or the list of instances to be considered for suggesting the test points.

Rules

The `no_test_point` constraint is used by the `Info_random_resistance` rule.

noclockcell_start

Purpose

Specifies start points (ports or nets) for checking.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `noclockcell_start` constraint is as follows:

```
current_design <du-name>
  noclockcell_start
    -name <port-net-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the start point

-name <port-net-name>

The hierarchical name of the start point port/net.

You can specify multiple start point signals using multiple `noclockcell_start` constraints.

NOTE: *You must specify at least one `noclockcell_start` constraint for the NoClockCell rule to perform rule-checking.*

Examples

The following example specifies net `clk1` of design unit `top` as the start point:

```
current_design top
  noclockcell_start -name top.clk1
```

Rules

The `noclockcell_start` constraint is used by the following rule:

SpyGlass CDC Solution
NoClockCell

noclockcell_stop_instance

Purpose

Specifies the instance where the NoClockCell rule should stop further traversal along the clock tree when the clock pin of the specified instance is hit.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `noclockcell_stop_instance` constraint is as follows:

```
current_design <du-name>
    noclockcell_stop_instance
        -name <inst-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the stop point instance

-name <inst-name>

The hierarchical name of the stop point instance.

You can specify multiple stop point instances using multiple `noclockcell_stop_instance` constraints.

Examples

The following example specifies instance I21 as the stop point instance under the design unit `top`:

```
current_design top
  noclockcell_stop_instance -name top.I21
```

Here, the NoClockCell rule stops further traversal along the clock tree when the clock pin of instance `top.I21` is hit.

The clock traversal automatically stops at instances of a stopped design unit (stopped using the `set_option stop <du-name>` command in the project file or the equivalent option in Atrenta Console GUI). Therefore, you need not specify such design unit instances with the `noclockcell_stop_instance` constraint.

Rules

The `noclockcell_stop_instance` constraint is used by the following rule:

SpyGlass CDC Solution
NoClockCell

noclockcell_stop_module

Purpose

Specifies the design unit where the NoClockCell rule should stop further traversal along the clock tree when the clock pin of an instance of the specified design unit is hit.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `noclockcell_stop_module` constraint is as follows:

```
current_design <du-name>
  noclockcell_stop_module
    -name <du-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the stop-point design unit.

-name <du-name>

Name of the stop-point design unit.

You can specify multiple stop point design units using multiple `noclockcell_stop_module` constraints.

Examples

The following example specifies design unit `m1` as the stop-point design unit under the design unit `top`:

```
current_design top
  noclockcell_stop_module -name m1
```

Here, the `NoClockCell` rule stops further traversal along the clock tree when the clock pin of an instance of design unit `m1` is hit.

The clock traversal automatically stops at instances of a stopped design unit (stopped using the `set_option stop <du-name>` command in the project file or the equivalent option in Atrenta Console GUI). Therefore, you need not specify such design units with the `noclockcell_stop_module` constraint.

Rules

The `noclockcell_stop_module` constraint is used by the following

rule:

SpyGlass CDC Solution

NoClockCell

noclockcell_stop_signal

Purpose

Specifies the design points (ports, pins, or nets) where the NoClockCell rule should stop further traversal along the clock tree.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the `noclockcell_stop_signal` constraint is as follows:

```
current_design <du-name>
  noclockcell_stop_signal
    -name <sig-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) under which you are specifying the stop point.

-name <sig-name>

The hierarchical name of the end point port/pin/net.

You can specify multiple stop points using multiple `noclockcell_stop_signal` constraints.

Example

The following example specifies pin `d1` of instance `I1` in design unit `top` as the stop point:

```
current_design top
  noclockcell_stop_signal -name top.I1.d1
```

Here, the `NoClockCell` rule stops further traversal along the clock tree when the pin `top.I1.d1` is hit.

The clock traversal automatically stops at the design points (ports, pins, or nets) that belong to instances of a stopped design unit (stopped using the `set_option stop <du-name>` command in the project file or the equivalent option in Atrienta Console GUI). Therefore, you need not specify such design points with the `noclockcell_stop_signal` constraint.

Rules

The `noclockcell_stop_signal` constraint is used by the following rule:

SpyGlass CDC Solution
NoClockCell

non_pd_inputcells

Purpose

The `non_pd_inputcells` constraint is used to specify the cells that should not be present at the input stage of the power/voltage domain.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `non_pd_inputcells` constraint is as follows:

```

current_design <du-name>
  non_pd_inputcells
    -names <cell-name-list>
    [ -pd <pd-name> ]
    [ -pins <pin_list> ]

```

Arguments

<du-name>

Name of the design unit under which you are specifying the input cells.

-names <cell-name-list>

Space-separated list of cell names. You can use wildcard characters while specifying cell names using the -names argument.

-pd <pd-name>

Name of the always-on domain/power domain for which the specified cells are to be checked.

If you do not specify the -pd argument, SpyGlass assumes that the specified cells are to be checked for all specified power/voltage domains.

<pin-list>

Name of the input pins of cells, which should not be present in at the input. When the pin list is not specified, all input pins are considered.

Rules

The `non_pd_inputcells` constraint is used by the following rule:

SpyGlass Power Verify Solution

LPSVM50

num_flops

Purpose

Specifies the minimum number of flip-flops required in synchronizer chain for the Conventional Multi-Flop Synchronization Scheme.

Product

SpyGlass CDC solution

Syntax

The syntax of the `num_flops` constraint is as follows:

```
num_flops
  -to_domain <dest-clk-domain>
  -value <num>
  [ -from_domain <src-clk-domain> ]
  [ -cells <library-cell-names> ]
  [ -lib <library-names> ]
  [ -reset ]
  [ -rdc ]
```

or

```
num_flops
  -to_clk <dest-clk-name>
  -value <num>
  [ -from_clk <src-clk-name> ]
  [ -cells <library-cell-names> ]
  [ -lib <library-names> ]
  [ -reset ]
  [ -rdc ]
```

or

```
num_flops
  -to_period <dest_clk_period>
  -value <num>
  [ -cells <library-cell-names> ]
  [ -lib <library-names> ]
  [ -reset ]
```

```
[ -rdc ]

or

num_flops -default <def_val>
[ -cells <library-cell-names> ]
[ -lib <library-names> ]
[ -reset ]
```

NOTE: Please note the following points:

- 📄 You must specify the `-to_clk` argument while specifying the `-from_clk` argument.
- 📄 You must specify the `-to_domain` argument while specifying the `-from_domain` argument.

Arguments

The syntax of using the `num_flops` constraint in a SpyGlass Design Constraints file is as follows:

`-from_domain <src-clk-domain>`

The domain name of the source clock.

`-to_domain <dest-clk-domain>`

The domain name of the destination clock.

`-from_clk <src-clk-name>`

Name of the source clock.

`-to_clk <dest-clk-name>`

Name of the destination clock.

`-to_period <dest-clk-period>`

Period of the destination clock. The value is applicable to all the crossings with destination clocks that have period less than or equal to `<dest-clk-period>`.

-value <num>

The minimum number of flip-flops required in a synchronizer chain for the crossing specified through:

- The source clock specified by *-from_clk <src-clk-name>* or the source clock of the domain specified by *-from_domain <src-clk-domain>*.
- The destination clock specified by *-to_clk <dest-clk-name>* or the destination clock of the domain specified by *-to_domain <dest-clk-domain>*.

The synchronizer chain in this case is the structure of the *Conventional Multi-Flop Synchronization Scheme*.

NOTE: *The minimum value for this argument is 1. However, if you specify the *-reset* argument, the minimum value for this argument should be 2.*

For domain pairs not mentioned with this constraint, the number of flip-flops to be used for multi-flop synchronization is the value of the parameter, `num_flops`, which is 2 by default.

-default <def_val>

The number of flip-flops for crossings not specified with the `num_flops` constraint.

Some of the examples of the `num_flops` constraint are as follows:

```
num_flops -from_clk clk1 -to_clk clk2 -value 3
```

The above specification specifies that for multi-flop synchronization, at least three flip-flops should be used for the crossings between the clocks, `clk1` and `clk2`.

```
num_flops -from_domain D1 -to_domain D2 -value 3
```

The above specification specifies that for multi-flop synchronization, at least three flip-flops should be used for the crossings between all the clocks in domains, `D1` and `D2`.

```
num_flops -to_period 20 -value 4
```

The above specification specifies that for all the crossings with destination clocks that have period less than or equal to 20, at least four flip-flops should be used for multi-flop synchronization.

```
num_flops -default 2
```

The above specification specifies that for all the clock crossings for which the `num_flops` constraint is not specified, at least two flip-flops should be used for multi-flop synchronization.

-cells <library-cell-names>

Space-separated list of library cell names. You can use wildcard characters while specifying library cell names.

The following example shows the usage of the `-cells` argument:

```
num_flops -from_domain D1 -to_domain D2 -cells X_CELL  
-value 3
```

In this case, for crossings from D1 to D2, the `X_CELL` cell is allowed to be a part of the synchronizer chain and there should be minimum of three such cells in the chain, including the destination instance.

-lib <library-names>

Space-separated list of library names.

The following example shows the usage of the `-lib` argument:

```
num_flops -from_domain D1 -to_domain D2 -lib LIB1  
-value 3
```

In this case, for crossings from D1 to D2, only the cells from the `LIB1` library are allowed to be a part of the synchronizer chain and there should be minimum three such cells in the chain, including the destination instance.

-reset

Applies the `num_flops` constraint on reset synchronizers with respect to a clock, domain, or frequency.

When you specify this argument, the minimum value specified to the `-value <num>` argument should be 2.

NOTE: You cannot specify the `-from_clk/-from_domain` arguments along with the `-reset` argument.

-rdc

Enables the `Ar_resetcross01` rule to apply the `num_flops` constraints on

reset domain crossings.

It is used to set a limit on the number of synchronizer flip-flops.

When you specify this argument, the minimum value specified to the *-value <num>* argument should be 2.

Rules

The `num_flops` constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_sync01	Ac_sync02	Ac_unsync01	Ac_unsync02
Ar_resetcross_matrix01	Ar_cross_analysis01	Clock_sync08	Clock_sync03a
Clock_sync03b	Clock_sync08a	Clock_sync09	Ac_handshake01
Ac_handshake02	Ac_glitch03	Ac_cdc01a	Ac_cdc01b
Ac_cdc01c	Ac_cdc08	Ac_conv01	Ac_conv02
Ac_conv03	Ar_sync01	Reset_sync04	Reset_sync01
Ar_unsync01	Reset_sync03	Ar_resetcross01	Ac_conv04

operating_mode_set

Purpose

The `operating_mode_set` constraint is used to assign modes to a mode set.

For example, you can create a mode set, `SPEED`, that contains a set of two modes, `FAST` and `SLOW`.

Refer to the [Example](#) section for more details.

See also the PESAE08 rule documentation

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `operating_mode_set` constraint is as follows:

```
operating_mode_set
  -name <mode_set_name>
  -values <mode_names>
```

Arguments

-name <mode_set_name>

Specifies the name of a mode set

-values <mode_names>

Specifies mode names that should be included in the mode set

Example

A design can operate in multiple modes at a time. For example, it can operate in TX mode, RX mode, SLOW mode, or FAST mode.

You can group all related modes under one category, where each category is known as a mode set.

For example, you can group the TX and RX mode under one mode set, STATE by specifying the following constraint:

```
operating_mode_set -name STATE -values TX RX
```

The following specification shows how to create a mode set, SPEED, with SLOW and FAST modes by using the `operating_mode_set` constraint:

```
operating_mode_set -name SPEED -values SLOW FAST
```

Rules

The `operating_mode_set` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

All rules running in EST mode

output

Purpose

The `output` constraint is used to specify clock domain at output ports. If you want to check synchronization for paths ending on output ports, you need to provide information about the destination clocks associated with those ports. The `output` constraints are used for this purpose.

You need to specify effective source clocks or destination clocks using the `clock` keyword and port-clock pairs using the `output` keyword in a SpyGlass Design Constraints file.

Product

SpyGlass CDC solution

Syntax

The syntax of using the `output` constraint in a SpyGlass Design Constraints file is as follows:

```
current_design <du-name>
  clock -name <clk-name> -domain <domain-name>
  output -name <output-name-list>
  -clock <dest-clk-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-name <clk-name>

The clock signal name or a virtual clock name.

-domain <domain-name>

The clock domain name.

-name <output-name-list>

Port names that can be scalar ports, bus ports, or wildcard names (matching against all top-level ports of appropriate type).

If you specify the same output port in multiple output constraint specifications, SpyGlass considers the last output constraint specification. Consider the following example in which the same output port, out1, is specified in two different output constraint specifications:

```
output -name out1 -clock clk2
output -name out1 -clock clk4
```

In the above example, SpyGlass considers the last output constraint specification, that is, output out1 clocked by clock clk4.

-clock <dest-clk-name>

The name of the destination clock for a primary output port or a virtual clock specified by the *-tag <logical-clock-name>* argument of the *clock* constraint. For virtual clock specification, see [Example 2](#).

You can use the match many (*) and match one (?) wildcard characters with the -name argument of the output constraints by specifying the regular expression enclosed in double quotes ("").

NOTE: Do not specify clock domain names to this argument.

Examples

Example 1

Consider the following output constraint specification:

```
output -name "out?" -clock clk1
```

The above specification matches all output ports whose names start with out followed by zero or one character, such as out, out1.

Example 2

Consider the following constraints:

```
clock -name clk1 -domain d1
clock -name clk2 -domain d2 -tag c1
clock -tag c2 -domain d3

output -name out1 -clock c2
```

In the above specification, the virtual clock `c2` is specified to the `-clock` argument of the `output` constraint. However, note that you cannot specify `c1` to the `-clock` argument as `c1` is not a virtual clock.

Rules

The `output` constraint is used by the following rules:

SpyGlass CDC Solution

All clock synchronization rules except the `Clock_sync06` rule

output_not_used

Purpose

In a multi-flop synchronizer, if the synchronizing flip-flops feed more than one destination, the value seen by the destination may be subject to cycle uncertainty or even go metastable, making the synchronizer ineffective. Synchronization check will fail if synchronizing flip-flops feed multiple destinations.

It is often possible that a design has synchronizing flip-flops with multiple fan-outs where only one fan-out is active at a time. Provide case analysis that will enable only one fan-out thereby preventing *Ac_unsync01/* *Ac_unsync02* violations.

There can be cases in which a fan-out is feeding a primary output (ignoring buffers and inverters) and case-analysis cannot be used, as shown in the

following figure.

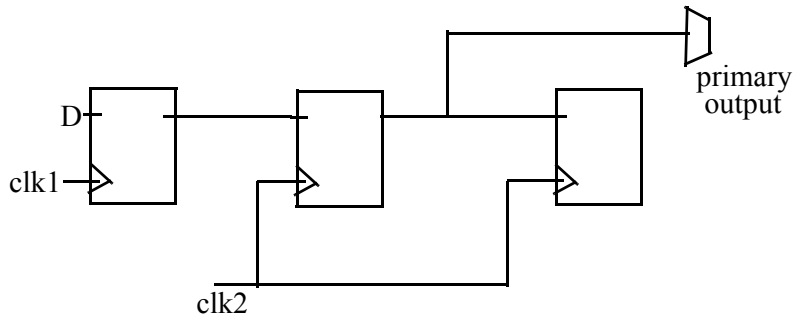


FIGURE 37. Fan-out Feeding Primary Output

In such cases, you may use the constraint, `output_not_used`.

Product

SpyGlass CDC solution

Syntax

Use the `output_not_used` constraint as follows to specify the name of the primary output port so that the connection is ignored while checking for synchronization:

```
current_design <du-name>
  output_not_used -name { <port-name> }
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-name <port-name>

The name of the primary output port.

You can use a combination of wildcard characters ('*' and '?') when

specifying port names.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

Rules

The `output_not_used` constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_sync01	Ac_sync02	Clock_sync08	Clock_sync03a
Clock_sync03b	Clock_sync08a	Clock_sync09	Propagate_clocks
Ac_cdc01a	Ac_cdc01b	Ac_cdc01c	Ac_cdc08
Ac_conv02	Ac_conv03	Ac_handshake01	Ac_handshake02
Ac_conv01	Ac_unsync01	Ac_unsync02	

pg_cell

Purpose

The `pg_cell` constraint is used to specify the names of power/ground pins for cells present in the input netlist, which are missing from the respective PLIB/LIB/LEF libraries.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `pgcell`.

Product

SpyGlass Power Verify solution, SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `pg_cell` constraint is as follows:

```
current_design <du-name>
  pg_cell
    -name <lib-group-name>
    [ -powerTerms <term-name-list> ]
```

```
[ -groundTerms <term-name-list> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the PG cell pins.

-name <lib-group-name>

Specifies an identifier for a PG cell.

You can use wildcard characters while specifying cell names by using the -name argument.

-powerTerms <term-name-list> / -groundTerms <term-name-list>

Space-separated list of member power/ground terminal names.

Examples

The following example declares a PG cell named PX_120_LS having power terminal named VDDEXT and ground terminal named VSSEXT:

```
pg_cell -name PX_120_LS
       -powerTerms VDDEXT -groundTerms VSSEXT
```

The following example declares a PG cell named PX_150_LS having power terminals named VDDEXT and VDDBULK and ground terminals named VSSEXT and VSSBULK:

```
pg_cell -name PX_150_LS
       -powerTerms VDDEXT VDDBULK
       -groundTerms VSSEXT VSSBULK
```

Rules

The pg_cell constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPPLIB04	LPPLIB05	LPPLIB06	LPPLIB07
LPPLIB08	LPPLIB11	LPPLIB15	LPPLIB16

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

All rules running in EST mode

pg_pins_naming

Purpose

Specifies power/ground pin names for single supply cells that need to be checked.

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was pgpins_naming.*

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `pg_pins_naming` constraint is as follows:

```
current_design <du-name>
  pg_pins_naming
    [ -power <pwr-pin-names>]
    [ -ground <gnd-pin-names>]
    [ -biaspower <bias-pwr-pin-names>]
    [ -biasground <bias-gnd-pin-names>]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the power/ground pin names for single supply cells.

-power <pwr-pin-names> & -ground <gnd-pin-names>

Simple pin name lists or lists of regular expressions. You can use wildcard characters while specifying power/ground pin names.

-biaspower <bias-pwr-pin-names> & -biasground <bias-gnd-pin-names>

Simple bias pin name lists or lists of regular expressions. You can use wildcard characters while specifying power/ground pin names.

Rules

The pg_pins_naming constraint is used by the following rules:

SpyGlass Power Verify Solution		
LPSVM49	LPPLIB18A	LPPLIB18B

pin_voltage

Purpose

Specifies voltage/power domain for primary ports, pins of design units, or instances in the design.

NOTE: The *voltage_domain* constraint allows you to specify the voltage/power domains of design units and instances. Use the *pin_voltage* constraint to specify voltage/power domains of pins.

The different ways to specify the pin_voltage constraint are as follows:

- Specify voltage/power domain for all pins of all instances of a specified design unit

Use the -module and -default arguments, as in the following example:

```
current_design top
  voltage_domain -name VD1 ...
  pin_voltage -voltage VD1 -module my_block -default
```

The above specification indicates that the voltage/power domain of all pins of all instances of design unit my_block is VD1.

- Specify voltage/power domain for named pins of all instances of a specified design unit

Use the -module and -names arguments, as in the following

example:

```
current_design top
  voltage_domain -name VD1 ...
  pin_voltage -voltage VD1 -module my_block -names A D
```

The above specification indicates that the voltage/power domain of pins named A and D of all instances of design unit `my_block` is VD1.

- Specify the voltage/power domain for all pins of the specified instance
Use the `-instance` and `-default` arguments as in the following example:

```
current_design top
  voltage_domain -name VD1 ...
  pin_voltage -voltage VD1 -instance top.U1 -default
```

The above specification indicates that the voltage/power domain of all pins of instance U1 under design unit `top` is VD1.

- Specify the voltage/power domain for named pins of the specified instance
Use the `-instance` and `-names` arguments, as in the following example:

```
current_design top
  voltage_domain -name VD1 ...
  pin_voltage -voltage VD1 -instance top.U1 -names A D
```

The above specification indicates that the voltage/power domain of pins named A and D of instance U1 under design unit `top` is VD1.

- Specify the voltage/power domain for selected pins of the bus of the specified instance
Use the `-instance` and `-names` arguments as in the following example:

```
current_design top1
  voltage_domain -name Vtop -value 1.2 -modname top1
  pin_voltage -voltage Vtop -instance top1.mid1 \
    -names in[0:2]
```

The above specification indicates that the voltage/power domain of pins named `in[0]`, `in[1]`, and `in[2]` of instance `mid1` under design unit

top1 is Vtop.

Please note the following:

- A `pin_voltage` specification using the `-instance` argument overrides the `pin_voltage` constraint using the `-module` argument for the same instance.

For example, assume design unit M1 has two instances U1 and U2 in design unit top. If you specify the following `pin_voltage` constraints, the inferred voltage/power domain for pin D of instance U2 under design unit top is VD2:

```
...
pin_voltage -voltage VD1 -module M1 -default
...
pin_voltage -voltage VD2 -instance top.U2 -names D
...
```

Here, the inferred voltage/power domain for all pins of instance U1 under design unit top and all pins except pin D of instance U2 under design unit top is VD1.

- The inferred voltage/power domain for all pins for which a `pin_voltage` constraint is not specified is the same as voltage/power domain of their parent instance.
- All voltage/power domain checking is applicable to pins specified with the `pin_voltage` constraint just like for design units and instances specified with the [voltage_domain](#) constraint.
- You can also specify external voltage/power domains (created with the standalone `-external` argument of the [voltage_domain](#) constraint) with the `-voltage` argument of the `pin_voltage` constraint.

The `pin_voltage` constraint overrides the [voltage_domain](#) constraint for pins. Therefore, if you have specified the voltage/power domain for a pin with both these constraints, the `pin_voltage` constraint information is used and the `voltage_domain` constraint information is ignored.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `pinvoltage`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `pin_voltage` constraint is as follows:

```
current_design <du-name>
  pin_voltage
    -voltage <vpd-name>
    [ -module <du-name> -default ]
    | [ -module <du-name> -names <pin-name-list> ]
    | [ -instance <du-name> -default ]
    | [ -instance <du-name> -names <pin-name-list> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the voltage/power domain for pins.

-voltage <vpd-name>

Name of a voltage domain or a power domain already specified using the [voltage_domain](#) constraint.

You can use wildcard characters while specifying module names (using the `-module` argument) and instance names (using the `-instance` argument).

Rules

The `pin_voltage` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM04A	LPSVM04B	LPSVM04C	LPSVM04D

pll

Purpose

The `pll` constraint is used to specify the PLL (Phase Lock Loops) modules.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `pll` constraint is as follows:

```
pll -name <mod-name>  
    -clkin <port-name>  
    -clkout <port-name>  
    [-reset <rst-port-name>  
     -value <value-that-resets-the-pll>]
```

NOTE: *The `pll` constraint supports wildcard characters.*

Arguments

The `pll` constraint has the following arguments:

-name <mod-name>

The PLL module name.

NOTE: *Any module declared as a PLL will automatically be created as a black box in SpyGlass DFT DSM solution.*

-clkin <port-name> / -clkout <port-name>

Input/Output port name of the `pll` module.

Consider the following example:

```
...  
-clkin in1 -clkout qbar  
...
```

NOTE: *Only one `clkin` port name is allowed.*

-reset <rst-port-name>

(Optional) Name of the reset pin on the design unit (black box). You can specify only a single pin name.

-value <value-that-resets-the-pll>

(Optional) The active value (0 or 1) for the pin <rst-port-name>.

Rules

This pll constraint is used by the following rules:

SpyGlass DFT Solution	
All rules	
SpyGlass DFT DSM Solution	
PLL_01	PLL_02

port_time_delay

Purpose

Specifies the design units to be checked by the PortTimeDelay rule.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was porttimedelay.

Product

SpyGlass CDC solution

Syntax

The syntax for specifying the port_time_delay constraint is as follows:

```
current_design <du-name>
  port_time_delay
    -name <ptddu-name>
    [ -ignore_instances <instance-list> ]
    [ -ignore_ports <port-name-list> ]
```

```
[ -notimedelay_ports <ntdport-name-list> ]
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in *<entity-name>.<arch-name>* format (for VHDL designs) under which you are specifying the design unit *<ptddu-name>* to be checked.

-ignore_instances <instance-list>

(Optional) Allows you to specify a space-separated hierarchical name list *<instance-list>* of instances that should be ignored by the PortTimeDelay rule, if found in the path.

-ignore_ports <port-name-list>

(Optional) Allows you to specify a space-separated name list *<port-name-list>* of ports of the design unit *<ptddu-name>* specified with the -name argument that should be ignored by the PortTimeDelay rule.

NOTE: Set the *hier_wild_card* parameter to yes to match the expression specified in this argument with hierarchies.

-notimedelay_ports <ntdport-name-list>

(Optional) Allows you to specify a space-separated name list *<ntdport-name-list>* of ports of the design unit *<ptddu-name>* specified with the -name argument that should be checked for unexpected time delay value by the PortTimeDelay rule.

NOTE: Set the *hier_wild_card* parameter to yes to match the expression specified in this argument with hierarchies.

Rules

The *port_time_delay* constraint is used by the following rule:

SpyGlass Design Constraints

SpyGlass CDC Solution

PortTimeDelay

power_data

Purpose

Specifies the details of files from which power data is to be taken.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `power_data` constraint is as follows:

```
power_data
  -format <CPF | UPF> ]
  -file <file-name-list>
  -version <version-number>
```

Arguments

-file <file-name-list>

List of files that contain power data.

NOTE: For the CPF or UPF flow, SpyGlass also honors the following SGDC commands:

activity	always_on_buffer	assume_path
cell_hookup	cell_pin_info	cell_tie_class
multivt_lib	non_pd_inputcells	power_data
pg_pins_naming	power_down_sequence	ram_instance
set_case_analysis	switchoff_wrapper_instance	
special_cell	assertion_signal	clock
power_down	ram_switch	

-version <version-number>

Specify this argument as 1 or 1.0 (for UPF 1.0) and 2 or 2.0 (for UPF 2.0). This would set the default UPF version.

Rules

The `power_data` constraint is used by the following rules:

SpyGlass Low Power			
LPPLIB04	LPPLIB07	LPLSH01	LPLSH02
LPLSH03	LPLSH04	LPLSH05	LPSVM04A
LPSVM04B	LPSVM04C	LPSVM04D	LPSVM04E
LPSVM17	LPSVM24	LPISO01	LPISO02
LPISO03	LPISO04A	LPISO04B	LPISO04C
LPISO05	LPISO06A	LPISO06B	LPSVM08
LPSVM09	LPSVM010	LPSVM12A	LPSVM12B
LPSVM22	LPSVM26	LPSVM31	LPSVM47
LPSVM51	LPSVM60	LPPLIB10	LPSVM38
LPSVM56B	LPSVM57	LPSVM59	LPCONN01
LPCONN02	LPCONN03	LPPLIB06	LPPLIB15
LPPLIB16	LPPLIB17	LPPLIB18A	LPPLIB18B
LPPLIB19A	LPSUP03	LPPSW01	LPPSW02
LPPSW03	LPPSW04	LP_DECOMPILE_ CONSTR	UPF_lowpower02
UPF_lowpower03	UPF_lowpower04	UPF_lowpower05	UPF_lowpower08
UPF_lowpower11	UPF_lowpower12	UPF_lowpower13	UPF_lowpower14
UPF_lowpower16			

power_down

Purpose

Specifies power-down conditions as used by the LPSVM28 rule.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `power_down` constraint is as follows:

```
current_design <du-name>
  power_down
    [ -domain <pd-name> ]
    -signame <sig-name-list>
    -value <value-list>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the power-down conditions.

-domain <pd-name>

Power domain name.

When you specify a power domain name using the `-domain` argument, the LPSVM28 rule of the *SpyGlass Power Verify* solution checks whether the specified pin(s)/net(s) attain the expected value when the specified power domain is switched off. If you do not specify the `-domain` argument of the `power_down` constraint, the LPSVM28 rule checks whether the specified pin(s)/net(s) attain the expected value when all power domains are switched off.

-signame <sig-name-list>

Space-separated list of pin/net names that are to be checked.

You can use wildcard characters (*) while specifying the pin/net names.

-value <value-list>

Space-separated list of expected values.

NOTE: For the `-value` argument of the `power_down` constraint, use only 0 or 1 to specify expected values. The constraint check reports any other value specified.

Examples

The following specification indicates that all pins of instance `top.mod1` are to be expected to attain a value of 1 when the power domain V3 is switched off:

```
power_down -domain V3 -signame "top.mod1.*" -value 1
```

The following specification indicates that all pins of instance `top.I1` with names starting with `vd_in` are to be expected to attain a value of 0 when the power domain V3 is switched off:

```
power_down -domain V3 -signame "top.I1.vd_in*" -value 0
```

You can also specify vector nets using the `power_down` constraint. These can be specified in different ways. For example, a 4-bit input signal can be specified as follows:

```
power_down -domain V3 -signame top.I1.vd_in[0:3] -value 0
```

```
power_down -domain V3 -signame top.I1.vd_in -value 0
```

```
power_down -domain V3 -signame top.I1.vd_in[0] -value 0 -
signame top.I1.vd_in[1] -value 1 -signame top.I1.vd_in[2] -
value 0 -signame top.I1.vd_in[3] -value 1
```

Rules

The `power_down` constraint is used by the following rule:

SpyGlass Power Verify Solution

LPSVM28

power_down_sequence

Purpose

The `power_down_sequence` constraint is used to specify the registers that should be connected to the specified power-down signal.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `pdsequence`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `power_down_sequence` constraint is as follows:

```
current_design <du-name>
  power_down_sequence
    -signalname <sig-name>
    -instnames <inst-name-list>
    [ -ignorecells <ignorecell-name-list> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the power-down signals.

-signalname <sig-name>

Name of the power-down signal to be checked.

-instnames <inst-name-list>

Space-separated list of register cell instance names. You can use wildcards while specifying the cell instance names.

-ignorecells <ignorecell-name-list>

Space-separated list of cells to be ignored while traversing the fan-out of the specified power-down signal. You can use wildcard characters while specifying cells to be ignored using the `-ignorecells` argument.

Examples

The following example specifies that signal `vdd` should be directly connected to instances `top.mid.inst1`, `top.mid.inst2` and `top.mid.inst3` only (ignoring instances of cells `isoBuf` and `isoAnd` while traversing the fan-out of the power-down signal):

SpyGlass Design Constraints

```
power_down_sequence
  -signalname vdd
  -instnames top.mid.inst1 top.mid.inst2 top.mid.inst3
  -ignorecells isoBuf isoAnd
```

Rules

The `power_down_sequence` constraint is used by the following rule:

SpyGlass Power Verify Solution

LPSVM41

power_management_test_control_cell**Purpose**

A `power_management_test_control_cell` is used to specify PMUWRs in the design.

Product

SpyGlass DFT DSM solution

Syntax

The syntax to specify the `power_management_test_control_cell` constraint is as follows:

```
power_management_test_control_cell
  -name <module-name>
  -control_output <control_pin>
```

Arguments

-name <module-name>

The name of the module to be declared as PMUWR.

-control_output <control_pin>

Output control pin of PMUWR.

Rules

The `power_management_test_control_cell` constraint is used by the following rules:

SpyGlass DFT DSM Solution			
SP_02	SP_03	SP_04	SP_05

power_management_unit

Purpose

A `power_management_unit` is used to specify PMUs in the design.

Product

SpyGlass DFT DSM solution

Syntax

The syntax to specify the `power_managament_unit` constraint is as follows:

```
power_management_unit
  -name <module-name>
```

Arguments

-name <module-name>

The name of the module to be declared as PMU. This is a mandatory field.

Rules

The `power_management_unit` constraint is used by the following rules:

SpyGlass DFT DSM Solution		
SP_02	SP_03	SP_04

power_rail_mapping

Purpose

The `power_rail_mapping` constraint specifies a mapping between the supply rail(s) in design and power rails in the technology library for an instance of the design.

If there are multiple technology libraries, you need to specify a mapping with each library separately.

NOTE: *The supply rails in design can be specified by using the [supply](#) constraint.*

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `power_rail_mapping` constraint is as follows:

```
current_design <top-du-name>
  power_rail_mapping
    [ -design <inst-name> ]
    -lib <lib-name>
    -lib_rail <lib-rail-name>
    -design_rail <inst-rail-name>
    -default_lib_rail
```

Arguments

<top-du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-design <inst-name>

Name of the instance (under the environment *<top-du-name>*) for which you are specifying the instance-to-supply rail mapping. If you do not specify an instance name using the `-design` argument, the specified instance-to-supply rail mapping is applicable to all instances under the environment *<top-du-name>* unless overridden by another `power_rail_mapping` constraint specifying a particular instance.

NOTE: *The -design argument is optional.*

-lib <lib-name>

Name of library from which some gates are mapped to the gates in instance *<inst-name>*.

-lib_rail <lib-rail-name>

Name of library supply applicable for instance *<inst-name>*.

-default_lib_rail

The `-default_lib_rail` argument is used to specify default supply rail for the libraries where `default_power_rail` attribute is not specified.

NOTE: *You should not use -default_lib_rail along with the -lib_rail argument. This will result in a fatal error generated by the SGDC_power_est25 rule.*

-design_rail <inst-rail-name>

Name of the supply rail (defined using the `supply` constraint) that maps to the library supply *<lib-rail-name>* for instance *<inst-name>*.

Example

Consider a library, `lib1`, which has three power rails, `rail1`, `rail2`, and `rail3`. Also, consider a top-level design, `top`, and an instance, `inst`, instantiated in that design. Design, `top`, has two supply rails, `VDD1` and `VDD2`, specified by using the *supply* constraint.

Now, if you want to map the design rail, `VDD1`, with power rails, `rail1`

and rail2, and map design rail, VDD2, with power rail, rail3, use the power_rail_mapping constraint, as shown below:

```
power_rail_mapping -design top -design_rail VDD1 -lib lib1 -
lib_rail rail1
power_rail_mapping -design top -design_rail VDD1 -lib lib1 -
lib_rail rail2
power_rail_mapping -design top -design_rail VDD2 -lib lib1 -
lib_rail rail3
```

The above commands connect library rails, rail1 and rail2, to design rail, VDD1, and library rail, rail3, to design rail, VDD2. The power values corresponding to rail1 and rail2 are reported in VDD1. Similarly, power values corresponding to rail3 are reported in VDD2.

If you want to specify a separate mapping for inst, then specify the following commands:

```
power_rail_mapping -design top.inst -design_rail VDD1 lib
lib1 -lib_rail rail1
power_rail_mapping -design top.inst -design_rail VDD2 lib
lib1 -lib_rail rail3
```

NOTE: *If a mapping between library rail and supply rail is not defined, the power corresponding to the library rail is reported in an internally generated supply rail, OTHER_RAILS.*

Rule

The power_rail_mapping constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions		
PEPWR01	PEPWR02	poweraudit

power_state

Purpose

Specifies the legal combinations of domain states (voltage values), that is, those combinations of domain states that can exist at the same time during operation of the design.

NOTE: *The voltage specified in the `power_state` constraint is applied for the respective domain. However, if you do not specify a domain in any of the defined power states, the corresponding voltage value is taken from the [voltage_domain](#) constraint specification.*

NOTE: *For the SpyGlass Power Estimation and SpyGlass Power Reduction solutions, only one power state is honored at a time, currently.*

Product

SpyGlass Power Verify solution, SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `power_state` constraint is as follows:

```
current_design <du-name>
  power_state
    -name <pd-condition-name>
    -domains <domain-name-list>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the power state conditions.

-name <pd-condition-name>

Name of the power state condition.

-domains <domain-name-list>

Domain name@voltage-value. For example:

```
power_state -name LL -domains VA@0.8 VB@0.8
```

NOTE: *A domain that is declared as voltage domain (always-on) is always on. However, a*

domain declared as power domain (switchable) can be either on or off. Each valid state for the full design will have some domains on and some domains off. The power_state constraint describes one power state. You can use the constraint to specify all power domains that are on in a specific power mode. Each power domain specified by the -domains argument list is powered on, and the other power domains that are not listed are powered off.

Rules

The power_state constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM08	LPSVM09	LPSVM10	LPSVM23
LPSVM47	LPSVM48	LPSVM60	
SpyGlass Power Estimate			
All rules that use voltage information			

power_switch

Purpose

The power_switch constraint is used to specify the power switches in power domains.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was powerswitch.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the power_switch constraint is as follows:

```
current_design <du-name>
  power_switch
    -name <name>
```

```
[ -pwroutpin <out-pin-name> ]
[ -pwrinpin <in-pin-name> ]
[ -en_inv_in <en-in-pin-name> ]
[ -en_inv_in_2 <en-in-pin2-name> ]
[ -en_inv_out <en-out-pin-name> ]
[ -en_buf_in <en-buf-pin-name> ]
[ -enableval <0 | 1> ]
[ -enableval_2 <0 | 1> ]
[ -exclude <pin-name-list> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the power switch.

-name <name>

Name of the power switch. You can specify names using wildcards.

-pwroutpin <out-pin-name>

Name of the power-out terminal

-pwrinpin <in-pin-name>

Name of the power-in terminal

-en_inv_in <en-in-pin-name>

Name of the enable input terminal

-en_inv_in_2 <en-in-pin2-name>

Name of the enable input terminal

NOTE: *This argument is used in case of dual enable power switch.*

-en_inv_out <en-out-pin-name>

Name of the enable output terminal

-en_buf_in <en-buf-pin-name>

Name of the buffer enable input terminal.

If you specify the -en_inv_out and -en_buf_in arguments, the power switch is assumed as dual enable port power switch.

-enableval <0 | 1>

Specifies the value of the signal at enable pin. You can specify the argument as 0 (active low) or 1 (active high).

-enableval_2 <0 | 1>

Specifies the value of the signal at the second enable pin. You can specify the argument as 0 (active low) or 1 (active high). This option is used in case of dual enable port power switch.

-exclude <pin-name-list>

Specifies a list of supply pins that are not to be checked for power and ground connections.

NOTE: *You can specify the same design unit as the power switch for multiple power domains.*

Rules

The power_switch constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM06	LPSVM45	LPPLIB17	LPPLIB15

pr_safe_clocks

Purpose

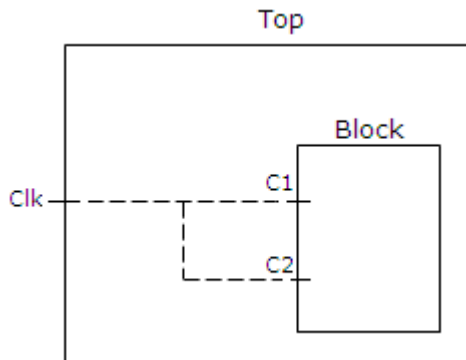
Power reduction rules recommend the RTL changes that do not introduce any clock domain issues in the design. Therefore, the rules need to ensure that every transformation is *safe* from the clock crossing perspective.

A design may contain clocks that are physically different but have the same

domain, frequency, and phase. Such clocks are termed as *safe* clocks.

The `pr_safe_clocks` constraint is used to specify a list of such clocks that can be used by SpyGlass to report power reduction recommendations.

For example, consider the following figure:



In the above figure, when you run SpyGlass for the `Block` block, SpyGlass sees two clocks, `C1` and `C2`. However, these clock nets are connected to the `Clk` clock at the chip level. In this case, you should specify the following command in the SGDC file:

```
pr_safe_clocks -name C1 C2
```

The above command specifies that `C1` and `C2` are actually same and any transformation across them is *safe*.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `pr_safe_clocks` constraint is as follows:

```
pr_safe_clocks
  -name <clk-list>
```

Arguments

-name <clk-list>

List of clocks that are physically different but have the same domain, frequency, and phase

Rules

The `pr_safe_clocks` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions	
Rules of SpyGlass Power Reduce	Rules of SEC

pulldown

Purpose

The `pulldown` constraint is used to specify the pull-down design units so that various rules can take appropriate actions.

The `Scan_21` rule of the SpyGlass DFT solution uses the `pulldown` constraint to check whether a scan flip-flop exists in the fan-in of the enable pin. The `Tristate_09` rule uses the `pulldown` constraint to check whether the enable pin of the pull-down design unit gets the expected value in the capture mode. The `Topology_12` rule uses the `pulldown` constraint to check whether primary inputs are connected to a pull-down device.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `pullDown`.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `pulldown` constraint is as follows:

```
pulldown
```

```
-name <du-name>
[ -pin <pin-name> ]
[ -value <value> ]
```

NOTE: *The pulldown constraint supports wildcard characters.*

Arguments

<du-name>

The pull-down design unit (black box) name.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are considered.

You can specify a single design unit name or a space-separated list of design unit names.

-pin <pin-name>

(Optional) The enable pin on the pull-down design unit (black box).

You can specify only a single pin name.

If you do not specify the enable pin, the design unit (black box) is always assumed as an enabled pull-down.

-value <value>

(Optional) The expected value (0, 1, X, or Z) on the enable pin *<pin-name>* under the shift mode.

You need to specify the value only if you have specified the pin name.

Rules

The pulldown constraint is used by the following rules:

SpyGlass DFT Solution			
Scan_21	Tristate_06	Topology_12	Tristate_09

SpyGlass DFT DSM Solution

 All rules

pullup

Purpose

The `pullup` constraint is used to specify the pull-up design units so that various rules of SpyGlass DFT solution can take appropriate actions.

The `Scan_21` rule uses the `pullup` constraint to check whether a scan flip-flop exists in the fan-in of the enable pin. The `Tristate_09` uses the `pullup` constraint to check whether the enable pin of the design unit gets the expected value in the capture mode. The `Topology_12` rule uses the `pullup` constraint to check whether primary inputs are connected to a pull-up device.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `pullUp`.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `pullup` constraint is as follows:

```
pullup
  -name <du-name>
  [ -pin <pin-name> ]
  [ -value <value> ]
```

NOTE: The `pullup` constraint supports wildcard characters.

Arguments

<du-name>

The pull-up design unit (black box) name.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are considered.

You can specify a single design unit name or a space-separated list of design unit names.

-pin <pin-name>

(Optional) The enable pin on the pull-up design unit (black box).
You can specify only a single pin name.
If you do not specify the enable pin, the design unit (black box) is always assumed as an enabled pull-up.

-value <value>

(Optional) The expected value (0, 1, X, or Z) on the enable pin *<pin-name>* under the shift mode.
You need to specify the value only if you have specified the pin name.

Rules

The pullup constraint is used by the following rules:

SpyGlass DFT Solution			
Scan_21	Tristate_06	Topology_12	Tristate_09
SpyGlass DFT DSM Solution			
All rules			

qualifier

Purpose

Specifies a qualifier for synchronizing a clock domain crossing by the *Qualifier Synchronization* scheme.

Product

SpyGlass CDC solution

Syntax

The syntax of the `qualifier` constraint is as follows:

Usage 1:

```
qualifier
  -name <qualifier-name>
  -from_clk <src_clk_list>
  -to_clk <dest_clk_list>
  -type <type> -strict
  [ -crossing ]
  [ -ignore ]
  [ -reset ]
```

Usage 2:

```
qualifier
  -name <qualifier-name>
  -from_domain <src_domain_list>
  -to_domain <dest_domain_list>
  -type <type> -strict
  [ -crossing ]
  [ -ignore ]
  [ -reset ]
```

Usage 3:

```
qualifier
  -name <qualifier-name>
  -ignore
```

Usage 4:

```
qualifier
  -name <qualifier-name>
  -from_obj <src-obj-list>
  -to_obj <dest-obj-list>
  -ignore
```

Usage 5:

```
qualifier
  -enable <hierarchical-verilog-expression>
  -from_obj <src-obj-list>
  -to_obj <dest-obj-list>
```

Usage 6:

```
qualifier
  -name <qualifier-name>
  -from_obj <src-obj-list>
  -to_obj <dest-obj-list>
```

Usage 7:

```
qualifier
  -name <qualifier-name>
  -rdc
  [-from_obj <src-obj-list> -to_obj <dest-obj-list>]
  [-to_clk <dest-clock-list>]
  [-to_domain <dest-domain-list>]
  [-from_rst <source_reset_list> -to_rst <dest_reset_list>]
```

Usage 8:

```
qualifier
  -src_qual <source-qual-name>
  -dest_qual <des-qual-name>
  -from_clk <src_clk_list>
  -to_clk <dest_clk_list>
```

Usage 9:

```

qualifier
  [-src_qual <src_qual_name>]
  [-dest_qual <dest_qual_name>]
  -from_domain <src_domain_list>
  -to_domain <des_domain_list>

```

Usage 10:

```

qualifier
  [-src_qual <src_qual_name>]
  [-dest_qual <dest_qual_name>]
  -from_obj <src_obj_of_crossing>
  -to_obj <des_obj_of_crossing>

```

Arguments

-name <qualifier-name>

Name of the qualifier (port, net, hierarchical terminal). You can also specify the output of the destination in a control synchronized crossing to this argument.

You can also specify a pin name (of a master design unit) in the `<du-name>/<pin-name>` format.

You can also use wildcard characters ('*' and '?') while specifying names.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression with the hierarchies. For example, the `top.*.n1` expression is matched to `top.u1.n1` and `top.u1.u2.n1`. By default, the qualifier constraint matches only `top.u1.n1`.

Setting the value of the `hier_wild_card` parameter to `yes`, run-time performance of the qualifier constraint is impacted.

-from_clk <src-clk-list>

Space-separated list of clock names that drive the source of a crossing to be synchronized by the qualifier specified by `-name <qualifier-name>`.

Specify this argument with `-to_clk <dest-clk-list>`.

See [Example 1](#).

NOTE: While specifying the list of clock names, either specify individual clock names in double quotes or do not use double quotes at all. See [Example 8](#).

-to_clk <dest-clk-list>

Space-separated list of clock names that drive the destination of a crossing to be synchronized by the qualifier specified by `-name <qualifier-name>`.

Specify this argument with `-from_clk <src-clk-list>`.

See [Example 1](#).

NOTE: While specifying the list of clock names, either specify individual clock names in double quotes or do not use double quotes at all. See [Example 8](#).

-from_domain <src-domain-list>

Space-separated list of domain names that drive the source of a crossing to be synchronized by the qualifier specified by `-name <qualifier-name>`.

Specify this argument with `-to_domain <dest-domain-list>`.

See [Example 2](#).

-to_domain <dest-domain-list>

Space-separated list of domain names that drive the destination of a crossing to be synchronized by the qualifier specified by `-name <qualifier-name>`.

Specify this argument with `-from_domain <src-domain-list>`.

See [Example 2](#).

-from_obj <src-obj-list>

Space-separated list of objects (hierarchical net, pin, or port) of a source such that the crossings containing such sources are not synchronized by the qualifier specified by `-name <qualifier-name>`.

Specify this argument with `-to_obj <dest-obj-list>` and `-ignore`.

See [Example 3](#).

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

-to_obj <dest-obj-list>

Space-separated list of objects (hierarchical net, pin, or port) of a destination such that the crossings containing such destinations are not synchronized by the qualifier specified by *-name <qualifier-name>*.

For library cells and black boxes, specify their input pin or input net names to this argument.

Specify this argument with *-from_obj <src-obj-list>* and *-ignore*.

See [Example 3](#).

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

-type <type>

Can be `des` if the qualifier reaches to the destination of a crossing or it can be `src` if it reaches to the source of a crossing.

By default, the type is `des`.

See [Example 4](#) and [Example 5](#).

-strict

Whether to allow strict synchronization checks for qualifier synchronization scheme. Specify *-strict* option to allow these checks.

Qualifier propagation stops on arithmetic macros and memories when the *-strict* option is specified in the *qualifier* constraint.

NOTE: Refer to the *Qualifier Synchronization Scheme* section of the *SpyGlass CDC Rules Reference Guide* for details.

See [Example 6](#).

-crossing

Specify this argument to consider a crossing as synchronized if the specified qualifier defines a crossing output that contains only a single destination flip-flop.

-ignore

Marks the qualifier specified by the *-name <qualifier-name>* argument as an invalid qualifier for the specified objects, domains, or clock crossings in the

design.

This argument is useful when SpyGlass infers a qualifier in the design but you do not want SpyGlass to consider it as a valid qualifier.

See [Example 1](#), [Example 2](#), [Example 3](#), and [Example 4](#).

NOTE: *This argument should not be used with the `-type <type>`, `-strict`, and `-crossing` argument of the qualifier constraint.*

-src_qual <source-qual-name>

Name of the qualifier (port, net, hierarchical terminal). You can also specify a pin name (of a master design unit) in the `<du-name>/<pin-name>` format. You can use wildcard characters ('*' and '?') while specifying names. See [Example 9](#) and [Example 10](#).

NOTE: *Set the `hier_wild_card` parameter to `yes` to match the expression with the hierarchies. For example, the `top.*.n1` expression is matched to `top.u1.n1` and `top.u1.u2.n1`. By default, the qualifier constraint matches only `top.u1.n1`. Note that setting the value of the `hier_wild_card` parameter to `yes` impacts the run-time performance of the qualifier constraint.*

NOTE: *The `-src_qual` and `-dest_qual` arguments cannot be specified with the following arguments of the qualifier constraint: `-name`, `-enable`, `-from_rst`, `-to_rst`, `-thru_obj`, `-type`, `-crossing`, `-ignore`, and `-rdc`.*

-dest_qual <des-qual-name>

Name of the qualifier (port, net, hierarchical terminal). You can also specify the output of the destination in a control synchronized crossing to this argument.

In addition, you can specify a pin name (of a master design unit) in the `<du-name>/<pin-name>` format. You can use wildcard characters ('*' and '?') while specifying names.

NOTE: *NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression with the hierarchies. For example, the `top.*.n1` expression is matched to `top.u1.n1` and `top.u1.u2.n1`. By default, the qualifier constraint matches only `top.u1.n1`. Note that setting the value of the `hier_wild_card` parameter to `yes` impacts the run-time performance of the qualifier constraint.*

NOTE: *The `-src_qual` and `-dest_qual` arguments cannot be specified with the following arguments of the qualifier constraint: `-name`, `-enable`, `-from_rst`, `-to_rst`, `-thru_obj`, `-type`, `-crossing`, `-ignore`, and `-rdc`.*

-enable <hierarchical-verilog-expression>

Specifies the expression representing an enable condition of a qualifier. The expression uses Verilog hierarchical names and operators.

The names specified to this argument should be valid design objects and should have the same domain as that of *-to_obj <dest-obj-list>* in their fan-in.

NOTE: *It is mandatory to specify the both the *-to_obj <dest-obj-list>* and *-from_obj <src-obj-list>* arguments with the *-enable* argument.*

The following example shows the usage of this argument:

```
qualifier -enable "top.en_out" -from_obj top.src.q -to_obj
top.des.q
```

-reset

Specify this argument to consider a crossing as synchronized if the specified qualifier is present on a reset path.

For example, specify the following constraint to synchronize a clock domain crossing only on reset paths.

```
qualifier -name qual -from_clk clk1 -to_clk clk2 -reset
```

-rdc

Specify this argument to consider a rdc crossing as synchronized if the specified qualifier is present on a data or control path.

For example, specify the following constraint to synchronize a reset domain crossing.

```
qualifier -name qual -from_rst r1 -to_rst r2 -rdc
```

Examples

Consider the scenario shown in the following figure:

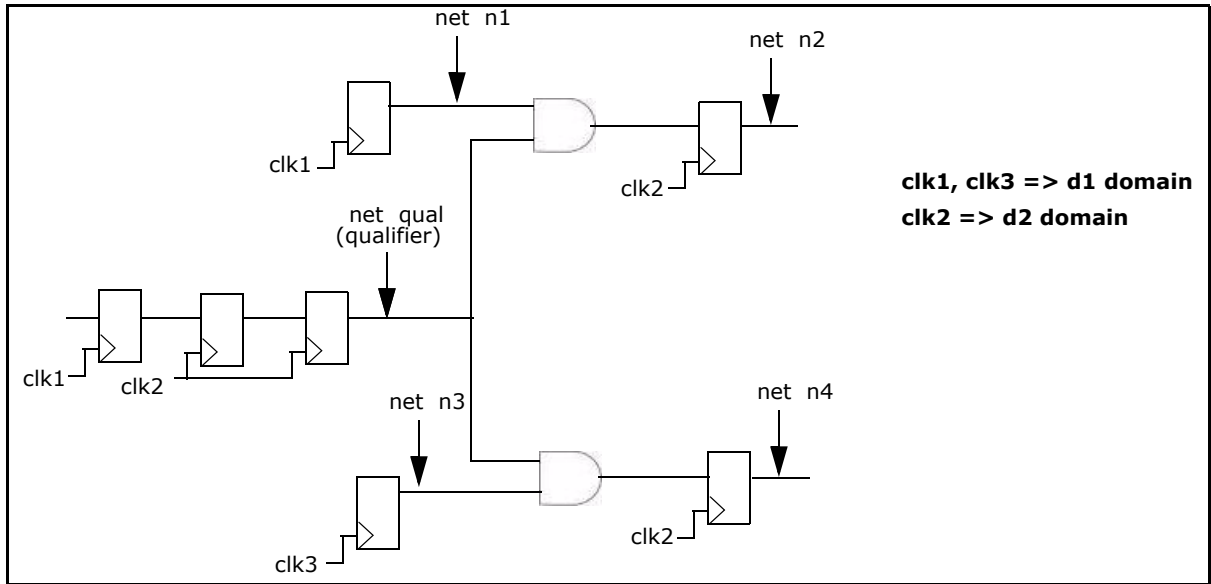


FIGURE 38. Specifying crossings synchronized by a qualifier

In the above scenario, you can specify the crossings that should or should not be synchronized by the `qual` qualifier. This is described in the examples below.

Example 1

In the scenario shown in [Figure 38](#), to enable `qual` to synchronize the crossing between the source driven by the `clk1` clock and destination driven by the `clk2` clock, specify the following constraint:

```
qualifier -name qual -from_clk clk1 -to_clk clk2
```

To ignore this crossing such that `qual` does not synchronize this crossing, specify **`-ignore`** to this constraint, as shown below:

```
qualifier -name qual -from_clk clk1 -to_clk clk2 -ignore
```

Example 2

In the scenario shown in [Figure 38](#), to enable `qual` to synchronize the crossing between the source driven by the `d1` domain clock and destination driven by the `d2` domain clock, specify the following constraint:

```
qualifier -name qual -from_domain d1 -to_domain d2
```

To ignore this crossing such that `qual` does not synchronize this crossing, specify *-ignore* to this constraint, as shown below:

```
qualifier -name qual -from_domain d1 -to_domain d2 -ignore
```

Example 3

In the scenario shown in [Figure 38](#), if you do not want `qual` to synchronize the crossing between the source containing the `n1` net and a destination containing the `n2` net, specify the following constraint:

```
qualifier -name qual -from_obj n1 -to_obj n2 -ignore
```

Example 4

Consider the following constraint:

```
qualifier -name qual -from_clk c1 -to_clk c2 c3 -type src
```

The above constraint specifies a qualifier for synchronization of the crossings where the source is driven by `c1` and destination by `c2` and `c3`. The type `src` specifies that `qual` is reaching the source of the crossings.

Example 5

Consider the following constraint:

```
qualifier -name qual -from_domain d1 d2 -to_domain d3 d4  
-type des
```

The above constraint specifies a qualifier for synchronization of the crossings where the source is driven by clocks from domain `d1` and `d2` and destination by clocks of domain `d3` and `d4`. The type `des` specifies that `qual` is reaching the destination of the crossings.

Example 6

Consider the following constraint:

```
qualifier -name qual -from_clk c1 c2 -to_clk c3 -strict
```

The above constraint specifies a qualifier for synchronization of the crossings where the source is driven by `c1` and `c2` and destination by `c3` and `qual` is reaching the destination of the crossings. In addition, the constraint specifies that strict synchronization checks should be allowed for qualifier synchronization scheme.

Example 7

The following constraint shows the usage of the `-enable` argument with respect to the scenario shown in [Figure 38](#):

```
qualifier -enable "qual" -from_obj "n1" -to_obj "n2"
```

Example 8

The following example shows the correct way to specify clock names in `-from_clk` and `-to_clk` where either individual clock names are specified in double quotes or no double quotes are used at all:

```
qualifier -name "xyz" -from_clk "clk1" "clk2" "clk3" -to_clk  
"ck1" "ck2"
```

```
qualifier -name "xyz" -from_clk clk1 clk2 clk3 -to_clk ck1  
ck2
```

The following example shows the incorrect way of specifying clock names in `-from_clk` and `-to_clk`:

```
qualifier -name "xyz" -from_clk "clk1 clk2 clk3" -to_clk  
"ck1 ck2"
```

Example 9

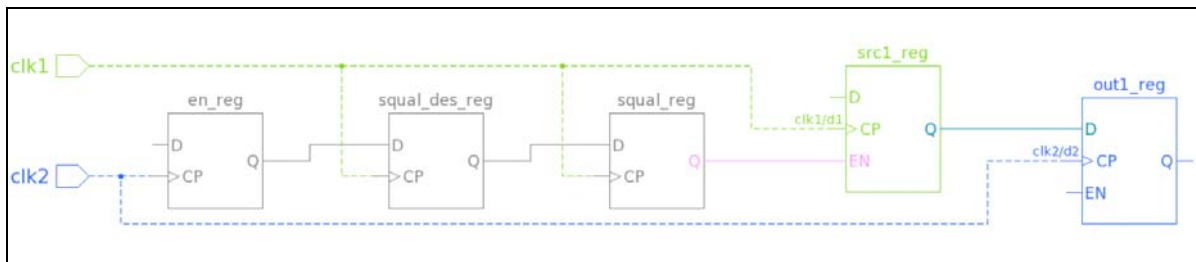
This example shows the **qualifier** constraint specification for synchronizing a crossing from the `clk1` clock to the `clk2` clock using the source qualifier.

SpyGlass Design Constraints

```
qualifier -src_qual squal -from_clk clk1 -to_clk clk2
```

This constraint specification synchronizes the crossing if the `squal` source qualifier is reaching the source of the crossing at the enable pin or the select pin of the recirculation multiplexer at that source or enable pin of the clock-gating cell driving the clock pin of the source.

In the following schematic, the `squal` source qualifier is reaching the enable pin of the source of the crossing `src1_reg`.



Synchronization reason in this case is reported as:

User-defined qualifier at source

Example 10

This example shows the **qualifier** constraint specification for synchronizing a crossing from the `clk1` clock to the `clk2` clock.

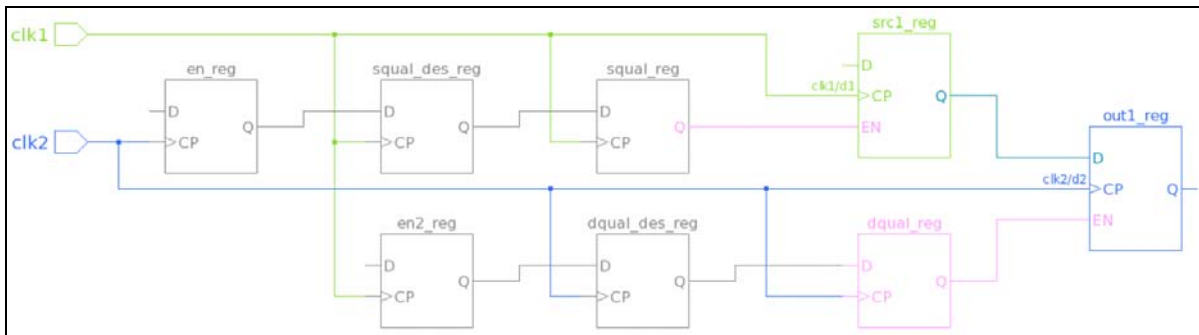
```
qualifier -src_qual squal -dest_qual dqual -from_clk clk1
-to_clk clk2
```

This constraint specification synchronizes the crossing if:

The `squal` source qualifier is reaching the source of that crossing at the enable pin or select pin of the recirculation multiplexer at that source or enable pin of the clock-gating cell driving the clock pin of the source.

The `dqual` destination qualifier is blocking the source before reaching the destination.

In the following schematic, the `squal` source qualifier is reaching the enable pin of the source of the crossing `src1_reg` and the `dqual` destination qualifier is reaching the enable pin of the destination flip-flop `out1_reg`.



Example 11

This example shows the qualifier constraint specification for synchronizing a crossing from the `clk1` clock to the `clk2` clock using the destination qualifier.

```
qualifier -des_qual dqual -from_clk clk1 -to_clk clk2
```

The `dqual` destination qualifier is blocking the source before reaching the destination. In the following schematic, the `dqual` destination qualifier is reaching the enable pin of the destination flip-flop `out1_reg`.

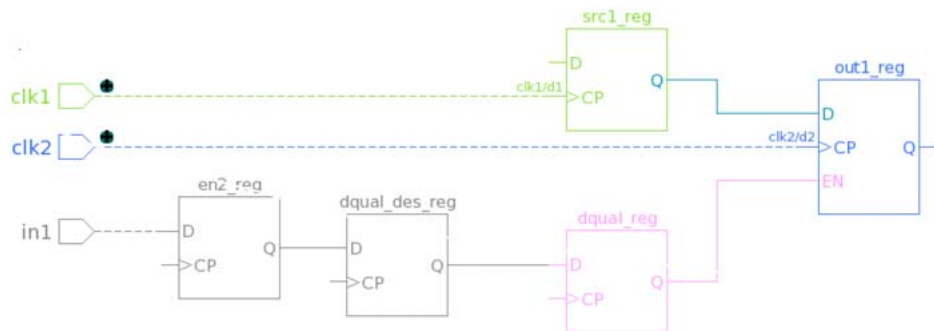


FIGURE 39.

Rules

The `qualifier` constraint is used by the following rules:

SpyGlass CDC Solution			
Clock_sync08	Clock_sync08a	Clock_sync03a	Clock_sync03b
Clock_sync09	Ac_cdc01a	Ac_cdc01b	Ac_cdc01c
Ac_cdc08	Propagate_Clocks	Ac_conv02	Ac_sync01
Ac_sync02	Ac_unsync01	Ac_unsync02	Ac_conv04
Ar_cross_analysis01	Ar_resetcross01		

quasi_static

For SpyGlass TXV solution

Purpose

There are some primary inputs, flip-flops, nets, or terms present in the design, which may change briefly at the start but assume a static value of 0 or 1 for the rest of the circuit operation.

You can specify the primary inputs, flip-flops, nets, or terms as static signals using the `quasi_static` constraint to skip the verification of paths that involve such signals.

Product

SpyGlass TXV solution

Syntax

The syntax to specify the `quasi_static` constraint is as follows:

```
quasi_static -name <sig-name-list>
```

Arguments

-name <sig-name-list>

List of hierarchical net/term names of the static signal

Example

If A is a static signal then you need to specify the `quasi_static` constraint in the `.sgdc` file as follows:

```
quasi_static -name top.A
```

Note

- Wildcard characters “*” and “?” can be used with the `quasi_static` constraint.
- If the hierarchical net/term name of a static signal specified with the `-name` argument does not exist as a net/term in the current design, `SGDC_quasi_static01rule` reports a violation.

Rules

The `quasi_static` constraint is used by the following rules:

SpyGlass TXV Solution		
FP_Pass_Verif05	FP_Skip_Verif02	MCP_Pass_Verif03
MCP_As_FP_Verif07	MCP_Skip_Verif03	

For SpyGlass CDC solution

Purpose

The `quasi_static` constraint is used to specify signals whose value is predominantly static. Clock domain crossings containing such static signals are reported as synchronized under the *Delay Signals Synchronization Scheme*.

You can use this constraint if any of the following conditions hold true:

- If the value of signals is predominantly static
- If a clock on the destination flip-flop is stopped
- If a reset is active on a destination flip-flop

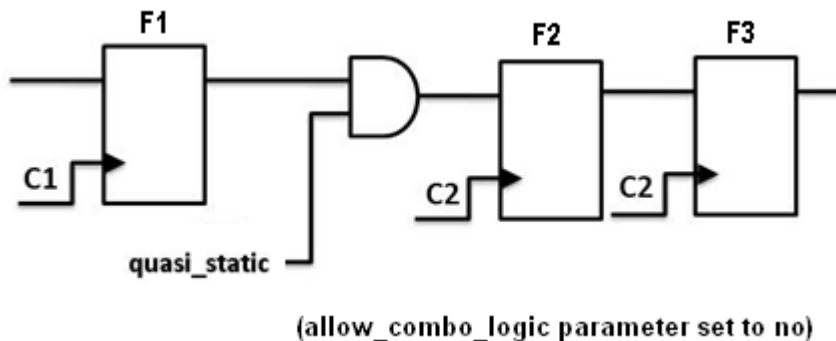
- If the logic in the clock domain crossing path is not sensitive to any metastability issue

Signals specified by this constraint are propagated beyond buffers, inverters, and transparent latches. In addition, a signal is propagated if all inputs of an RTL gate or all related inputs of a black box that has the [assume_path](#) constraint set on it are quasi-static.

If all inputs of a combinational logic present in a crossing path are quasi-static except the source path, the logic is considered as safe even when the `allow_combo_logic` parameter is set to `no`. Therefore, the logic is allowed while checking in the *Conventional Multi-Flop Synchronization Scheme*.

NOTE: The *quasi_static* constraint impacts the behavior of the *Reset_check07* rule. Refer to the help on *Reset_check07* rule for details.

For example, consider the following figure:



In the above figure, crossing between the C1 and C2 clocks is reported as synchronized by using the *Conventional Multi-Flop Synchronization Scheme*.

NOTE: Please note the following points:

- Quasi-static signals are not propagated beyond flip-flops, normal latches, and pure black boxes.
- This constraint is only applicable to Clock Domain Crossing (CDC) data paths. It is NOT applicable to either clock paths or side paths/inputs to CDC data paths.

- It is recommended that you use this constraint very carefully. If not used carefully, this constraint may mask some of the real design issues.
- If a crossing contains signals other than static signals, it is recommended that you use the `cdc_false_path` constraint to specify false paths.
- By default, the `quasi_static` constraint does not propagate through flops/sequential elements. Use the `num_quasi_seq_elem` parameter to specify the depth of flops/sequential element up to which the `quasi_static` constraint should propagate. Set the `num_quasi_seq_elem` parameter to -1 to propagate the constraint through infinite number of flops/sequential elements.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `quasi_static` constraint is as follows:

```
current_design <du-name>
    quasi_static -name <sig-name-list>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs)

-name <sig-name-list>

The list of port names, hierarchical net/term names, and/or hierarchical pin names of predominantly static signals.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression with the hierarchies. For example, the `top.*.n1` expression is matched to `top.u1.n1` and `top.u1.u2.n1`. By default, the `quasi_static` constraint matches only `top.u1.n1`. Setting the value of the `hier_wild_card` parameter to `yes` runtime performance of the `quasi_static` constraint is impacted.

Example

You can specify bit-selects and part-selects with the `quasi_static` constraint, as in the following example:

```
current_design top
  quasi_static -name "top.I1.NetA2B[0]"
  quasi_static -name "top.I1.NetA2B[1]"
  clock -name clkA -domain A -value rtz
  clock -name clkB -domain B -value rtz
```

You can also use regular expressions and wildcard characters while specifying signal names. For example, consider that your design contains buses, such as `netBus1[0:7]` and `netBus2[0:15]`. In this case, if you want to specify both these buses, specify the following constraint:

```
quasi_static -name "top.netBus*"
```

For details on using regular expressions and wildcard characters, refer to the *Using Regular Expressions and Wildcard Characters* topic of the *Atrenta Console User Guide*.

Rules

The `quasi_static` constraint is used by the following rules:

SpyGlass CDC Solution			
Clock_info05b	Reset_check07	Clock_sync03a	Clock_sync03b
Clock_sync05	Clock_sync06	Clock_sync08	Clock_sync08a
Clock_sync09	Ac_cdc01a	Ac_cdc01b	Ac_cdc01c
Ac_cdc08	Ac_handshake01	Ac_handshake02	Ac_conv01
Ac_conv02	Ac_conv03	Ac_sync01	Ac_sync02
Ac_unsync01	Ac_unsync02	Reset_sync02	Setup_quasi_static 01
Ac_coherency06	Propagate_Clocks		

For SpyGlass Power Family

Purpose

Specifies quasi static signals in a design.

Quasi static signals are the signals that are constant for most of the time and toggle for the remaining time. This feature of these signals can be used to find clock gating opportunities in a design.

Product

SpyGlass Power Family

Syntax

The syntax of the `quasi_static` signal is as follows:

```
current_design <du-name>  
  quasi_static -name <sig-name-list>  
  [ -value <value> ]
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs)

-name <signal-name>

Name of the signal that is considered as quasi static.

The signal name can be the name of a port, hierarchical net/term, or hierarchical pin.

-value <value>

The static value on the quasi static signal.

If you do not specify this argument, the value is picked from the simulation file.

Rules

The `quasi_static` constraint is used by the following rules:

SpyGlass Power Family			
PEPWR20	PEPWR21	PEPWR22	PEPWR23
PEPWR24	PEPWR25	PEPWR33	

quasi_static_style

Purpose

Specifies a criterion based on which SpyGlass infers quasi-static signals in a design.

If a signal matches the specified criteria, SpyGlass infers it as a quasi-static signal.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `quasi_static_style` constraint is as follows:

```
current_design <du-name>
  quasi_static_style
    [ -min_seq_fanouts <fanout-count> ]
    [ -min_domain_fanouts <domain-count> ]
    [ -names <pattern-list> ]
    [ -check_all_signals ]
    [ -report_full_count ]
```

Arguments

-min_seq_fanouts <fanout-count>

Specifies the minimum number of sequential elements a signal must drive.
The default value is 10.

-min_domain_fanouts <domain-count>

Specifies the minimum number of clock-domains in the fan-out cone of a signal.
The default value is 1.

-names <pattern-list>

Specifies a naming pattern for signals.

If a signal matches the specified pattern, only then SpyGlass checks if that signal is a quasi-static signal based on the criteria specified by other arguments of this constraint.

For details, see [Example 1](#), [Example 2](#), and [Example 3](#).

NOTE: *You can use wildcard expressions while specifying patterns.*

-check_all_signals

Specifies that all the output of sequential elements, output pins of black boxes, or input ports should be checked to see if they are quasi-static signals.

By default, SpyGlass considers only the sources of clock-domain crossings to detect quasi-static signals.

-report_full_count

Specify this argument to report the complete sequential fan-out count and domain count of the inferred `quasi_static` signal in the spreadsheet report.

Examples

Example 1

Consider the following example:

```
quasi_static_style -names "cfg*" "*_cfg" "*_stable?"
```

When you specify the above constraint, SpyGlass infers the signals that match all the following criteria:

- The signal is the source of a clock-domain crossing.
- The signal name matches any of the following criteria:
 - ☐ The name starts with the string `cfg`.
 - ☐ The name ends with `_cfg`.
 - ☐ The name is succeeded by a character at the end of `_stable`.
- The fan-out count of the signal is at least 10.

NOTE: In the absence of the `-min_seq_fanouts <fanout-count>` argument, SpyGlass considers the default fan-out count as 10.

- The domain count of the signal is at least 1.

NOTE: In the absence of the `-min_domain_fanouts <domain-count>` argument, SpyGlass considers the default domain count as 1.

Example 2

Consider the following example:

```
quasi_static_style -names "*cfg*" -min_seq_fanouts 20
-check_all_signals
```

When you specify the above constraint, SpyGlass infers the signals that match all the following criteria:

- The signal name contains the string `cfg`.
- The fan-out count of the signal is at least 20.
- The domain count of the signal is at least 1.

NOTE: In the absence of the `-min_domain_fanouts <domain-count>` argument, SpyGlass considers the default domain count as 1.

Example 3

Consider the following example:

```
quasi_static_style -names "cfg" -min_seq_fanouts 20
-min_domain_fanouts 3
```

When you specify the above constraint, SpyGlass infers the signals that match all the following criteria:

- The signal is the source of a clock-domain crossing.
- The signal name is `cfg`, such as `top.SB1.cfg` and `top.cfg`.
- The fan-out count of the signal is at least 20.
- The domain count of the signal is at least 3.

Example 4

Consider the following example:

```
quasi_static_style -min_seq_fanouts 20 -min_domain_fanouts 3
```

When you specify the above constraint, SpyGlass infers the signals that match all the following criteria:

- The fan-out count of the signal is at least 20.
- The domain count of the signal is at least 3.

Rules

The `quasi_static_style` constraint is used by the following rules:

SpyGlass CDC Solution

Setup_quasi_static01	SGDC_quasi_static_style01	SGDC_quasi_static_style02
----------------------	---------------------------	---------------------------

ram_instance

Purpose

Specifies RAM switch instance-to-RAM instance connections.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `raminstance`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `ram_instance` constraint is as follows:

```
current_design <du-name>
    ram_instance
        -switch_instance <swt-inst-name>
        -ram_instance <ram-inst-name>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the connection.

-switch_instance <swt-inst-name>

Name of the RAM switch instance.

-ram_instance <ram-inst-name>

Name of the RAM instance.

Rules

The `ram_instance` constraint is used by the following rule:

SpyGlass Power Verify Solution
LPPLIB12

ram_switch

Purpose

Specifies the RAM switches as checked by the LPSVM46 and LPPLIB12 rules.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `ramswitch`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `ram_switch` constraint is as follows:

```
current_design <du-name>
  ram_switch
    -switch_name <swt-name>
    -switch_enable_pin <swt-en-pin-name>
    [ -switch_vss_outpin <swt-vss-outpin-name> ]
    -memory_cellnames <mem-cell-name-list>
    [ -memory_vss_pin <mem-vss-pin-name> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the RAM switch.

-switch_name <swt-name>

Name of the RAM switch

-switch_enable_pin <swt-en-pin-name>

Name of the enable pin of the RAM switch.

-switch_vss_outpin <swt-vss-outpin-name>

Name of the VSS pin of the RAM switch.

-memory_cellnames <mem-cell-name-list>

Space-separated cell name list of memory cells

-memory_vss_pin <mem-vss-pin-name>

Name of the memory cell VSS pin.

Rules

The `ram_switch` constraint is used by the following rules:

SpyGlass Power Verify Solution	
LPPLIB12	LPSVM46

rdc_false_path

Purpose

NOTE: *This constraint is deprecated. Use the [reset_filter_path](#) constraint instead of this constraint.*

The `rdc_false_path` constraint is used to specify false paths so that reset crossings along these paths are ignored from rule checking.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `rdc_false_path` constraint is as follows:

```
current_design <du-name>
rdc_false_path
  -from_rst <frm-rst-list>
  -to_rst <to-rst-list>
  -clock <clk-obj-list>
  -from_obj <from-obj-list>
  -to_obj <to-obj-list>
  -type <rdc | sync | deassert>
```

Arguments

-from_rst <frm-rst-list>

Space-separated list of objects (hierarchical net, pin, or port) of a source such that the reset crossings containing such sources are not reported by *Ar_resetcross01* rule.

-to_rst <to-rst-list>

Space-separated list of objects (hierarchical net, pin, or port) of a destination such that the reset crossings containing such destinations are not reported by *Ar_resetcross01* rule.

-clock <clk-obj-list>

Space-separated list of source or destination clocks.

-from_obj <from-obj-list>

Space-separated list of hierarchical nets, pins, or ports of the source in the reset crossing so that the reset crossings containing such sources are not reported by the *Ar_resetcross01* rule.

-to_obj <to-obj-list>

Space-separated list of hierarchical nets, pins, or ports of the destination in the reset crossing so that the reset crossings containing such destinations are not reported by the *Ar_resetcross01* rule.

-type <rdc | sync | deassert>

(Optional) The default value is *rdc*. This value is for backward compatibility.

Set this argument to *deassert* to filter the *Ar_asyncdeassert01* and *Ar_syncdeassert01* rule violations reported for the reset paths specified by this constraint.

Set this argument to *sync* to filter the *Ar_unsync01* and *Ar_sync01* rule violations reported for the reset paths specified by this constraint.

NOTE: *If you specify the **sync** or **deassert** value to this argument then you can specify only **-from_rst** and **-clock** arguments to the *reset_filter_path* constraint.*

Examples

Example 1

Consider the following *Ar_resetcross01* spreadsheet showing violations related to invalid crossings:

A ID	B SOURCE	C SOURCE RESET	D DEST.	E DEST. RESET	F CLOCKS	G WAIVED
2C	top.F1.q	top.r1	top.F2.q	top.r2	top.c	No
2D	top.F3.q	top.r2	top.F4.q	top.r1	top.c	No
33	top1.F7.q	top1.r1	top1.F8.q	top1.r2	top1.c	No
39	top2.F3.q	top2.r1	top2.F4.q	top2.r2	top2.c	No
3A	top2.F5.q	top2.r1	top2.F6.q	top2.r2	top2.c	No

FIGURE 40.

From the above set of violations, if you do not want to report the reset crossings between the top.r1 and top.r2 resets, specify the following constraint:

```
rdc_false_path -from_rst r1 -to_rst r2
```

After specifying the above constraint, the violations reported in the cells 2C and 33 in [Figure 40](#) are not reported.

Example 2

To suppress the *Ar_resecross01* violations for the reset crossings between r1 to r2, r1 to r3, and r1 to r4 resets, specify the following constraint:

```
rdc_false_path -from_rst r1 -to_rst r2 r3 r4
```

Rules

The `rdc_false_path` constraint is used by the following rules:

SpyGlass CDC Solution	
Ar_resetcross01	Ar_resetcross_matrix01

ref_power_data

Purpose

The `ref_power_data` constraint is used to specify values for reference power numbers in the Correlation View of the Power Explorer.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `ref_power_data` constraint is as follows:

```
current_design <du-name>
ref_power_data
  -type <component-type>
  -<power-type> <value>
```

Arguments

-type <component-type>

The type can be total, combinational, sequential, blackbox, others, iopad, memory, clock, or megacell.

-<power-type> <value>

This argument can be leakage, internal, switching, or total. The value can be specified in floating point format and not in scientific format.

Examples

The following SGDC specification defines combinational, sequential, memory, and total power attributes and their values. These values are populated in the SpyGlass Power Reference Matrix.

```
ref_power_data -type combinational -leakage 0.001
ref_power_data -type sequential -internal 0.008
```

```
ref_power_data -type memory -total 0.0002  
ref_power_data -type total -total 0.10
```

Rules

The `ref_power_data` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions

PEPWR01

PEPWR02

reference_toplevel_isolation_signal

Purpose

The `reference_toplevel_isolation_signal` constraint is used to specify reference top-level isolation signal at SoC level.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `reference_toplevel_isolation_signal` constraint is as follows:

```
reference_toplevel_isolation_signal  
-name <isolation-control-signal-name-at-soc>  
-supply <supply-name>
```

Arguments

-name <isolation-control-signal-name-at-soc>

Use this argument to specify the name of the correct isolation enable signal at the SoC level.

-supply <supply-name>

Use this argument to specify the correct supply of the driver of isolation data pin. This is matched against the actual supply of the driver of the isolation cell data pin.

Examples

The following SGDC specification defines the `top.iso1` signal as a reference top-level isolation signal for supply VDD1.

```
reference_toplevel_isolation_signal -name top.iso1 -supply  
VDD1
```

Rules

The `reference_toplevel_isolation_signal` constraint is used by the following rules:

SpyGlass Power Verify Solutions	
LPSVM22	LPISO01

repeater

Purpose

Specifies the name of repeater modules or library cells inserted between sequential elements to meet timing requirements of a design.

Product

SpyGlass CDC solutions

Syntax

The syntax to specify the `repeater` constraint is as follows:

```
repeater -names <object-names>
```

Arguments

-names <object-names>

Space-separated list of repeaters.

NOTE: Set the `hier_wild_card` parameter to `yes` to match the expression specified in this argument with hierarchies.

Examples

Example 1

The following constraint specifies two repeater modules, `MOD1` and `MOD2`:

```
repeater -names MOD1 MOD2
```

Example 2

The following example uses wildcard expression to indicate that all instances starting with the string Rep should be considered as repeaters:

```
repeater -names "Rep*"
```

Rules

The `repeater` constraint is used by the following rules:

SpyGlass CDC Solution
Ac_repeater01

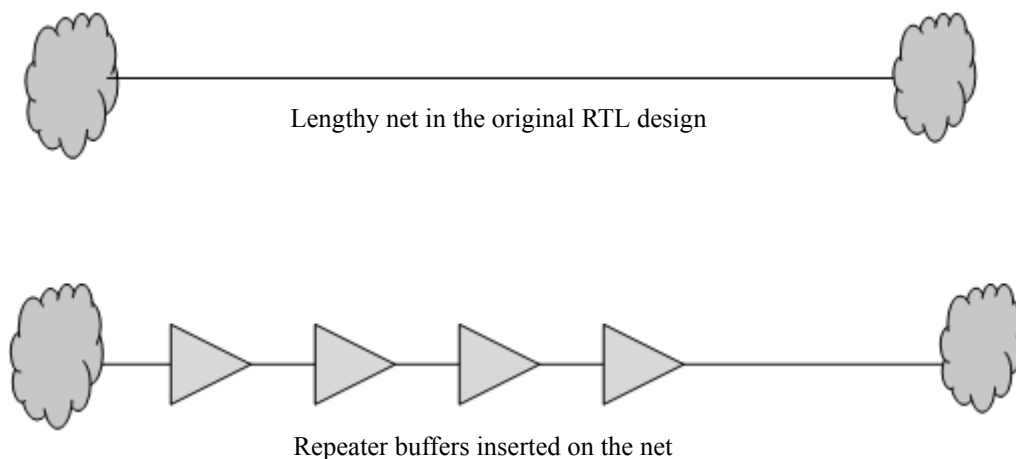
repeater_buffer

Purpose

A design may contain some very long nets that may or may not have a very high fan-out. After placement of such designs, repeater buffers can be inserted on such lengthy nets to ensure that the high capacitance of these nets can be switched properly.

The `repeater_buffer` constraint is used to specify information of such repeater buffers that can be inserted on a lengthy net.

Repeater buffers are inserted in series on a net, as shown in the following figure:

**FIGURE 41.** Repeater Buffers Inserted

Please note the following points:

- This constraint is considered independent of the value of the `pe_infer_clock_net_bufs` and `pe_infer_high_fanout_bufs` parameters.
- When the `pe_infer_clock_net_bufs` and/or `pe_infer_high_fanout_bufs` parameter is set to `yes`, only the `repeater_buffer` constraint information is used for the nets for which it is provided.
For the nets for which the `repeater_buffer` constraint information is not provided, the default virtual buffer algorithm is used.
- This constraint honors the `mix_vt_constraint` and `syn_set_dont_use` commands.
- Power of such lengthy nets is reported in the same hierarchy because it happens for nets with virtual buffers.
- Capacitance of additional nets is calculated based on wire-load. So even if the capacitance of the original net is set using SPEF or SDC (`set_load`), `repeater_buffer` constraints will be applied.

Note that this is a different behavior with respect to virtual buffers.

- Power of nets with virtual buffers is proportionately distributed in the hierarchies where the fan-out elements of that net are present. This is because the assumption there is that a buffer tree is getting created. Therefore, a majority of buffers are in the side where the fan-out elements are present. However, in case of a `repeater_buffer` structure, power of repeater buffers will be assigned and the nets in the hierarchy which completely encloses that net. Such nets will not honor the `pe_report_virtual_power_at_driving_instance` parameter.
- Consider a net that has a fan-out of more than one, as shown in the following figure:



FIGURE 42. Net with Multiple Fan-outs

Now consider that you set three repeater buffers on the above net. In this case, SpyGlass will assume the following structure:

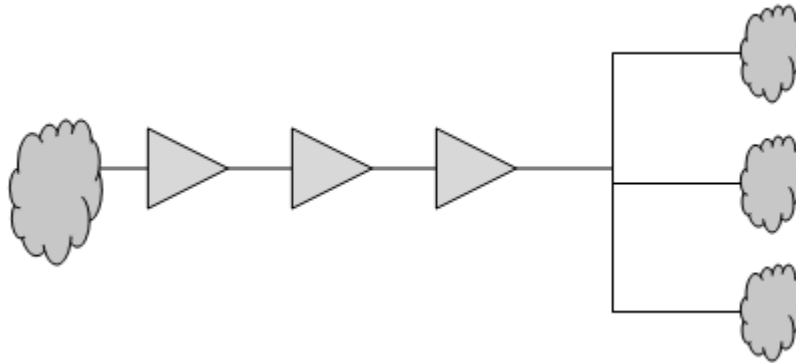


FIGURE 43. Net with Three Repeater Buffers

Therefore, in this case also, the additional elements are N buffers and N nets with a single fan-out.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `repeater_buffer` constraint is as follows:

```
repeater_buffer
  -name <net-name-list>
  -cell <cell-name>
  [ -lib <library-name> ]
  -count <count>
```

Arguments

-name <net-name-list>

Specifies a space-separated list of nets in a design for which repeater buffers should be considered.

-cell <cell-name>

Specifies a cell name that should be repeated on a net.

-lib <library-name>

Specifies the library from which the cell should be picked.

-count <count>

Specifies the number of buffers that should be considered.

Examples

The following command specifies two buffers of type BUF1 that should be used as repeater buffers, each on NET1 and NET2:

```
repeater_buffer -name "NET1 NET2" -cell BUF1 -count 2
```

You can specify the same information separately for each net, as shown in the following example:

```
repeater_buffer -name NET2 -cell BUF1 -count 2
repeater_buffer -name NET1 -cell BUF1 -count 2
```

Rules

The `repeater_buffer` constraint is used by the following rule:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions
PEPWR02

require_constraint_message_tag

Purpose

The *require_constraint_message_tag* checks whether *constraint_message_tag_expression* is used for the design nodes or not. It reports violation, if specific combination of tags are absent or present in the design for a particular node.

Syntax

The syntax for the *require_constraint_message_tag* constraint is as follows:

```
require_constraint_message_tag
  [-name <nodename>]
  [-except <except_nodename>]
  [-except_type <exceptDo-nodename>]
  [-type <DO_nodename>]
  [-constraint_message_tag_expression
  <constraint_message_tag_expression>]
```

Arguments

-name <nodename>

Specifies the name of the top-module port, or any internal net or terminal or leaf instance for which the specified tag expression must be satisfied.

-except <except_nodename>

Specifies the name of the top-module port, any internal net, terminal, or leaf instance name, which needs to be excluded from rule checking.

-except_type <exceptDo-nodename>

Specifies the name of macro for which the specified tag expression must be satisfied.

-type <DO_nodename>

Specifies the name of the macro, which needs to be excluded from rule

checking.

-constraint_message_tag_expression
<constraint_message_tag_expression>

Signifies the message tag expression specified using logical '||' and '&&' operator, their combinations, and :PASS and :FAIL values of tags. You can also use braces ('(',')') when specifying message tag expression.

Examples

Example 1

```
require_value -name "top.l1_1.out" -value 0
-constraint_message_tag LATCH_VALUE_CHECK
require_path -from_type LATCH_OUT -to_type FLIP_FLOP_RESET
-constraint_message_tag LATCH_CHECK
require_constraint_message_tag -type LATCH
-constraint_message_tag_expression "LATCH_CHECK:PASS &&
LATCH_VALUE_CHECK:PASS"
```

In the above example, the `require_constraint_message_tag` looks for absence of any violation on `LATCH_CHECK` and `LATCH_VALUE_CHECK` constraint_message_tags. It will report info message, if the specified expression is met otherwise gives error.

Example 2

```
require_constraint_message_tag -type FLIP_FLOP
-constraint_message_tag_expression "( CHECK_1:PASS ||
CHECK_2:FAIL) && CHECK_3:PASS"
```

In the above example, the `require_constraint_message_tag` looks for the absence of violation on `CHECK_1` and `CHECK_3` and presence of violation `CHECK_2`.

require_path

Purpose

The `require_path` constraint checks for the existence of the path between the specified nodes. However, the destination node may have paths existing to other nodes.

To check for the existence of the path originating from the source node that does not have any other destination node, except for the specified one, use the [require_strict_path](#) constraint.

NOTE: *Buffers and inverters are single-input, single-output devices. Therefore, there is no interference from external logic while tracing a net connectivity.*

Product

SpyGlass DFT solution

Syntax

The syntax for `require_path` constraint is as follows:

```
require_path
[ -tag <condname> | -use_shift |
  -use_capture | -use_captureATspeed ]
[ -from <frompinlist> ]
[ -from_one_of <fromoneof_pinlist> ]
[ -except_from <exceptfrom_pinlist> ]
[ -from_type <from_DO_pinlist> ]
[ -from_one_of_type <fromoneof_DO_pinlist> ]
[ -except_from_type <exceptfrom_DO_pinlist> ]
[ -to <topinlist> ]
[ -to_one_of <tooneof_pinlist> ]
[ -except_to <exceptto_pinlist> ]
[ -to_type <to_DO_pinlist> ]
[ -to_one_of_type <tooneof_DO_pinlist> ]
[ -except_to_type <exceptto_DO_pinlist> ]
[ -invert ] [ -noinvert ]
[ -parallel ]
[ -path_type <buffered | sensitized | sensitizable |
```

```

direct>]
  [-sequential_depth <value>]
  [-constraint_message_tag <value>]
  [-min_to_paths <value>]
  [-max_to_paths <value>]
  [-report_failure_as_info]

```

Arguments

-tag <condname>

(Optional) Specifies a condition name that is previously defined by using the [define_tag](#) constraint.

This condition describes stimulation of circuit into a desired state. Please note only one condition name can be defined in a `require_path` specification. However, simulation for a given condition name simulates all pin-value specification simultaneously, whether defined directly by pin-value specification, or merge of other `define_tag` specifications. The built-in power-ground connections are also simulated in this process automatically.

If the `-tag` argument is specified, an un-blocked path must exist when the defined condition is simulated. If the `-tag` argument is not specified, the `require_path` constraint defines a connectivity check that requires a net connection to exist from pin(s) specified with the `-from` argument to pin(s) specified with the `-to` argument. A net connection means a net connection across hierarchical boundaries between the specified start point and end-point(s) and may only contain buffers or inverters.

-use_shift | -use_capture | -use_captureATspeed

For any of these modifiers, `require_path` simulates test mode of that particular mode.

If `-use_shift`, `-use_capture`, or `-use_captureATspeed` argument is specified, the constraint simulates all, shift, capture, or captureATspeed test_mode constraints, respectively.

NOTE: *If more than one of the `-tag`, `-use_shift`, `-use_capture`, or `-use_captureATspeed` arguments is specified, an error condition occurs. You should specify only one or none of these modifiers with `require_path`*

constraint.

-from <frompinlist>, -to <topinlist>

These are the start-point and end-point nodes, respectively, in a circuit (as controlled by effective `current_design` specification) for which a path is searched after the circuit has been simulated by LE into the desired state (with the `-tag` argument specified) or a net connection is to be checked (without the `-tag` argument specified).

Either of these can be a list of top-module port names, internal net names, or terminal names. It is effectively read as a concise description of connectivity check from the each node in the *<frompinlist>* list to each node in the *<topinlist>* list.

-from_one_of <fromoneof_pinlist>

(Optional) Specifies that no error message is reported, if there is at least one success case among the specified nodes.

-except_from <exceptfrom_pinlist>

(Optional) Excludes the specified objects from nodes.

-from_type <from_DO_pinlist>

(Optional) Same as *<frompinlist>* but it takes only macros as inputs.

-from_one_of_type <fromoneof_DO_pinlist>

(Optional) Same as *<fromoneof_pinlist>* but it takes only macros as inputs

-except_from_type <exceptfrom_DO_pinlist>

(Optional) Same as *<exceptfrom_pinlist>* but it takes only macros as inputs.

-to_one_of <tooneof_pinlist>

(Optional) Specifies that no error message is reported, if there is at least one success case among the specified nodes.

-except_to <exceptto_pinlist>

(Optional) Excludes the specified objects from nodes.

-to_type <to_DO_pinlist>

(Optional) Same as <topinlist> but it takes only macros as inputs.

-to_one_of_type <tooneof_DO_pinlist>

(Optional) Same as <tooneof_pinlist> but it takes only macros as inputs.

-except_to_type <exceptto_DO_pinlist>

(Optional) Same as <tofrom_pinlist> but it takes only macros as inputs.

-invert | -noinvert

(Optional) These additional qualifiers mean that check is performed to check a positive polarity or negative polarity path between the two nodes. If neither of these is specified, polarity is not relevant for such paths.

-parallel

(Optional) Specifies that paths should be searched from a node in the <frompinlist> list to a node at the same relative position in the <topinlist> list. Thus, the paths should be searched from the first node in the <frompinlist> list to the first node in the <topinlist> list, and so on.

When you specify the -parallel argument, you need to specify the exact same number of nodes in both the <frompinlist> list and the <topinlist> list. Otherwise, the constraint is not processed.

If the -parallel argument is specified, the require_path constraint assumes that paths should be searched from a node specified with the -from argument to a node specified with the -to argument at the same relative position.

-path_type

The `-path_type` argument accepts only the following predefined list of values: `buffered`, `sensitized`, `sensitizable`, and `direct`. The default value of this qualifier is `sensitizable`.

-sequential_depth <value>

Specifies the number of sequential elements between end points specified on a success path. This means that the `require_path` check will go through the specified number of sequential elements. You can specify an integer value as an input to this argument.

-constraint_message_tag <value>

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

NOTE: *This argument accepts only alpha-numeric characters and underscore.*

-min_to_paths <value>

Specifies minimum number of expected successful paths. You can not specify this argument with `-from_one_of` and `-from_one_of_type` arguments.

-max_to_paths <value>

Specifies maximum number of expected successful paths. Following are the rules for using this parameter:

- If you are using both the `-min_to_paths` and `-max_to_paths` arguments, then the value of the `-max_to_paths` argument should be greater than the `-min_to_paths` argument.
- You can not specify this argument with `-from_one_of` and `-from_one_of_type` arguments.

-report_failure_as_info/-report_failures_as_info

Reports all the failures as info severity message.

NOTE: *The `require_path` constraint supports wildcard expressions. The supported meta-characters are `*` (star) and `?` (question mark) where `*` matches any number of characters and `?` matches only one character. The wildcard support is applicable*

for non-escaped names only. If the meta-characters appear inside an escaped name, they are treated as literals. For example, in the expression "top.\mid*\bottom", mid* is considered as a literal and does expand to "mid1, mid2, and so on. In addition, if you specify a hierarchical path using a wildcard, any sub-portion of this path that contains the wildcard does not cross the module boundary while searching for the expression in the design. This means that each level in the hierarchy path should be mentioned explicitly in the wildcard string. For example, the expression "top.mid*.bottom" will expand to "top.mid1.bottom", and not to "top.mid2.bottom".

NOTE: The expression on which a wildcard is used should always be enclosed within double quotes. For example, "top.mid*.bottom".

NOTE: The wildcard support is applicable for design objects only. For non-design objects, the support is not applicable.

Supported Macros

SpyGlass DFT solution supports the following macros:

Macros Representing Collection Of Pins/ports		
FLIP_FLOP_DATA	FLIP_FLOP_OUT	FLIP_FLIP_CLOCK
FLIP_FLOP_SET	FLIP_FLOP_RESET	FLIP_FLOP_ENABLE
SCAN_FLIP_FLOP_DATA	SCAN_FLIP_FLOP_OUT	SCAN_FLIP_FLOP_CLOCK
SCAN_FLIP_FLOP_SET	SCAN_FLIP_FLOP_RESET	SCAN_FLIP_FLOP_ENABLE
LATCH_DATA	LATCH_OUT	LATCH_SET
LATCH_RESET	LATCH_ENABLE	MUX_SELECT
CGC_CLOCK_IN	CGC_CLOCK_OUT	CGC_SYSTEM_ENABLE
CGC_TEST_ENABLE	BLACK_BOX	BLACK_BOX_OUTPUT
BLACK_BOX_INPUT	INPUT_PORTS	OUTPUT_PORTS
INOUT_PORTS	ALL_PORTS	TIED_0
TIED_1	TIED_SGDC	TIED_0_SGDC
TIED_1_SGDC	MEMORY_ADDRESS	MEMORY_CLOCK
MEMORY_DATA	MEMORY_ENABLE	MEMORY_OUT
Macros Representing Collection of Instances		
FLIP_FLOP	SCAN_FLIP_FLOP	LATCH
MUX	LATCH	ICG
CGC	BLCAK_BOX	MEMORY

NOTE: *TIED_** macros are used to filter out already selected set using the *-except_type*, *-except_from_type*, or *-except_to_type* arguments.

For more information on using macros in SGDC, see the **Using Macros in SGDC** section in the *SpyGlass DFT Rules Reference Guide*.

Examples

Some examples of the `require_path` constraint are as follows:

```
require_path -tag s1 -from top.SEF
            -to top.U1.U3.inst_add.D
require_path -tag s100mode -from top.U10.inst_mdd.Q
            -to top.Q1
require_path -from top.EN
            -to top.U1.U2.DIF top.U1.U4.DIF
```

Some examples of the `require_path` constraint with the `-parallel` argument are as follows:

```
require_path -tag s11 -from top.SEF top.SFF
            -to top.U1.D, top.U2.D -parallel
```

Here, the paths will be searched from the `top.SEF` node to the `top.U1.D` node and from the `top.SFF` node to the `top.U2.D` node.

```
require_path -tag s101 -from top.SEF[2:0] top.SFF
            -to top.FGT[3:0] - parallel
```

The above specifications imply that the following paths should be searched:

- From the `top.SEF[2]` node to the `top.FGT[3]` node
- From the `top.SEF[1]` node to the `top.FGT[2]` node
- From the `top.SEF[0]` node to the `top.FGT[1]` node
- From the `top.SFF` node (exists as a scalar node) to the `top.FGT[0]` node

Rules

The `require_path` constraint is used by the following rules:

SpyGlass Connectivity Verify Solution

 Soc_02

require_pulse

Purpose

The `require_pulse` constraint defines a check that requires a pulse sequence to be established on a certain node, when the circuit is simulated using the condition specified by the `-tag` argument.

Product

SpyGlass DFT DSM solution

Syntax

```
require_pulse
  -tag <condition_name>
  [-name <pin/net hier name>]
  [ -count <num-of-pulses> ]
  [ -except <except_pin_hier_name> ]
  [ -except_type <except_DO> ]
  [ -after <observe_after_bit_number> ]
  [ -before <observe_before_bit_number> ]
  [ -high_width <number_of_bits_during_high_phase> ]
  [ -low_width <number_of_bits_during_low_phase> ]
  [ -type <DO_objlist>]
  [-constraint_message_tag <value>]
```

Arguments

-tag <condition-name>

(Mandatory) Tag name of the [define_tag](#) constraint under which simulation is to be done.

-name <pin/net hier name>

(Optional) Pin or net hier name, where pulse pattern is to be checked.

-count <num-of-pulses>

(Optional) Total number of pulses required. The default value of this argument is 1.

-after <observe_after_bit_number>

(Optional) Pulse pattern checking starts after this bit number. The default value of this argument is 0.

-before <observe_before_bit_number>

(Optional) Pulse pattern checking ends before this bit number. The default value of this argument is the value of the last simulation cycle.

-high_width <number_of_bits_during_high_phase>

(Optional) Number of bits during high phase. The default value of this argument is 1.

-low_width <number_of_bits_durin_low_phase>

(Optional) Number of bits during low phase. The default value of this argument is 1.

-type <DO_objlist>

(Optional) Specifies a collection of design objects, which require a pulse. This argument takes macros as input.

-except <except_pin_hier_name>

same as -name but defines design nodes which are not to be used as name.

-except_type <except_DO>

same as <except_pin_hier_name> but it takes only macros as inputs.

`-constraint_message_tag <value>`

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

NOTE: *This argument accepts only alpha-numeric characters and underscore.*

Supported Macros

To view the list of macros supported by the `require_pulse` constraint, see [Supported Macros](#).

Rules

The `require_pulse` constraint is used by the following rules:

SpyGlass DFT DSM Solution	
Atspeed_21	Info_Atspeed_21

require_stable_value

Purpose

The `require_stable_value` constraint specifies nodes whose value is expected to be stable.

Product

SpyGlass DFT solution

Syntax

The syntax for the `require_stable_value` constraint is as follows:

```
require_stable_value
[ -name <nodename> ]
[ -except <except_nodename> ]
[ -type <DO_nodename> ]
[ -except_type <exceptDO_nodename> ]
[ -value <value> ]
[ -tag <condname> | -use_shift | -use_capture |
  -use_captureATspeed ]
```

```
[-constraint_message_tag <value>]
[-report_failure_as_info]
```

Arguments

-name <nodename>

(Optional) The name can be a top-module port, or any internal net name, or terminal name. More than one pin name can be specified, and it is effectively read as a concise description of as many individual value checks.

NOTE: Specify either *-name* or *-type* argument.

-except <except_nodename>

(Optional) Same as *<nodename>* but defines design nodes that are not to be used as name.

-type <DO_nodename>

Same as *<nodename>* but it takes only macros as inputs.

NOTE: Specify either *-name* or *-type* argument.

To view the list of macros supported by the `require_stable_value` constraint, see [Supported Macros](#).

-except_type <exceptDO_nodename>

Same as *<DO_nodename>* but it takes only macros as inputs.

-value <value>

The value is a logic value string of 0, 1, X, Z, 1_or_0, and 0_or_1. A single-bit value means check at end of complete simulation. The X value is treated as do-not-compare. A multi-bit value means check on cycle-by-cycle simulation basis. For specification of a vector value, SGDC multi-bit specification format (same as used for the `test_mode` constraint) should be used.

You can specify repeat sequences for the `require_stable_value` constraint.

For fields that require repeat sequence, you can specify the values as

<I*S>. Here, S is any string that does not contain the <, >, and * characters. However, S can contain another <I*S> expression. I is an integer that is always interpreted as a decimal value. The expression <I*S> means that the sequence S will be repeated I number of times.

-constraint_message_tag <value>

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

NOTE: *This argument accepts only alpha-numeric characters and underscore.*

-tag <condname>

(Optional) A condition previously defined by using the [define_tag](#) constraint. It describes a stimulation condition.

Note that only one condition name can be defined in a `require_stable_value` specification. However, simulation for a given condition name simulates all pin-value specifications simultaneously. The built-in power-ground connections are also simulated in this process.

-use_shift | -use_capture | -use_captureATspeed

For any of these modifiers, `require_stable_value` simulates test mode of that particular mode.

If `-use_shift`, `-use_capture`, or `-use_captureATspeed` argument is specified, the constraint simulates all, shift, capture, or captureATspeed test_mode constraints, respectively.

NOTE: *If more than one of the -tag, -use_shift, -use_capture, or -use_captureATspeed arguments is specified, an error condition occurs. You should specify exactly one of these modifiers with require_stable_value constraint.*

-report_failure_as_info/-report_failures_as_info

Reports all the failures as info severity message.

Examples

Currently Unavailable

Rules

The `require_stable_value` constraint is used by the following rule:

SpyGlass Connectivity Verify Solution

Soc_14

require_strict_path

Purpose

The `require_strict_path` constraint is used to check for the existence of path between the specified nodes. However, no other path should exist apart from the specified path.

To check for the existence of the path between the specified nodes, which may have paths to other nodes, use the [require_path](#) constraint.

NOTE: *Buffers and inverters are single-input, single-output devices. Therefore, there is no interference from external logic while tracing a net connectivity.*

Product

SpyGlass DFT solution

Syntax

The syntax for the `require_strict_path` constraint is as follows:

```
require_strict_path
  [-from_one_of_type <fromoneof_DO_pinlist> ]
  [-to_one_of_type <tooneof_DO_pinlist> ]
  [-except_from_type <exceptfrom_DO_pinlist> ]
  [-except_to_type <exceptto_DO_pinlist> ]
  [-from_type <from_DO_pinlist> ]
  [-to_type <to_DO_pinlist>]
  [-from <from-pinlist> ]
  [-except_from <exceptfrom_pinlist>
  [-from_one_of <fromoneof_pinlist> ]
  [-to <to-pinlist> ]
  [-except_to <exceptto_pinlist> ]
  [ -to_one_of <tooneof_pinlist> ]
  [ -tag <cond-name> | -use_shift | -use_capture |
    -use_captureATspeed ]
  [-constraint_message_tag <value>]
  [-min_to_paths <value>]
  [-max_to_paths <value>]
  [-report_failure_as_info]
```

Arguments

-from_one_of_type <fromoneof_DO_pinlist>

(Optional) Same as <fromoneof_pinlist> but it takes only macros as inputs.

-to_one_of_type <tooneof_DO_pinlist>

(Optional) Same as <tooneof_pinlist> but it takes only macros as inputs.

-except_from_type <exceptfrom_DO_pinlist>

(Optional) Same as <exceptfrom_pinlist> but it takes only macros as inputs.

-except_to_type <exceptto_DO_pinlist>

(Optional) Same as <tofrom_pinlist> but it takes only macros as inputs.

-from_type <from_DO_pinlist>

(Optional) Same as <frompinlist> but it takes only macros as inputs.

-to_type <to_DO_pinlist>

(Optional) Same as <topinlist> but it takes only macros as inputs.

-except_from <exceptfrom_pinlist>

(Optional) Excludes the specified objects from nodes.

-from_one_of <fromoneof_pinlist>

(Optional) Specifies that no error message is reported, if there is at least one success case among the specified nodes.

-except_to <exceptto_pinlist>

(Optional) Excludes the specified objects from nodes.

-to_one_of <tooneof_pinlist>

(Optional) Specifies that no error message is reported, if there is at least one success case among the specified nodes.

-from <from-pinlist>, -to <to-pinlist>

These are the start-point and end-point nodes, respectively, in a circuit (as controlled by effective `current_design` specification) for which a path is searched after the circuit has been simulated by LE into the desired state (with the `-tag` argument specified) or a net connection is to be checked (without the `-tag` argument specified).

Either of these can be a list of top-module port names, internal net names, or terminal names. It is effectively read as a concise description of connectivity check from each node in the *<from-pinlist>* list to each node in the *<to-pinlist>* list.

-tag <cond-name>

(Optional) Specifies a condition name that is previously defined by using the [define_tag](#) constraint.

This condition describes stimulation of circuit into a desired state. Please note only one condition name can be defined in a `require_strict_path` specification. However, simulation for a given condition name simulates all pin-value specification simultaneously, whether defined directly by pin-value specification, or merge of other `define_tag` specifications. The built-in power-ground connections are also simulated in this process automatically.

If the `-tag` argument is specified, an un-blocked path must exist when the defined condition is simulated. If the `-tag` argument is not specified, the `require_strict_path` constraint defines a connectivity check that requires a net connection to exist from pin(s) specified with the `-from` argument to pin(s) specified with the `-to` argument. A net connection means a net connection across hierarchical boundaries between the specified start point and end-point(s) and may only contain buffers or inverters.

-use_shift | -use_capture | -use_captureATspeed

(Optional) For any of these modifiers, the `require_strict_path` constraint simulates the test mode of that particular mode.

If the `-use_shift`, `-use_capture`, or `-use_captureATspeed` argument is specified, the constraint simulates the shift, capture, or captureATspeed mode, respectively.

NOTE: *Specify only one of the `-tag`, `-use_shift`, `-use_capture`, or `-use_captureATspeed` fields with the `require_strict_path` constraint.*

-constraint_message_tag <value>

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

NOTE: *This argument accepts only alpha-numeric characters and underscore.*

-min_to_paths <value>

Specifies minimum number of expected successful paths. You can not specify this argument with `-from_one_of` and `-from_one_of_type` arguments.

-max_to_paths <value>

Specifies maximum number of expected successful paths. Following are the rules for using this parameter:

- If you are using both the `-min_to_paths` and `-max_to_paths` arguments, then the value of the `-max_to_paths` argument should be greater than the `-min_to_paths` argument.
- You can not specify this argument with `-from_one_of` and `-from_one_of_type` arguments.

Supported Macros

To view the list of macros supported by the `require_strict_path` constraint, see [Supported Macros](#).

-report_failure_as_info/-report_failures_as_info

Reports all the failures as info severity message.

Examples

Some examples of the `require_strict_path` constraint are as follows:

```
require_strict_path -tag s1
    -from top.SEF -to top.U1.U3.inst_add.D

require_strict_path -tag s100mode
    -from top.U10.inst_mdd.Q -to top.Q1

require_strict_path
    -from top.EN -to top.U1.U2.DIF top.U1.U4.DIF
```

Rules

The `require_strict_path` constraint is used by the following rule:

SpyGlass Connectivity Verify Solution
Soc_08

require_structure

Purpose

The `require_structure` constraint is used to define a structure check for all paths from source pins to destination pin.

The source pins are the hierarchical pins, which are specified using the `-from` and `-from_one_of` arguments. You can specify multiple values for the source pin.

The destination pins are the hierarchical pins, which are specified using the `-to` argument. The destination pin can assume only one value.

The structure type is controlled by the `-structure` argument, which may take one of the following values: `and`, `or`, or `xor`.

The type of check is controlled by the `-type` argument, which may take one of the following values: `Structural` or `Simulation`

Product

SpyGlass DFT solution

Syntax

```
require_structure
-[from <src_node_names>]
[-from_one_of <src_node_names>
-to <destn_node_name>
-structure <and | or | xor>
[ -type <STRUCTURAL | SIMULATION> ]
[ -cycle <number> ]
[-constraint_message_tag <value>]
```

Arguments

-from <src_node_names>

Name of hierarchical source pin. You can specify multiple values for the source pin.

-from_one_of <src_node_names>

Name of hierarchical source pin. You can specify multiple values for the source pin. However, rule checks whether at least one of the values specified using this argument is present in the fan-in.

-to <destn_node_name>

Name of hierarchical destination pin.

-structure <and | or | xor>

Specifies the structure, which may take one of the following values: and, or, xor.

-type <STRUCTURAL | SIMULATION>

Specifies the type of check as Structural or Simulation. Default type is Structural.

SpyGlass Design Constraints

-cycle <number>

Specifies number of simulation cycles

-constraint_message_tag <value>

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

NOTE: *This argument accepts only alpha-numeric characters and underscore.*

Examples

Consider the following example:

```
require_structure -from top.pin1 top.pin2 top.pin3 -to  
top.pinX -structure and
```

Rules

The `require_structure` constraint is used by the following rules:

SpyGlass Connectivity Verify Solution
Soc_07

require_value

Purpose

The `require_value` constraint defines a check that requires a logic value to be established on a certain node when the circuit has been simulated using the condition specified by the `-tag` argument.

The `require_value` constraint has two optional fields: `-allowInversion` and `matchNBits`.

When `require_value` is used twice for the same node, `spyglass` interprets as neither 0 nor 1.

Product

SpyGlass DFT solution

Syntax

The syntax for `require_value` constraint is as follows:

```
require_value
  [-name <nodename> ]
  [ -except <except_nodename> ]
  [ -type <DO_nodename> ]
  [ -except_type <exceptDO_nodename> ]
  [ -value <value> ]
  [ -value_type <type> ]
  [ -tag <condname> | -use_shift |
    -use_capture | -use_captureATspeed ]
  [ -allowInversion ]
  [ -matchNBits <num> ]
  [-constraint_message_tag <value>]
  -[report_failure_as_info]
```

Arguments

-name <nodename>

(Optional) The name can be a top-module port, or any internal net name,

or terminal name. More than one pin name can be specified, and it is effectively read as a concise description of as many individual value checks.

NOTE: *Specify either -name or -type argument.*

-except <except_nodename>

(Optional) Same as <nodename> but defines design nodes that are not to be used as name.

-type <DO_nodename>

Same as <nodename> but it takes only macros as inputs.

NOTE: *Specify either -name or -type argument.*

To view the list of macros supported by the `require_value` constraint, see [Supported Macros](#).

-except_type <exceptDO_nodename>

Same as <DO_nodename> but it takes only macros as inputs.

-value <value>

The value is a logic value string of 0, 1, X, Z, 1_or_0, and 0_or_1. A single-bit value means check at end of complete simulation. The X value is treated as do-not-compare. A multi-bit value means check on cycle-by-cycle simulation basis. For specification of a vector value, SGDC multi-bit specification format (same as used for the `test_mode` constraint) should be used.

You can specify repeat sequences for the `require_value` constraint.

For fields that require repeat sequence, you can specify the values as <I*S>. Here, S is any string that does not contain the <, >, and * characters. However, S can contain another <I*S> expression. I is an integer that is always interpreted as a decimal value. The expression <I*S> means that the sequence S will be repeated I number of times.

value_type <type>

Specify one the following values:

■ **active:** Implies 1 for non-inverting pin and 0 for inverting clock-pin.

■ **inactive:** Implies 0 for non-inverting pin and 1 for inverting clock-pin.

NOTE: *Specify either -value or -value_type argument. Otherwise, the require_value constraint is ignored for analysis.*

List of Macros Supported by the require_value and illegal_value Constraints

You can use only the following set of macros along with the -value_type argument of the require_value and illegal_value constraints:

- FLIP_FLOP_RESET
- SCAN_FLIP_FLOP_RESET
- LATCH_RESET
- FLIP_FLOP_SET
- SCAN_FLIP_FLOP_SET
- LATCH_SET
- FLIP_FLOP_ENABLE
- SCAN_FLIP_FLOP_ENABLE
- LATCH_ENABLE
- FLIP_FLOP_CLOCK
- SCAN_FLIP_FLOP_CLOCK

-constraint_message_tag <value>

Specifies a string value that gets prefixed in the violation message generated by the respective rule for the said constraint.

NOTE: *This argument accepts only alpha-numeric characters and underscore.*

-tag <condname>

(Optional) A condition previously defined by using the [define_tag](#) constraint. It describes a stimulation condition.

Note that only one condition name can be defined in a `require_value` specification. However, simulation for a given condition name simulates all pin-value specifications simultaneously. The built-in power-ground connections are also simulated in this process.

-use_shift | -use_capture | -use_captureATspeed

For any of these modifiers, `require_value` simulates test mode of that particular mode.

If `-use_shift`, `-use_capture`, or `-use_captureATspeed` argument is specified, the constraint simulates all, shift, capture, or captureATspeed test_mode constraints, respectively.

NOTE: *If more than one of the -tag, -use_shift, -use_capture, or -use_captureATspeed arguments is specified, an error condition occurs. You should specify exactly one of these modifiers with require_value constraint.*

-allowInversion

(Optional) Indicates that the inverted simulated pattern is also considered as valid pattern.

-matchNBits <num>

(Optional) Specifies that only the `<num>` number of least significant bits are to be considered. If `<num>` is greater than `<value>` (specified with `-value` argument), the latter is padded with `X` to match the former's width.

-report_failure_as_info/-report_failures_as_info

Reports all the failures as info severity message.

Examples

Consider the following examples:

Example 1

```
require_value -name abc -value "<5*10>"
```

The above example will be expanded as follows:

```
require_value -name abc -value 1010101010
```

Example 2

```
require_value -name abc -value "11<5*10>010"
```

The above example will be expanded as follows:

```
require_value -name abc -value 111010101010010
```

Example 3

```
require_value -name abc -value "<50*11<5*10>>010"
```

The above example will be expanded as follows:

```
require_value -name abc -value 111010101010...(repeated 50 times
followed by 010)
```

You can also set a variable using the command `setvar` to obtain the above result as follows:

```
setvar x 11<5*10>
require_value -name abc -value "<50*${x}>010"
```

The above example will be expanded as follows:

```
require_value -name abc -value 111010101010...(repeated 50 times
followed by 010)
```

NOTE: *Tagging for nesting is not allowed. For example, the following `require_value` statements are not allowed:*

```
require_value -name sub_seq -value <5*01>
require_value -name main_seq -value <100*sub_seq>
```

However, you can achieve the same result by using the `setvar` command.

Example 4

```
require_value -tag s1 -name top.U1.U2.SEF
-value 1010 -allowInversion -matchNBits 2
```

Example 5

Consider the following example:

```
-require_value -name p1 -value 0X110_or_110
```

The above constraint specification means:

- first bit must not be 0
- second bit is don't care
- third and fourth bits must not be 1
- fifth should be neither 0 nor 1
- sixth should not be 1
- seventh bit should not be 0

Rules

The `require_value` constraint is used by the following rule:

SpyGlass Connectivity Verify Solution	
Soc_01	Soc_01_Info
SpyGlass DFT Solution	
Conn_01	

reset

Purpose

Rules that check for resets require you to specify the names of reset signals using the `reset` keyword in a SpyGlass Design Constraints file.

Product

SpyGlass Auto Verify solution, SpyGlass CDC solution, SpyGlass DFT DSM solution, SpyGlass TXV solution, and SpyGlass ERC product

Syntax

The syntax of using the `reset` keyword in a SpyGlass Design Constraints file is as follows:

```
current_design <du-name>
  reset -name <rst-name>
    [ -async | -sync ]
    [ -value <0 | 1> ]
    [ -soft ]
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-name <rst-name>

The reset port/pin name.

The pin can be a top-level port/pin/net as well as an internal pin.

You can specify a single port/pin/net name or a space-separated list of port/pin/net names.

For top-level port/pin/nets, `<rst-name>` can be the port/pin/net's full hierarchical name or its simple name. For internal pins, `<rst-name>` must be the pin's full hierarchical name.

-async, -sync

(Optional) Specifies the reset type as asynchronous or synchronous.

If neither the `-async` argument nor the `-sync` argument is specified, the reset is assumed to be an asynchronous reset.

Any combinational gate in the data transfer path between flip-flops at the clock domain crossing or the data paths around the synchronizing flip-flops is allowed if one of the inputs to the gate is a signal declared as synchronous reset using the `reset` constraint with the `-sync` argument.

All advanced clock rules and the `Reset_Check03` and `Reset_Check04` rules also work with synchronous resets and therefore require you to specify these signals using the `reset` constraint with the `-sync` argument.

-value <0 | 1>

(Optional) Specifies the reset value as 1 or 0.

If the value is specified as 0, the reset is active low. Otherwise, the reset is active high.

If the `-value` option is not specified, the SpyGlass TXV solution assumes the reset to be active high for auto initialization.

-soft

NOTE: *This option is not applicable for the SpyGlass DFT DSM solution.*

(Optional) Specifies if the reset is a soft reset.

If this option is specified in the `reset` constraint, the reset signal is marked as a soft reset; otherwise it is marked as a hard reset.

Active state of the reset is used during initialization irrespective of hard or soft reset. However, if reset is defined as hard, it will be deactivated during functional analysis while soft reset will be used as other signals during functional analysis.

NOTE: *The -soft argument is only applicable to SpyGlass CDC formal rules. This argument has no impact on SpyGlass CDC structural rules. If a reset is mentioned by using the -soft argument, that reset is used for initialization and then ignored for functional analysis.*

Bus Support

SpyGlass CDC honors all bits of a bus in the reset SGDC constraint. The

following example shows the violations reported by the Propagate_Resets rule when the reset SGDC constraint is specified on a bus. The Verilog design file and corresponding SGDC specification is shown as follows.

Verilog

```
module top(in1, in2, clk1, clk2, rst1, rst2, out);
input in1, in2, clk1, clk2;
input [0:2] rst1;
input [0:2] rst2;
output out;

FLOP src(in1, clk1, rst1[1], src_out);
FLOP src2(src_out, clk1, rst1[1], src2_out);
FLOP src3(in, clk2, rst2[1], src3_out);

FLOP des(in, clk1, src2_out, out);
endmodule

module FLOP(input d, clk, rst, output reg q);
always@(posedge clk or negedge rst)
    if(!rst)
        q <= 1'b0;
    else
        q <= d;
endmodule
```

SGDC Specification

```
current_design top
clock -name clk1
clock -name clk2
reset -name rst1
reset -name rst2
```

The Propagate_Resets rule reports the following violations. Notice that all bits of rst1 and rst2 are reported.

Propagate_Resets (6) : Propagates resets and displays a portion of the reset tree			
For top, reset 'top.rst1[2]' not propagated	top.v	3	
For top, reset 'top.rst1[1]' propagated	top.v	3	
For top, reset 'top.rst1[0]' not propagated	top.v	3	
For top, reset 'top.rst2[2]' not propagated	top.v	4	
For top, reset 'top.rst2[1]' propagated	top.v	4	
For top, reset 'top.rst2[0]' not propagated	top.v	4	

For SpyGlass TXV Solution

The SpyGlass TXV solution honors the SGDC `reset` constraint as follows:

- If you do not provide the `-soft` option, the SpyGlass TXV solution deactivates the corresponding reset signal during verification by placing “~(active-value)” in it. You can specify the `-soft` option to stop the deactivation so that the signal is considered during verification.
- The `-sync/-async` options do not have any impact in the verification. The SpyGlass TXV solution deactivates all reset signals unless the `-soft` option is specified.

Rules

The `reset` constraint is used by the following rules:

SpyGlass Auto Verify Solution

All rules

SpyGlass CDC solution

Clock_Reset_check01	Clock_Reset_Info01	Propagate_resets	Reset_check03
Reset_check04	Reset_check06	Reset_check10	Reset_check11
Reset_sync02	Reset_sync03	Reset_sync04	Reset_info02
Ac_cdc01a	Ac_cdc01b	Ac_cdc01c	Ac_cdc08
Ac_fifo01	Ac_handshake01	Ac_handshake02	Ac_conv01
Ac_conv02	Ac_conv03	Ar_resetcross_matrix01	Reset_sync01
Setup_quasi_static01	Ac_unsync01	Ac_unsync02	Ac_conv04
Ac_conv05	Ar_resetcross01		

SpyGlass DFT DSM Solution

PLL_02

SpyGlass ERC Solution

resetPinConnectedToResetNet

SpyGlass TXV Solution

All rules

reset -async

Purpose

NOTE: `reset -async` is an alias for the `reset` constraint used by the SpyGlass DFT solution.

The `reset -async` constraint is used to specify asynchronous set or reset pins in test mode as used by the `Async_10` rule of the SpyGlass DFT solution.

Product

SpyGlass DFT solution

Syntax

```
reset -async
      -name <port-name>
```

Arguments

-name <port-name>
The asynchronous port name.

Rules

The `reset -async` constraint is used by the following rules:

SpyGlass DFT Solution	
Async_10	Info_blackboxDriver

reset_filter_path

Purpose

The `reset_filter_path` constraint specifies reset paths so that the reset crossings on these paths are ignored from SpyGlass analysis.

For example, the `Ar_resetcross01` and `Ar_sync_group` rules will not report violations for the destination reset and destination clock specified by this constraint.

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `reset_filter_path` constraint is as follows:

```
current_design <du-name>
reset_filter_path
  [-from_rst <frm-rst-list>]
  [ -to_rst <to-rst-list> ]
  [ -to_clock <destination-clock-obj-list> ]
  [ -from_clock <source-clock-obj-list> ]
  [ -from_obj <from-obj-list> ]
  [ -to_obj <to-obj-list> ]
  [ -type <rdc | sync | deassert | reset_sync02> ]
```

Arguments

-from_rst <frm-rst-list>

Space-separated list of objects (hierarchical net, pin, or port) of a source such that the reset crossings containing such sources are not considered for SpyGlass analysis.

-to_rst <to-rst-list>

Space-separated list of objects (hierarchical net, pin, or port) of a destination such that the reset crossings containing such destinations are not reported.

-from_clock <source-clk-list>

Space-separated list of source clocks.

-to_clock <dest-clk-list>

Space-separated list of destination clocks.

-no_des_rst

Optional argument to waive sequential instances that are involved in RDC but do not have any resets.

-from_obj <from-obj-list>

Space-separated list of hierarchical nets, pins, or ports of the source in the reset crossing so that the reset crossings containing such sources are not reported.

-to_obj <to-obj-list>

Space-separated list of hierarchical nets, pins, or ports of the destination in the reset crossing so that the reset crossings containing such destinations are not reported.

-clock <destination-clock-obj-list>

(Optional) Space-separated list of destination clocks.

-type <rdc | sync | deassert | reset_sync02>

(Optional) The default value is `rdc`. This value is for backward compatibility. This option is applicable only for the `Ar_resetcross01` rule.

Set this argument to `deassert` to filter the `Ar_asyncdeassert01` and `Ar_syncdeassert01` rule violations reported for the reset paths specified by this constraint.

Set this argument to `sync` to filter the `Ar_unsync01` and `Ar_sync01` rule violations reported for the reset paths specified by this constraint.

Set this argument to `reset_sync02` to filter the `Reset_sync02` rule violations reported for the reset paths specified by this constraint.

NOTE: If you specify the ***sync*** or ***deassert*** value to this argument then you can specify only ***-from_rst*** and ***-clock*** arguments to the `reset_filter_path` constraint.

Handling Multiple Clocks or Resets

For example, if a destination flip-flop receives the R1 and R2 resets and the C1 and C2 clocks, specify the following constraints to filter the *Ar_unsync01*, *Ar_sync01*, *Ar_asyncdeassert01*, and *Ar_syncdeassert01* rule violations:

```
reset_filter_path -from_rst R1 R2 -clock C1 C2 -type sync
reset_filter_path -from_rst R1 R2 -clock C1 C2 -type deassert
```

Examples

Example 1

Consider the following *Ar_asyncdeassert01* spreadsheet showing violations related to invalid crossings:

A ID	B SOURCE	C SOURCE RESET	D DEST.	E DEST. RESET	F CLOCKS	G WAIVED
2C	top.F1.q	top.r1	top.F2.q	top.r2	top.c	No
2D	top.F3.q	top.r2	top.F4.q	top.r1	top.c	No
33	top1.F7.q	top1.r1	top1.F8.q	top1.r2	top1.c	No
39	top2.F3.q	top2.r1	top2.F4.q	top2.r2	top2.c	No
3A	top2.F5.q	top2.r1	top2.F6.q	top2.r2	top2.c	No

FIGURE 44.

From the above set of violations, if you want to suppress reporting *Ar_asyncdeassert01* and *Ar_syncdeassert01* violations from the `top.r1` reset to `top.c` clock, specify the following constraint:

```
reset_filter_path -from_rst r1 -clock c -type deassert
```

After specifying the above constraint, the violations reported in the cells 2C and 33 in [Figure 44](#) are not reported.

Rules

The `reset_filter_path` constraint is used by the following rules:

SpyGlass CDC Solution		
Ar_unsync01	Ar_sync01	Ar_asyncdeassert01
Ar_sync_group	Ar_syncdeassert01	Ar_resetcross01
RFPSetup	Ar_resetcross_matrix01	

reset_pin

Purpose

The `reset_pin` constraint is used to specify black box pins that should be assumed to be reset pins.

Then, the `Async_07` rule of the *SpyGlass DFT* solution that uses asynchronous source analysis treats a pin specified with the `reset_pin` constraints just like it does the reset pins on flip-flops. The source of such a pin must be test mode controlled just as any other asynchronous source.

Product

SpyGlass DFT solution

Syntax

The syntax of the `reset_pin` constraint is as follows:

```
reset_pin
  -name <du-name>.<port-name>
  [ -value <value> ]
  [ -synchronous ]
  [ -asynchronous ]
```

Arguments

<du-name>

The name of the design unit (black box) for which you are specifying the reset pin.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are processed.

You can specify a single design unit name or a space-separated list of design unit names.

<port-name>

Name of the asynchronous reset port on the design unit (black box).
You can specify only a single port name.

-value <value>

The active value (0 or 1) for this asynchronous reset port *<port-name>*.

-synchronous

Implies that the reset pins are synchronous.

-asynchronous

Implies that the reset pins are asynchronous.

Rules

The `reset_pin` constraint is used by the following rules:

SpyGlass DFT Solution	
Async_07	Async_07Lssd

reset_synchronizer

Purpose

The `reset_synchronizer` constraint is used to specify a reset synchronizer signal along with its asserted reset value.
During SpyGlass analysis, the tool uses this signal as a synchronizing point as if a multi-flop reset synchronizer is present with its initial state value.

Product

SpyGlass CDC solution

Syntax

The syntax of the `reset_synchronizer` constraint is as follows:

```
reset_synchronizer
  -name <synchronizer-name>
  -reset <source-reset>
  -clock <synchronizer-clock> | <tag-name>
  -value <synchronizer-reset-active-value>
  -ignore
```

Arguments

-name <synchronizer-name>

Specifies the name of the synchronizer output.

You can specify a hierarchical net name, pin name, or top port name to this argument.

NOTE: *You can use a combination of wildcard characters, such as * and ? while specifying the name of a synchronizer output.*

-reset <source-reset>

Specifies the name of the source reset for which *<synchronizer-name>* is acting as a synchronizer.

You can specify a hierarchical net name, pin name, or top port name to this argument.

NOTE: *You can use a combination of wildcard characters, such as * and ? while specifying the name of a reset source.*

-clock <synchronizer-clock>

Specifies the clock of the synchronizer.

You can specify a hierarchical net name, pin name, or top port name to this argument.

NOTE: *You can use a combination of wildcard characters, such as * and ? while specifying the clock of a synchronizer.*

-clock <tag-name>

Specifies the tag name specified by the `-tag` argument of the clock constraint.

-value <synchronizer-reset-active-value>

Specifies 0 or 1 to indicate the active assertion value of the reset synchronizer provided by the user by using this constraint.

SpyGlass uses this value for de-assertion verification purposes. This value does not have any impact on the `Ar_sync01` and `Ar_unsync01` rules.

-ignore

Specifies that all the synchronizers from the object specified in the `-name` argument be ignored for synchronization of reset crossings.

NOTE: *If you use the `-ignore` argument along with the `-name` argument, the other arguments of the `reset_synchronizer` constraint are not required.*

Rules

The `reset_synchronizer` constraint is used by the following rules:

SpyGlass CDC Solution		
Ar_sync01	Ar_unsync01	Ar_syncdeassert01
Ar_asyncdeassert01	Reset_sync02	Reset_check07
Reset_check10		

retention_cell

Purpose

Specifies the retention latch cells as used by the LPSVM33, LPSVM33A, LPSVM34, LPSVM56, LPSVM57, LPSVM58, LPSVM59 rules or the SRPG cells as used by the LPSVM38 and LPPLIB10 rules of the SpyGlass Power Verify solution.

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was `retencell`.*

Product

SpyGlass Power Verify solution

Syntax

Defining Retention Latch Cells

The syntax to specify the `retention_cell` constraint for LPSVM33, LPSVM33A, LPSVM34, and LPSVM58 rules of the *SpyGlass Power Verify* solution is as follows:

```
current_design <du-name>
  retention_cell
    -name <cell-name-list>
    [ -save <save-pin-name> ]
    [ -restore <restore-pin-name> ]
    [ -clk <clk-pin-name> ]
    [ -clkval <0 | 1> ]
    [ -qTerm <q-pin-name> ]
    [-memory]
```

Defining SRPG Cells

The syntax to specify the `retention_cell` constraint for the LPSVM38 and LPPLIB10 rules of the *SpyGlass Power Verify* solution is as follows:

```
current_design <du-name>
  retention_cell
    -name <cell-name-list>
    [ -domains <domain-name-list> ]
    [ -vddpin <vdd-pin-name> ]
    [ -vddcpin <vddc-pin-name> ]
```

Defining Retention Cells with Control Pins

The syntax to specify the `retention_cell` constraint for LPSVM56, LPSVM57 and LPSVM59 rules of the *SpyGlass Power Verify* solution is as follows:

```
current_design <du-name>
  retention_cell
    -name <cell-name-list>
```

```
[ -sleep <sl-pin-name> ]
[ -save <sa-pin-name> ]
[ -restore <rst-pin-name> ]
[ -sleepval <0 | 1> ]
[ -saveval <0 | 1> ]
[ -restoreval <0 | 1> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying retention latch cells, SRPG cells, or retention cells with control pins.

-name <cell-name-list>

Space-separated name list of the retention latch cells or SRPG Cells or retention cells with control pins.

-save <save-pin-name> | -restore <restore-pin-name> | -clk <clk-pin-name> | -qTerm <q-pin-name>

Names of the save pin, restore pin, clock pin, and the output pin of the retention latch cell, respectively.

The LPSVM33 rule of the *SpyGlass Power Verify* solution uses the output pin name specified using the **-qTerm** argument. If you do not specify the **-qTerm** argument, the LPSVM33 rule assumes the output pin name to be Q.

The LPSVM33A rule of the *SpyGlass Power Verify* solution uses only the retention cell names.

The LPSVM34 rule of the *SpyGlass Power Verify* solution uses the save, restore, and clock pin names specified using the **-save**, **-restore**, and **-clk** arguments, respectively. You can specify any combination of these arguments. If you do not specify any of these arguments, the LPSVM34 rule does not perform any checks.

-clk <clk-pin-name> | -clkval <0 | 1>

The LPSVM58 rule of the *SpyGlass Power Verify* solution uses the **-clk**

argument defining the clock pin of the retention cell and the `-clkval` argument defining the active value of that clock.

-memory

The LPSVM57 rule of the *SpyGlass Power Verify* solution uses the `-memory` argument to define the retention memories. Other retention cell violations are not reported when this Boolean variable is given with the `retention_cell` constraint.

-domains <domain-name-list>

Space-separated voltage domain name list.

-vddpin <vdd-pin-name> | -vddcpin <vddc-pin-name>

Names of the Vdd pin (the pin to be connected to the OFF supply) and the Vddc pin (the pin to be connected to the always-on supply) of the SRPG cell, respectively. These values are used by the LPPLIB10 rule of the *SpyGlass Power Verify* solution.

-sleep <sl_pin-name> | -save <sa-pin-name> | -restore <rst-pin-name>

Name of the sleep pin, save pin, and restore pin.

-sleepval <0|1> | -saveval <0|1> | -restoreval <0|1>

The `-sleepval`, `-saveval`, and `-restoreval` arguments specify the active value of the respective control pins.

The LPSVM59 rule of the *SpyGlass Power Verify* solution uses the `-sleep` argument defining the sleep pin of the retention cell and the `-sleepval` argument defining the active value of that sleep pin.

The LPSVM59 rule of the *SpyGlass Power Verify* solution also uses `-save` argument defining the save pin of the retention cell and the `-saveval` argument defining the active value of that save pin. The rule also uses `-restore` argument defining the restore pin of the retention cell and the `-restoreval` argument defining the active value of that restore pin.

Rules

The `retention_cell` constraint is used by the following rules:

SpyGlass Power Verify Solution			
LPSVM33	LPSVM33A	LPSVM34	LPSVM56
LPSVM57	LPSVM58	LPSVM59	LPSVM38
LPPLIB10			

retention_instance

Purpose

Specifies the hierarchy information of retention cell instances.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `retain_instance`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `retention_instance` constraint is as follows:

```
current_design <du-name>
  retention_instance
    -name <inst-name-list>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the retention cell instances.

-name <inst-name-list>

Space-separated hierarchical instance name list (name of any instance or the top-level design unit) indicating the regions to be checked.

NOTE: Wildcard support is provided for `-name` argument of the

`retention_instance` *constraint*.

Rules

The `retention_instance` constraint is used by the following rules:

SpyGlass Power Verify Solution

LPSVM38	LPSVM58
---------	---------

rme_config

Purpose

The `rme_config` constraint is used to configure the output of RME.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions,
SpyGlass DFT solution

Syntax

```
rme_config
  -module_prefix <module-prefix>
  | -module_suffix <module-suffix>
  | -instance_prefix <instance-prefix>
  | -instance_suffix <instance-suffix>
  | -port_prefix <port-prefix>
  | -wire_prefix <wire-prefix>
  | -file_prefix <file-prefix>
  | -file_suffix <file-suffix>
  | -tm_port <tm-port-name>
  | -tclk_port <tclk-port-name>
  | -logic_at_top
  | -insert_disable_logic
  | -port_type <port_type>
  | -wire_naming_style <wire_naming_style>
  | -delay_set_reset_type <delay_set_reset_type>
```

```
| -delay_clock_type <delay_clock_type>  
| -verilog_delay_text <verilog_delay_string>  
| -vhdl_delay_text <vhdl_delay_string>  
| -insert_undef_in_clone_module <undef_value>
```

Arguments

-module_prefix <module-prefix>

Specifies the prefix of modified design modules and entities.

For example, if you want to specify the prefix of modified design modules as `atrenta_modified_`, specify the `rme_config` constraint as follows:

```
rme_config -module_prefix atrenta_modified_
```

The default value is "".

NOTE: The `-module_prefix` argument of the `rme_config` constraint is not applicable for the design *top* module.

-module_suffix <module-suffix>

Specifies the suffix of modified design modules and entities.

The default value is "".

NOTE: The `-module_suffix` argument of the `rme_config` constraint is not applicable for the design *top* module.

-instance_prefix <instance-prefix>

Specifies the prefix of modified instances.

The default value is "".

-instance_suffix <instance-suffix>

Specifies the suffix of modified instances.

The default value is "".

-port_prefix <port-prefix>

Specifies the prefix of new ports that are added to design modules and

instances.

For input ports, the default value is `atrenta_iport_`. For output ports, the default value is `atrenta_oport_`.

-wire_prefix <wire-prefix>

Specifies the prefix of new wires that are added to design modules.

The default value is `atrenta_wire`.

-file_prefix <file-prefix>

Specifies the prefix of modified and newly generated files.

For modified files, the default value is `atrenta_modified_`. For newly generated files, the default value is `atrenta_generated_`.

The user-specified prefix is applied only if the following conditions are satisfied:

- The RTL file contains description for single design module only.
- The name of RTL file (without extension) is the same as that of the design module.

-file_suffix <file-suffix>

Specifies the suffix of modified and newly generated files.

The default value is "".

-tm_port <tm-port-name>

Test mode port name. This argument is specific to the SpyGlass DFT solution.

-tclk_port <tclk-port-name>

Test clock port name. This argument is specific to the SpyGlass DFT solution.

-logic_at_top

Specifies that all glue logic should be created in the top module. This argument is specific to the SpyGlass Power Estimation and SpyGlass Power Reduction solutions.

-insert_disable_logic

Specifies that disable logic should be inserted. This argument is specific to the SpyGlass Power Estimation and SpyGlass Power Reduction solutions.

-port_type <port_type>

Specifies the data type of port in V2K and SystemVerilog modules. Possible values are wire and reg_wire.

-wire_naming_style <wire_naming_style>

Specifies the naming style of wires inserted by the AutoFix feature. The possible values are as follows:

Value	Naming Style of Inserted Wires
prefix_only	<prefix>_<count>
prefix_and_pin	<prefix>_<driver_instance_pin>_<count>

-delay_set_reset_type <delay_set_reset_type>

Specifies whether the flip-flop inserted by the AutoFix feature is a set flip-flop or a reset flip-flop. In addition, it also specifies the polarity of the set/reset pin of the flip-flop.

The possible values are as follows:

Value	Type of the flip-flop
active_high_set	An active high set flip-flop
active_low_set	An active low set flip-flop
active_high_reset	An active high reset flip-flop
active_low_reset	An active low reset flip-flop
active_high_set_reset	An active high set/reset flip-flop
active_low_set_reset	An active low set/reset flip-flop

-delay_clock_type <delay_clock_type>

Specifies the polarity of the clock of the flip-flop inserted by the AutoFix

feature.

The possible values are as follows:

Value	Type of the clock
active_high	An active high clock
active_low	An active low clock

-verilog_delay_text <verilog_delay_string>

Specifies the delay string to be inserted in a flip-flop definition in Verilog. The string is inserted in the following format:

```
q <= <verilog_delay_string> d;
```

Where *q* and *d* are the output pin and the input pin, respectively, of the flip-flop.

For example, if you want to add a numeric delay in a flip-flop definition in Verilog, specify the delay string as follows:

```
-verilog_delay_text = "# 10"
```

In this case, the modified RTL of the flip-flop will be as follows:

```
q <= # 10 d;
```

As another example, if you want to add a tick define delay in a flip-flop definition in Verilog, specify the delay string as follows:

```
-verilog_delay_text = "# `MY_DELAY"
```

In this case, the modified RTL of the flip-flop will be as follows:

```
q <= # `MY_DELAY d;
```

Please note that you have to specify a proper declaration of *MY_DELAY* in this case.

-vhdl_delay_text <vhdl_delay_string>

Specifies the delay string to be inserted in a flip-flop definition in VHDL. In addition, the *after* keyword is also inserted in the flip-flop definition.

The delay string and the *after* keyword are inserted in the following format:

```
q <= d after <vhdl_delay_string>;
```

Where, *q* and *d* are the output pin and the input pin, respectively, of the flip-flop.

For example, if you want to add a numeric delay in a flip-flop definition in VHDL, specify the delay string as follows:

```
-vhdl_delay_text = "10ns"
```

In this case, the modified RTL of the flip-flop will be as follows:

```
q <= d after 10ns;
```

As another example, if you want to add a tick define delay in a flip-flop definition in VHDL, specify the delay string as follows:

```
-vhdl_delay_text = "MY_DELAY"
```

In this case, the modified RTL of the flip-flop will be as follows:

```
q <= d after MY_DELAY;
```

Please note that you have to specify a proper declaration of *MY_DELAY* in this case.

-insert_undef_in_clone_module <undef_value>

Specify to add the `undef of macros in the Atrenta uniquified blocks. The possible values are 0 & 1.

If *undef_value* is 1, then `undef macros are inserted in Atrenta uniquified blocks. Default *undef_value* is 0.

Rules

The *rme_config* constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction solutions			
PEPWR20	PEPWR21	PEPWR22	PEPWR23
PEPWR24	PEPWR25	PESTR20	PESTR21
PESTR22	PESTR23	PESTR24	PESTR25
SpyGlass DFT Solution			
Async_07	Clock_11	Clock_11_capture	Latch_08
TA_06	Topology_01		

set_slew

Purpose

The `set_slew` constraint is used to specify slew value of all the nets of a clock domain of type data or clock. Apart from the `set_slew` constraint, the slew value can be specified in various ways. The following list shows the order from the highest priority to the lowest priority:

1. The `set_annotated_transition` SDC constraint
2. The `set_slew` SGDC constraint
3. The value from SpyGlass Physical in the SpyGlass Physical - Power Estimate flow
4. The `pe_auto_infer_slew` parameter
5. The `pe_slew` parameter

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `set_slew` constraint is as follows:

```
set_slew
-value <val>
[-clock <clock-net-name>]
[-type <data | clock>]
[-rise]
[-fall]
```

NOTE: *If both -rise and -fall arguments are defined, the value is considered as both the value of the rise slew and the fall slew.*

Arguments

-value <val>

Specifies positive float value with the unit of time. By default, the unit is nanoseconds.

-clock <clock-net-name>

Specifies the clock net name. List is supported.

-type <data | clock>

Specifies the type of net.

-rise

Specifies the rise slew value.

-fall

Specifies the fall slew value.

Examples

Example 1

In the following SGDC specification, the value `v1` is applicable for all the nets of the design.

```
set_slew -value v1
```

Example 2

You can overwrite the value for clock or data nets by specifying the type argument. In the following SGDC specification, the value `v2` is applicable to all the data nets of the design.

```
set_slew -value v2 -type data
```

Example 3

In the following SGDC specification, the value `v3` is applicable for all the clock nets of the clock domain `c1`

```
set_slew -value v3 -type clock -clock c1
```

Example 4

In the following SGDC specification, value `v1` is applied to nets of type `clock` and value `v2` is applied to nets of type `data`. The value `v3` is not applied to any net and is ignored.

```
set_slew -value v1 -type clock -clock clk1
set_slew -value v2 -type data -clock clk1
set_slew -value v3 -clock clk1
```

Example 5

In the following SGDC specification, the first `set_slew` SGDC specification is taken. Therefore, value `v1` is applied. All other SGDC constraint specifications of the same constraint are ignored.

```
set_slew -type data -value v1
set_slew -type data -value v2
```

Example 6

For nets in a merged clock domain, the value specified for the fastest clock domain is used. Suppose, clock net `n1` is a clock net in a merged clock domain of clocks `c1` and `c2`. In addition, the clock `c1` is faster than clock `c2`. In the following specification, the value `v1` is taken.

```
set_slew -clock c1 -value v1
set_slew -clock c2 -type clock -value v2
```

Rules

The `set_slew` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions

All rules

force_scan

Purpose

The `force_scan` constraint is used to declare flip-flops as scannable even if they do not so qualify.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `force_scan` constraint is as follows:

```
force_scan
  -name <du-name> | <reg-name> |
  -clock_control <signal-name> |
  -set_control <signal-name> |
  -reset_control <signal-name>
  -register_suffix <suffixes>
  -module_suffix <suffixes>
  [-latch]
  [-flip_flop]
```

NOTE: *The `force_scan` constraint supports wildcard characters. Using wildcards, expression is expanded only within the hierarchy.*

Arguments

-name <du-name>

Specifies the name of the design unit from which scan is included.

You can specify design units that are single flip-flops or design units where one or more flip-flops are described besides other logic. Then, all flip-flops in the specified design unit are included in scan.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are considered.

You can specify a single design unit name or a space-separated list of design unit names.

-name <reg-name>

The name of a net that is connected to the output pin of a flip-flop.

Then, the corresponding flip-flop is included in scan.

You can specify a simple net name or a hierarchical net name. The net specified as simple net name is searched at the top-level.

You can specify a single net name or a space-separated list of net names.

NOTE: *You can specify design unit names, net names, or a combination of both.*

<signal-name>

Flip-flops whose control pins (clock, set, or reset) are driven by this signal are included in the scan.

NOTE: *The traversal involved in this is structural only.*

The <signal-name> argument can have the following values:

- **CLOCK:** For the clock_control option, if the <signal-name> argument is driving CLOCK pin of the flip-flop.
- **SET:** For the set_control option, if the <signal-name> argument is driving SET pin of the flip-flop.
- **RESET:** For the reset_control option, if the <signal-name> argument is driving RESET pin of the flip-flop.

-register_suffix <suffixes>

Space-separated list of suffixes to be specified as scannable. The -register_suffix argument should not be used along with other arguments of the force_scan constraint, that is, -name, -clock_control, -set_control, or -reset_control.

If the value of the dft_treat_suffix_as_pattern parameter is set to on, the register_suffix value is used as a pattern to be matched with the register name. The pattern may be present anywhere in the register name, excluding the path.

If the value of the dft_check_path_name_for_register_suffix parameter is on, the value of the -register_suffix field will be matched with the register name along with the path in which the register is present.

-module_suffix <suffixes>

Define this field to use suffix based pattern match for all module names.

If the value of the `dft_treat_suffix_as_pattern` parameter is on, the value of the `-module_suffix` field will be matched with the module name along with the path in which the module is present.

-latch

Marks only latches as scannable.

-flip_flop

Marks only flip-flops as scannable.

NOTE: *If you do not specify either -latch or -flip-flop options, then, both latches and flip-flops are marked as scannable.*

Examples

You can use the `force_scan` constraint in the following ways:

Specifying only the design unit names with the -name argument

By specifying a design unit name using the `-name` argument only, all instances of this design unit are considered scannable. The following `force_scan` constraint indicates that all flip-flops within all instances of `modName1` will be considered scannable:

```
force_scan -name modName1
```

Specifying only the register names with the -name argument

By specifying a net name using the `-name` argument only, the corresponding flip-flop will be considered scannable. The following `force_scan` constraint indicates that the flip-flop whose output pin is connected to net `reg_123` (at the top-level) will be considered scannable:

```
force_scan -name reg_123
```

Specifying list of suffixes using the -register_suffix argument

Consider the following example:

```
current_design top_scan
  force_scan -register_suffix ffl
```


In the above example, All flip-flops with name ending with ff1 will be marked as scan.

Rules

The `force_scan` constraint is used by the following rules:

SpyGlass DFT Solution

All rules

force_stable_value

Purpose

The `force_stable_value` constraint specifies nodes that will have a stable value during the scan shift and capture modes.

Product

SpyGlass DFT solution

Syntax

The syntax for the `force_stable_value` constraint is as follows:

```
force_stable_value
  [-name <nodename> ]
  [ -except <except_nodename> ]
  [ -type <DO_nodename> ]
  [ -except_type <exceptDO_nodename> ]
  [ -value <value> ]
  [ -tag <condname> | -use_shift | -use_capture |
    -use_captureATspeed ]
```

Arguments

-name <nodename>

(Optional) The name can be a top-module port, or any internal net name, or terminal name. More than one pin name can be specified, and it is

effectively read as a concise description of as many individual value checks.

NOTE: Specify either *-name* or *-type* argument.

-except <except_nodename>

(Optional) Same as <nodename> but defines design nodes that are not to be used as name.

-type <DO_nodename>

Same as <nodename> but it takes only macros as inputs.

NOTE: Specify either *-name* or *-type* argument.

To view the list of macros supported by the `force_stable_value` constraint, see [Supported Macros](#).

-except_type <exceptDO_nodename>

Same as <DO_nodename> but it takes only macros as inputs.

-value <value>

The value is a logic value string of 0, 1, X, Z, 1_or_0, and 0_or_1. A single-bit value means check at end of complete simulation. The X value is treated as do-not-compare. A multi-bit value means check on cycle-by-cycle simulation basis. For specification of a vector value, SGDC multi-bit specification format (same as used for the `test_mode` constraint) should be used.

You can specify repeat sequences for the `force_stable_value` constraint.

For fields that require repeat sequence, you can specify the values as <I*S>. Here, S is any string that does not contain the <, >, and * characters. However, S can contain another <I*S> expression. I is an integer that is always interpreted as a decimal value. The expression <I*S> means that the sequence S will be repeated I number of times.

-tag <condname>

(Optional) A condition previously defined by using the [define_tag](#) constraint. It describes a stimulation condition.

Note that only one condition name can be defined in a `force_stable_value` specification. However, simulation for a given condition name simulates all pin-value specifications simultaneously. The built-in power-ground connections are also simulated in this process.

-use_shift | -use_capture | -use_captureATspeed

For any of these modifiers, `force_stable_value` simulates test mode of that particular mode.

If `-use_shift`, `-use_capture`, or `-use_captureATspeed` argument is specified, the constraint simulates all, shift, capture, or captureATspeed test_mode constraints, respectively.

NOTE: *If more than one of the -tag, -use_shift, -use_capture, or -use_captureATspeed arguments is specified, an error condition occurs. You should specify exactly one of these modifiers with require_stable_value constraint.*

Examples

Rules

The `force_stable_value` constraint is used by the following rule:

SpyGlass Connectivity Verify Solution
Soc_14

force_unstable_value

Purpose

The `force_unstable_value` constraint specifies nodes that will have an unstable value during the scan shift and capture modes.

Product

SpyGlass DFT solution

Syntax

The syntax for the `force_unstable_value` constraint is as follows:

```
force_unstable_value
  [-name <nodename> ]
  [ -except <except_nodename> ]
  [ -type <DO_nodename> ]
  [ -except_type <exceptDO_nodename> ]
  [ -value <value> ]
  [ -tag <condname> | -use_shift | -use_capture |
    -use_captureATspeed ]
```

Arguments

-name <nodename>

(Optional) The name can be a top-module port, or any internal net name, or terminal name. More than one pin name can be specified, and it is effectively read as a concise description of as many individual value checks.

NOTE: Specify either *-name* or *-type* argument.

-except <except_nodename>

(Optional) Same as *<nodename>* but defines design nodes that are not to be used as name.

-type <DO_nodename>

Same as *<nodename>* but it takes only macros as inputs.

NOTE: Specify either *-name* or *-type* argument.

To view the list of macros supported by the `force_unstable_value` constraint, see [Supported Macros](#).

-except_type <exceptDO_nodename>

Same as *<DO_nodename>* but it takes only macros as inputs.

-value <value>

The value is a logic value string of 0, 1, X, Z, 1_or_0, and 0_or_1. A single-bit value means check at end of complete simulation. The X value is treated as do-not-compare. A multi-bit value means check on cycle-by-cycle simulation basis. For specification of a vector value, SGDC multi-bit specification format (same as used for the `test_mode` constraint) should be used.

You can specify repeat sequences for the `force_unstable_value` constraint.

For fields that require repeat sequence, you can specify the values as `<I*S>`. Here, S is any string that does not contain the `<`, `>`, and `*` characters. However, S can contain another `<I*S>` expression. I is an integer that is always interpreted as a decimal value. The expression `<I*S>` means that the sequence S will be repeated I number of times.

-tag <condname>

(Optional) A condition previously defined by using the [define_tag](#) constraint. It describes a stimulation condition.

Note that only one condition name can be defined in a `force_unstable_value` specification. However, simulation for a given condition name simulates all pin-value specifications simultaneously. The built-in power-ground connections are also simulated in this process.

-use_shift | -use_capture | -use_captureATspeed

For any of these modifiers, `force_unstable_value` simulates test mode of that particular mode.

If `-use_shift`, `-use_capture`, or `-use_captureATspeed` argument is specified, the constraint simulates all, shift, capture, or captureATspeed test_mode constraints, respectively.

NOTE: *If more than one of the -tag, -use_shift, -use_capture, or -use_captureATspeed arguments is specified, an error condition occurs. You should specify exactly one of these modifiers with require_stable_value constraint.*

Examples

Currently Unavailable

Rules

The `force_unstable_value` constraint is used by the following rule:

SpyGlass Connectivity Verify Solution

Soc_14

scan_cell

Purpose

The `scan_cell` constraint is used to define scan cell design units and their data pins as used by the Scan_18 rule.

Product

SpyGlass DFT solution

Syntax

The syntax of the `scan_cell` constraint is as follows:

```
scan_cell
  -module <du-name>
  -datapin <dpin-name>
```

Arguments

-module <du-name>

The name of the scan cell design unit.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are considered.

You can only specify a single design unit name.

-datapin <dpin-name>

Name of the data pin of the scan cell design unit.

Rules

The `scan_cell` constraint is used by the following rule:

SpyGlass DFT Solution
Scan_18

scan_chain

Purpose

The `scan_chain` constraint is used to specify the scan chains for the rules mentioned in the [Rules](#) section.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `scanchain`.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `scan_chain` constraint is as follows:

```
scan_chain
[ -module <name> ]
-scanin <pin-name>
-scanout <pin-name>
[ -scanenable <tag-name> ]
[ -scanin_clock_pin <pin-name> ]
[ -scanout_clock_pin <pin-name> ]
[ -scanin_clock_pin_phase_inverted ]
[ -scanout_clock_pin_phase_inverted ]
```

Arguments

-scanin <pin-name> / -scanout <pin-name>

Complete hierarchical name of starting/ending port/pin.

The starting/ending points can be primary ports as well as internal pins.

NOTE: *The -scanin and -scanout arguments support wildcard expressions.*

For primary ports, you can also specify the simple port name as in the following example:

```
current_design top
  scan_chain -scanin in15 -scanout out23 ...
```

-scanenable <tag-name>

(Optional) Specifies the simulation condition *<tag-name>* (specified using the `define_tag` constraint) required for scan chain detection.

If the `-scanenable` argument is not specified, the default simulation conditions are assumed.

-module <name>

(Optional) Name of the black box module.

Specifying this option implies that the software should assume a valid scan chain from its specified scan-in through the scan-out port.

-scanin_clock_pin <pin-name>

(Optional) Specifies the scanin clock pin name.

-scanout_clock_pin <pin-name>

(Optional) Specifies the scanout clock pin name.

-scanin_clock_pin_phase_inverted

(Optional) Specifies whether the scan-in clock pin phase is inverted or not.

-scanout_clock_pin_phase_inverted

(Optional) Specifies whether the scan-out clock pin phase is inverted or not.

NOTE: The SpyGlass DFT product honors the `-scanin_clock_pin`, `-scanout_clock_pin`, `-scanin_clock_pin_phase_inverted`, and `-scanout_clock_pin_phase_inverted` arguments only when you have specified a black-box module using the `-module` argument.

Rules

The `scan_chain` constraint is used by the following rules:

SpyGlass DFT Solution			
Info_schain	Info_stilFile	Diagnose_ScanChain	Scan_22
Scan_24	Scan_25	Scan_26	Scan_29
Scan_32	Scan_33	Scan_34	Scan_35
Scan_36	Scan_38	Scan_39	Scan_40
Scan_41	Topology_15	dftSGDCSTX_069	
SpyGlass DFT DSM Solution			
SP_01	SP_05		
SpyGlass Built-in Solution			
SGDC_schain01	SGDC_schain02	SGDC_schain03	

Examples

The following examples show the usage of the `scan_chain` constraint.

Example 1

Consider the following example:

```
module test (input [1:0] si, output [1:0] so);
  scan_chain -scanin si -scanout so
```

The above example will be expanded as follows:

```
scan_chain -scanin si[1] -scanout so[1]
scan_chain -scanin si[0] -scanout so[0]
```

Example 2

Consider the following example:

```
module test (input [19:0] scanin_bus, output [63:0]
scanout_bus);
```

```
scan_chain -scanin_bus[15:0] -scanout_bus[31:16]
```

The above example will be expanded as follows:

```
scan_chain -scanin scanin_bus[15] -scanout scanout_bus[31]
```

```
scan_chain -scanin scanin_bus[14] -scanout scanout_bus[30]
```

```
scan_chain -scanin scanin_bus[13] -scanout scanout_bus[29]
```

```
...
```

```
scan_chain -scanin scanin_bus[1] -scanout scanout_bus[17]
```

```
scan_chain -scanin scanin_bus[0] -scanout scanout_bus[16]
```

Example 3

Consider the following example:

```
scan_chain -module bbox_scan_chain -scanin si1 -scanout so1
-scanin_clock_pin si1clk -scanout_clock_pin solclk_inv
-scanin_clock_pin_phase_inverted false
-scanout_clock_pin_phase_inverted true
```

In the above example, the black box module, `bbox_scan_chain`, has a scan chain from its scanin pin, `si1`, to its scanout pin, `so1`. In this case, the clock pin of scan in flop is driven by pin, `si1clk`, with no phase inversion. Whereas, the scanout flop clock pin is driven by pin, `solclk_inv`, with inversion.

scan_enable_source

Purpose

This constraint is used to define the source signal for scan chain enables. Such sources are then used by various Scan Enable rules.

If this constraint is not used, potential nodes are determined as candidates for use in a `scan_enable_source` constraint. In that case, Scan Enable rules will use these potential nodes as the basis for checking rule compliance.

Product

SpyGlass DFT DSM solution

Syntax

The syntax of the `scan_enable_source` constraint is as follows:

```
scan_enable_source
  -name <SE_name>
  [ -active_value <0 | 1> ]
  [ -mode <combinational | sequential> ]
  [ -clock <clock-name> ]
```

Arguments

-name <SE_name>

Specifies the scan enable name

-active_value <0 | 1>

(Optional) Check that when this `scan_enable` source has `-active_value` (and any specified define tags are also simulated), the scan enable pins on flip-flops fed by this source are enabled for scan.

-mode <combinational | sequential>

(Optional) Specifies how to treat the associated source node, that is, whether to consider it as combinational or sequential.

-clock <clock-name>

(Optional) Specifies the name of the clock.

Rules

The `scan_enable_source` constraint is used by the following rules:

SpyGlass DFT DSM Solution

All Scan Enable rules

scan_ratio

Purpose

The `scan_ratio` constraint is used to determine what fraction of the flip-flops should be scanned or scannable. You can calculate `scan_ratio` using the following formula:

$$\text{scan_ratio} = (\text{scannable flip-flops} + \text{forced scan flip-flops}) / (\text{total number of flip-flops} - \text{forced_no_scan flip-flops} - \text{synthesis_redundant flip-flops})$$

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `scanratio`.

Product

SpyGlass DFT solution

Syntax

The syntax of the `scan_ratio` constraint is as follows:

```
scan_ratio -value <value>
```

Arguments

-value <value>

A scan ratio as a decimal value less than 1 such as 0.95.

Rules

The `scan_ratio` constraint is used by the following rule:

SpyGlass DFT Solution
Scan_11

scan_type

Purpose

The `scan_type` constraint is used to specify the SpyGlass DFT type (LSSD or MUXSCAN).

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `scantype`.

Product

SpyGlass DFT solution

Syntax

The syntax of the `scan_type` constraint is as follows:

```
scan_type
[ -lssd | -clock | -muxscan | -allscan ]
```

Arguments

The `scan_type` constraint has the following arguments:

-lssd

(Optional) SpyGlass DFT solution scannability type.

The `-lssd` argument specifies that scannability depends only on sets and resets being inactive in the shift mode. Scannability no longer depends on test clock controllability since, in LSSD, a separate test clock that is independent of all system clocks is inserted during scan replacement and

synthesis.

-clock

The `-clock` argument specifies that scannability depends only on the test clock controllability.

-muxscan

The `-muxscan` argument (default) specifies that scannability depends on the test clock controllability and set/reset controllability.

NOTE: *If both `-lssd` argument and `-clock` argument are specified, it is assumed that the `-muxscan` argument has been specified.*

-allscan

The `-allscan` argument specifies that all flip-flops except those declared as no scan flip-flops (using the `force_no_scan` constraint) are to be considered as scannable.

Rules

The `scan_type` constraint is used by the following rules:

SpyGlass DFT Solution			
Async_07Lssd	Async_09	Bist_04	clock_23
Info_uncontrollable	Info_unobservable	Info_undetecCause	Info_coverage
Info_untestable	Info_unused	Info_noscanFlopsTextReport	Info_schain
Coverage_audit	Latch_15	Scan_08	Scan_11
Scan_16	Scan_17	Scan_18	Scan_19
Scan_20	Scan_21	Scan_22	Scan_24
Scan_25	Scan_26	TA_06	

scan_wrap

Purpose

The `scan_wrap` constraint is used to specify black box design units or instances that will be designed with scan wrappers.

Use of the `scan_wrap` constraint causes the input pins of the designated black box design unit or instance to be treated as observable, output pins to be treated as controllable and inout pins to be treated as both controllable and observable.

You can use the [dftSetAllBBSscanwrapped](#) parameter to make all the black boxes scan-wrapped.

You can use [force_ta](#) constraint to override the controllability and observability of individual pins of the scan wrapped module. Please refer to the [Specifying force_ta constraint on the individual pins of the scan wrapped module](#) section in the Examples section for more information.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `scanwrap`.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The syntax of the `scan_wrap` constraint is as follows:

```
scan_wrap
  -name <du-name> | <inst-name>
  [ -enable <en-pin-list> ]
  [ -envalue <en-value-list>]
```

NOTE: The `scan_wrap` constraint supports wildcard characters.

Arguments

-name <du-name>

The scanwrap design unit (black box) name.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are considered.

You can specify a single design unit name or a space-separated list of design unit names.

-name <inst-name>

The scanwrap design unit (black box) hierarchical instance name.

The master design unit of the specified instance must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

You can specify a single instance name or a space-separated list of instance names.

-enable <en-pin-list>

List of enable pins of design unit. These pins should be activated for scan wrap to be enabled.

-envalue <en-value-list>

The enable value of the enable pins. It is the list of 0 or 1 values. These values specify the signals that activate the corresponding enable pins.

Examples

You can use the `scan_wrap` constraint in the following ways:

Specifying only the design unit names with the -name argument

By specifying a black box design unit name using the `-name` argument only, all instances of this design unit are considered as scan-wrapped. The following `scan_wrap` constraint indicates that all instances of `modName1` will be considered scan-wrapped:

```
scan_wrap -name modName1
```

Specifying only the instance names with the -name argument

By specifying a hierarchical instance name using the `-name` argument,

only the corresponding instance will be considered scan-wrapped. The following `scan_wrap` constraint indicates that the instance `BB_I123` (at the top-level) will be considered scan-wrapped:

```
scan_wrap -name BB_I123
```

Specifying enable pins of the scan wrapped design unit

Consider the following command:

```
scan_wrap -name bb_inst1 -enable en1 en2 -envalue 0 1
```

The above command describes that enable pins, `en1` and `en2`, should have values 0 and 1 respectively for scan wrap to be enabled for `bb_inst1`.

Specifying `force_ta` constraint on the individual pins of the scan wrapped module

Consider the following constraint description file, `test.sgdc`:

```
current_design top
  test_mode -name en -value 1

  scan_wrap -name bbox_en      -enable en -envalue 1
  scan_wrap -name non_bbox
  scan_wrap -name bbox

  force_ta -name "bbox::op2" -control nyn
  force_ta -name "bbox::in2" -observe n
```

In the above example, `scan_wrap` constraint is applied on the module, `bbox`. Then, `force_ta` constraint is applied on black box instances, `op2` and `in2`. Here, `force_ta` constraint takes precedence over the `scan_wrap` constraint. Therefore, `op2` and `in2` are uncontrollable and unobservable, respectively.

[Figure 45](#) and [Figure 46](#) illustrate the uncontrollable and unobservable conditions after specifying the `force_ta` constraint.

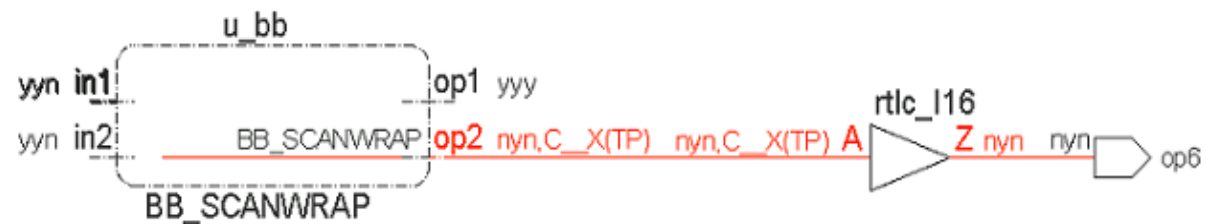


FIGURE 45. force_ta: Uncontrollable condition

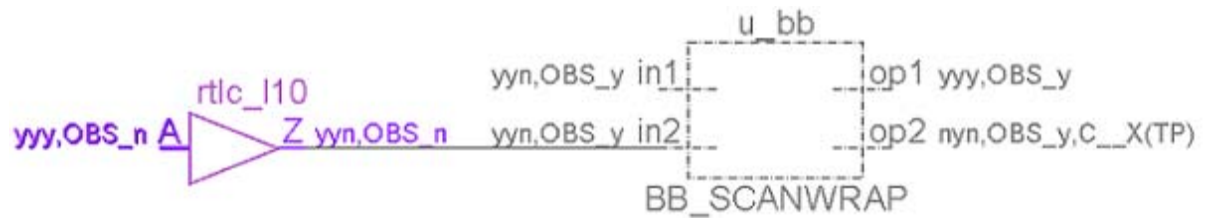


FIGURE 46. force_ta: Unobservable condition

Rules

The scan_wrap constraint is used by the following rules:

SpyGlass DFT Solution			
Async_02_shift	Async_02_capture	Bist_04	Info_uncontrollable
Info_unobservable	Info_undetectedCause	Info_coverage	Info_untestable
Info_unused	Coverage_audit	RAM_01	RAM_02
RAM_03	RAM_04	RAM_05	RAM_06
RAM_07	RAM_08	RAM_09	RAM_10
Scan_17	TA_06		
SpyGlass DFT DSM Solution			
All rules			

sdc_data

Purpose

The `sdc_data` constraint is used to specify the SDC file.

NOTE: *Prior to SpyGlass 4.3.0 release, the name of this constraint was `sdcschema`.*

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions, SpyGlass Constraints solution, SpyGlass DFT DSM solution, SpyGlass CDC solution

For the SpyGlass Power Estimation and SpyGlass Power Reduction Solutions

Syntax

The syntax to specify the `sdc_data` constraint is as follows:

```
current_design <du-name>  
  sdc_data -file <file-list>
```

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-file <file-list>

Space-separated list of SDC files or a single SDC file.

Rules

The `sdc_data` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	poweraudit	

For SpyGlass Constraints solution

Syntax

The syntax to specify the `sdc_data` constraint is as follows:

```
current_design <du-name>
  sdc_data -file <file-list>
[ -level <level> ]
[ -mode <mode> ]
[ -corner best | worst ]
```

Arguments

<du-name>

The top-level module name (for Verilog designs) or the top-level entity name (for VHDL designs) or a synthesis partition name specified using the `block` keyword.

-file <file-list>

Specifies a list of associated SDC files. You can also specify compressed SDC files in the gzip format with this argument.

If in the SGDC file, multiple SDC files are specified in different SDC schemas with identical values of `mode/corner` options or if no value is specified for these options, SpyGlass reports a violation message. Therefore, the correct way to specify multiple SDC files is based on the following conditions:

- If the SDC files are dependent, specify all the files with a single SDC-schema as shown in the following example:

```
sdc_data -file bar1.sdc bar2.sdc
```

- If the SDC files are independent, specify each file in a different schema with its respective mode as shown in the following example:

```
sdc_data -file bar1.sdc -mode test -corner best
sdc_data -file bar2.sdc -mode func -corner typical
```

If you are specifying a list of SDC files with the `-file` argument, ensure that the SDC files are specified in the correct order so that the definition of a variable is available before it is used.

All of the following arguments are attached to these SDC files to qualify the scope of these SDC files.

If rules that require SDC are run and no `sdc_data` constraint is specified in SGDC, the following fatal message is reported.

```
SGDC_sdc_data02: SDC_data is required to run some of the
selected rule(s).
```

NOTE: *The `-type` argument that was used to specify SDC files is replaced by the `-file` argument. While the `-type` argument is still available for backward compatibility, its functionality is expected to change in future.*

-level <level>

(Optional) Specifies the scope of the SDC File type by design phase. You can specify either `rtl`, `prelayout`, `postlayout`, or `all`.

Use this argument to specify the design phase to which a particular SDC file is applicable. In the SpyGlass Constraint solution, most rules work in a specific, pre-defined design phase. Pre-defined design phase is configurable by specifying goals. Therefore, you do not need to specify the level argument.

The rules that work across design phases are: `Const_Struct03` and `Clk_Uncert04`. Therefore, these rules leverage the design phase specified in the `level` argument. The following example illustrates the use of the `level` argument.

```
#Constraints.sgdc
current_design top

sdc_data -file top_function_pre.sdc -level prelayout -mode
function
```

```
sdcd_data -file top_function_post.sdc -level postlayout -mode
function

sdcd_data -file top_scan_all.sdc -level all -mode scan

#top_function_pre.sdc
set_case_analysis 1 [get_ports "MODE1"]
set_case_analysis 1 [get_ports "MODE2"]

#top_function_post.sdc
set_case_analysis 1 [get_ports "MODE1"]
set_case_analysis 0 [get_ports "MODE2"]

#top_scan_all.sdc
set_case_analysis 1 [get_ports "MODE1"]
set_case_analysis 1 [get_ports "MODE2"]
```

After you run the Const_Struct03 rule, violation messages are reported because:

- the `set_case_analysis` settings for the function mode, which is specified in `top_function_pre.sdc`, and scan mode, which is specified in `top_scan_all.sdc`, are identical.
- the `set_case_analysis` settings for the same function mode in different phases, which is `top_function_pre.sdc` file for the Pre-layout phase and `top_function_post.sdc` file for the Post-layout phase, are not the same.

-mode <mode>

(Optional) Specify the mode name to indicate a functional mode of operation, such as test mode, functional mode, or bypass mode, for the design. The rules in the SpyGlass Constraints solution are independently evaluated for each mode of operation and coverage is generated for each specified mode.

For example, you can specify the functional mode or the scan shift mode. As the mode name is user-defined, it does not follow any pattern; its only purpose is to differentiate one mode from another. If omitted, the current schema applies to all modes.

In addition, you can specify a seed SDC file for the SDC_GenerateIncr rule by specifying `seed` as the mode name.

When using constraints management of the SpyGlass Constraints solution, the mode values have specific meanings. For example, mode could be `flatTop` or `block` for the `Equiv_SDC_block` rule.

-corner [best | worst]

(Optional) Identifies the analysis corner for the `Const_Struct08`, `Const_Struct09`, `Inp_Trans05`, and `Inp_Trans06` rules.

For all other rules, the associated files will be assumed valid for both best and worst corners, where applicable.

block -name <name-list>

(Optional) Specifies sub-blocks that are partitions from synthesis if the current design is actually a top-level design.

Each name in `<name-list>` should name a module (for Verilog) or entity (for VHDL) appearing in the design that should be treated as a top-level partition. More than one `block` constraint may appear, or all blocks may be specified in one constraint.

NOTE: *The top design is also considered as a block and all block-level rules are run on it unless explicitly mentioned otherwise.*

blocksize [-min <num>] [-max <num>]

(Optional) Specify the minimum and maximum allowed block sizes in terms of gate count.

You must specify at least one of the `-min` and `-max` arguments with valid integer values.

By default, the minimum block size is 5000 gates and maximum block size is 100000 gates.

Rules

The `sdc_data` constraint is used by the following rules.

SpyGlass Constraints Solution

All rules except the ParamSanityCheck01a, ParamSanityCheck01b, Const_Struct02, SDC_GenerateIncr, and Block02 rules

For SpyGlass DFT DSM solution

Syntax

The syntax to specify the `sdc_data` constraint is as follows:

```
current_design <du-name>
  sdc_data -file <sdc-file-name>
```

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-file <sdc-file-name>

Name of the SDC file.

Rules

The `sdc_data` constraint is used by the following rule:

SpyGlass DFT DSM Solution

Atspeed_05

validation_filter_path

Purpose

The `validation_filter_path` constraint is used to filter data domain violations reported during block validation. For example, consider the following schematic:

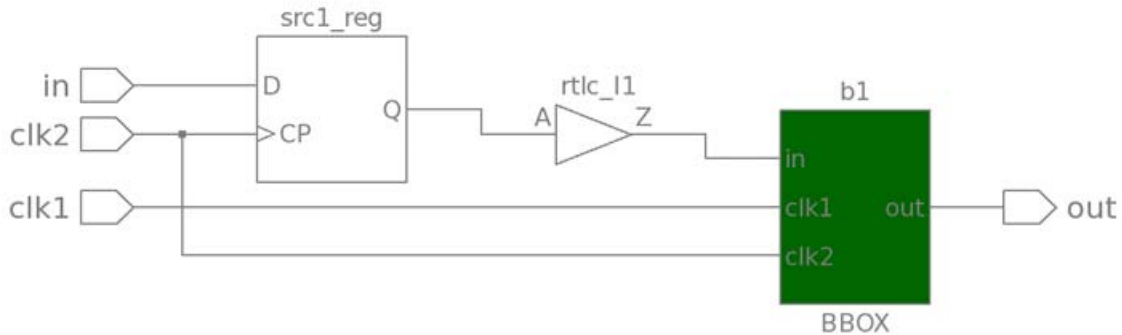


FIGURE 47.

In addition, consider the following SGDC constraints at the top level and block level respectively:

Sgdc constraint at top level

```

clock -name clk1 -domain d1
clock -name clk2 -domain d2
validation_filter_path -from_obj "test.src1" -to_clock
"clk1"
  
```

Sgdc constraint at block level

```

clock -name clk1 -domain d1
clock -name clk2 -domain d2
abstract_port -ports in -clock clk1
  
```

In the above scenario, data domain violations between `src1_reg` and `b1.in` are filtered because the `validation_filter_path` constraint has been specified.

Product

SpyGlass CDC solution

For SpyGlass CDC Solution

Syntax

The syntax to specify the `validation_filter_path` constraint is as follows:

```
validation_filter_path -from_obj <src-object-name>  
-to_clock <clock-of-destination-port>
```

Arguments

-from_obj <src-object-name>

Name of the source object that is causing the data domain violation.

-to_clock <clock-of-destination-port>

Name of the block's clock port associated with the destination block port that is causing the data domain violation.

Rules

The `validation_filter_path` constraint is used by the following rules:

SpyGlass CDC Solution
Ac_abstract_validation01

select_wireload_model

Purpose

The `select_wireload_model` constraint is used to specify the wire-load model for the design.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `selectwireloadmodel`.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `select_wireload_model` constraint is as follows:

```
select_wireload_model
[ -instname <inst-name-list> ]
  -wireload <wl-name> | -wireloadtable <wltable-name>
  -net_type < clock | clock_leaf | clock_non_leaf |
hard_macro | std_logic >
  -size <float>
  -start_size <float>
  -end_size <float>
  -tvlg <string>
```

Arguments

-wireload <wl-name>

Name of the `wire_load` structure in the associated technology library.

-wireloadtable <wltable-name>

Name of the `wire_load_table` structure in the associated technology library.

-instname <inst-name-list>

The instance names must be hierarchical names with respect to the `current_design <du-name>`. This requirement is checked by the `SGDC_power_est17` rule. If you do not specify instance names using the optional `-instname` argument, the specified wire-load model condition is applicable to all instances under the environment `<du-name>` unless

overridden by another `select_wireload_model` constraint specifying particular instance(s).

You can specify multiple `select_wireload_model` constraints.

If you do not specify any `select_wireload_model` constraint, the `default_wire_load_selection` library attribute is used for inferring the wire-load models. If both are not specified or available, the `default_wire_load` library attribute is used. This requirement is checked by the `SGDC_power_est10` rule.

The actual wire-loads used by these rules are reported in the `pe_wireload` Report.

NOTE: *The `-instname` argument cannot be specified while specifying the wire-load model for the top-level design unit. Until now, you needed to specify the name of the top-level design unit with the `-instname` argument. Therefore, you must modify your existing `select_wireload_model` constraints for the top-level design units accordingly. Otherwise, SpyGlass will exit with a fatal error (`SGDC_power_est17`).*

-net_type < `clock|clock_leaf|clock_non_leaf|hard_macro|std_logic` >

Defines the type of net:

- **clock:** Clock nets
- **clock_leaf:** Clock nets, which are directly connected to the clock pin of latches and registers, are considered as `clock_leaf` nets.
- **clock_non_leaf:** Clock nets, which are directly connected to buffers, inverters, or ICGCs, are considered as `clock_non_leaf` nets.
- **hard_macro:** Nets are the input of cells with no functionality (memories, blackboxes etc.)
- **std_logic:** Combinational and sequential nets

For `clock_leaf` and `clock_non_leaf` types of nets, additional capacitance tables are generated when the `power/power_calibration` goal is run on the reference netlist.

-size <float>

Specifies the relative size of a cell.

SpyGlass Design Constraints

The smaller the specified value, the smaller is the size.

The specified value must be greater than 0. In addition, float values that are less than 1 are supported. For any value greater than 1, specify an integer. If you specify an unsupported value, a violation message is reported.

After the SpyGlass Power Estimate product run, a `pe_cell_sizing_info.csv` file is created. Open this file to review the sizing interpretation of the libraries.

-start_size <float>

Defines a specific start size or a size range.

The specified value must be greater than 0. In addition, float values that are less than 1 are supported. For any value greater than 1, specify an integer. If you specify an unsupported value, a violation message is reported.

-end_size <float>

Defines a specific start size or a size range.

The specified value must be greater than 0. In addition, float values that are less than 1 are supported. For any value greater than 1, specify an integer. If you specify an unsupported value, a violation message is reported.

-tvsg <string>

Defines the string which corresponds to vt of the design.

Rules

The `select_wireload_model` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	poweraudit	

seq_atpg

Purpose

The `seq_atpg` constraint is used when sequential ATPG tools are planned.

Sequential ATPG will not require all flip-flops to be scanned. Instead, the state of non-scan flip-flops is controlled by properly controlling their next state function as well as their clocks. Repeated actions of this type, which will results in multi vector tests, allow various states to be loaded into the non-scan flip-flops.

The default condition for SpyGlass DFT solution is combinational ATPG. In that case, the state of non-scan flip-flops is controllable only if their set and reset pins, if any, are controllable. When the `seq_atpg` constraint is set to sequential mode, the state of non-scan flip-flops is controllable only if their data pins are controllable.

The `seq_atpg` constraint causes controllability of non-scannable designs to accurately model the power of an ideal sequential ATPG and, therefore, provide a more realistic estimate of fault coverage. The `seq_atpg` constraint may be applied whether or not scan is intended.

Product

SpyGlass DFT solution

Syntax

The syntax of the `seq_atpg` constraint is as follows:

```
seq_atpg -value 1
```

Arguments

-value

The `seq_atpg` constraint is specified with value 1 to enable the feature.

Rules

The `seq_atpg` constraint is used by the following rules:

SpyGlass DFT Solution	
Info_undetectedCause	Info_coverage

set

Purpose

The `set` constraint is used to specify the names of set signals.

Product

SpyGlass ERC Product

Syntax

The syntax of using the `set` keyword in a SpyGlass Design Constraints file is as follows:

```
current_design <du-name>
    set -name <set-name>
```

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in *<entity-name>.<arch-name>* format (for VHDL designs).

-name <set-name>

The set port/pin name.

The pin can be a top-level port/pin as well as an internal pin.

You can specify a single port/pin name or a space-separated list of port/pin names.

For top-level port/pins, *<set-name>* can be the port/pin's full hierarchical name or its simple name. For internal pins, *<set-name>* must be the pin's full hierarchical name.

Rules

The `set` constraint is used by the following rule:

SpyGlass ERC Solution

`setPinConnectedToSetNet`

set_case_analysis

Purpose

The `set_case_analysis` constraint specifies the case analysis conditions.

NOTE: For SpyGlass DFT and SpyGlass DFT DSM products, the `set_case_analysis` constraint is treated as the [test_mode](#) -functional constraint.

Product

SpyGlass Auto Verify solution, SpyGlass Power Verify solution, SpyGlass Power Estimation and SpyGlass Power Reduction solutions, SpyGlass ERC product, SpyGlass CDC solution, SpyGlass latch product, SpyGlass OpenMore product, and SpyGlass STARC product., SpyGlass DFT solution, SpyGlass DFT DSM solution

Syntax

The `set_case_analysis` constraint uses the following syntax:

```
current_design <du-name>
set_case_analysis
  -name {<name>}
  -value <value>
```


Arguments

<du-name>

The module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs) in the design

-name <name>

A primary port name or hierarchical name of a pin/net. The pin can be a primary pin or an internal pin.

You can also specify a space-separated list of primary port names or hierarchical pin/net names.

The following example shows that the `set_case_analysis` constraint has been defined on internal net `top.U1.U13.tm1` and `top.U3.tm3` with value 0:

```
current_design top
  set_case_analysis
    -name top.U1.U13.tm1 -value 0
```

For primary ports, you can also specify the simple port name as in the following example:

```
current_design top
  set_case_analysis -name in15 -value 1
```

NOTE: The value set on the signal specified with the `set_case_analysis` keyword is automatically propagated through the design. You do not need to set values for other signals in the path.

NOTE: The `set_case_analysis` constraints support wildcard characters. The supported meta-characters are `*` (star) and `?` (question mark), where `*` matches any number of characters and `?` matches only one character. The wildcard support is applicable for non-escaped names only. If the meta-characters appear inside an escaped name, they are treated as literals. For example, in the expression `"top.\mid* .bottom"`, `\mid*` is considered as a literal and does not expand to `mid1`, `mid2`, and so on. In addition, if you specify a hierarchical path by using a wildcard, any sub-portion of this path that contains the wildcard does not cross the module boundary while searching for the expression in the design. This means that each level in the hierarchy path should be mentioned explicitly in the wildcard

string. For example, the expression "top.mid.bottom" will expand to top.mid1.bottom", and not to "top.mid1.u1.bottom".*

NOTE: *The expression on which a wildcard is used should always be enclosed within double quotes. For example, "top.mid*.bottom".*

NOTE: *The wildcard support is applicable for design objects only. For non-design objects, the support is not applicable.*

-value <value>

Value list for a pin. The value list is the sequence of one or more values (each value being 0, 1, X, Z, or a combination).

You can also specify repeat sequences for the value as <I*S>. Here, S is any string that does not contain the <, >, and * characters. However, S can contain another <I*S> expression. I is an integer that is always interpreted as a decimal value. The expression <I*S> means that the sequence S will be repeated I number of times.

Consider the following examples:

Example1:

```
set_case_analysis -name abc -value "<5*10>"
```

The above example will be expanded as follows:

```
set_case_analysis -name abc -value 1010101010
```

Example2:

```
set_case_analysis -name abc -value "11<5*10>010"
```

The above example will be expanded as follows:

```
set_case_analysis -name abc -value 111010101010010
```

Example3:

```
set_case_analysis -name abc -value "<50*11<5*10>>010"
```

The above example will be expanded as follows:

```
set_case_analysis -name abc -value 111010101010...(repeated
50 times followed by 010)
```

You can also set a variable using the command `setvar` to obtain the above result as follows:

```
setvar x 11<5*10>
set_case_analysis -name abc -value "<50*${x}>010"
```

The above example will be expanded as follows:

```
set_case_analysis -name abc -value 111010101010...(repeated
50 times followed by 010)
```

NOTE: *If the `set_case_analysis` keyword is not specified or is incorrectly specified, the feature is not enabled.*

To specify the values for a vector signal, you can specify the value as a binary/decimal/hexadecimal number enclosed in curly brackets as in the following example:

```
set_case_analysis -name a[7:0] -value {h 0F}
```

The above specification can also be written with different value bases as follows:

```
set_case_analysis -name a[7:0] -value {d 15}
set_case_analysis -name a[7:0] -value {h F}
set_case_analysis -name a -value {b 00001111}
set_case_analysis -name a[7:0] -value {b 00001111}
```

Values are assigned to vectors from left to right. If you have not given values for all the bits of the vector, the remaining bits will all be assigned a value of 0.

For example,

```
set_case_analysis -name a[7:0] -value {b 0101}
```

The above code is equivalent to the following bit-value assignment:

```
a[0] = 1
a[1] = 0
a[2] = 1
a[3] = 0
a[4] = 0
a[5] = 0
a[6] = 0
a[7] = 0
```

You can also specify the `set_case_analysis` constraint using .sdc file as follows:

```
set_case_analysis 0|1 [get_ports <name>]
```

While specifying the `set_case_analysis` constraint using the .sdc file, specify the following command in the SGDC file:

```
sdc_data -file <sdc-file-name>
```

For SpyGlass ERC Product and SpyGlass CDC solution

For SpyGlass ERC product and SpyGlass CDC solution, if any combinational gate is found between a starting point and destination point, it is assumed that the signal would reach the destination depending on the other inputs to these gates. If the signal is found to be blocked due to the scalar values specified for other inputs at these gates, by using the `set_case_analysis` constraint, you can stop the further processing of that particular path.

NOTE: *For the SpyGlass CDC solution, the information supplied by the `set_case_analysis` constraint is also printed in the Section A: Case Analysis Settings section of the SpyGlass CDC-Summary Report.*

Examples

For SpyGlass CDC solution

The following examples show usage of `set_case_analysis` constraint. Consider the following example:

```
module test(clk1, clk2, testclock, tm, data, out);
    input clk1, clk2, testclock, tm, data;
    output out;

    reg out, temp;

    wire clock1, clock2;

    assign clock1 = tm ? testclock : clk1;
    assign clock2 = tm ? testclock : clk2;

    always @(posedge clock1)
        temp <= data;
```

SpyGlass Design Constraints

```
always @(posedge clock2)
    out <= temp;
endmodule
```

Normally, SpyGlass will mark the clock crossing between flip-flops `temp` and `out` if the clock names `clk1` and `clk2` are specified using the `clock` keyword in a design constraints file.

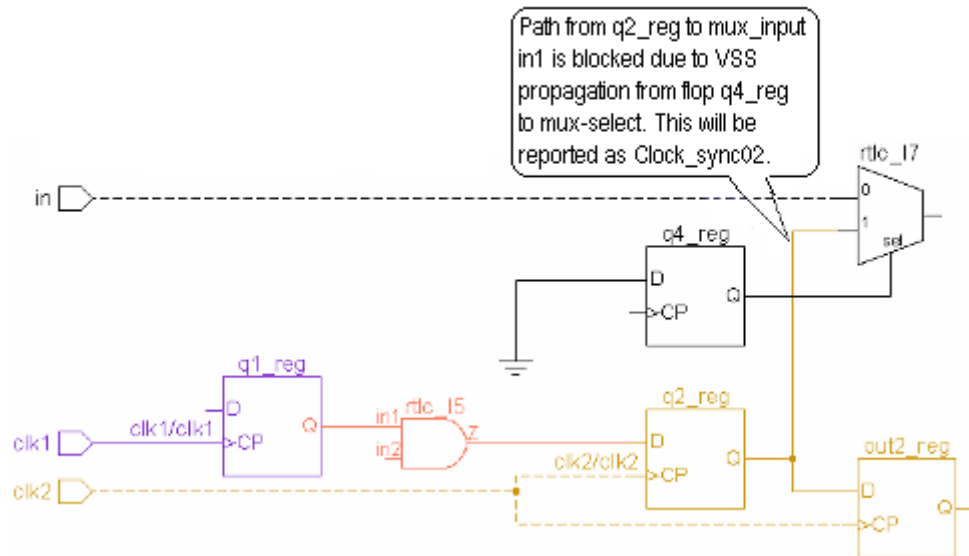
However, SpyGlass will not mark the clock crossing between flip-flops `temp` and `out` if you also supply a design constraint file as follows:

```
current_design test
    set_case_analysis -name tm -value 1
```

If you supply `set_case_analysis` constraints, the information is also printed in the Section A: Case Analysis Settings section of the SpyGlass CDC Summary Report.

Set-case analysis and power-ground can be propagated through flip-flops without specifying clock events when the `set_option enable_const_prop_thru_seq yes` command is specified in the project file. With this option, flip-flops will be considered as feed through buffers during case analysis and constant propagation, regardless of the clock, set, and reset feeding the flip-flops. The same process is applied to latches and other sequential elements.

This is illustrated in the following figure:

**FIGURE 48.** Flip-Flops as Feed-Through Buffers

For SpyGlass Power Verify solution

NOTE: For the SpyGlass Power Verify solution, the value for `set_case_analysis` will not be propagated, if the combination logic lies in power domain only for rules based on simulation (use LE for simulation).

Examples

As shown in the following figure, the value for `test_mode` is specified as 0 by `set_case_analysis` constraint and it goes to the logic (OR gate) `rtlci_I3` is in power domain, the value of `set_case_analysis` is not propagated beyond OR gate and the value at output net will be X.

SpyGlass Design Constraints

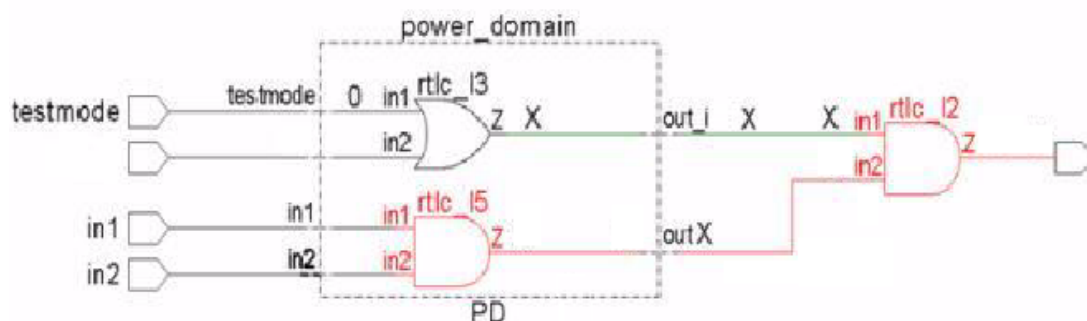


FIGURE 49. The test_mode constraint used by the set_case_analysis constraint

Rules

The set_case_analysis constraint is used by the following rules:

SpyGlass Auto Verify Solution

All rules

SpyGlass CDC Solution

All rules except the DeltaDelay01, NoClockCell, and PortTimeDelay rules.

SpyGlass DFT Product

All rules

SpyGlass DFT DSM Product

All rules

SpyGlass ERC Product

FloatingInputs	FlopClockConstant	FlopClockUndriven	FlopDataConstant
FlopDataUndriven	FlopSRConst	FlopEConst	FlopSR
LatchEnableUndriven	LatchEnableConstant	LatchDataUndriven	LatchDataConstant
MuxSelConst	DisabledAnd	DisabledOr	TristateConst
FlopClockX	FlopDataX	FlopSREX	LatchEnableX
LatchDataX	checkNetDriver	checkIOPinConnectedToNet	noCombinatorialFeedback

clockPinsConnecte dToClkNets	resetPinConnectedTo ResetNet	setPinConnec tedToSetNet	
SpyGlass latch Product			
LatchFeedback			
SpyGlass OpenMore Product			
CombLoop			
SpyGlass STARC Product			
STARC05-1.2.1.3			
SpyGlass Power Estimation and SpyGlass Power Reduction Solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	PESTR05	PESTR06
PESTR07	PESTR08	PESTR09	PESTR10
PESTR11	PESTR12	PESTR03	PESTR13
poweraudit			
SpyGlass Power Verify Solution			
LPSVM08	LPSVM09	LPSVM10	LPSVM28
LPSVM31	LPSVM47	LPSVM51	LPSVM52
LPSVM53	LPSVM56	LPSVM57	LPSVM59
LPPLIB17	LPSVM12	LPSVM15	LPSVM60

NOTE: For the SpyGlass Power Estimation and SpyGlass Power Reduction solutions, the `set_case_analysis` constraint set on a net, overrides the simulation file (VCD/FSDB) and activity constraint information. In addition, if you want the `set_case_analysis` constraint to propagate through the sequential cells, set the `set_option enable_const_prop_thru_seq yes` command in the project file.

set_cell_allocation

Purpose

The `set_cell_allocation` constraint is used to achieve a distribution across multiple sizes for the same cell types in the different clock domains in the design.

The sizes are relative, that is, size 1 refers to smallest size of a particular type and 2 the second higher size. In addition, each size has an associated percentage.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `set_cell_allocation` constraint is as follows:

```
set_cell_allocation
  -type <combinational | sequential>
  -size <float> | -alphabet_size <char>
  -area
  -percentage <float>
  [ -clock_period <float> ]
  [ -clock <string> ]
  [ -instname <string> ]
```

NOTE: The `-clock_period`, `-clock`, and `-instname` arguments are mutually exclusive.

Arguments

-type <combinational | sequential>

Specifies the type of cell.

Possible values are `combinational` and `sequential`.

-size <float>

Specifies the relative size of a cell.

The smaller the specified value, the smaller is the size.

The specified value must be greater than 0. In addition, float values that are less than 1 are supported. For any value greater than 1, specify an integer. If you specify an unsupported value, a violation message is reported.

After the SpyGlass Power Estimate product is run, a `pe_cell_sizing_info.csv` file is created. Open this file to review the sizing interpretation of the libraries.

See [Example 1](#), [Example 2](#), and [Example 3](#) for more information.

NOTE: You cannot specify this argument with the `-alphabet_size` argument.

-alphabet_size <char>

Specifies the relative size of a cell in character size format, such as L or M. If you specify an unsupported value, a violation message is reported.

After the SpyGlass Power Estimate product is run, a `pe_cell_sizing_info.csv` file is created. Open this file to review the sizing interpretation of the libraries.

See [Example 3](#) for more information.

NOTE: You cannot specify this argument with the `-size` argument.

-area

Specifies that the size of the cell will be calculated based on the area of the library cell. By default, the `max_cap` attribute of the library cell is used to calculate the size of the cell.

-percentage <float>

Specifies the percentage of cell size specified in the `-size` argument.

The specified percentage can be between 0 and 100 or 100. For example, you can specify the percentage as 0.1, 20, and 100. However, please note that the percentage cannot be 0.

NOTE: The sum of the values specified for a type of cell (sequential / combinational) in different `set_cell_allocation` constraints for different clocks periods and clock names must be equal to 100.

-clock_period <float>

(Optional) Specifies the period of the clock for which the constraints is to be applied.

NOTE: The *-clock_period*, *-clock*, and *-instname* arguments are mutually exclusive.

-clock <string>

(Optional) Specifies the clock name for which the constraint is to be applied. The type of clock nets supported are:

- Port clocks
- Clocks driven by black boxes
- Clocks specified using the `clock` SGDC constraint

NOTE: The *-clock_period*, *-clock*, and *-instname* arguments are mutually exclusive.

-instname <string>

(Optional) Specifies the hierarchical instance name for which the constraint is to be applied. See [Example 2](#) for more information.

NOTE: The *-clock_period*, *-clock*, and *-instname* arguments are mutually exclusive.

Examples

Example 1

Consider that you have a set of sequential cells of four different sizes in your SGLIB file. Also, consider the following:

- 80% of the cells in clock domain CLK1 should use cells of size 2 and 20% of the cells in clock domain CLK1 should use size 4
- 75% of the cells in clock domain CLK2 should use cells of size 4 and 25% of the cells in clock domain CLK2 should use size 8

To specify the above in SpyGlass, specify the following set of constraints:

```
set_cell_allocation -type sequential -size 2
  -percentage 80 -clock CLK1
```

```
set_cell_allocation -type sequential -size 4
  -percentage 20 -clock CLK1
```

```
set_cell_allocation -type sequential -size 4  
-percentage 75 -clock CLK2
```

```
set_cell_allocation -type sequential -size 8  
-percentage 25 -clock CLK2
```

Please note that in this case, cells of size 1 and 3 will not be present in the design.

Example 2

You can specify the cell distribution as per the hierarchical instance. For example, suppose there is an instance called **top.u1**. In this instance, consider the following:

- 40% of sequential cells should use cells of size 2
- 60% of sequential cells should use cells of size 4

To specify the above in SpyGlass, specify the following set of constraints:

```
current_design top  
  
set_cell_allocation -instname top.u1 -type sequential -size 2  
-percentage 40  
  
set_cell_allocation -instname top.u1 -type sequential -size 4  
-percentage 60  
  
...
```

Example 3

You can specify the cell distribution as per the hierarchical instance. For example, suppose there is an instance called **top.u1**. In this instance, consider the following:

- 40% of sequential cells should use cells of size L
- 60% of sequential cells should use cells of size 4

To specify the above in SpyGlass, specify the following set of constraints:

```
current_design top  
  
set_cell_allocation -instname top.u1 -type sequential -  
alphabet_size L -percentage 40
```

SpyGlass Design Constraints

```
set_cell_allocation -instname top.u1 -type sequential -size 4
-percentage 60
...
```

Rules

The set_cell_allocation constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions		
PECELLDIST	PECHECK37	PECHECK40

set_cell_name_pattern

Purpose

The `set_cell_name_pattern` constraint is used to identify the sizing information by using the naming convention of cells.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `set_cell_name_pattern` constraint is as follows:

```
set_cell_name_pattern
-name <pattern>
```

Arguments

-name <pattern>

Specifies the predefined values that form the naming convention of cells.

The pattern takes the following syntax `%sx%d`, where

- `%s` stands for the string part of the name that does not convey the size
- `x` in this case is a constant string that exists in each cell name
- `%d` stands for the drive of the cell; it is not restricted to a number and can be some predefined characters, such as:
 - ☐ L: Low strength
 - ☐ M: Medium strength
 - ☐ V: High strength

See [Example 1](#), [Example 2](#), and [Example 3](#).

Alternatively, this pattern can take the following syntax `%sx[%d/<alphabets>]`, where

- `%s` stands for the string part of the name that does not convey the size

- `x` in this case is a constant string that exists in each cell name
- `[%d/<alphabets>]` represents the drive of the cell. Brackets are used to denote size of the cell. Therefore, either `%d` or brackets can be inside a cell name pattern.

The Brackets are introduced to match some specific type of cells with the drive strength given inside the brackets. This extends to match the drive not only to L,M, or V but any valid size in alphabets such as P, Q etc. (if the library supports such type of cells). For example, if you specify `%sx[LPQ]`, it matches only those cells which have drive strength L, P or Q.

Only alphabetic characters and `%d` is allowed inside brackets. The following are examples of wrong usage:

- ❑ `%sX[12]`
- ❑ `%sX[_0]`

See [Example 4](#), [Example 5](#), and [Example 6](#).

Examples

Example 1

In the following `set_cell_name_pattern` constraint specification, the cell named **dti_hvt_28m_and2x2** is represented.

```
set_cell_name_pattern -name "%sx%d"
```

In this example, the

- `%s` corresponds to **dti_hvt_28m_and2**
- `x` corresponds to **x**
- `%d` corresponds to the drive, which is **2**

Example 2

In the following `set_cell_name_pattern` constraint specification, the cell named **dti_hvt_28m_and2xp5** is represented. In this cell, the size is 0.5.

```
set_cell_name_pattern -name "%sx%d"
```

In this example:

- %s corresponds to **dti_hvt_28m_and2**
- x corresponds to **x**
- %d corresponds to the drive, which is **p5**. The **p** signifies a decimal value.

However, for the following `set_cell_name_pattern` constraint specification, the **p** is considered as a character. Therefore, the size is 5.

```
set_cell_name_pattern -name "%sxp%d"
```

Example 3

In the following `set_cell_name_pattern` constraint specification, the cell named **abc_x2_def_x4** is represented. In this cell, the drive is 4.

```
set_cell_name_pattern -name "%sx%d"
```

In this example, the

- %s corresponds to **abc_x2_def_**
- x corresponds to **x**
- %d corresponds to the drive, which is **4**.

In this example, if the drive was represented by 2 in the cell name **abc_x2_def_x4**, the following `set_cell_name_pattern` constraint specification can be used:

```
set_cell_name_pattern -name "%sx%d%s"
```

Example 4

In the following `set_cell_name_pattern` constraint specification, the **ABC2FGXP** (drive P) and **ABCDE2333LMXQ** (drive Q) cells are represented.

```
set_cell_name_pattern %sX[PQ]
```

In this example, the

- %s corresponds to **ABC2FG** and **ABCDE2333LM**
- x corresponds to **X**

- [PQ] corresponds to cells with drive strengths P or Q matching the pattern %sXP or %sXQ.

Example 5

In the following `set_cell_name_pattern` constraint specification, all the cells with drive strengths L or M and with patterns as **%sL%s** or **%sM%s** are matched.

```
set_cell_name_pattern %s[LM]%s
```

Therefore, the following cells are matched:

- **ABC2LPQR**: This cell is matched with cell strength as L and not 2.
- **ABCTT2MW**: This cell is matched with cell strength as M and not 2.

However, a cell named **ABCPQR2L** is not matched because the cell might have L as drive strength, but it does not match the pattern.

Example 6

In the following `set_cell_name_pattern` constraint specification, all cells that have the **%sX%d** or **%sXT** patterns are matched:

```
set_cell_name_pattern %sX[%dT]
```

%d and brackets cannot be used in one cell name pattern. However, you can use %d inside brackets. For example, %sX[PQR%d] means to match %sXP, %sXQ, %sXR, or %sX%d.

Rules

The `set_cell_name_pattern` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions		
PECELLDIST	PECHECK40	SGDC_power_est67

set_clock_gating_type

Purpose

The `set_clock_gating_type` constraint is used to specify the name of the cell that should be used as the clock-gating cell.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `set_clock_gating_type` constraint is as follows:

```
current_design <du-name>
  set_clock_gating_type
    [ -libname <lib-name> ]
    -cellname <cell-name>
    [ -control_location <control-locations> ]
    [ -control_function <control-function> ]
    [ -posedge ]
    [ -negedge ]
```

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-libname <lib-name>

(Optional) Name of the library in which the ICGC is located.

-cellname <cell-name>

Name of the cell to be used as the clock-gating cell.

NOTE: *You can also specify cell names using regular expressions.*

-control_location <control_location>

(Optional) Specifies the control point on the clock gate cell. This field can have the following enumeration values:

none | before | after

If you specify the value as *before* / *after*, a clock gate cell having an OR gate with the original register enable and a test signal is picked up. These values (*before/after*) signify the position of the OR gate with respect to the register.

-control_function <control_function>

(Optional) Specifies the control signal on the clock gate cells. It can have the following enumeration values:

test_mode | scan_enable

-posedge

(Optional) Specifies the polarity of the clock-edge logic. This field means that a clock-gate cell having positive edge clock logic should be picked-up.

-negedge

(Optional) Specifies the polarity of the clock-edge logic. This field means that a clock-gate cell having negative edge clock logic should be picked-up.

NOTE: *If you do not specify either of the -posedge or -negedge options, the best ICG cell present in the library is chosen. Further, if you specify either of the two options along with an unknown/invalid combination for the other two values, the best ICG cell with posedge or negedge polarity is selected, respectively, along with an appropriate warning.*

NOTE: *The clock-gate name fields (cellname and libname) and clock-gate characteristic fields (control_location, control_function, posedge, negedge) should not be specified together.*

Consider the following example:

```
set_clock_gating_type -control_location before -posedge -
control_function scan_enable
```

Here, a clock-gating cell having positive edge logic at clock and an OR gate before the register with *scan_enable* signal and enable signal of register

is picked up.

SpyGlass selects the best ICGC cell based on the following table:

Clock-edge polarity	Control signal	Control point	ICGC type
posedge	-	-	latch_posedge
negedge	-	-	latch_negedge
posedge	scan_enable	before	latch_posedge_precontrol
posedge	scan_enable	after	latch_posedge_postcontrol
posedge	test_mode	before	latch_posedge_precontrol_obs
posedge	test_mode	after	latch_posedge_postcontrol_obs
negedge	scan_enable	before	latch_negedge_precontrol
negedge	scan_enable	after	latch_negedge_postcontrol
negedge	test_mode	before	latch_negedge_precontrol_obs
negedge	test_mode	after	latch_negedge_postcontrol_obs

Rules

The `set_clock_gating_type` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR14	poweraudit		

set_fully_decoded_bus

Purpose

The `set_fully_decoded_bus` constraint is used to specify three state buses for which if one tristate enable is active, it ensure that enables of all the other tristate devices driving it are disabled.

Product

SpyGlass DFT DSM Solution

Syntax

The syntax of `set_fully_decoded_bus` is as follows:

```
set_fully_decoded_bus  
  -name <net_name_list>
```

Arguments

-name <node_name>

List of fully decoded three state buses.

Rules

The `set_fully_decoded_bus` constraint is used by the *Info_random_resistance* rule of the SpyGlass DFT DSM solution.

Examples

The following example shows the usage of the `set_fully_decoded_bus` constraint:

```
set_fully_decoded_bus -name top.net
```

set_mega_cell

Purpose

The `set_mega_cell` constraint is used to defined megacells in a design.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

```
set_mega_cell  
  -name <cell-names>
```

Arguments

-name <cell-names>

The names of the cells that you want SpyGlass to consider as megacells. You can only specify black boxes, grey boxes, or library cells that have no functionality as megacells.

To specify a range of cells as megacells, use wildcards.

Examples

Example 1: Specifying Multiple Megacells

The following constraint specification defines three cells as megacells:

```
set_mega_cell -name MPX41PND MPX42PND MPX43PND
```

Example 2: Specifying Multiple Megacells by Using Wildcards.

The following constraint specification defines all cells that begin with the prefix MPX as megacells:

```
set_mega_cell -name MPX*
```

set_power_info

Purpose

The `set_power_info` constraint is used to specify user attributes on the pins of library cell.

NOTE: Wildcard support is provided for the `set_power_info` constraint. Supported meta-characters are `*` (star) and `?` (question mark), where `*` matches any number of characters and `?` matches only one character.

Product

SpyGlass Power Verify

Syntax

The syntax to specify the `set_power_info` constraint is as follows:

```
current_design <du-name>
  set_power_info
    -cell <cell-name>
    -pin <pin-name>
    -attribute <attribute-name>
    -value < true | false>
```

Arguments

current_design <du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

cell <cell-name>

Name of the library cell.

pin <pin-name>

Name of the pin of the library cell.

attribute <attribute-name>

Name of the user specified attribute that needs to be applied on the pin of the library cell.

value <true | false>

Value of the user specified attribute applied on the pin of the library cell.

Examples

In this example, the `has_pass_gate` attribute is applied to the macro cell at `in1` pin.

```
set_power_info -cell macro -pin in1 -attribute has_pass_gate
-value true
```

Rules

The `set_power_info` constraint is used by the following rule:

SpyGlass Power Verify
LPCONN06

sg_multicycle_path

Purpose

The `sg_multicycle_path` constraint enables you to specify multi-cycle paths, which you want to exclude from the at-speed testing.

NOTE: *You can convert the `set_multicycle_path` SDC command to `sg_multicycle_path` SGDC command using the SDC to SGDC conversion. For details, refer to the Atrenta Console Reference Guide.*

Product

SpyGlass DFT DSM

Syntax

The syntax to specify the `sg_multicycle_path` constraint is as follows:

```
sg_multicycle_path
  [-from <from_list>]
  [-to <to_list>]
  [-through <through_list>]
  [-path_multiplier <integer_value>]
```

Arguments

`-from <from_list>`

Specifies a list of objects that act as the start point for the multi-cycle path. A valid start point is a clock, a primary input or inout port, a sequential cell, a clock pin of a sequential cell. If a clock is specified, all registers and primary inputs related to the clock are considered as start points.

`-to <to_list>`

Specifies a list of objects that act as endpoint for the multi-cycle path. A valid endpoint is a primary output or inout port, a sequential cell, a data pin of a sequential cell.

`-through <through_list>`

Specifies a list of pins or nets that you want to disable.

NOTE: *The DFT DSM policy does not support the -through option.*

`-path_multiplier <integer_value>`

Specifies the number of cycles that a path must have for setup or hold, relative to the start point or end point clock, before data is required at the endpoint. For example, specifying a `path_multiplier` of 2 for setup implies a 2-cycle data path.

Examples

Consider the following example:

SpyGlass Design Constraints

```
sg_multicycle_path -from FF1 -to FF2 -path_multiplier 2
```

Here, path from FF1 to FF2 is a 2-cycle path.

syn_set_dont_use

Purpose

The `syn_set_dont_use` constraint is used to exclude selected cells from technology mapping and timing optimization.

NOTE: Wildcard support is provided for the `syn_set_dont_use` constraint. Supported meta-characters are `*` (star) and `?` (question mark), where `*` matches any number of characters and `?` matches only one character.

The SpyGlass Power Estimation and SpyGlass Power Reduction solutions allow overrides for the `syn_set_dont_use` constraint. However, the override statements are taken collectively and no order is followed while processing them.

Let us see the implication on the `syn_set_dont_use` constraint using the following example:

```
syn_set_dont_use -libname TSMC -cellname "*"
syn_set_dont_use -libname TSMC -cellname "*VT*" -exclude 1
syn_set_dont_use -libname TSMC -cellname "*HVT*" -exclude 1
```

Here, the first command tells SpyGlass to ignore all cells (specified with an `*`). The second command excludes `*VT*` cells from the ignored set. Third one excludes `*HVT*` cells (that are already included in the second statement). SpyGlass processes all these statements collectively and uses only `*VT*` cells for technology mapping and timing optimization.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `syn_set_dont_use` constraint is as follows:

```
current_design <du-name>
  syn_set_dont_use
    [-libname <lib-name>]
    -cellname <cell-names-list>
```

`-exclude <0|1|yes|no>`

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

-libname <lib-name>

(Optional) Name of the library in which the cells are located.

-cellname <cell-names-list>

A space-separated list of cells that are to be excluded.

NOTE: *You can also specify cell names using regular expressions.*

-exclude <0|1|yes|no>

(Optional) In some cases, where a cell in the technology library has `dont_use` attribute set as `true`, and you want to use that cell for mapping, set the `-exclude` argument to 1. The specified cell gets included as a candidate for mapping, when this argument is set to 1.

You should use the `-exclude` argument for including cells like integrating clock-gating cells that have the `dont_use` attribute set as `true` in the technology library for mapping.

Rules

The `syn_set_dont_use` constraint is used by the following rules:

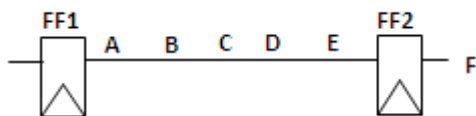
SpyGlass Power Estimation and SpyGlass Power Reduction Solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	poweraudit	

ignore_nets

Purpose

The `ignore_nets` constraint is used to exclude nets from the expression of a new/strengthened enable.

For example, consider the following figure:



In the above figure, the FF1 flip-flop is connected to the A net, which is connected to the B net. The B net is connected to the C net, which is then connected to the D net. The D net is connected to the E net, which is connected to the FF2 flip-flop. The FF2 flip-flop is connected to the F net.

Now consider that the enable expression in this case is $C \ \& \ X$, where X is a net in the design (not shown in the above figure).

Consider that you have specified the following `ignore_nets` constraint:

```
ignore_nets -net C
```

In this case, the nets to be excluded from the enable expression are A, B, C, D, and E. Therefore, the enable expression in this case will be X.

The search for the net to be excluded is done forward and backward. However, the navigation from one net to another net is limited to going through buffers and inverters. For example, consider the following expression:

```
C = A & B
```

For the above expression, if the constraint is set on the B net, the C net will not be excluded from the expression.

NOTE: If the specified net is not available after synthesis, the net is not removed from the enable expression.

The `ignore_nets` constraint is applied on a net directly when the net name is specified. However, the constraint can also be indirectly applied on

the net by specifying a buffer name or an inverter name.

NOTE: *Wildcard support is provided for the ignore_nets constraint. Supported meta-characters are * (star) and ? (question mark), where * matches any number of characters and ? matches only one character.*

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the ignore_nets constraint is as follows:

```
current_design <du-name>
  ignore_nets
    -net <net-names-list>
```

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

-net <net-names-list>

A space-separated list of nets that are to be excluded from the expression of a new/strengthened enable.

NOTE: *You can also specify net names using regular expressions.*

Rules

The ignore_nets constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions			
PEPWR20	PEPWR21	PEPWR22	PEPWR23
PEPWR24	PEPWR25	PEPWR28	PESTR20

PESTR21	PESTR22	PESTR23	PESTR24
PESTR25	PESTR28		

set_lib_timing_mode

Purpose

The `set_lib_timing_mode` constraint is used to select the active timing mode of cell groups.

Product

SpyGlass Core

Syntax

The syntax to specify the `set_lib_timing_mode` constraint is as follows:

```
set_lib_timing_mode
  -modes <timing-mode-names>
  -instances <instances-list>
```

Arguments

-modes <timing-mode-names>

List of the names of timing modes defined in a liberty file.

-instances <instance-list>

List of instances of liberty cells.

Example

Consider the following cell definition in a liberty file:

```
// Define mode definition
mode_definition (Mode_def1) {
    mode_value( model1) {      /* empty when-sdf_cond */ }
    mode_value( model2) {      /* empty when-sdf_cond */ }
}

pin(Q) {
    direction : output;
```

```

timing() {
  related_pin : "clk1";
  mode (Mode_def1, model);
}
timing() {

```

Now consider U1 as the instance of the library cell. In this case, if you define the following constraint, SpyGlass enables the timing arc only for clk1 and therefore, only clk1 will be propagated for U1:

```
set_lib_timing_mode -modes model -instances U1
```

Rules

The `set_lib_timing_mode` constraint is used by the following rules:

SpyGlass Core

SGDC_set_lib_timing_mode01

SGDC_set_lib_timing_mode02

set_lib_name

Purpose

The `set_lib_name` constraint is used to specify the cell name defined in the corresponding liberty file.

Product

SpyGlass Power Verify

Syntax

The syntax to specify the `set_lib_name` constraint is as follows:

```
set_lib_name -cell <cell-name>
```

Arguments

-cell <cell-name>

Name of the cell defined in the liberty file.

Example

Consider the following command:

```
...
set_lib_name -cell cellA -cell cellB
...
```

If this constraint is not specified in the SGDC file, the UPF_lowpower26 rule does not run and reports following message:

```
set_lib_name is not specified in SGDC file, it is mandatory for
the rule to run"
```

Rules

The `set_lib_name` constraint is used by the following rule:

SpyGlass Power Verify
UPF_lowpower26

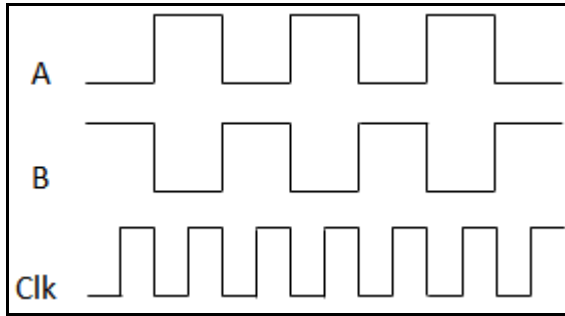
set_monitor_cell

Purpose

The `set_monitor_cell` constraint is specified to estimate the power of special cells, such as mega cells, which consume huge power.

By default, the SpyGlass Power Estimate solution estimates the power of a cell based on the activity and probability values. However, when the cell is specified using this constraint, all the instances of the cell are calculated in a high accuracy mode. In this mode, all the `when` conditions of the internal and leakage power of the cell are calculated by considering individual signals and the temporal behavior of the cell.

Consider that A and B are pins of an AND cell and their signals are as follows:



Now consider that the activity and probability for the A and B pins are as follows:

Pin	Activity	Probability
A	1	0.5
B	1	0.5

The following table shows the activity and probability of the AND condition when the calculations are based on the activity and probability of the A and B pins:

AND Condition	Activity	Probability
A&B	1	0.25

Now, if you apply the `set_monitor_cell` constraint on these pins, the activity and probability of the AND condition will be as follows:

AND Condition	Activity	Probability
A&B	0	0

The activity and probability values in the above table are more accurate as compared to the values based on the activity and probability of the A and B

pins.

NOTE: *The run time and memory consumption increases if the `set_monitor_cell` constraint is applied on all the cells in a design.*

NOTE: *This constraint is applied by default for all memories and hard macros. Set the value of the `pe_enable_monitor_for_mem` parameter to 0 to disable automatic power estimation by monitors on memories and hard macros.*

NOTE: *If you are using the `set_monitor_cell` constraint along with the `pe_num_clock_cycles_avg_power` parameter in the `PECHECK34` rule, the value of the `pe_num_clock_cycles_avg_power` parameter should be reasonably high. It is recommended that you set the value of the parameter to a value higher than 50.*

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `set_monitor_cell` constraint is as follows:

```
set_monitor_cell
  -name <cell-name>
```

Arguments

-name <cell-name>

Name of the library cell for which power accuracy calculation is to be used. You can also use wildcard characters while specifying cell names.

Examples

Consider the following example:

```
set_monitor_cell -name "RSHL*"
```

The above example implies that all the cell names that match with the wildcard expansion will be evaluated in the high accuracy mode.

Rules

The `set_monitor_cell` constraint is used by the following rules:

**SpyGlass Power Estimation and SpyGlass
Power Reduction Solutions**

 PEPWR02 PECHECK34

set_pin

Purpose

The `set_pin` constraint is used to specify black box pins that should be assumed to be set/reset pins.

Then, the `Async_07` rule of the SpyGlass DFT solution treats the pin specified with the `set_pin` constraints as if it is the set/reset pin on a flip-flop. The source of such a pin must be a test mode controlled to the inactive state.

Product

SpyGlass DFT solution

Syntax

The syntax of the `set_pin` constraint is as follows:

```
set_pin
  -name <du-name>.<port-name>
  [ -value <value> ]
  [ -synchronous ]
  [ -asynchronous ]
```

Arguments

<du-name>

The name of the design unit (black box) for which you are specifying the set pin.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are processed.

You can specify a single design unit name or a space-separated list of design unit names.

<port-name>

Name of the asynchronous set port on the design unit (black box).

You can specify only a single port name.

-value <value>

The active value (0 or 1) for this asynchronous set port *<port-name>*.

-synchronous

Implies that the set pins are synchronous.

-asynchronous

Implies that the set pins are asynchronous.

Rules

The `set_pin` constraint is used by the following rules:

SpyGlass DFT Solution	
Async_07	Async_07Lssd

set_power_scaling

Purpose

The `set_power_scaling` constraint is used to scale the power estimation values of the design.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax of the `set_power_scaling` constraint is as follows:

```
set_power_scaling
  -type <design-element-type>
  [ -leakage <leakage-power-factor> ]
  [ -internal <internal-power-factor> ]
  [ -switching <switching-power-factor> ]
```

Arguments

-type <design-element-type>

The type of design element for which you are specifying the power scaling factor.

The possible values are as follows:

Type of Design Element	Value Specified in <design-element-type>
Black box	blackbox
Clock	clock
Combinational element	combinational
IO pad	iopad
Memory	memory
Sequential element	sequential
Megacell	megacell

NOTE: *The values specified in the -type argument of the `set_power_scaling` constraint are case-insensitive.*

-leakage <leakage-power-factor>

Power scaling factor for leakage power of the design element (in Watts).

Default value is 1.0, which specifies that no scaling is required.

-internal <internal-power-factor>

Power scaling factor for internal power of the design element (in Watts).

Default value is 1.0, which specifies that no scaling is required.

-switching <switching-power-factor>

Power scaling factor for switching power of the design element (in Watts).

Default value is 1.0, which specifies that no scaling is required.

NOTE: *You must specify at least one of the -leakage, -internal, or -switching arguments of the set_power_scaling constraint.*

Examples

Example 1

Consider the following example:

```
set_power_scaling -type combinational -leakage 0.95
```

The above example specifies that the leakage power of the combinational elements in the design should be scaled by 0.95 (95%).

Example 2

Consider the following SGDC file snippet:

```
set_power_scaling -type combinational
    -leakage 0.5 -internal 1.5 -switching 2.0
set_power_scaling -type sequential
    -leakage 0.5 -internal 1.5 -switching 2.0
set_power_scaling -type iopad
    -leakage 0.5 -internal 1.5 -switching 2.0
set_power_scaling -type memory
    -leakage 0.5 -internal 1.5 -switching 2.0
set_power_scaling -type blackbox
    -leakage 0.5 -internal 1.5 -switching 2.0
set_power_scaling -type clock
    -leakage 0.5 -internal 1.5 -switching 2.0
set_power_scaling -type megacell
```

```
-leakage 0.5 -internal 1.5 -switching 2.0
```

In this case, the pe_summary report will contain the following section for the power scaling values:

```
## Power scaling factors used to scale the power consumed
## by each logical component of the design. This information
## is specified using 'set_power_scaling' constraint
##
## Power Scaling Factor:
## -----
##
##           Leakage      Internal      Switching
## Combinational Power : 0.50        1.50        2.00
## Sequential Power    : 0.50        1.50        2.00
## Black Box Power     : 0.50        1.50        2.00
## Memory Power        : 0.50        1.50        2.00
## IO PAD Power        : 0.50        1.50        2.00
## Clock Power         : 0.50        1.50        2.00
## Mega Cell Power     : 0.50        1.50        2.00
```

Power Summary:

```
-----
           Leakage      Internal      Switching      Total
Total Power      : 67.8nW      55.1uW      35.6uW      90.8uW
Combinational Power : 19.8nW      3.25uW      6.82uW      10.1uW
Sequential Power   : 46.6nW      46.9uW      6.13uW      53.1uW
Black Box Power    : 0W          0W          0W          0W
Memory Power       : 0W          0W          0W          0W
IO PAD Power       : 0W          0W          0W          0W
Clock Power        : 1.40nW      4.94uW      22.7uW      27.6uW
Mega Cell Power    : 2.26pW      0W          0W          2.26pW
```

The above section of the pe_summary report has the following tables:

- **Power Scaling Factor:** Shows the power scaling factors as comments
- **Power Summary:** Shows the scaled power values

For details on this report, refer to *pe_summary Report* in the *SpyGlass Power Estimation and SpyGlass Power Reduction Rules Reference Guide*.

Rules

The `set_power_scaling` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR20	PEPWR21	PEPWR22	PEPWR23
PEPWR24	PEPWR25	PEPWR28	SGDC_power_est 62

set_supply_node

Purpose

The `set_supply_node` constraint is used to specify PG pin of a library cell that needs to be considered as a valid driver during the execution of the *UPF_lowpower09* rule.

Product

SpyGlass Power Verify solution

Syntax

The syntax of the `set_supply_node` constraint is as follows:

```
set_supply_node -cell <cell-name> -pin <pin-name>
```

Arguments

-cell <cell-name>

(Mandatory) Specify the name of the cell to be considered as supply.

-pin <pin-name>

(Mandatory) Specify the PG pin of the cell to be considered as a supply port.

Examples

Consider the following example:

```
set_supply_node -cell C28SOI_BBGEN -pin vddcore
```

In the above example, the pin `vddcore` of cell `C28SOI_BBGEN` is being considered as a valid driver.

Rules

The `set_supply_node` constraint is used by the following rules:

SpyGlass Power Verify Solution

UPF_lowpower09

sg_clock_group

Purpose

The `sg_clock_group` constraint defines asynchronous relationship between clocks.

NOTE: *This constraint works only if the `sta_based_clock_relationship` parameter is set to either `only_scg` or `scg_functional`. For details on this parameter, refer to the *SpyGlass CDC Rules Reference Guide*.*

Syntax

The syntax of the `sg_clock_group` constraint is as follows:

```
current_design <du-name>
  sg_clock_group
    -group1 <clk-tag-list1>
    -group2 <clk-tag-list2>
```

Arguments

The `sg_clock_group` constraint has the following arguments:

-group1 <clk-tag-list1>

The clock tag list for which asynchronous relationship is needed.

-group2 <clk-tag-list2>

The clock tag list for which asynchronous relationship is needed.

Example

Consider the following example:

```
clock -name top.clk1 -period 10 -tag T1
clock -name top.clk2 -period 12 -tag T2
sg_clock_group -group1 { T1 } -group2 { T2 }
```

In the above example, the clk1 and clk2 clocks are made asynchronous to each other.

Rules

The sg_clock_group constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_sync_group rules	Ac_glitch03	Clock_sync05	Clock_sync06

sgdc

Purpose

The sgdc constraint is used to specify a block-level SGDC file to be imported or specify blocks for which block-level SGDC file is to be generated.

For SpyGlass CDC Solution

Syntax

The syntax of the `sgdc` constraint is as follows:

Usage 1

```
sgdc -import <block-name> <SGDC-file>  
      -instance_list <instance-list>  
      -bbox_instance_list <instance-list>
```

Usage 2

```
sgdc -export <blocks>
```

Arguments

The `sgdc` constraint has the following arguments:

-import <block-name>

Specifies the block name.

<SGDC-file>

Specifies an SGDC file that needs to be imported for the specified block, <block-name>.

It is recommended that you specify an absolute path of the file.

-instance_list <instance-list>

Specifies the list of instances (hierarchical name) that should be bound to the abstract model specified in the abstracted SGDC file (<SGDC-file>).

-bbox_instance_list <instance-list>

Specifies the list of instances (hierarchical name) that should be treated as black boxes.

-export <blocks>

Specifies a space-separated list of blocks for which a block-level SGDC file

should be generated.

Rules

The `sgdc` constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_blksgdc01	SGDC_clock_validation01	SGDC_clock_validation02	SGDC_clock_domain_validation01
SGDC_clock_domain_validation02	SGDC_set_case_analysis_validation01	SGDC_set_case_analysis_validation02	SGDC_reset_validation01
SGDC_reset_validation02	SGDC_reset_validation03	SGDC_reset_validation04	SGDC_virtualclock_validation01
SGDC_input_validation01	SGDC_input_validation02	SGDC_num_flops_validation01	SGDC_num_flops_validation02
SGDC_output_validation01	SGDC_output_validation02	SGDC_abstract_port_validation01	SGDC_abstract_port_validation02
SGDC_abstract_port_validation03	SGDC_abstract_port_validation04	SGDC_qualifier_validation01	SGDC_qualifier_validation02
SGDC_cdc_false_path_validation01	SGDC__validation01	SGDC__validation02	

Examples

Example 1

Consider the following `sgdc` constraint specification:

```
sgdc -import B1 "/u/user1/B1.sgdc"
```

The above specification implies that the block-level SGDC file, `/u/user1/B1.sgdc`, is to be imported for the B1 block.

Suppose the B1 block has three instances: `i1`, `i2`, and `i3`. In this scenario, consider that you want to:

- Bind `i1` with an abstract model.
- Treat `i2` as a black box.

- Use the complete synthesized view for i3.

To accomplish the above requirements, specify the following command:

```
sgdc -import B1 "/u/user1/B1.sgcd" -instance_list i1  
-bbox_instance_list i2
```

Example 2

Consider the following sgdc constraint specification:

```
sgdc -export B1 B2 B3
```

The above specification implies that a block-level SGDC file would be generated for the B1, B2, and B3 blocks.

For All Products

Purpose

The sgdc constraint is used in hierarchical methodology flow in which an abstracted view of an IP is used during SoC-level verification. For details, refer to the *Using the Hierarchical Methodology* chapter of the *Atrenta Console User Guide*.

Use this constraint to specify an IP and the path where the abstracted view of that IP is saved.

Syntax

The syntax of the sgdc constraint is as follows:

```
sgdc -import <IP-name> <abstracted-view-path>  
-instance_list <instance-list>  
-bbox_instance_list <instance-list>
```

Arguments

The sgdc constraint has the following arguments:

-import <IP-name>

Specifies the IP name.

<abstracted-view-path>

Specifies the path at which the abstracted view of the specified IP is saved.

-instance_list <instance-list>

Specifies the list of instances that should be bound to the abstract model specified in the abstracted SGDC file (<SGDC-file>).

-bbox_instance_list <instance-list>

Specifies the list of instances that should be treated as black boxes.

shadow_ratio

Purpose

The `shadow_ratio` constraint is used to specify the fraction of the total logic that is allowed to be shadowed by memories.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `shadowratio`.

Product

SpyGlass DFT solution

Syntax

The syntax of the `shadow_ratio` constraint is as follows:

```
shadow_ratio -value <value>
```

Arguments

The `shadow_ratio` constraint has the following arguments:

-value <value>

The shadow ratio as a decimal value less than 1 such as 0.95.

Rules

The `shadow_ratio` constraint is used by the following rule:

SpyGlass DFT Solution

RAM_01

show_power_calc_details

Purpose

The `show_power_calc_details` constraint is used to specify instances for which power calculation debug data is required. Wildcard and list is supported for this constraint.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

```
show_power_calc_details
    -instname <leaf-level-instances>
```

Arguments

instname <leaf-level-instances>

The instance names for which you want to generate power calculation debug data.

To specify a range of instances, use wildcards.

Examples

Example 1: Specifying Multiple Instances

The following constraint specification defines two instances:

```
show_power_calc_details -instname top.A.i1 top.A.i2
```

Example 2: Specifying Multiple Instances by Using Wildcards.

The following constraint specification:

```
show_power_calc_details -instname top.A.*
```

Here, debug data is generated for all the leaf-level instances under the `top.A` hierarchy.

signal_in_domain

Purpose

By default, the schemes Synchronized Enable Synchronization Scheme, Recirculation MUX Synchronization Scheme, and MUX-Select Sync (Without Recirculation) Synchronization Scheme, fail if a black box is hit while traversing back from a flip-flop enable pin or a MUX select pin, respectively.

The `signal_in_domain` constraint is used in case a black box instance where more than one clock is reaching is encountered while marking clock domains. By default, all the input/output pins of such black boxes are considered in the domains of all the clocks reaching to the black box as it is not possible to ascertain the exact clock domain of black box input/output pins.

NOTE: Use this constraint if you want to associate or specify a clock on an input or output port, whereas, use the [assume_path](#) constraint if you want SpyGlass rules to consider combinational path from an input to output port of a black box.

Product

SpyGlass CDC solution

Syntax

Use the `signal_in_domain` constraint to specify that the black box input/output pins are in the same clock domain as the specified black box clock input pin or to specify that the black box output pin is synchronized with respect to the specified black box clock input pin:

```
current_design <du-name>
  signal_in_domain
    -name <bb-name>
    -domain <pin-name>
    -signal <sig-list>
```

-synchronized

Arguments

<du-name>

The module name (for Verilog designs) or the design unit name in *<entity-name>.<arch-name>* format (for VHDL designs).

-name <bb-name>

The name of the black box module. You can use wildcard characters while specifying the black box module name.

-domain <pin-name>

The name of the black box clock input or output pin.
You can also specify virtual clocks to this argument.

-signal <sig-list>

A list of black box input/output pin names. You can use wildcard characters while specifying pin names.

NOTE: *You must specify only relative path of black box input/output pins. This argument does not accept a complete hierarchical path.*

-synchronized

The `-synchronized` argument indicates that black box output pins in the signal list *<sig-list>* are synchronized in the domain of a clock specified through the *<pin-name>* argument. This information can be useful where the specified black box output is used as a synchronized enable controlling a data crossing. In such a case, the data crossing is considered synchronized.

NOTE: *It is recommended to use [abstract_port](#) to specify the clocks and related information for black box output ports.*

Examples

Consider the following example:

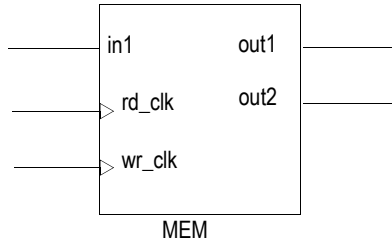


FIGURE 50. Signal_in_domain Constraint Example

The output pins out1 and out2 are in rd_clk and wr_clk domains, respectively. The input pin in1 is in the wr_clk domain. This is specified using the signal_in_domain constraint as follows:

```
signal_in_domain -name MEM -domain rd_clk -signal out1
signal_in_domain -name MEM -domain wr_clk -signal out2
signal_in_domain -name MEM -domain wr_clk -signal in1
```

In the example given above, if the output pin out2 is considered to be synchronized in domain wr_clk, this is specified using the signal_in_domain constraint as follows:

```
signal_in_domain -name MEM -domain wr_clk -signal out2 -
synchronized
```

NOTE: Multiple signal_in_domain should not be assigned on the same output pin of a black box.

NOTE: The assume_path and signal_in_domain constraints should not propagate different domains on the same output pin of a black box.

Rules

The signal_in_domain constraint is used by the following rules:

SpyGlass CDC Solution			
Ac_unsync01	Ac_unsync02	Clock_sync08	Clock_sync03a
Clock_sync03b	Ac_handshake01	Clock_sync08a	Clock_sync09
Ac_handshake02	Ac_cdc01a	Ac_cdc01b	Ac_cdc01c

Ac_cdc08	Propagate_clocks	Ac_conv01	Ac_conv02
Ac_conv03	Ac_sync02	Ac_sync01	

signal_type

The `signal_type` constraint specifies if a signal is a *Control Signal* or *Data Signal*.

For information on using this constraint, refer to its *Syntax* and *Arguments*.

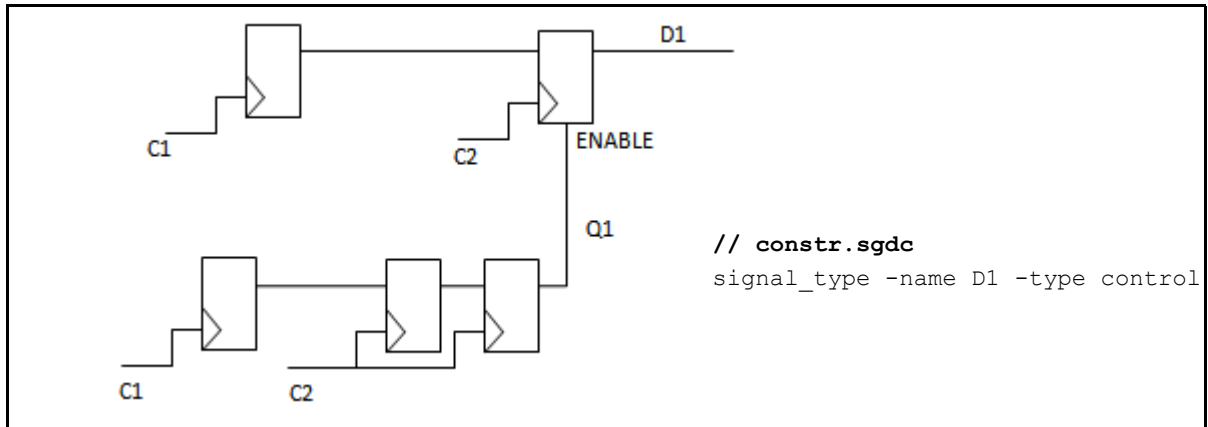
Control Signal

A control signal is a signal that is:

- The output of a clock-domain crossing, the source of a clock-domain crossing, or an input port acting as the source for a clock-domain crossing.
- Considered as synchronized by any of the following synchronization schemes (also known as control synchronization schemes):
 - ☐ Conventional multi-flop synchronization scheme
 - ☐ Synchronizing cell synchronization scheme
 - ☐ Qualifier synchronization scheme by using the *qualifier-crossing* constraint
 - ☐ Recirculation MUX synchronization scheme

This means that even if a control signal has a valid qualifier present in its input cone but it is not synchronized by any of the control synchronization schemes, it will be reported as unsynchronized by *Ac_unsync01/*
Ac_unsync02 rules.

For example, consider the scenario shown in the following figure:

**FIGURE 51.**

In the above figure, although the Q1 qualifier exists on the enable pin, the control signal D1 is not synchronized by any of the control synchronization schemes. Therefore, SpyGlass reports the D1 signal as unsynchronized.

The *Ac_glitch03* Rule Behavior for Control Signals

The *Ac_glitch03* rule performs glitch checks on the unsynchronized crossings having a virtual domain on the output port when both the following conditions hold true:

- The output port is specified as `-type control` in the `signal_type` constraint.
- The `glitch_on_vck_port` value is specified to the `cdc_reduce_pessimism` parameter.

For details on the *Ac_glitch03* rule behavior based on the above conditions, refer to the documentation of the *Ac_glitch03* rule.

Considering the Output of Convergence as a Valid Qualifier

When a qualifier merges with a source, the output of convergence is not considered as a valid qualifier to qualify other sources if the `allowed_merged_qualifier` parameter is set to `no`. However, if the source is a control signal, the convergence output may still act as a qualifier.

By default, the `allowed_merged_qualifier` parameter is set to `yes`, and such convergence outputs are considered as valid qualifiers.

Data Signal

A data signal is a signal that is:

- The output of a clock-domain crossing, the source of a clock-domain crossing, or an input port acting as the source for a clock-domain crossing.
- Considered as synchronized by a data synchronization scheme, which is a scheme other than the following control synchronization schemes:
 - ☐ Conventional multi-flop synchronization scheme
 - ☐ Synchronizing cell synchronization scheme
 - ☐ Qualifier synchronization scheme by using the *qualifier*-crossing constraint

This means that even if a data signal is synchronized by a multi-flop or a synchronizer cell (specified by the *sync_cell* constraint), it will be reported as unsynchronized by *Ac_unsync01/Ac_unsync02* rules.

For example, consider the scenario shown in the following figure:

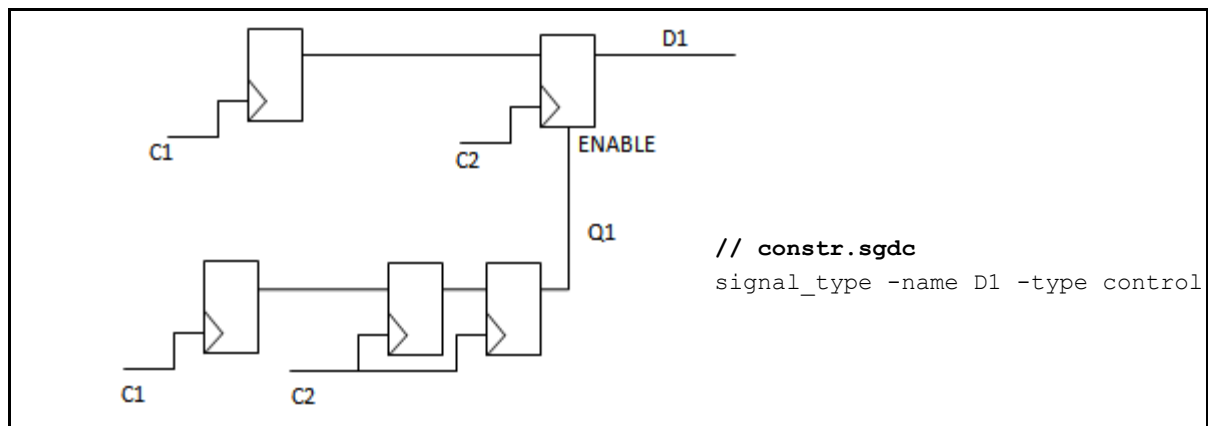


FIGURE 52.

In the above figure, the data signal D1 has a multi-flop structure in its

path, but it is not synchronized by any of the data synchronization schemes. Therefore, SpyGlass reports the D1 signal as unsynchronized.

Product

SpyGlass CDC solution

Syntax

The syntax of the `signal_type` constraint is as follows:

```
signal_type
  -name <signal-name>
  -type <control | data>
```

Arguments

-name <signal-name>

Name of a signal that can be any of the following:

- Output of a clock-domain crossing
- Source of a clock-domain crossing
- Input port acting as the source for a clock-domain crossing

You can use wildcard characters while specifying a signal name.

-type <control | data>

Signal type as data or control.

Rules

The `signal_type` constraint is used by the following rules:

SpyGlass CDC Product			
Ac_sync01	Ac_sync02	Ac_unsync01	Ac_unsync02
Ac_glitch03			

simulation_data

The `simulation_data` command is used to specify the initial state sequence for the design.

You can specify the initial state sequence in a Tcl file or have the product read it from a VCD file.

NOTE: *The `simulation_data` command is not required for combinational analysis.*

Product

SpyGlass TXV, SpyGlass CDC, and SpyGlass Auto Verify

Syntax

The syntax of the `simulation_data` constraint is as follow:

```
current_design <du-name>
  simulation_data
    -type <tcl | vcd>
    -file <file-name>
    [ -mode <mode-name> ]
    [ -time <value> ]
    [ -scopename <block-name> ]
    [ -modulename <module-name> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the initial state sequence.

-type <tcl | vcd>

The `-type` argument specifies the input file type as `tcl` for Tcl file or `vcd` for VCD file.

-file <file-name>

The `-file` argument specifies the Tcl file or the VCD file.

-mode <mode-name>

The `-mode` argument is optional and is used to specify the applicable mode (one of the mode values specified with the `-mode` argument of the `sdc_data` constraint.) If you do not specify the `-mode` argument, the SpyGlass assumes that the specified initial state sequence is applicable for all modes.

-time <value>

The `-time` argument is optional and is used to read the initial state by using a timestamp.

-scopename <block-name>

The `-scopename` argument is used if the given VCD file is for a full chip but you want to perform verification for a sub-block module. The hierarchical path of the block module's instantiation (in VCD) must be specified to the `simulation_data` constraint.

For example, consider a VCD file with a top module `top` and a sub module `block2` that lies within another block `block1`. Therefore, while performing the verification of `block2`, the following must be defined to initialize the `block2` module from the VCD file generated for the full chip design:

```
current_design block2
simulation_data -type vcd -scopename
"top.block1_inst.block2_inst" -file chip.vcd
```

-modulename <module-name>

The `-modulename` argument is used to specify the hierarchical name of the module in the design corresponding to the module instance in the VCD file specified in the `-scopename` option.

For example, if `top.block1_inst.block2_inst` in the VCD hierarchy corresponds to `block1.block2` in the design, specify the `simulation_data` constraint as:

```
current_design block2
simulation_data
```

```
-type vcd
-scopeName "top.block1_inst.block2_inst"
-moduleName "block1.block2" -file chip.vcd
```

NOTE: For sequential analysis, it is recommended that you either provide sufficient reset and set_case_analysis constraints so that the SpyGlass TXV solution automatically reads the initial state sequence or specify the simulation_data constraint. If the number of sequential elements initialized is high, the quality of the result produced by the SpyGlass TXV solution is good.

Examples

Consider the example below:

```
simulation_data -type vcd -moduleName A -scopeName B -file
design.vcd
```

In the above example, A is the name of the top module and B is the sub block module name. Here, the top module name A will replace the sub block module name B and initial state sequence is read for the sub block module.

An example of the Tcl File is as follows:

```
set sig1 0xffffffff
incr a
for {set i 1} {$i<=4} {incr i} {
    force xyz [incr a 4]
    simulate 2 -clk clk1
}
force sig2 [incr a]
simulate 2
```

Rules

The initState constraint is used by the following rules:

SpyGlass TXV Solution

All rules

SpyGlass CDC Solution

Ac_initstate01

SpyGlass Auto Verify Solution

Av_initstate01

special_cell

Purpose

The `special_cell` constraint is used to specify special cells as checked by the LPSVM37 and LPPLIB13 rules.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `specialcell`.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `special_cell` constraint is as follows:

```
current_design <du-name>
  special_cell
    -name <cell-name-list>
    [ -regions <inst-name-list> ]
    [ -supplies <supply-name-list> ]
```

Arguments

<du-name>

Name of the design unit under which you are specifying the special cells.

-name <cell-name-list>

Space-separated list of special cell names. You can use wildcard characters while specifying cells using the `-name` argument.

-regions <inst-name-list>

Space-separated hierarchical instance name list (name of any instance or the top-level design unit) indicating the regions to be checked as used by the LPSVM37 rule of the *SpyGlass Power Verify* solution.

-supplies <supply-name-list>

Space-separated supply name list as used by the LPPLIB13 rule.

Rules

The `special_cell` constraint is used by the following rules:

SpyGlass Power Verify Solution	
LPSVM37	LPPLIB13

special_module

Purpose

The `special_module` command is used to define property and constraint modules. All OVL modules are supposed to be specified as special modules. If not, these modules are treated as regular design modules.

NOTE: Prior to *SpyGlass 4.3.0* release, the name of this constraint was `smodule`.

Product

SpyGlass Auto Verify solution

Examples

The following command in a SpyGlass Design Constraints file will treat all modules whose name starts with "assert_" as assertion or constraint modules.

```
special_module -type ASSERTION -name "assert_*" -regex
```

In the presence of OVL modules, this constraint is a requirement to run

SpyGlass Auto Verify solution (otherwise the OVL checks are not performed). It is included as a constraint in the goal for SpyGlass Auto Verify solution. Therefore, you need not specify this, unless an OVL assertion is written which does not start with the keyword "assert". For example, if you define new OVL assertions, all starting with the prefix "user1...", the following command should be given in the SpyGlass Design Constraints file:

```
special_module -type ASSERTION -name "user1_*" -regexp
```

Rules

The `special_module` constraint is used by the following rule:

SpyGlass Auto Verify Solution
Av_ovl01

spef_data

Purpose

The `spef_data` constraint specifies the SPEF (Standard Parasitic Extraction Format) file.

For post-route ("signoff") power estimation, accuracy improves if the actual routed parasitics are used to get the net capacitances instead of wire-load models. The actual routed parasitics are stored in the SPEF file, which is an accepted industry standard for storing extracted parasitics.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `spef_data` constraint is as follows:

```
current_design <du-name>
    spef_data -file <spef-file-name>
    [ -modname <du-name> ]
```

```
[ -instname <inst-name> ]  
[ -spef_topname <top-name> ]
```

NOTE: *If the capacitance of a net is specified using both `spef` file and `set_load` command, the `set_load` is given preference.*

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

-file <spef-file-name>

Name of the SPEF file.

NOTE: *You can also specify compressed SPEF file that has been generated by using the `gzip` utility.*

-modname <du-name>

(Optional) Name of the design unit (specified as module name or the entity name) for which SPEF file is specified.

-instname <inst-name>

(Optional) Name of the instance for which SPEF file is specified.

In case neither `modname` nor `instname` is given, the SPEF file will be applied for the design unit specified using `DESIGN` in the SPEF file.

-spef_topname <top-name>

(Optional) Path of the hierarchy in the SPEF file corresponding to the design.

This argument is useful if you want to use SPEF information for a specific instantiated module from a top-level SPEF file. In this case, the `spef_topname` argument is used to specify the instantiated name of the module in the SPEF file.

Examples

Example 1 - Specifying modname and instname

An example of `spef_data` constraint is given below:

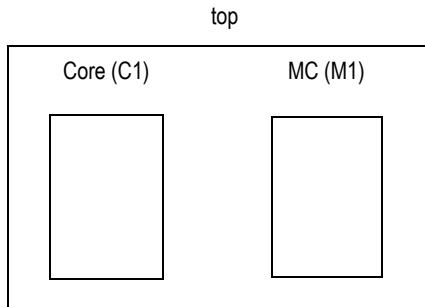


FIGURE 53. `spef_data` Constraint Example

Consider a top design module `top` consisting of two blocks - Core (instance `C1`) and MC (instance `M1`). There are three SPEF files - `top.SPEF` (for `top`), `Core.SPEF` (for instance `top.C1`), and `MC.SPEF` (for instance `top.M1`).

The SPEF files can be specified as follows:

```

current_design top
    spef_data -file top.SPEF -modname top
    spef_data -file Core.SPEF -instname top.C1
    spef_data -file MC.SPEF -instname top.M1
  
```

NOTE: While calculating the capacitance value for any net from the spef file, the coupling capacitance value can be included or excluded using `pe_include_coupling_capacitance` switch.

Example 2 - Specifying spef_topname

Suppose a design has an instance `U0` of module `MC_RF`. This module is inside the TOP module. For the design, the associated SPEF file is `TOP.SPEF`.

From the `TOP.SPEF` file, you can use the SPEF data only for the instance `U0`

by specifying the following SGDC constraints:

```
current_design MC_RF
spef_data -file TOP.SPEF -spef_topname U0
```

NOTE: Hierarchical format is supported for specifying *spef_topname* argument. The hierarchy separator used in the argument must match the separator defined in the SPEF file.

Rules

The `spef_data` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions			
PEPWR01	PEPWR02	PEPWR03	PEPWR05
PEPWR13	PEPWR14	poweraudit	

supply

Product

SpyGlass Power Verify solution, SpyGlass Power Estimation and SpyGlass Power Reduction solutions

For SpyGlass Power Verify solution

Purpose

The `supply` constraint is used to specify the supply and ground port names for all the LPPLIB rules.

Syntax

The syntax to specify the `supply` constraint is as follows:

```
current_design <du-name>
  supply
    -name <name>
```

```

-value <fvalue>
[ -net <net-name> ]
[ -on <sig-name-list> ]
[ -on_2 <sig-name-list> ]
[ -parent <supply-name> ]
[ -alwayson 0 | 1 ]
[ -onval 0 | 1 ]
[ -onval_2 0 | 1 ]
[ -isosig <sig-name-list> ]

```

Arguments

<du-name>

Name of the design unit under which you are specifying the power and ground ports

-name <name>

Simple name of the power/ground port (post-synthesis pin)

-value <fvalue>

Voltage value at the specified port.

-net <net-name>

(Optional) Name of the supply nets that are inside a block. For example, the name `ua.VDDX` will be used to refer to supply net inside the block `top.ua`.

-on <sig-name>

(Optional) Space-separated list of `-on` signals. For each power switch instance, the enable is connected to the `-on` signal. If an expression involving several terms is given, an error will be reported. Also, there is an interaction between the `-on` switch and the `-alwayson` switch. If the `-on` argument is specified, `-alwayson` should not be specified as '1'. If `-alwayson` is 1 and `-on` is also specified, an error will be reported.

-on_2 <sig2-name>

(Optional) Space-separated list of `-on_2` signals. In case of dual enable power switch, for each power switch instance, the second enable is connected to the `-on_2` signal.

-parent <supply-name>

(Optional) Name of the parent supply net. The input and output supply net for power switches have a parent-child relationship. That is, the supply net outside the domain is the parent, and the supply net inside the domain is the child. To derive the parent-child relationship the `-parent` switch is specified. When the option is specified, for any supply X that has power switches, the parent supply should be connected to the input power pin of the power switch, and the supply X should be connected to the output power pin of the power switch.

-always on <0 | 1>

(Optional) The type of supply. Can be set to 0 (switched off) or 1 (always on).

If `-always on` (and also `-on` and `-parent` arguments) is not specified with the `supply` constraint, supply is considered an always-on supply (`-always on` is set to 1). If `-on` and `-parent` arguments are specified, supply is considered a switched off supply (`-always on` is set to 0).

-onval <0 | 1>

(Optional) Space-separated list of values for the `-on` signals. Can be set to 0 (active low) or 1 (active high).

-onval_2 <0 | 1>

(Optional) Space-separated list of values for the `-on_2` signals. Can be set to 0 (active low) or 1 (active high).

-isosig <sig-name-list>

(Optional) Name of the isolation signal(s) for the supply being defined as a space-separated list.

Rules

The supply constraint is used by the following rules:

SpyGlass Power Verify Solution

All LPPLIB rules

For the SpyGlass Power Estimation and SpyGlass Power Reduction Solutions

Purpose

The supply constraint is used to specify the supply rails information.

NOTE: *Use the supply constraint for multiple voltage domain designs only. For single voltage domain designs, the supply rails information is inferred from the associated technology library.*

Syntax

The syntax to specify the supply constraint is as follows:

```
current_design <du-name>
  supply
    -name <supply-name>
    -value <fvalue>
    [ -on <on-expr> ]
    [ -isnetdriver ]
```

Arguments

<du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

-name <supply-name>

Name of the supply rail.

-value <fvalue>

The voltage value for the supply rail.

Use the `-on` argument to specify the condition for supply rails that may be off under certain conditions. `<on-expr>` is the signal-based Boolean expression that indicates the supply rail ON condition.

-isnetdriver

The `-isnetdriver` argument specifies whether the supply rail is a net driver. You can use this argument to specify the net drivers.

Examples

The following value indicates that supply rail S1 has a voltage value of 1.2:

```
supply -name S1 -value 1.2
```

The following example specifies a supply rail named A1 that has a supply voltage of 1.2 and is ON only when both `in1` and `in2` signals are high:

```
supply -name A1 -value 1.2 -on top.in1 & top.in2
```

The following example specifies a supply rail named G123 that has a supply voltage of 1.2 and is a net driver:

```
supply -name G123 -value 1.2 -isnetdriver
```

Rules

The `supply` constraint is used by the following rules:

SpyGlass Power Estimation and SpyGlass Power Reduction Solutions		
PEPWR01	PEPWR02	poweraudit

switchoff_wrapper_instance

Purpose

The `switchoff_wrapper_instance` constraint is used to specify the

instance hierarchy.

Product

SpyGlass Power Verify solution

Syntax

The syntax to specify the `switchoff_wrapper_instance` constraint is as follows:

```
current_design <du-name>
    switchoff_wrapper_instance
        -name <hier-list>
```

Arguments

<du-name>

Name of the design unit under which you are specifying the instance hierarchies.

-name <hier-list>

Space-separated list of hierarchical references.

Examples

The following example specifies `top.SOW1` and `top.SOW2` as the instance hierarchies:

```
switchoff_wrapper_instance -name top.SOW1 top.SOW2
```

Rules

The `switchoff_wrapper_instance` constraint is used by the following rule:

SpyGlass Power Verify Solution
LPSVM55

sync_cell

Purpose

The `sync_cell` constraint is used to specify synchronizer cells that should be considered valid for control crossings that contain specified frequencies, source/destination clocks, or domains.

NOTE: *If you use the `synchronize_cells` or `synchronize_data_cells` parameter along with this constraint, the constraint is used for the specified clocks and frequencies and the parameter will be applicable for the rest of the crossings.*

NOTE: *If the `synchronize_cells` parameter is defined and the `sync_cell` constraint is specified with the `-from_clk/to_clk/from_domain/to_domain` arguments, then the `sync_cell` constraint is honored for the specific clock/domain pair. For other clock/domain pairs, the `synchronize_cells` parameter is honored. However if the `sync_cell -name` constraint is used without any argument, then it is honored by default and the parameter is ignored.*

NOTE: *This constraint is not applicable for data crossings. It is applicable for control crossings.*

Product

SpyGlass CDC solution

Syntax

The syntax to specify the `sync_cell` constraint is as follows:

Usage 1:

```
sync_cell
  -name <synchronizer-modules>
  [ -to_clk <dest-clk>
    [ -from_clk <src-clk> ]
  ]
  [ -user_tag <user-tag-string> ]
  [ -rdc ]
  [ -reset ]
```


Usage 2:

```
sync_cell
  -name <synchronizer-modules>
  [-to_domain <dest-domain>
    [ -from_domain <src-domain> ]
  ]
  [ -user_tag <user-tag-string> ]
  [ -rdc ]
  [ -reset ]
```

Usage 3:

```
sync_cell
  -name <synchronizer-modules>
  [ -to_period <dest-period>
    [-from_period <src-period>]
  ]
  [ -user_tag <user-tag-string> ]
  [ -rdc ]
  [ -reset ]
```

Usage 4:

```
sync_cell
  -name <synchronizer-modules>
  [ -user_tag <user-tag-string> ]
  [ -rdc ]
  [ -reset ]
```

NOTE: Please note the following points:

- You can specify any one of the following combination of arguments:

- ☐ -from_clk and -to_clk
- ☐ -from_domain and -to_domain
- ☐ -from_period and -to_period

You cannot mix the above combinations.

- You must specify the `-to_clk` argument while specifying the `-from_clk` argument.
- You must specify the `-to_domain` argument while specifying the `-from_domain` argument.
- You must specify the `-to_period` argument while specifying the `-from_period` argument.

Arguments

`-name <synchronizer-modules>`

Space-separated list of synchronizer cells/modules. You can use wildcard characters while specifying such modules.

Details of such modules are discussed in the following points:

- Such modules can either be user-defined modules containing at least one sequential element or it can be a sequential library cell.

NOTE: *Such modules cannot be black boxes. For black boxes, use the [qualifier](#) constraint along with the `synchronize_cells` parameter.*

- You can specify multiple synchronizer cells for the same clock, domain, or period, as shown in the following example:

```
sync_cell -name SYNC1 SYNC2 SYNC3 -to_period 10
```

- If clock names are specified in some `sync_cell` constraints and in other constraints, domain or period is specified for those clocks, all the `sync_cell` constraints will be considered.

Consider the following example:

```
clock -name top.clk1 -domain d1 -period 10
clock -name top.clk2 -domain d2 -period 20
sync_cell -name SYNC1 -from_clk top.clk1 -to_clk top.clk2
sync_cell -name SYNC2 -from_domain d1 -to_domain d2
sync_cell -name SYNC3 -to_period 20
```

In this case, all the SYNC1, SYNC2, and SYNC3 synchronizer cells will be considered valid for crossings from `clk1` (`d1`) to `clk2` (`d2`).

- For the same synchronizer cell, if multiple `sync_cell` constraints are specified for different clocks, domains, and clock periods, the synchronizer cell is considered valid for all the specified clocks, domains, and clock periods.

This is explained in the following examples:

```
// SYNC1 cell is valid for crossings from
// CLK1 to CLK2 and crossings with
// destination CLK3

sync_cell -name SYNC1 -from_clk CLK1 -to_clk CLK2
sync_cell -name SYNC1 -to_clk CLK3

// SYNC2 cell is valid for crossings with destination
// clock period 20 and 40

sync_cell -name SYNC2 -to_period 20
sync_cell -name SYNC2 -to_period 40

// SYNC3 cell is valid for crossings from domain
// D1 -> D2 and also for crossings from D3 to D4

sync_cell -name SYNC3 -from_domain D1 -to_domain D2
sync_cell -name SYNC3 -from_domain D3 -to_domain D4
```

-from_clk <src-clk>

Name of source clock.

You can use wildcard characters while specifying a source clock.

-to_clk <dest-clk>

Name of destination clock.

You can use wildcard characters while specifying a destination clock.

-from_domain <src-domain>

Name of source domain.

-to_domain <dest-domain>

Name of destination domain.

-from_period <src-period>

Source clock period value.

-to_period <dest-period>

Destination clock period value.

-user_tag <user-tag-string>

A string that is appended to the control-crossings-related violation messages of rules using this constraint.

Use this string to filter messages as per your requirement. For example, you add a custom filter in a spreadsheet to show or hide messages containing the specified string.

-rdc

Enables the *Ar_resetcross01* apply the `sync_cell` constraints on reset domain crossings.

-reset

Specifies that a clock domain crossing on reset path be considered as synchronized.

Examples

Example 1

Consider the following constraints:

```
sync_cell -name SYNC1 -from_period 50 -to_period 10
sync_cell -name SYNC2 -to_period 20
```

In this case, `SYNC1` is the valid synchronizer module/cell for crossings with destination clock period 10 and source clock period 50. However, for crossings with destination clock period 20, `SYNC2` is the valid synchronizer.

Rules

The `sync_cell` constraint is used by the following rules:

SpyGlass CDC Solution			
Clock_sync08	Clock_sync03a	Clock_sync03b	Ac_handshake01
Ac_handshake02	Ar_resetcross_matrix01	Ar_cross_analysis01	Clock_sync08a
Clock_sync09	Ac_cdc01a	Ac_cdc01b	Ac_cdc01c
Ac_cdc08	Propagate_Clocks	Ac_conv01	Ac_conv02
Ac_conv03	Ac_glitch02	Ac_sync01	Ac_sync02
Ac_unsync01	Ac_unsync02	Ac_glitch03	Ar_resetcross01
Ac_conv04			

sync_reset_style

Purpose

The `sync_reset_style` constraint is used by SpyGlass CDC solution during detection and validation of a synchronous reset in a design.

Product

SpyGlass CDC solution

Syntax

The following is the syntax of the `sync_reset_style` constraint:

```
sync_reset_style
[ -max_load <max-sequential-elements> ]
[ -min_load <min-sequential-elements> ]
[ -combo <yes | no | buf_inv> ]
[ -active <low | high | both> ]
[ -pragma <yes | no | mixed> ]
[ -first_if <yes | no> ]
```

Arguments

-max_load <max-sequential-elements>

Specifies the maximum load allowed on a synchronous reset.

This load is specified in terms of the total number of the following types of terminals driven by a synchronous reset:

sequential element terminals	tristate terminals
mux instance control terminals	black box terminals

The default value of this argument is -1, which means that any load is allowed on a synchronous reset. You can specify the load ranging from 0 to any positive integer value.

-min_load <min-sequential-elements>

Specifies the minimum load allowed on a synchronous reset.

The default value of this argument is 0, which means that no load is allowed on a synchronous reset. You can specify the load ranging from 0 to any positive integer value.

-combo <yes | no | buf_inv>

Specifies if combo logic is allowed between the reset source and data pin of a flip-flop.

You can specify any of the following values for this argument:

Value	Description
yes	(Default) Specifies that all combinational logic is allowed
no	Specifies that no combo logic, except SpyGlass-generated buffer, is allowed in the reset path
buf_inv	Specifies that any number of buffers, inverters, or combinational logic equivalent to buffer

-active <low | high | both>

Specifies the polarity of the usage path of a synchronous reset signal.

SpyGlass Design Constraints

You can specify any of the following values for this argument:

Value	Description
low	Specifies that the reset is active-low
high	Specifies that the reset is active high
both	(Default) Specifies that the reset can be used as active-high or active-low

NOTE: *SpyGlass ignores the `-active` argument when flip-flops are created by using a library cell instance.*

-pragma <yes | no | mixed>

Specifies if the `synopsys_set_reset` pragma should be present in the RTL.

You can specify any of the following values for this argument:

Value	Description
yes	Specifies that pragma should be present in all usage of RTL
no	(Default) Specifies that pragma may or may not be present in the synchronous usage of RTL. SpyGlass CDC solution does not check the presence of pragma in such cases.
mixed	Specifies that pragma should be present in at least one usage of synchronous reset at RTL

NOTE: *Please note the following points:*

-  *VHDL does not have any pragmas, therefore, all flip-flops created using VHDL RTL are considered with presence of pragmas to avoid reporting for pragma mismatch in VHDL code.*
-  *SpyGlass CDC solution this argument in case of netlist design.*

-first_if <yes | no>

Specifies the modeling of synchronous reset usage in the first `if` condition of a synchronous block.

If you set this argument to yes, synchronous reset should be used in the first `if` condition.

By default, this argument is set to no, which means that no rule-checking is done. In this case, synchronous reset may or may not be used in the first `if` block in the RTL design.

Rules

The `sync_reset_style` constraint is used by the following rules:

SpyGlass CDC Solution		
Reset_info01	Ar_syncrstactive01	Ar_syncrstcombo01
Ar_syncrstload01	Ar_syncrstload02	Ar_syncrstpragma01
Ar_syncrstrtl01		

test_mode

Purpose

The `test_mode` constraint specifies the set of conditions, both pins and values, that when simulated, will force the circuit in test mode.

NOTE: Prior to SpyGlass 4.3.0 release, the name of this constraint was `testmode`.

Product

SpyGlass DFT solution, SpyGlass DFT DSM solution, SpyGlass STARC product, SpyGlass STARC02 product, SpyGlass STARC05 product, and SpyGlass STARCad-21 product

For SpyGlass DFT solution and SpyGlass DFT DSM solution

In SpyGlass DFT solution, the `test_mode` constraint when specified forces the circuit into a state for scan shifting or for capture. In mux-scan, for example, during scan shift, all internally generated clocks should be bypassed with a test clock and all flip-flop sets and resets should be held inactive. In capture mode, all system latches should be made transparent.

Generally, there are cases in which the same pin must have different values to differentiate scan shift from capture. To support such cases, the

syntax of the `test_mode` constraint enables you to provide the following types of values:

- Values that are the same in shift and capture (no optional syntax required)
- Values that complement from shift to capture (use the `-invertInCapture` argument)
- Values that are non-X in shift and X in capture (use the `-scanshift` argument)
- Values that are X in shift and non-X in capture (use the `-capture` argument)

When `test_mode`, `force_ta`, or `test_point` constraints are specified on the same node, following is the priority among different constraints:

- `Test_mode`
- User-specified specific *`force_ta` / `test_point`*
- Effect of *`dft_treat_primary_inputs_as_x_source`* and *`dft_treat_primary_outputs_as_unobservable`* parameters

For example, if `test_mode 1` and `test_point control` are applied on the same node then the `test_mode` constraint will be considered.

Also, if the `test_mode`, `force_ta`, or `test_point` constraints are found in the fanout of each other, following is the priority among different constraints:

- The constraint in the fanout gets the priority
- Fanin effect is blocked by the specified / resolved constraint on the node

For example, consider that `test_mode 1` is applied on the input of buffer and `test_point control` is applied on the output of the same buffer. In this case, input will have simulation value 1 and nyn controllability but output will have yyn controllability and no simulation value.

Syntax

The syntax of the `test_mode` constraint is as follows:

```
test_mode
  -name <name>
  -value <value>
```

```
[ -scanshift ]  
[ -capture ]  
[ -functional ]  
[ -captureATspeed ]  
[ -invertInCapture ]  
[ -initialize_for_bist]  
[ -memory_force]
```

NOTE: The `test_mode` constraint supports wildcard characters.

Arguments

-name <name>

Complete hierarchical name of a test mode port/pin.

The pin can be a primary pin as well as an internal pin.

You can specify a single port/pin's full hierarchical name or a space-separated list of full hierarchical port/pin names.

The following example shows that the `test_mode` constraint has been defined on internal net `top.U1.U13.tm1` and `top.U3.tm3`:

```
current_design top  
  test_mode -name top.U1.U13.tm1, top.U3.tm3 ...
```

For primary ports, you can also specify the simple port name as in the following example:

```
current_design top  
  test_mode -name in15 ...
```

-value <value>

Value list for a test mode pin.

The value list is the sequence of one or more of the following values:

- 0
- 1
- X
- Z
- Combination

■ 'p'(010) / 'P'(010)

■ 'n'(101) / 'N'(101)

The 'p'(010) denotes a rising edge or a positive edge. The value 'n'(101) denotes a falling edge or a negative edge. Here a bit corresponds to a pulse. Therefore, if you specify, '1' or '0' or 'X' or 'Z', they will be repeated three times in the command.

See the [Usage of Different Clock Edges](#) section, to understand the usage of different clock edges.

Applying these values to the test mode pin causes the circuit to enter the test mode.

You can specify repeat sequences for the `test_mode` constraint.

For fields that require repeat sequence, you can specify the values as `<I*S>`. Here, S is any string that does not contain the `<`, `>`, and `*` characters. However, S can contain another `<I*S>` expression. I is an integer that is always interpreted as a decimal value. The expression `<I*S>` means that the sequence S will be repeated I number of times.

-scanshift

(Optional) Indicates that the test mode pin is only required during the scan shifting operations. Such a pin may be used for ATPG purposes when not scanning.

NOTE: *You can specify both the -scanshift argument and the -capture argument together.*

NOTE: *Use of the -scanshift argument will generally increase the estimated fault coverage since both 0 and 1 values are allowed on such pins, whereas without this argument only the specified value is allowed.*

-capture

(Optional) Indicates that the test mode pin is only required during the capture operations.

NOTE: *You can specify both the -scanshift argument and the -capture argument together.*

-functional

(Optional) Indicates that the test mode signal is only required during the

functional mode.

NOTE: The `set_case_analysis` constraint is treated as the `test_mode` -functional constraint.

-captureATspeed

NOTE: This option is not applicable for the SpyGlass DFT solution.

(Optional) You should use this option to define test mode signals exclusively for the at-speed rules.

This option helps you to apply the test mode condition exclusively for at-speed capture cycle that can be different from the capture used by stuck-at testing or from scan shifting. Consider the following figure:

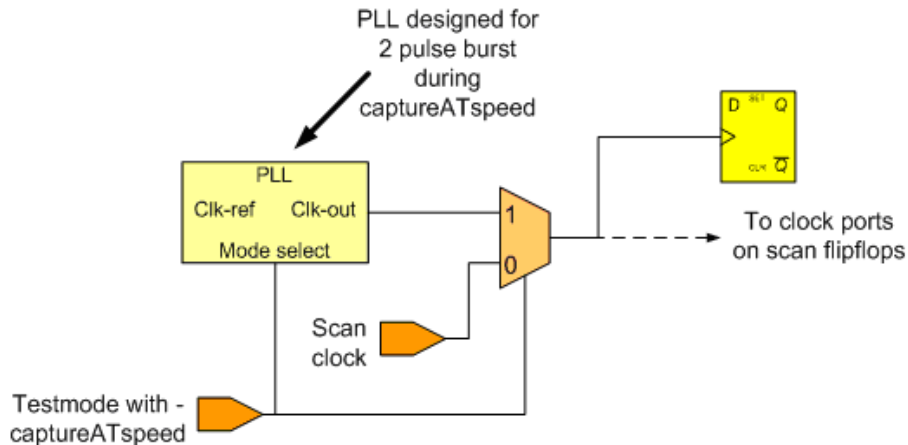


FIGURE 54. PLL burst during captureATspeed

-invertInCapture

(Optional) Indicates that the value specified with the `-value` argument should be simulated in the shift mode as is whereas the inverse of this value (complement of the value for single-bit values or last-bit complement of pattern values) should be simulated in the capture mode.

-initialize_for_bist

Defines a way to drive all registers to a known state.

You can drive registers to a known state asynchronously, sequentially, or in some combination of both.

If there is a global reset signal, an `initialize_for_bist` argument specifies that signal and the value that makes all asynchronous sets or resets ACTIVE.

-memory_force

Lists the primary ports and values that, when simulated, will force all memories used in a circuit to have non-x values on their outputs.

Notes

1. More than one `test_mode` constraint may be necessary for your design. If more than one `test_mode` constraint is used, they will all be simulated in parallel for determining scannability but only the `test_mode` constraints without the `-scanshift` argument will have fixed values when determining test coverage.
2. The values supplied through the `test_mode` constraint are 0, 1, Z, and X (note case-sensitivity for Z and X). Since the simulator built into SpyGlass DFT solution is designed for sequential circuits, the value lists may contain sequences of values. For example, the following constraint directive will cause three rising clock edges on a signal `tclk` and involve six simulation cycles:

```
test_mode -name tclk -value 010101
```

3. Special attention is necessary for a signal that is used both as a test mode signal (that is, used in a `test_mode` constraint) and as a test clock signal (used in a `clock` constraint with `-testclock` argument). In such a case, the last value listed in the `test_mode` constraint must be an X so that the signal is free to be pulsed as a test clock.
 4. The last value in all `test_mode` value lists is used by controllability analysis and is preserved for all rules that depend on the `test_mode` constraint. For example, the following constraint will cause the signal `tclk` to be uncontrollable to a logic zero since the last value is 1:
- ```
test_mode -name tclk -value 010101
```
5. Test mode signals that are required to maintain the test mode state must not have a value list that ends in X because the X-value, just as 0s and 1s, will be simulated and be the final value on this pin. This will

often cause the propagation of X values into the circuit and override other simulation results established by prior values.

## Examples

### General Usage

Consider the following examples:

#### **Example1**

```
test_mode -name abc -value "<5*10>"
```

The above example will be expanded as follows:

```
test_mode -name abc -value 1010101010
```

#### **Example2**

```
test_mode -name abc -value "11<5*10>010"
```

The above example will be expanded as follows:

```
test_mode -name abc -value 111010101010010
```

#### **Example3**

```
test_mode -name abc -value "<50*11<5*10>>010"
```

The above example will be expanded as follows:

```
test_mode -name abc -value 111010101010...(repeated 50 times followed by 010)
```

You can also set a variable using the command `setvar` to obtain the above result as follows:

```
setvar x 11<5*10>
```

```
test_mode -name abc -value "<50*${x}>010"
```

The above example will be expanded as follows:

```
test_mode -name abc -value 111010101010...(repeated 50 times followed by 010)
```

**Example 4**

Tagging is not allowed during nesting. For example, the following `test_mode` statements are not allowed:

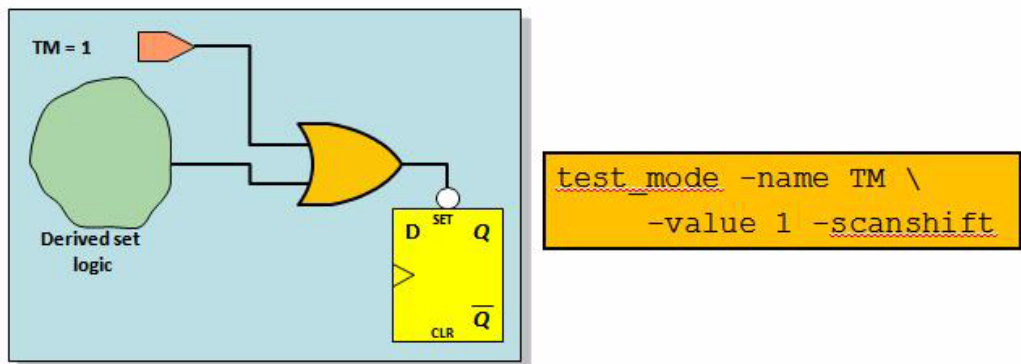
```
test_mode -name sub_seq -value <5*01>
test_mode -name main_seq -value <100*sub_seq>
```

However, you can achieve the same result by using the `setvar` command.

**NOTE:** For information and examples on specifying values for vector signals, refer to "Specifying Values for Vector Signals" section in the *SpyGlass DFT Rules Reference Guide*.

**Example 5**

Consider the following figure:



In the above example, the value, 1, is applied on the `SET` pin of the flip-flop ensuring that this pin is inactive during shift mode, that is, when the scan data is shifted in the scan chain. However, during capture phase, the combo logic applies the value, 1, on the same pin, if appropriate inputs are fed to it.

## Usage of Different Clock Edges

### *Usage of Positive Edge*

Consider the following sample input values:

```
test_mode -name tm1 -value 1 0 0 p
test_mode -name tm2 -value 0 0 1 1
test_mode -name tm3 -value 1 0
test_mode -name tm4 -value 0 0 1
test_mode -name tm5 -value 0 1 1 0
```

The above input is expanded as shown below:

```
test_mode -name tm1 -value 1 0 0 010
test_mode -name tm2 -value 0 0 1 111
test_mode -name tm3 -value 1 0
test_mode -name tm4 -value 0 0 1
test_mode -name tm5 -value 0 1 1 000
```

### *Usage Of Negative Edge*

Consider the following sample input values:

```
test_mode -name tm1 -value 1 0 0 N
test_mode -name tm2 -value 0 0 1 1
test_mode -name tm3 -value 1 0
test_mode -name tm4 -value 0 0 1
test_mode -name tm5 -value 0 1 1 0
```

The above input is expanded as shown below:

```
test_mode -name tm1 -value 1 0 0 101
test_mode -name tm2 -value 0 0 1 111
test_mode -name tm3 -value 1 0
test_mode -name tm4 -value 0 0 1 000
```



**Usage Of Positive Edge In Between**

Consider the following sample input values:

```
test_mode -name tm1 -value 1 0 0 p 0 1
test_mode -name tm2 -value 0 0 1 1 0 1
test_mode -name tm3 -value 1 0
test_mode -name tm4 -value 0 0 1
test_mode -name tm5 -value 0 1 1 0 1 1
```

The above input is expanded as shown below:

```
test_mode -name tm1 -value 1 0 0 010 0 1
test_mode -name tm2 -value 0 0 1 111 0 1
test_mode -name tm3 -value 1 0
test_mode -name tm4 -value 0 0 1
test_mode -name tm5 -value 0 1 1 000 1 1
```

**Usage Of Multiple Edges****Example 1**

Consider the following sample input values:

```
test_mode -name tm1 -value 1 0 0 p 0 1 n
test_mode -name tm2 -value 0 0 1 1 0 1 0 0 1
test_mode -name tm3 -value 1 0
test_mode -name tm4 -value 0 0 1 1 1 0 0 0 0
test_mode -name tm5 -value 0 1 1 0 1 1 0 1 0
```

The above input is expanded as shown below:

```
test_mode -name tm1 -value 1 0 0 010 0 1 101
test_mode -name tm2 -value 0 0 1 111 0 1 000 0 1
test_mode -name tm3 -value 1 0
test_mode -name tm4 -value 0 0 1 111 1 0 000 0 0
test_mode -name tm5 -value 0 1 1 000 1 1 000 1 0
```

**Example 2**

Consider the following sample input values:

```
test_mode -name clk1 -value P P P P 0 N N N
test_mode -name din1 -value 0 0 1 1 0 1 0 0
test_mode -name din2 -value 1 0 0 1
test_mode -name din3 -value 0 0 1 1 1 0 0 0
```

The above input is expanded as shown below:

```
test_mode -name clk1 -value 010 010 010 010 0 101 101 101
test_mode -name din1 -value 000 000 111 111 0 111 000 000
test_mode -name din2 -value 111 000 000 111
test_mode -name din3 -value 000 000 111 111 1 000 000 000
```

***Usage of pulse in bit-wise assignment***

Consider the following sample input values:

```
test_mode -name tm[3:0] -value { 4 b 0 n p 1 }
```

The above input, is expanded bit-wise as shown below:

```
test_mode -name tm[3] -value 0
test_mode -name tm[2] -value n
test_mode -name tm[1] -value p
test_mode -name tm[0] -value 1
```

The above input is expanded bit-wise with pulse expanded:

```
test_mode -name tm[3] -value 000
test_mode -name tm[2] -value 101
test_mode -name tm[1] -value 010
test_mode -name tm[0] -value 111
```

# Rules

The `test_mode` constraint is used by the following rules:

|                                  |
|----------------------------------|
| <b>SpyGlass DFT Solution</b>     |
| All rules                        |
| <b>SpyGlass DFT DSM Solution</b> |
| All rules                        |

## For SpyGlass STARC Product, SpyGlass STARC02 product, SpyGlass STARC05 product, and SpyGlass STARCad-21 product

### Purpose

In these products, the `test_mode` constraint causes certain rules of the products to work in test mode where all scan flip-flops are controlled by test clocks and all asynchronous set and reset pins are inactive

### Syntax

The syntax of the `test_mode` constraint is as follows:

```
current_design <du-name>
 test_mode
 -name <signame>
 -value <value>
```

### Arguments

**<du-name>**

Name of the design unit under which you are specifying the test mode pin.

**-name <signame>**

Hierarchical name of the test mode pin.

**-value <value>**

A logic pattern which, when applied on the test mode pin, will put the circuit in the test mode.

**Rules**

The `test_mode` constraint is used by the following rules:

|                                    |                  |                 |
|------------------------------------|------------------|-----------------|
| <b>SpyGlass STARC Product</b>      |                  |                 |
| STARC-3.3.2.2a                     | STARC-3.3.2.2.b  | STARC-3.3.2.3   |
| <b>SpyGlass STARC02 Product</b>    |                  |                 |
| STARC02-3.3.1.1                    | STARC02-3.3.1.4a | STARC02-3.3.2.3 |
| <b>SpyGlass STARC05 Product</b>    |                  |                 |
| STARC05-3.3.1.4a                   | STARC05-3.3.2.3  |                 |
| <b>SpyGlass STARCad-21 Product</b> |                  |                 |
| starcad_21_Prereq                  |                  |                 |

**test\_point****Purpose**

The `test_point` constraint specifies where a test point should be added in a design without the necessity of changing the source RTL.

The `test_point` constraint is used as seeding information for testability analysis. This seeding then influences coverage calculations just as if it was directly controllable, observable or both — depending on the value of the `-type` argument. The benefit is that when the `TA_01` or `TA_02` rules make recommendations, the source RTL description does not have to be modified.

When `test_mode`, `force_ta`, or `test_point` constraints are specified on the same node, following is the priority among different constraints:

- `Test_mode`
- User-specified specific [force\\_ta](#) / [test\\_point](#)

- Effect of *dft\_treat\_primary\_inputs\_as\_x\_source* and *dft\_treat\_primary\_outputs\_as\_unobservable* parameters

For example, if `test_mode 1` and `test_point control` are applied on the same node then the `test_mode` constraint will be considered.

Also, if the `test_mode`, `force_ta`, or `test_point` constraints are found in the fanout of each other, following is the priority among different constraints:

- The constraint in the fanout gets the priority
- Fanin effect is blocked by the specified / resolved constraint on the node

For example, consider that `test_mode 1` is applied on the input of buffer and `test_point control` is applied on the output of the same buffer. In this case, input will have simulation value 1 and `nyn` controllability but output will have `yyn` controllability and no simulation value.

**NOTE:** Prior to SpyGlass 4.3.0 release, the name of this constraint was `testpoint`.

## Product

SpyGlass DFT solution, SpyGlass DFT DSM solution

## Syntax

The syntax of the `test_point` constraint is as follows:

```
test_point
 -name <tp-name>
 -type <control | observe | full>
```

**NOTE:** The `test_point` constraint supports wildcard characters.

## Arguments

**-name <tp-name>**

Complete hierarchical name of a test point net, port, or pin.

The pin can be a primary pin as well as an internal pin.

You can specify a single port/pin/net's full hierarchical name or a space-separated list of full hierarchical port/pin/net's names.

For primary ports, you can also specify the simple port name as in the

following example:

```
current_design top
 test_point -name in15 ...
```

**-type <control | observe | full>**

Type of test point.

A value of `control` specifies that the test point is directly controllable, `observe` specifies that the test point is directly observable, and `full` specifies that the test point is directly controllable and observable.

Example

Consider the following example:

```
test_point -name "AND::b" -type control
```

The above command will apply `test_point` to all instantiations of the AND gate, and is equivalent to the following commands:

```
testpoint -name "test.inst2.b" -type control
testpoint -name "test.inst1.b" -type control
```

Rules

The `test_point` constraint is used by the following rules:

| SpyGlass DFT Solution     |                     |                      |               |
|---------------------------|---------------------|----------------------|---------------|
| Info_testclock            | Info_uncontrollable | Info_undetectedCause | Info_coverage |
| Coverage_audit            | TA_06               |                      |               |
| SpyGlass DFT DSM Solution |                     |                      |               |
| All rules                 |                     |                      |               |

tie\_x

Purpose

The `tie_x` constraint specifies the X-generator design unit (black box)

names. Then, all instances of these design units in the design are assumed as X-generator instances. The `tie_x` constraint support wildcards.

**NOTE:** Prior to SpyGlass 4.3.0 release, the name of this constraint was `TIE_X`.

## Product

SpyGlass DFT solution

## Syntax

The syntax of the `tie_x` constraint is as follows:

```
tie_x
 -name <bb-name>
 -xpin <xpin-name-list>
```

**NOTE:** The `tie_x` constraint supports wildcard characters.

## Arguments

The `tie_x` constraint has the following arguments:

**-name <bb-name>**

The X-generator design unit (black box) name.

The design unit must be a black box. That is, its definition must not exist in the design or in the specified libraries, if any.

The design unit name *<du-name>* can be specified as module name (for Verilog designs) or as entity name (for VHDL designs). For VHDL designs, all architectures of the specified entity are treated as X-generator design units.

You can specify a single design unit name or a space-separated list of design unit names.

**-xpin <xpin-name-list>**

A space-separated name list of X-generator design unit (black box) output pins that are X-generator pins.

## Notes

More than one `tie_x` constraint may be necessary. If more than one `tie_x` constraint is used, they all will be processed in parallel.

## Rules

The `tie_x` constraint is used by the following rule:

| SpyGlass DFT Solution |
|-----------------------|
| BIST_05               |

# tristate\_cell

## Purpose

The `tristate_cell` constraint is used to define the tristate cells.

## Product

SpyGlass DFT solution

## Syntax

The syntax of the `tristate_cell` constraint is as follows:

```
tristate_cell
 -name <mod-name>
 -zpin <inout-port>
 -enpin <input-port>
 [-envalue <tri-buf-val>]
 -dpin <input-data-pin>
 -oppin <output-port>
```

**NOTE:** The `tristate_cell` constraint supports wildcard characters.

## Arguments

The `tristate_cell` constraint has the following arguments:



**-name <mod-name>**

Tristate buffer module name.  
Names may refer to the modules where pins are generating X.

**-zpin <inout-port>**

Inout port driving the tristate buffer

**-enpin <input-port>**

Input port driving the tristate buffer

**-envalue <tri-buf-val>**

Value to make tristate cell act as a buffer. This value can be set as 1 or 0.

**-dpin <input-data-pin>**

Input data pin

**-oppin <output-port>**

Output port

## Rules

The `tristate_cell` constraint is used by the following rule:

| SpyGlass DFT Solution |
|-----------------------|
| Tristate_16           |

## ungroup\_cells

### Purpose

Use this constraint to ungroup instances in the current module. To override the default delimiter character (/) while ungrouping, use the `sgsyn_ungroup_delimiter` command. For more information on this command, see SpyGlass Explorer User Guide.

## Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

## Syntax

The syntax to specify the `ungroup_cells` constraint is as follows:

```
ungroup_cells
 [-inst <inst-list>]
 [-all]
 [-flatten]
 [-small <num>]
 [-level <depth>]
 [-mod <mod-list>]
 [-mod_all <mod-all-list>]
```

## Arguments

### **-inst <inst-list>**

Specifies a list of instances (in the current module) that needs to be ungrouped.

You can specify the current module by using the `current_design` constraint.

For Verilog, the value provided by this argument is case-sensitive. For VHDL, this value is not case-sensitive.

### **-all**

Specifies that all instances in the current module should be ungrouped.

### **-flatten**

Specifies that the specified instances should be ungrouped recursively until leaf-level cells. By default, the behavior is single level flattening.

**NOTE:** *The -all argument is automatically considered if you specify the -flatten argument.*

**-small <num>**

Specifies a number of leaf cells so that all instances in the current module that have less than the specified number of leaf cells should be ungrouped.

If you specify the `-small <num>` argument, all instances under the current design are considered. That is, the `-inst` argument cannot be specified in this case.

**NOTE:** Please note the following points:

- The `-all` argument is automatically considered if you specify the `-small` argument.
- If you specify the `-small` and `-all` arguments together, SpyGlass reports a fatal violation.

**-level <depth>**

Specifies a depth so that all instances at a depth greater or equal to the specified depth should be dissolved.

**NOTE:** Please note the following points:

- The `-all` argument is automatically considered if you specify the `-depth` argument.
- If you specify the `-level` and `-all` arguments together, SpyGlass reports a fatal violation.

**-mode <mod-list>**

Specifies a list of modules so that all instances of these modules within the current module are dissolved.

You can specify a current module by using the `current_design` command.

By default, only immediate instances of the specified modules are dissolved. To dissolve instances hierarchically until leaf level, specify the `-flatten` argument of this constraint along with the `-mod` argument.

**-mod\_all <mod-all-list>**

Specifies a list of modules so that all instances of these modules across the whole design are dissolved.

By default, only immediate instances of the specified modules are

dissolved. To dissolve instances hierarchically until leaf-level, specify the `-flatten` argument of this constraint along with the `-mod_all` argument.

**NOTE:** *If you specify the `-mod_all` argument along with the `-inst`, `-mod`, or `-all` argument of this constraint, SpyGlass reports a fatal violation.*

## Examples

The following examples show the usage of the `ungroup_cells` constraint:

- The following command is used to ungroup the `mid1` and `mid2` instances in the `top` module.

```
current_design top
ungroup_cells -inst mid1 mid2
```

- The following command is used to ungroup the `mid1` and `mid2` instances in the `top` module recursively till leaf-level:

```
current_design top
ungroup_cells -inst mid1 mid2 -flatten
```

- The following command is used to ungroup all instances of the `M1` and `M2` modules in the `top` module:

```
current_design top
ungroup_cells -mod M1, M2
```

- The following command is used to ungroup all instances in the `mid` module recursively till leaf-level:

```
current_design mid
ungroup_cells -all -flatten
```

- The following command is used to ungroup all instances in all hierarchies with respect to `top` that have less than `N` leaf cells:

```
current_design top
ungroup_cells -small N
```

## Rules

The `ungroup_cells` constraint is used by the following rules:

---

### **SpyGlass Power Estimation and SpyGlass Power Reduction Solutions**

---

All rules running in EST mode

---

## use\_library\_group

### Purpose

Helps in specifying the instance and library group mapping, that is, specifying the library group from which the cell definition of the instance needs to be picked.

### Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

### Syntax

The syntax to specify the `use_library_group` constraint is as follows:

```
current_design <du-name>
 use_library_group
 -name <lib-grp-name>
 -instname <inst-name-list>
```

### Arguments

#### <du-name>

Module name (for Verilog designs) or design unit name in `<entity-name>.<arch-name>` format (for VHDL designs).

#### -name <lib-grp-name>

Name of the library group(s) to which the cell definition of the specified instance belongs.

**NOTE:** You can also use the `-name` argument as `-names` to specify a single library or a group of libraries.

**-instname** <inst-name-list>

Hierarchical path and name of an instance or a top-level design unit name to which library groups should be bound.

Consider the following example, suppose the top design unit name, `top`, has two instances, `g1` and `g2`, with the same library cell definition but the cell definitions are in different library groups (say, `G1` and `G2`) at different PVT values.

Now, if you want to pick the cell definition for instance `g1` from library group `G1` and the cell definition for instance `g2` from library group `G2`, use the `use_library_group` constraint as follows:

```
use_library_group -name G1 -instname top.g1
use_library_group -name G2 -instname top.g2
```

**NOTE:** Refer to the SGDC command [define\\_library\\_group](#) to group all libraries that are characterized at same PVT together.

**Rules**

The `use_library_group` constraint is used by the following rules:

| SpyGlass Power Estimation and SpyGlass Power Reduction Solutions |         |            |         |
|------------------------------------------------------------------|---------|------------|---------|
| PEPWR01                                                          | PEPWR02 | PEPWR03    | PEPWR05 |
| PEPWR13                                                          | PEPWR14 | poweraudit |         |

**voltage\_domain**

**Product**

SpyGlass DFT DSM solution, SpyGlass Power Verify solution, SpyGlass Power Estimation and SpyGlass Power Reduction solutions

## For SpyGlass DFT DSM Solution

### Purpose

The `voltage_domain` constraint is used to specify the voltage/power domains in the design and its information is used by `SP_01` rule of the SpyGlass DFT DSM solution.

**NOTE:** *Prior to SpyGlass 4.3.0 release, the name of this constraint was `voltagedomain`.*

### Syntax

The syntax to specify the `voltage_domain` constraint is as follows:

```
current_design <du-name>
 voltage_domain
 [-modname <du-name>]
 [-instname <inst-name-list>]
```

### Arguments

#### <du-name>

Name of the design unit under which you are specifying the voltage/power domains. This is the environment for the voltage/power domains.

#### -modname <du-name>

(Optional) Name of top-level design unit, specified as module name or the entity name.

Use the `-modname` argument, as in the following examples:

```
current_design top
 voltage_domain -modname top
```

The above example applies to Verilog module named `top`.

```
current_design top.rtl
 voltage_domain -modname top
```

The above example applies to VHDL entity named `top` with an architecture

named `rtl`. If only the entity name is specified with `current_design` specification, the last compiled architecture is used in case of entities with multiple architectures.

**-instname** <inst-name-list>

(Optional) Space-separated list of instance names.

The instance name must be a hierarchical instance name with respect to the top-level design unit specified as the environment.

You can use wildcard characters while specifying instance names.

**NOTE:** *The voltage domain information is applicable hierarchically.*

### Rules

The `voltage_domain` constraint is used by the following rule:

| SpyGlass DFT DSM Solution |
|---------------------------|
| SP_01                     |

## For SpyGlass Power Verify solution

### Purpose

The `voltage_domain` constraint is used to specify the voltage/power domains in the design and its information is used by voltage and power domain rules.

**NOTE:** *Prior to SpyGlass 4.3.0 release, the name of this constraint was `voltagedomain`.*

### Syntax

The syntax to specify the `voltage_domain` constraint is as follows:

```
current_design <du-name>
 voltage_domain
 -name <name>
 -value <fvalue-list>
```



## SpyGlass Design Constraints

```

[-instname <inst-name-list>]
 [-portname <port-name-list>]
 | -modname <du-name>
 [-portname <port-name-list>]
 | -portname <port-name-list> -external
 [-netname <net-name-list>]
 | -netname <net-name-list> -external
 | -external
[-isosig <iso-sig-list> -isoval <iso-value-list>
 | -inisosig <iniso-sig-list>
 -inisoval <iniso-value-list>
 | -noisosig]
[-generate_iso_logic]
[-outputs <ss-condition-name>]
[-supplyname <port-name-list>]
[-clkdomain <clk-name-list>]
[-enableports <en-port-name-with-limit-list>]
[-inputs <pd-condition-name>]
[-isolatedinputs <pin-name-list>]
[-isolatedoutputs <pin-name-list>]
[-isolatedoutputs <pin-name-list>]
[-biaspowernet <bias-pwr-net>]
[-biasgroundnet <bias-gnd-net>]
 [-sleepnet <net-name>]
[-sleepval 0 | 1]
[-stopclock <net-name>]
[-stopclockval 0 | 1]
[-savenet <net-name>]
[-saveval 0 | 1]
[-restorenet <net-name>]
[-restoreval 0 | 1]

```

## Arguments

### <du-name>

Name of the design unit under which you are specifying the voltage/power domains. This is the environment for the voltage/power domains.

**-name <name>**

Name of the voltage/power domain.

**-value <fvalue-list>**

Voltage values (floating-point values) of the voltage/power domain.

For voltage domains, you should specify only one positive non-zero value.

For power domains, you should specify two values (space-separated) that indicate the ON and OFF voltage values of the power domain. The first value should be a positive non-zero value and the second value should be zero.

**-instname <inst-name-list>**

(Optional) Space-separated list of instances belonging to the voltage/power domain.

The instance name must be a hierarchical instance name with respect to the top-level design unit specified as the environment.

You can use wildcard characters while specifying instance names.

**NOTE:** *The voltage domain information is applicable hierarchically.*

**NOTE:** *If you do not use the -instname argument, you must specify the -modname argument or the -portname argument with the -external argument, or the -external argument.*

**-modname <du-name>**

(Optional) Name of top-level design unit (specified as module name or the entity name) belonging to the voltage/power domain. Then, all instances of the design unit are assumed to be in the voltage/power domain being specified.

Use the -modname argument as in the following examples:

```
current_design top
 voltage_domain -name vd1 -value 1.2 -modname top
```

The above example applies to Verilog module named top.

```
current_design top.rtl
 voltage_domain -name vd1 -value 1.2 -modname top
```

The above example applies to VHDL entity named `top` with an architecture named `rtl`. If only the entity name is specified with `current_design` specification, the last compiled architecture is used in case of entities with multiple architectures.

**NOTE:** You must specify the voltage domain information for top-level design unit using `-modname` argument.

**NOTE:** The voltage domain information is applicable hierarchically.

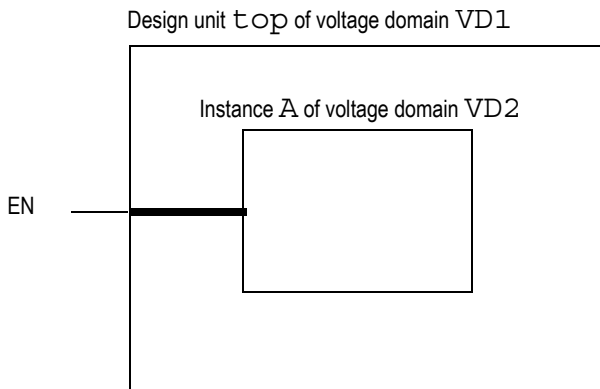
**NOTE:** If you do not use the `-modname` argument, you must specify the `-instname` argument, the `-portname` argument with the `-external` argument, or the `-external` argument.

#### **-portname** <port-name-list>

(Optional) Space-separated list of ports of other design units/instances that belong to a different voltage/power domain but are directly connected to an instance of the specified voltage/power domain.

You can use wildcard characters while specifying port names where ports can be top-level or of a user-defined module.

Consider the following example:



**FIGURE 55.** Port of a Design Unit Belonging to a Different Voltage Domain

Here, the port `EN` of top-level design unit `top` of voltage domain `VD1` is directly connected to the instance `top.A` of voltage domain `VD2`. If you do

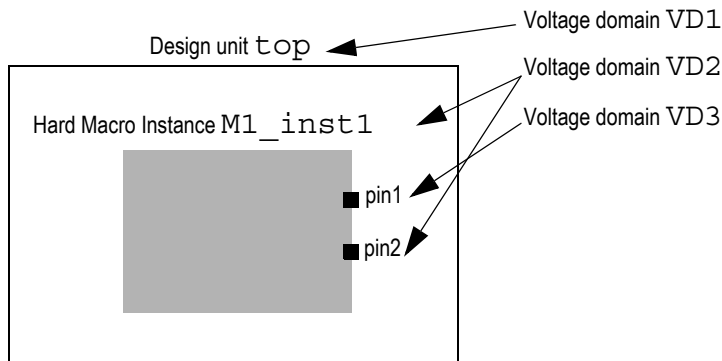
not specify the port `EN` to be in voltage domain `VD2`, the connection becomes a voltage domain crossing from `VD1` to `VD2` and you would need to insert a level shifter in the path.

The above situation is specified as follows:

```
voltage_domain -name VD1 -value 1.2 -modname top
voltage_domain -name VD2 -value 1.5
 -instname top.A -portname EN
```

You can also specify the individual pins of hard macro instances (black box instances) to be in a different voltage/power domain than the voltage/power domain of the hard macro instance.

Consider the following example where pin `pin1` is of a different voltage/power domain than its parent hard macro instance (black box instance) `M1_inst1`:



**FIGURE 56.** Children Pin of a Different Voltage Domain From its Parent Instance

The above situation can be specified as follows:

```
voltage_domain -name VD1 -value 1.2 -modname top
voltage_domain -name VD2 -value 1.5
 -instname top.M1_inst1
voltage_domain -name VD3 -value 1.7
 -portname top.M1_inst1.pin1
```

**NOTE:** You need to specify the full hierarchical name of the pin having a different voltage/power domain.

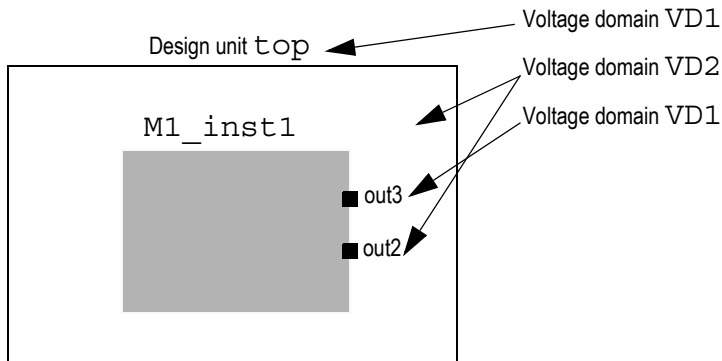
**NOTE:** Only the ports of top-level design units and pins of leaf level instances can be

*specified with the -portname argument.*

**-netname** <net-name-list>

(Optional) Space-separated list of nets of other design units that belong to a different voltage/power domain but are directly driven by an instance of the specified voltage/power domain.

Consider the following example where all the output nets of the design unit `top.M1_inst1` are in domain VD2 working at 1.5 volts except net `out3` which is being driven by 1.2 volts.



**FIGURE 57.** All Output Nets Belong to Same Voltage Domain

The above situation can be specified as follows:

```
voltage_domain -name VD1 -value 1.2 -modname top -
netname top.M1_inst1.out3
voltage_domain -name VD2 -value 1.5
 -instname top.M1_inst1
```

You can use wildcard characters while specifying net names.

The constraint also supports specification of vector nets. For example, a 4 bit net `top.M1_inst1.out3` can be specified as either:

```
top.M1_inst1.out3, top.M1_inst1.out3[0:3]
```

or

```
top.M1_inst1.out3[0] top.M1_inst1.out3[1]
```

```
top.M1_inst1.out3[2] top.M1_inst1.out3[3]
```

You need to specify the full hierarchical name of the net having a different voltage/power domain. The field is used by LPSVM04 rule of the *SpyGlass Power Verify* solution for checking correct level shifters.

### **-external**

(Optional) Specifies that the voltage domain specified for a port is external to the design.

You can use the `-external` argument with the `-portname` argument (to indicate that the voltage/power domain is applicable to the port and is external to the design) or as a stand-alone argument (to indicate that the voltage/power domain is external to the design).

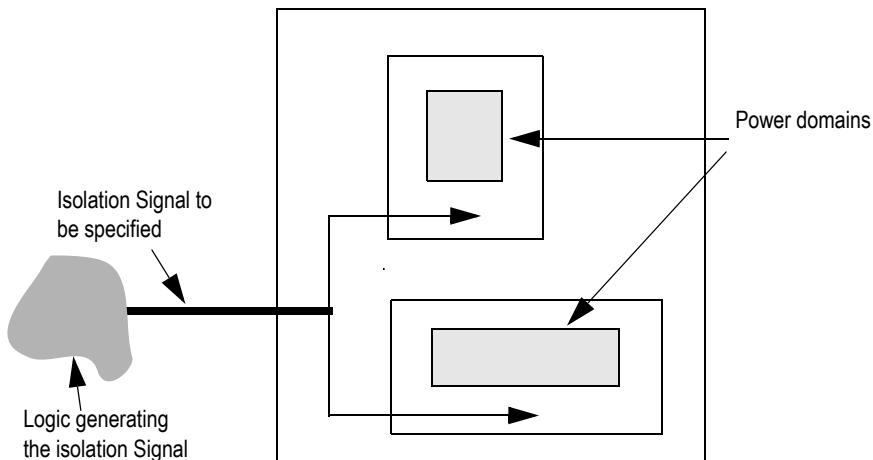
**NOTE:** *You can specify the external voltage/power domains (using the stand-alone -external argument of the voltage\_domain constraint). Then, the specified domain name can be used by the pin\_voltage constraint.*

### **-isosig** <iso-sig-list>

(Optional) Name of the isolation signal(s) for the power domain being defined as a space-separated list.

**NOTE:** *You must specify an isolation signal for a power domain for the related checks to be performed. If you are not specifying -isosig argument, it is mandatory to specify either -inisosig with the input isolation signal(s) for the power domain or -noisosig indicating that the power domain does not have an isolation signal.*

If you are specifying an internal signal, please ensure that you specify the signal name at the point of creation, as in the following example:



**FIGURE 58.** Signal Name At The Point Of Creation

#### **-isoval <iso-value-list>**

(Optional) Assert value(s) of the isolation signal(s) being specified with the `-isosig` argument as a space-separated list.

The assert value is the value that the isolation signal attains under the power-down conditions. In normal mode, the isolation signal gets the inverse of the assert value.

**NOTE:** The number of isolation values specified with the `-isoval` argument must match the number of isolation signals specified with the `-isosig` argument. Otherwise, SpyGlass reports a FATAL message.

#### **-inisosig <iniso-sig-list>**

(Optional) Name of the input isolation signal(s) for the power domain being defined as a space-separated list.

**NOTE:** The object(s) specified with the `-inisosig` argument should be a top-level port, net, or instance terminal in the design unit specified as current design of the voltage\_domain constraint

**-inisoval <iniso-value-list>**

(Optional) Assert value(s) of the input isolation signal(s) being specified with the `-inisosig` argument as a space-separated list.

The valid values for `-inisoval` argument are only 0 or 1.

**NOTE:** *The number of isolation values specified with the `-inisoval` argument must match the number of input isolation signals specified with the `-inisosig` argument. Otherwise, SpyGlass reports a FATAL message.*

**-noisosig**

(Optional) Indicates that the power domain does not have an isolation signal.

When you specify the `-noisosig` argument, the power domain checking rules do not perform isolation signal-related checking.

**NOTE:** *It is mandatory to specify either of `-isosig`, `-inisosig`, or `-noisosig` arguments.*

**-supplyname <port-name-list>**

(Optional) Specifies a space-separated simple name list of power/ground rails (ports) in the design.

These ports are associated with the supply constraint.

**-generate\_iso\_logic -outputs <ss-condition-name>**

(Optional) The `-generate_iso_logic` argument enables generation of missing isolation logic by the LPSVM23 rule of the *SpyGlass Power Verify* solution and the `-output` argument specifies the steady state conditions (any valid string) as specified using the `domain_outputs` constraint.

In addition, the LPSVM09 rule of the *SpyGlass Power Verify* solution uses the `-output` argument along with the `domain_outputs` constraint.

Consider the following example that defines the power domain V3 that does not have isolation logic:

```
voltage_domain
 -name V3 -value 1.2 0
 -instname top.u6
```



```
-isosig isosig6
-isoval 1
-generate_iso_logic
-outputs PD_V3_OUT
```

Here, the `-generate_iso_logic` argument enables generation of missing isolation logic and the `-outputs` argument defines the steady state condition as `PD_V3_OUT`. For defining `PD_V3_OUT`, use `domain_outputs` constraint as follows:

```
domain_outputs -name PD_V3_OUT -value top.d_o 0 top.ack_o 0
top.err_o 0 -default 1
```

#### **-clkdomain <clk-name-list>**

(Optional) Specifies the valid clock domains of the voltage domain for clocks described using the `clock` constraint.

#### **-enableports <en-port-name-with-limit-list>**

(Optional) Specifies a space-separated list of enable ports of a power domain along with the individual power switch enable connection limit as checked by the LPSVM45 rule of the *SpyGlass Power Verify* solution.

Consider the following constraint specification specifying power domain PD1:

```
voltage_domain -name PD1 -value 1.2 0
-instantiate "TOP.lower1" -isosig top.iso_sig
-isoval 1 -enableports 'EN1 1 EN2 100 EN3[2] 100'
```

The `-enableports` argument of the `voltage_domain` constraint specifies the ports of power domain through which enable signals of power switches must be connected. The enable port of the power domain and the maximum number of power switch enables allowed to be connected to it are specified as a space-separated pair. For example, `EN1 1` indicates that only one power switch enable can be connected to port `EN1`.

Specify `"-1"` as the limit to indicate that any number of power switch enables can be connected. For example, `EN12 "-1"`.

**-inputs <pd-condition-name>**

(Optional) Specifies the power domain input expected value condition name that is specified with the `domain_inputs` constraint.

**-isolatedinputs <pin-name-list>**

(Optional) Specifies the power domain input name list (hierarchical pin name list) when these power domain inputs are to be assumed as already isolated and, therefore, no isolation-related checking should be performed on these inputs.

You can also use wildcard characters and bus notation while specifying the pin names for this argument.

**-isolatedoutputs <pin-name-list>**

(Optional) Specifies the power domain output name list (hierarchical pin name list) when these power domain outputs are to be assumed as already isolated and, therefore, no isolation-related checking should be performed on these outputs.

You can also use wildcard characters and bus notation while specifying the pin names for this argument.

**-biaspowernet <bias-pwr-net>**

(Optional) Specifies the name of bias power net associated with the domain.

**-biasgroundnet <bias-gnd-net>**

(Optional) Specifies the name of bias ground net associated with the domain.

**-sleepnet <net-name>**

(Optional) Specifies a sleep net of a power domain as used by the LPSVM56, LPSVM57, LPSVM58, and LPSVM59 rules of the *SpyGlass Power Verify* solution.

**NOTE:** The `-sleepnet` argument should be specified along with the `-sleepval` argument. For example,

```
voltage_domain -instname P1 -sleepnet SL1 -sleepval 1
```

**-sleepval 0 | 1**

(Optional) Specifies the active value of a sleep net of a power domain (0 or 1) as used by the LPSVM58 and LPSVM59 rules of the *SpyGlass Power Verify* solution.

**NOTE:** *The -sleepval argument should be specified along with the -sleepnet argument.*

**-stopclock <net-name>**

(Optional) Specifies the stop clock net of a power domain.

**-stopclockval 0 | 1**

(Optional) Specifies the active value of a stop clock net of a power domain (0 or 1).

**-savenet <net-name>**

(Optional) Specifies the save net of a power domain as used by the LPSVM57 and LPSVM59 rules of the *SpyGlass Power Verify* solution.

**NOTE:** *The -savenet argument should be specified along with the -saveval argument. For example:*

`voltage_domain -instname P1 -savenet SA1 -saveval 1`

**-saveval 0 | 1**

(Optional) Specifies the active value of the save net of the power domain (0 or 1) as used by the LPSVM59 rule of the *SpyGlass Power Verify* solution.

**NOTE:** *The -saveval argument should be specified along with the -savenet argument.*

**-restorenet <net-name>**

(Optional) Specifies the restore net of a power domain as used by the LPSVM57 and LPSVM59 rules of the *SpyGlass Power Verify* solution.

**NOTE:** *The -restorenet argument should be specified along with the -restoreval argument. For example:*

`voltage_domain -instname P1 -restorenet RS1 -restoreval 1`

1

**-restoreval 0 | 1**

(Optional) Specifies the active value of the restore net of a power domain (0 or 1) as used by the LPSVM59 rule of the *SpyGlass Power Verify* solution.

**NOTE:** *The -restoreval argument should be specified along with the -restorennet argument.*

## Examples

Consider the following example:

```
module top(in1, in2, iso_sig, out1, out2);
 input in1, in2, iso_sig;
 output out1, out2;

 wire C1, D1;

 lower lower1(in1, C1);
 lower \lower2 (in2, D1);
 assign out1 = C1 & iso_sig;
 assign out2 = D1 & iso_sig;
endmodule

module lower(in, out);
 input in;
 output out;

 assign out = in;
endmodule
```

All constraint specifications are with respect to the top-level module `top`. Therefore, the environment is defined as follows:

```
current_design top
```

Now, the top-level module `top` is in the voltage domain named say `VD1`. This situation is specified as follows:

```
current_design top
```

```
voltage_domain -name VD1 -value 1.2 -modname top
```

Next, assume that the instance `lower1` is in a different voltage domain. This situation is specified as follows:

```
current_design top
...
voltage_domain -name VD2 -value 1.5
 -instname top.lower1
```

Since port `in1` of module `top` (which is in voltage domain `VD1`) is directly connected to instance `lower1` (which is in voltage domain `VD2`), you can also specify the port using the `-portname` argument to avoid crossing message for a voltage domain, as shown in the following example:

```
current_design top
...
voltage_domain -name VD2 -value 1.5
 -instname top.lower1
 -portname in1
```

Next, assume that the instance `\lower2` is in the power domain `PD1` having signal `iso_sig` as the isolation signal with assert value 1. This situation is described as follows:

```
current_design top
...
voltage_domain -name PD1 -value 1.2 0
 -instname "top.\lower2 "
 -isosig top.iso_sig -isoval 1
```

**NOTE:** When you use flattened netlists as inputs instead of RTL files, you can specify all flattened instance names in a scope by using the asterisk (\*) wildcard character instead of listing them individually by using the `-instname` argument. For example, when the scope `top.\U1` contains three flattened instances, `top.\U1/U11`, `top.\U1/U12`, and `top.\U1/U13`, specify them as `"top.\U1*"`.

## For SpyGlass Power Estimation and SpyGlass Power Reduction Solutions

### Purpose

The `voltage_domain` constraint is used to specify the design unit present in different `voltage_domain` as used by the rules in the SpyGlass Power Estimation and SpyGlass Power Reduction solutions.

### Syntax

The syntax to specify the `voltage_domain` constraint is as follows:

```
current_design <du-name>
 voltage_domain
 -name <name>
 -value <value>
 -instname <inst-name-list>
 | -modname <du-name>
 | -portname <port-name-list>
 | -netname <net-name-list>
```

### Arguments

#### <du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

#### -name <name>

Name of the voltage domain.

#### -value <value>

Voltage value (floating-point values) of the specified voltage domain.

**NOTE:** *The first voltage value in the `voltage_domain` constraint should be non-zero and the second should be zero (if defined). Multiple non-zero values are not allowed.*

**-instname <inst-name-list>**

(Optional) Space-separated list of instances belonging to the specified voltage domain.

The instance name must be a hierarchical instance name with respect to the top-level design unit specified as the environment.

**NOTE:** *The voltage domain information is applicable hierarchically.*

**NOTE:** *You can also specify the instance names using regular expressions.*

**-modname <du-name-list>**

(Optional) Space-separated list of top-level design units (specified as module name or the entity name) belonging to the specified voltage domain. Then, all instances of these design units are assumed in the specified voltage domain.

Use the -modname argument, as in the following examples:

```
current_design top
 voltage_domain -name vd1 -value 1.2 -modname top
```

The above example applies to Verilog module named top.

```
current_design top.rtl
 voltage_domain -name vd1 -value 1.2 -modname top
```

The above example applies to VHDL entity named top with an architecture named rtl. If only the entity name is specified with current\_design specification, the last compiled architecture is used in case of entities with multiple architectures.

**NOTE:** *The voltage domain information is applicable hierarchically.*

**-portname <port-name-list>**

(Optional) Space-separated name list of primary ports belonging to the specified voltage domain.

**NOTE:** *You can also specify the port names using regular expressions.*

**-netname <net-name-list>**

(Optional) Space-separated name list of nets belonging to the specified voltage domain.

**NOTE:** *You can also specify the net names using regular expressions.*

## Notes

You must specify at least one of the `-instname`, `-modname`, `-netname`, or `-portname` arguments. This requirement is checked by the `SGDC_power_est18` rule.

## Examples

Consider the following example:

```
module top(in1, in2, iso_sig, out1, out2);
 input in1, in2, iso_sig;
 output out1, out2;

 wire C1, D1;

 lower lower1(in1, C1);
 lower \lower2 (in2, D1);
 assign out1 = C1 & iso_sig;
 assign out2 = D1 & iso_sig;
endmodule

module lower(in, out);
 input in;
 output out;

 assign out = in;
endmodule
```

All constraint specifications are with respect to the top-level module `top`. Therefore, the environment is defined as follows:

```
current_design top
```

Now, the top-level module `top` is in the voltage domain named say `VD1`. This situation is specified as follows:

```
current_design top
 voltage_domain -name VD1 -value 1.2 -modname top
```

Next, assume that the instance `lower1` is in a different voltage domain. This situation is specified as follows:



```
current_design top
...
voltage_domain -name VD2 -value 1.5
 -instname top.lower1
```

**NOTE:** When you use flattened netlists as input instead of RTL files, you can specify all flattened instance names in a scope by using the asterisk (\*) wildcard character instead of listing them individually by using the `-instname` argument. For example, when the scope `top.\U1` contains three flattened instances, `top.\U1/U11`, `top.\U1/U12`, and `top.\U1/U13`, specify them as `"top.\U1*"`.

Rules

The `voltage_domain` constraint is used by the following rules:

| SpyGlass Power Estimation and SpyGlass Power Reduction Solutions |         |            |         |
|------------------------------------------------------------------|---------|------------|---------|
| PEPWR01                                                          | PEPWR02 | PEPWR03    | PEPWR05 |
| PEPWR13                                                          | PEPWR14 | poweraudit |         |

vt\_mix\_percentage

Purpose

The `vt_mix_percentage` constraint is used to specify the percentage number of cells of a threshold voltage group in a library or a complete library to be processed by the PEPWR01 and PEPWR02 rules.

Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

Syntax

The syntax to specify the `vt_mix_percentage` constraint is as follows:

```
current_design <top-du-name>
vt_mix_percentage
```

```

-group <grp-name> | -cellname <cell-name-pattern> |
-libname <lib-name>
[-weight <value>]
[-instname <inst-name>]
[-clock <clock-name>]
[-clock_period <clock-period>]

```

## Arguments

### <top-du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

### -group <grp-name>

Name of the threshold voltage group (name specified with the `default_threshold_voltage_group/`  
`threshold_voltage_group` attribute in the library).

This argument supports lists.

### -cellname <cell-name-pattern>

Specifies a pattern to match cell names of a certain group of cells.

This argument supports lists.

### -libname <lib-name>

Name of the library.

This argument supports lists.

### -weight <value>

The percentage value (a floating-point number between 0 and 100).

Do not specify this argument if the *PEVTDIST* rule is run.

### -instname <inst-name>

Name of the instance for which the percentage value is applicable.

**-clock <clock-name>**

Name of the clock for which the percentage values are applicable.

**-clock\_period <clock\_period>**

Clock period (in nanoseconds) for which the percentage values are applicable.

**NOTE:** *Please note the following:*

- ☞ *To specify values for a top-level design unit, do not use the -instname argument. However, to specify values for a specific instance hierarchy, use the -instname argument.*
- ☞ *You must at least define the percentage value for the top-level design unit. Specifying values for specific instance hierarchies is optional.*
- ☞ *For any specific run, you can use only one of the following arguments: -group, -libname, or -cellname. You cannot use a combination of two or more of these arguments to specify the VT mix information in the same run.*
- ☞ *You must use the -group argument to specify values when the associated library has a default\_threshold\_voltage\_group/threshold\_voltage\_group attribute set.*
- ☞ *The sum of values specified for the top-level design unit or an instance hierarchy in different vt\_mix\_percentage constraints must be equal to 100.*
- ☞ *Do not specify any two arguments together out of the -clock, -clock\_period, and -instname arguments.*
- ☞ *You can specify only one argument when using -clock, -clock\_period, and -instname arguments. You cannot use these three arguments together.*

The following example specifies percentage values for the top-level design unit top (first two vt\_mix\_percentage constraints) and for instance hierarchy top.inst1 (Next two vt\_mix\_percentage constraints):

```
current_design tophier
 vt_mix_percentage -group tvgl -weight 60
 vt_mix_percentage -group tvgl2 -weight 40
 vt_mix_percentage -group tvgl
 -instname top.inst1 -weight 30
```

```
vt_mix_percentage -group tvlg2
 -instname top.inst2 -weight 70
```

## Examples

- Consider a scenario in which you have a set of libraries with cells of two threshold voltage groups, DEFAULT1 and DEFAULT2. If you want to tell SpyGlass that 75% of the design should use cells from the DEFAULT1 group and 25% from the DEFAULT2 group, specify the following commands:

```
vt_mix_percentage -group DEFAULT1 -weight 75
vt_mix_percentage -group DEFAULT2 -weight 25
```

- Consider a scenario in which you have two libraries, typical1NOTVG and typical2NOTVG. Both the libraries have cells of a specific threshold voltage, but the `threshold_voltage_group` attribute is not specified in the library files. In this case, you can directly specify the library names, as shown below:

```
vt_mix_percentage -libname typical2NOTVG -weight 25
vt_mix_percentage -libname typical1NOTVG -weight 75
```

- You can specify the percentage usage of cells from different groups for specific hierarchical instances as well. This is shown in the following example:

```
select_wireload_model -wireload tsmc13_wl20
vt_mix_percentage -libname typical2NOTVG -weight 60
vt_mix_percentage -libname typical1NOTVG -weight 40
```

```
vt_mix_percentage -libname typical1 -instname
top.inst1 -weight 75
vt_mix_percentage -libname typical2 -instname
top.inst1 -weight 25
```

```
vt_mix_percentage -libname typical1 -instname
top.inst2 -weight 25
vt_mix_percentage -libname typical2 -instname
top.inst2 -weight 75
```

In the above example, for the top-level design, `top`, the ratio of the

usage of cells from the `typical2NOTVG` and `typical1NOTVG` libraries is 60:40.

In addition, for the `top.inst1` module within the top-level design, the ratio of the usage of cells from the `typical1` and `typical2` libraries is 75:25. Similarly, for the `top.inst2` module, the ratio of the usage of cells from the `typical1` and `typical2` libraries is 25:75.

- You can also specify the percentage usage directly based on the cell names, as shown in the following example:

```
vt_mix_percentage -cellname "VT2*" -weight 75
vt_mix_percentage -cellname "VT1*" -weight 25
```

In the above example, the percentage usage of all the cell names starting with the string, `VT2`, will be 75. Similarly, the percentage usage of all the cell names starting with the string, `VT1`, will be 25.

- You can specify the percentage usage of cells from different groups for specific clocks. This is shown in the following example:

```
select_wireload_model -wireload tsmc13_wl20
vt_mix_percentage -libname typical2NOTVG -weight 60 -clock
top.clk1
vt_mix_percentage -libname typical1NOTVG -weight 40 -clock
top.clk1

vt_mix_percentage -libname typical2NOTVG -weight 30 -clock
top.clk2

vt_mix_percentage -libname typical1NOTVG -weight 70 -clock
top.clk2
```

- You can specify the percentage usage of cells from different groups for specific clock periods. This is shown in the following example:

```
select_wireload_model -wireload tsmc13_wl20
vt_mix_percentage -libname typical2NOTVG -weight 60
-clock_period 10
vt_mix_percentage -libname typical1NOTVG -weight 40
-clock_period 10
vt_mix_percentage -libname typical2NOTVG -weight 30
-clock_period 20
```

```
vt_mix_percentage -libname typical1NOTVG -weight 70
-clock_period 20
```

- The following constraints specifications show how to specify multiple library names:

```
vt_mix_percentage -libname typical1 typical2 -weight 60
```

Rules

The vt\_mix\_percentage constraint is used by the following rules:

| SpyGlass Power Estimation and SpyGlass Power Reduction Solutions |         |            |         |
|------------------------------------------------------------------|---------|------------|---------|
| PEPWR01                                                          | PEPWR02 | PEPWR03    | PEPWR05 |
| PEPWR13                                                          | PEPWR14 | poweraudit |         |

watchpoint

Purpose

The watchpoint command can be used to specify watch points in a design.

In case of a functional rule-violation reported by SpyGlass Auto Verify solution or SpyGlass CDC solution, the simulation trace produced contains only the registers and primary inputs involved in the failure. The watchpoint constraint can be used to force SpyGlass Auto Verify solution and SpyGlass CDC solution to generate the waveform for an internal signal.

Product

SpyGlass Auto Verify solution, SpyGlass CDC solution

Syntax

The syntax of the watchpoint constraint is as follows:

```
watchpoint
-name <pin-name-list>
```

## Arguments

**-name <pin-name-list>**

Space-separated list of hierarchical pin names.

## Examples

The following constraint defines signals `sig3` in the design unit `top` as a watch point:

```
watchpoint -name top.sig3
```

## Rules

The `watchpoint` constraint is used by the following rules:

| SpyGlass CDC Solution         |                    |                    |               |
|-------------------------------|--------------------|--------------------|---------------|
| Ac_cdc01a                     | Ac_cdc01b          | Ac_cdc01c          | Ac_cdc08      |
| Ac_fifo01                     | Ac_handshake0<br>1 | Ac_handshake0<br>2 | Clock_sync03a |
| Ac_conv02                     |                    |                    |               |
| SpyGlass Auto Verify Solution |                    |                    |               |
| All rules                     |                    |                    |               |

## wireload\_selection

### Purpose

Specifies the default wire-load selection table to be used for power estimation by the rules in the SpyGlass Power Estimation and SpyGlass Power Reduction solutions. Normally, the `default_wire_load_selection` library attribute defines the default wire-load selection table to be used. You can overwrite this default value or specify the value if not available in the library using the `wireload_selection` constraint.

**NOTE:** Prior to SpyGlass 4.3.0 release, the name of this constraint was `wireloadselection`.

## Product

SpyGlass Power Estimation and SpyGlass Power Reduction solutions

## Syntax

The syntax to specify the `wireload_selection` constraint is as follows:

```
current_design <top-du-name>
 wireload_selection
 -name <wlst-name>
 [-libname <lib-name>]
```

## Arguments

### <top-du-name>

Module name (for Verilog designs) or design unit name in <entity-name>.<arch-name> format (for VHDL designs).

### -name <wlst-name>

Name of the wire-load selection table of the library <lib-name>.

### -libname <lib-name>

(Optional) Library name.

Specify a library name only when a wire-load selection table with the same name is available in more than one library.

**NOTE:** Please note the following points:

- If you do not use the `-name` argument, the default wire-load selection table name will be taken from the library specified with the `-libname` argument.
- If you do not use the `-libname` argument, the library that contains the specified name of the wire-load selection table will be taken.
- Though both the `-name` and `-libname` arguments are optional, you must specify at least one argument with the `wireload_selection` constraint. If you do not specify any one argument, the above rules report a syntax error.

The actual wire-loads used by these rules are reported in the `pe_wireload` Report in the SpyGlass Power Estimation and SpyGlass



Power Reduction solutions.

Rules

The wireload\_selection constraint is used by the following rules:

| SpyGlass Power Estimation and SpyGlass Power Reduction Solutions |         |            |         |
|------------------------------------------------------------------|---------|------------|---------|
| PEPWR01                                                          | PEPWR02 | PEPWR03    | PEPWR05 |
| PEPWR13                                                          | PEPWR14 | poweraudit |         |

